Grundgebiete der Informatik 1

Programmierung, Algorithmen und Datenstrukturen

Prof. Dr.-Ing. Tobias Gemmeke

Integrierte Digitale Systeme und Schaltungsentwurf (IDS)

gemmeke@ids.rwth-aachen.de





WS 2026/2027

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Inhalt

1. Einleitung
   - 1.1 Thematische Einordnung 16
   - 1.2 Hardware und Software 19
   - 1.3 Programmiersprachen 22
   - 1.4 Algorithmen und Programme 26
2. Einführendes Beispiel
   - 2.1 Temperaturumrechnung 31
   - 2.2 Analyse des Beispiels 35
   - 2.3 Ausführung und Fehlersuche 42
3. Elementare Datentypen und Anweisungen
   - 3.1 Einführung 47
   - 3.2 Variablen und Konstanten 53
   - 3.3 Arithmetische und logische Operationen und Zuweisungen 74
   - 3.4 Kontrollanweisungen 81
   - 3.5 Zeiger 59

Inhalt

4. Funktionen und Programmaufbau

4.1 Funktionen 97

4.2 Parameterübergabe 107

4.3 Polymorphie 112

4.4 Templates 116

4.5 Lambda Funktionen 118

4.6 Module und Programme 120

5. Datentypen

5.1 Arrays 123

5.2 Strukturen 133

5.3 Abstrakte Datentypen 137

5.4 Klassen 139

(De)Konstruktor 141

Public vs. private 143

Vererbung 146

Abstrakte Klassen 150

Überladen von Operatoren 152

Inhalt

6. Analyse von Algorithmen
   - 6.1 Laufzeitanalyse 159
   - 6.2 O-Notation für Wachstumsordnungen 165
   - 6.3 Beispiel: Exponentialfunktion 175
   - 6.4 Beispiel: Suchen in Arrays 180
7. Sortieralgorithmen
   - 7.1 Einfache Verfahren (Selection/Insertion/Bubblesort) 189
   - 7.2 Quicksort 205
   - 7.3 Heapsort 215
8. Lineare Datentypen
   - 8.1 Listen 229
   - 8.2 Stacks 242
   - 8.3 Queues 251
9. Bäume
   - 9.1 Definition und Darstellung 261
   - 9.2 Baumdurchläufe 265
   - 9.3 Suchbäume 269

Inhalt

10. Graphen
    - 10.1 Definition und Grapheigenschaften 277
    - 10.2 Berechnung kürzester Pfade 283
    - 10.3 Datenstrukturen zur Graphrepräsentation 294
    - 10.4 Graphdurchläufe 300
    - 10.5 Spannbäume 308
11. Techniken zum Algorithmenentwurf
    - 11.1 Rekursion und Iteration 316
    - 11.2 Backtracking 323
    - 11.3 Kombinatorische Optimierung 330
    - 11.4 Schwierige Probleme und Heuristiken 335
    - 11.5 Dynamische Programmierung 341
    - 11.6 Branch and Bound 346

RWTH-Studiengänge

- B.Sc. Elektrotechnik und Informationstechnik
- B.Sc. Computer Engineering
- B.Sc. Wirtschaftsingenieurwesen EET

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Übungsbetrieb

- **Koordination / Fachstudienbetreuung**
  - Dipl.-Inform. Tatjana Eiden



- **Gemeinsame Globalübung**
  - Do 12:15-13:15
  - Vor allem: c/C++-Programmierkurs
  - Michael Gansen



- **Kleingruppenübungen:**
  - Interaktive Vertiefung und Ergänzung des Vorlesungsstoffes
  - Tutoren: studentische Hilfskräfte
  - Ca. 16 wöchentliche Termine zur Auswahl
  - Vorauss. ab 2. Vorlesungswoche (siehe RWTHonline)

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Kleingruppenübungen

- Teilnahme nicht obligatorisch aber dringend empfohlen
- Anmeldungsinfo: **siehe RWTHonline**
- Anmeldeschluss beachten!
- **Gruppeneinteilung**: Benachrichtigung per email
- **Übungsaufgaben**: jeweils am Fr online über Moodle
- Vorbereitungsaufgaben, Präsenzaufgaben und Nachbereitungsaufgaben

Alle Studierende mit 100 von 200 Punkten in den Hausübungen **haben** in 21/22 **bestanden.**

Teilnahme an der **Probeklausur** führte in 21/22 zu folgender **Verbesserung:**

$$\mathbf{2} \xrightarrow{50\%} \mathbf{1} \qquad \mathbf{3} \xrightarrow{90\%} \mathbf{<3} \qquad \mathbf{4} \xrightarrow{90\%} \mathbf{<4} \qquad \mathbf{5} \xrightarrow{80\%} \mathbf{<5}$$

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Organisatorisches

- **Klausur**
  - 1. Versuch: siehe RWTHonline
  - 2. Versuch: siehe RWTHonline
- **Fragestunde GGI1** bei Fr. Eiden
  - Di 12:15-13:45 (UMIC, ab 20.10.)
  - Do 13:15-14:00 (H02, ab 15.10.) im Anschluss an die Globalübung
  - sowie nach vorheriger Absprache per E-Mail
- **Aktuelle Nachrichten**
  - Siehe RWTHonline
  - Sprechstunden bei Prof. Gemmeke n.V. per Email:

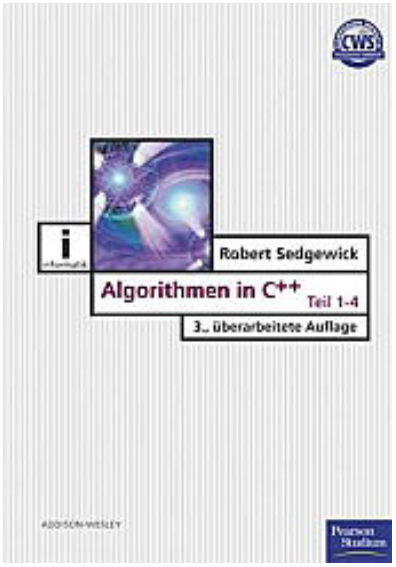
    info@ids.rwth-aachen.de

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Literatur

- B.W. Kernighan, D.M. Ritchie:
  Programmieren in C,
  2. Auflage, Hanser, 1990, ISBN 3-446-15497-3
- R. Sedgewick: Algorithmen in C++,
  Addison-Wesley, 2002, ISBN 3-8273-7026-4
- In der Lehrbuchsammlung
- Kommentierte Vorlesungsfolien
- Grundsätzlich auch andere Einführungen in die Programmierung als Hilfsmittel geeignet

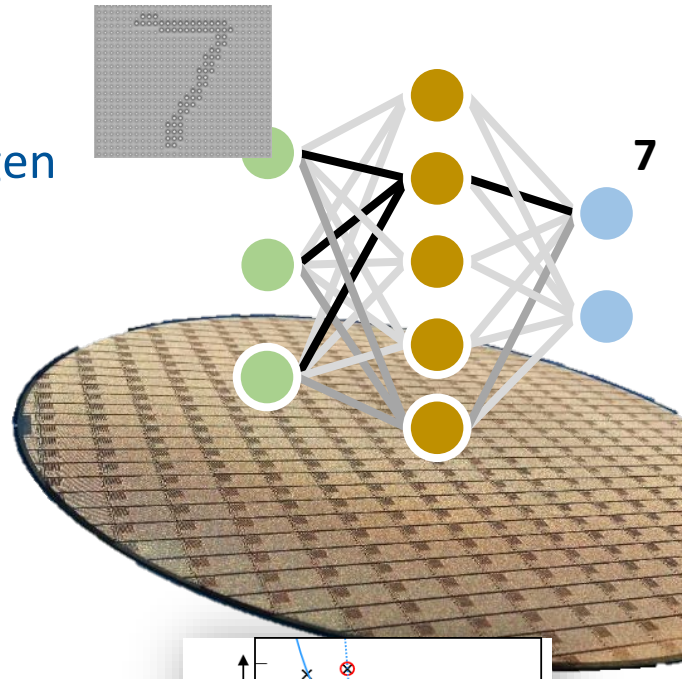


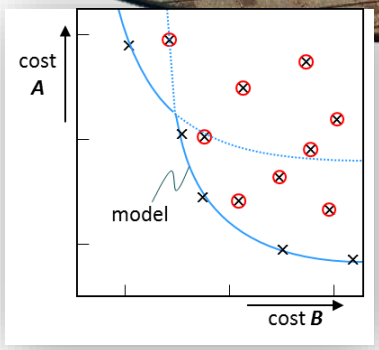


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Über das IDS (www.ids.rwth-aachen.de)

- Lehrstuhl für Integrierte Digitale Systeme und Schaltungsentwurf (IDS)
- Schwerpunkte:
  - Entwurfsmethodik integrierte digitale Schaltungen
  - Beschleuniger für verschiedene Anwendungen:
    - Simulation biologischer neuronaler Netzwerke
    - Inferenz in künstlichen neuronalen Netzen
    - Kryptographie
    - …
  - Testmethoden
- Standort: UMIC Gebäude
  - Mies-van-der-Rohe Str. 15, RWTH Hörn Campus

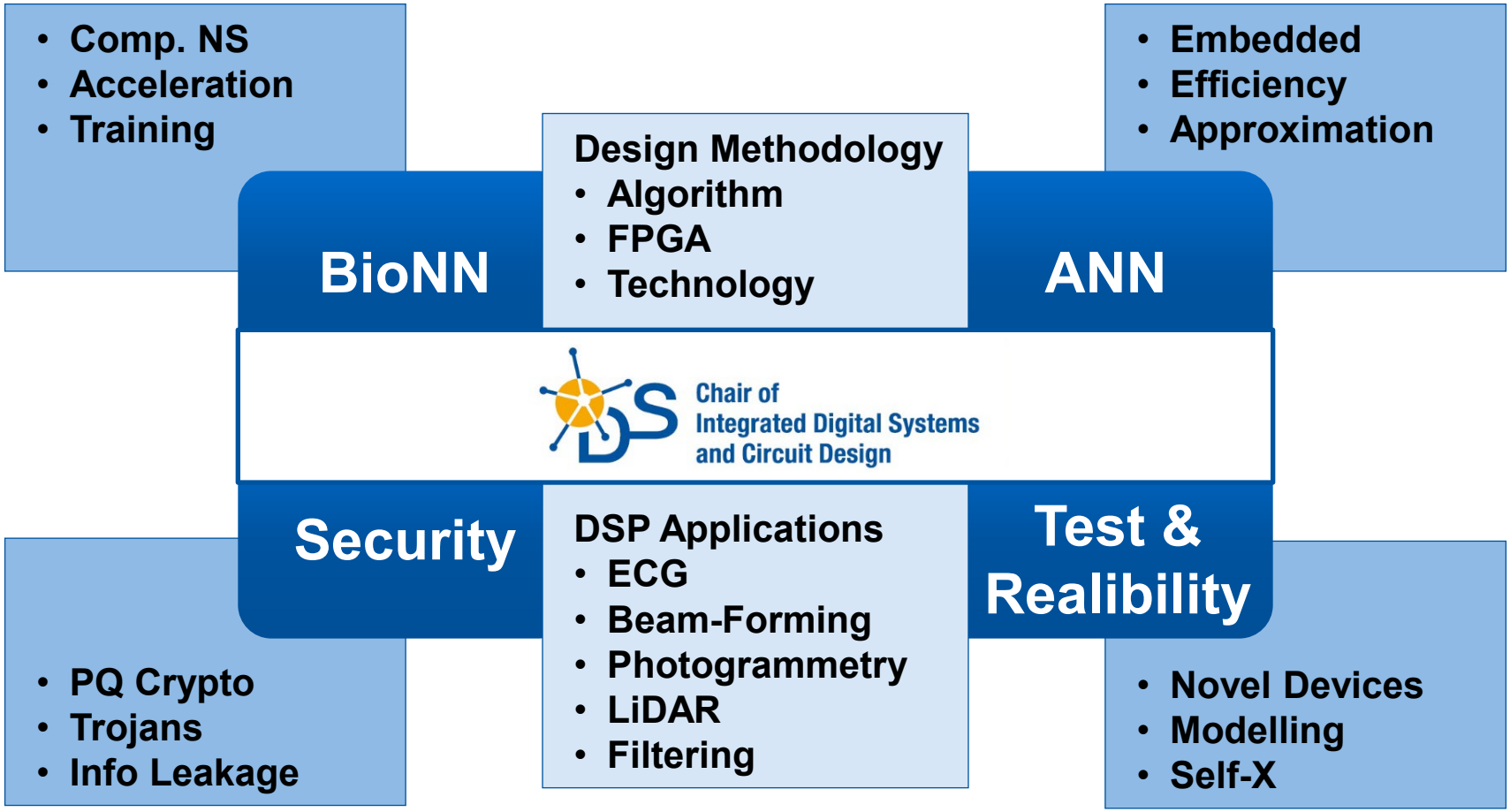






© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

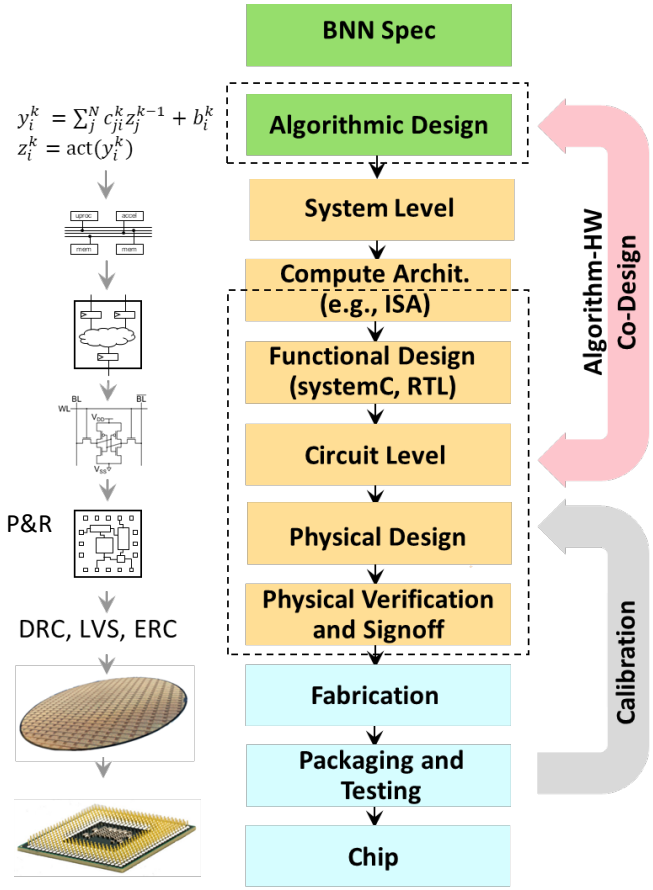
Forschungsgebiete am Lehrstuhl IDS



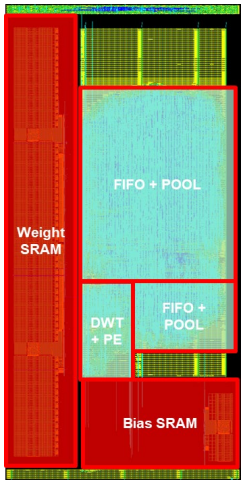
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

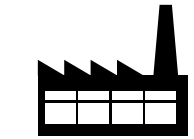
Entwurfsablauf

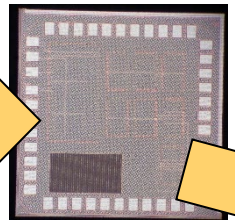
Chip-Entwurf

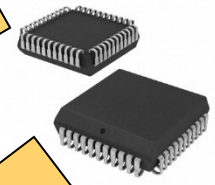


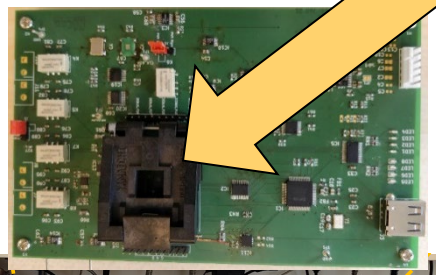
vom Layout zum Bring-Up

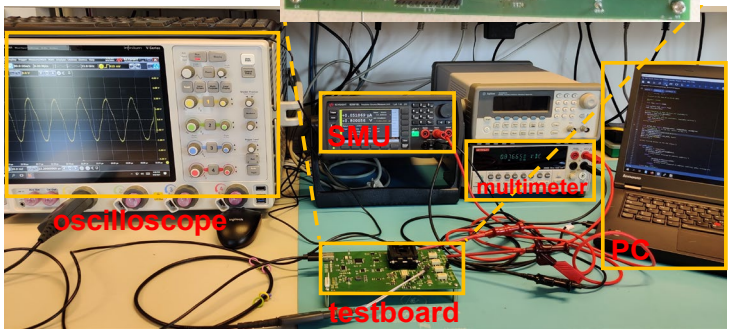












© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Kooperationsprojekte (Auswahl)

Advanced Computing Architectures (ACA)

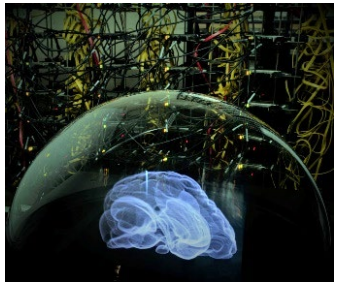




Neuro Sys KI

NEUROTEC I&II

towards multi-scale natural-density Neuromorphic Computing



neuroAI^x



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

IDS-Lehrveranstaltungen

* Mehrere Lehrstühle tragen zur Durchführung der Veranstaltung bei

- **Bachelor:**
  - Grundgebiete der Informatik 1 (GGI1) 1. Sem
  - *Projekt PETIT ("MATLAB meets LEGO Mindstorms„) 1. Sem
  - *Praktikum Informatik 2 Teile
  - VLSI Schaltungen & Architekturen (VLSI-CA) 4. Sem Pflicht CE bzw. 6. Sem Wahl ET&IT
  - *Grundlagen Integrierter Schaltungen und Systeme (GISS) 5. Sem Wahl ME
  - Institutsprojekt „Anwendung von künstlichen neuronalen Netzen zur Linienverfolgung mit Braitenberg-Vehikeln“ 4. Sem
  - *Praktikum Mikroelektronik 5. Sem Pflicht in ME
  - Bachelor-Arbeiten zu aktuellen Forschungsthemen
- **Master**
  - VLSI Design for Digital Signal Processing – Fundamentals (VLSI-DF)
  - VLSI Design for Digital Signal Processing – Architecturs (VLSI-DA)
  - Computer Arithmetic – Fundamentals (CAR-F)
  - Computer Arithmetic – Advanced Topics (CAR-A)
  - Laboratory: VLSI Design
  - Laboratory: FPGA Design
  - Master-Arbeiten zu aktuellen Forschungsthemen

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

1. Einleitung

1.1 Thematische Einordnung

1.2 Hardware und Software

1.3 Programmiersprachen

1.4 Algorithmen und Programme

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

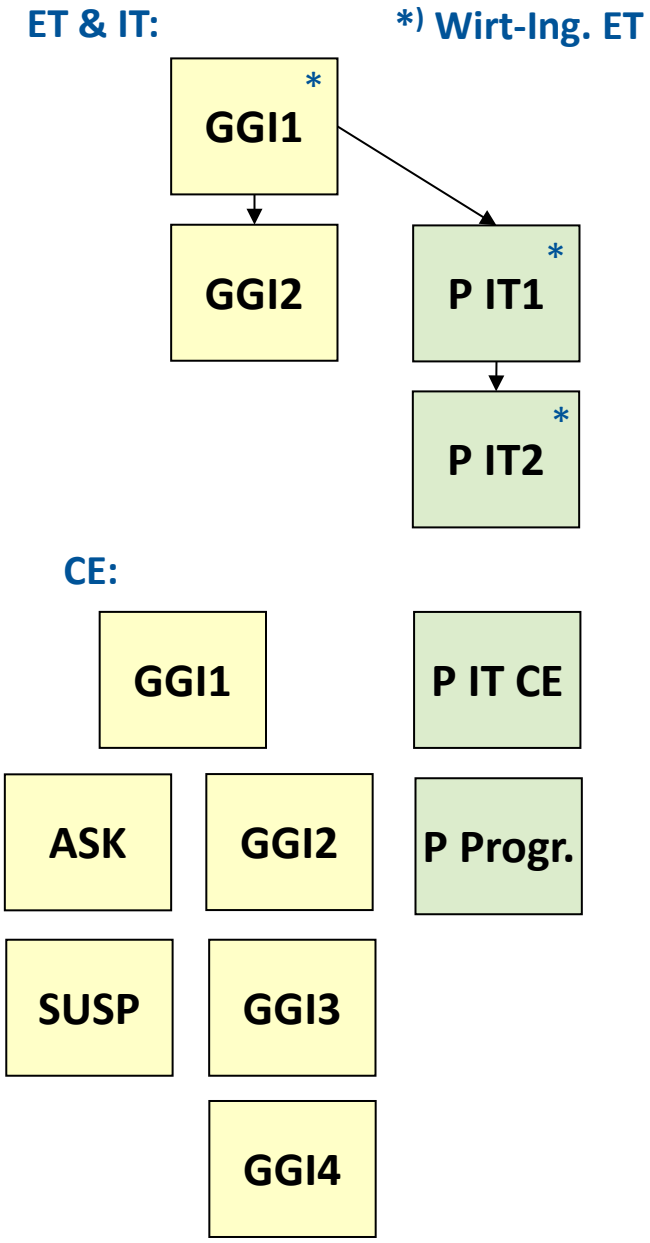
Was ist Informatik?

- **Informatik** (Computer Science) ist die **Wissenschaft**, die sich mit der Entwicklung und Nutzung von Computersystemen beschäftigt
- Wurzeln in der **Mathematik** und **Elektrotechnik**
- **Informatikkenntnisse** (speziell: Programmierkenntnisse) gehören heute zum Handwerkszeug jedes Ingenieurs
- Einteilung in drei große **Teilgebiete**:
  1. Theoretische Informatik: Computermodelle, Sprachen, Boolesche Funktionen, Komplexitätsanalyse, ...
  2. Praktische Informatik: Programmiersprachen, Compiler, Betriebssysteme, SW Engineering, Datenbanken, ...
  3. Technische Informatik: Aufbau von Computer-Hardware, Chip-Entwurf, Netzwerke, Mobilfunk, ...

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Einordnung GGI1

- **GGI1** ist in erster Linie in der **praktischen Informatik** angesiedelt
- **GGI2**: Technische Informatik: Prinzipien des Digitalrechners
- **Automaten, Sprachen, Komplexität** (CE): Theoretische Informatik
- **GGI3**: Praktische und Technische Informatik: Optimierung, Modellierung und Parallelität
- **SUSP:** Praktische Informatik: System-SW und systemnahe Program.
- **GGI4:** Prakt. Inf.: Betriebssysteme
- Daneben: **Projekte** und **Praktika**
- Grundlagen für weitere **Spezialisierung** ab 5. Semester



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

1. Einleitung

1.1 Thematische Einordnung

1.2 Hardware und Software

1.3 Programmiersprachen

1.4 Algorithmen und Programme

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Hardware und Software

- **HW** = Physikalisch existierender, programmierbarer Rechner (Computer)
  → Modifikation aufwendig
- **SW** = Auf HW ausführbares Programm (Textdatei) für gegebene Anwendung
  → Modifikation einfach
- In GGI1 wird die HW als **Black Box** betrachtet, die Programmcode ausführen kann
- Innerer HW-Aufbau: siehe Vorlesung **GGI2**

User



SW

```
void rgb2ycc1(
  signed char r, signed char g, signed char b,
  signed char *y, signed char *cb, signed char *cr)
{
  *y  = ( 77*r + 150*g +  29*b       ) >> 8;
  *cb = (-43*r -  85*g + 128*b + 32768) >> 8;
  *cr = (128*r - 107*g -  21*b + 32768) >> 8;
}
void rgb2ycc(int NP, signed char *RGB, signed char *YCC)
{
  int i;
  for (i = 0; i < 3 * NP; i += 3) {
    rgb2ycc(RGB[i], RGB[i+1], RGB[i+2], &YCC[i], &YCC[i+1], &YCC[i+2]);
  }
}
```

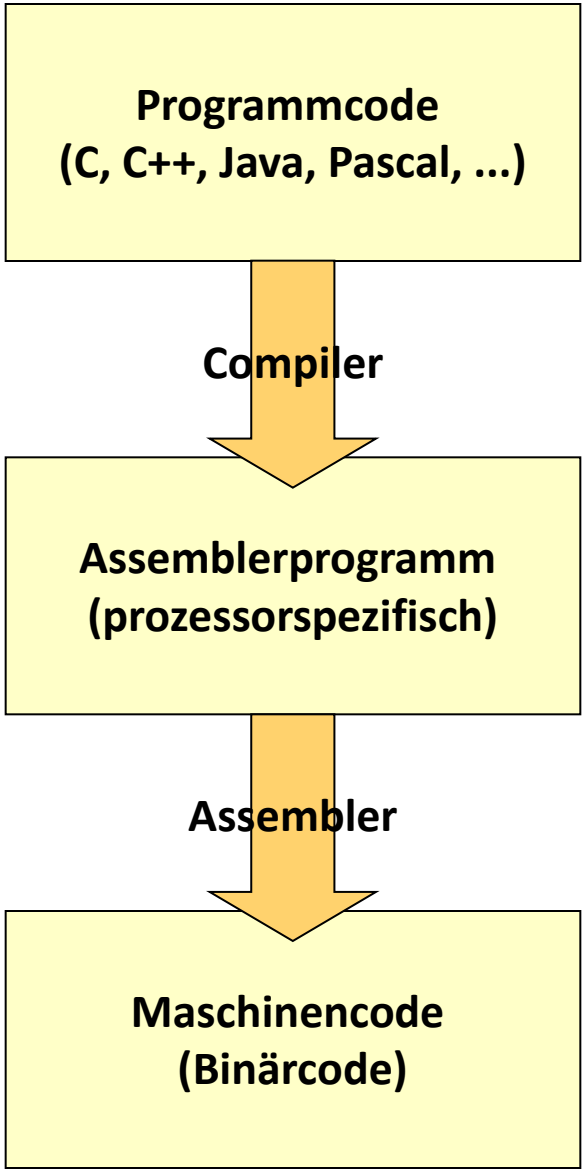
HW



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Abbildung von SW auf HW

- SW wird meist in höheren **Programmiersprachen** entwickelt
- Programmiersprachen als **Schnittstelle** sind sowohl für Menschen als auch für Computer verständlich
- **Compiler** übersetzt Hochsprachen-Programm in **Assemblerprogramm**, bestehend aus einzelnen (noch lesbaren) Prozessorbefehlen
- Übersetzung von **Quellcode** in **Zielcode**
- Auch die direkte Entwicklung von **Assemblerprogrammen** ist möglich
- Assembler übersetzt Assemblerprogramm in binären **Maschinencode**, der direkt auf der HW ausgeführt werden kann



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

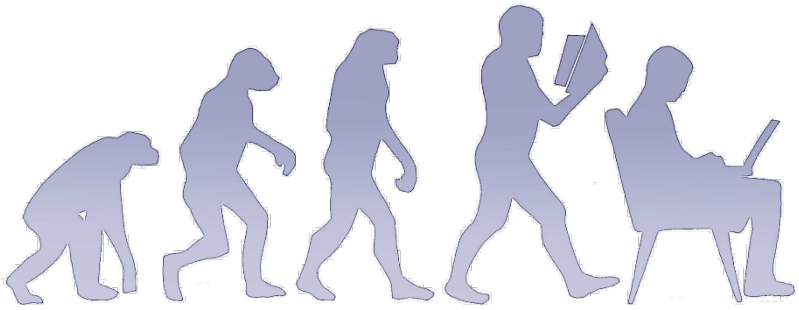
1. Einleitung

1.1 Thematische Einordnung
1.2 Hardware und Software
1.3 Programmiersprachen
1.4 Algorithmen und Programme

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Programmiersprachen

- Computer wurden zu Anfang direkt in **Maschinencode** programmiert
- Später Übergang zu **symbolischem** Maschinencode (Assembler)
- Dann Einführung von (höheren) **Programmiersprachen**



- **Vorteile**:
  - lesbar und wartbar (bei gutem Programmierstil)
  - portierbar (nicht unbedingt zu 100%)
- „Evolution“ der Sprachen setzt sich ständig fort
  - Zukünftig: für parallele **Multiprozessorsysteme**
- Eine neue Sprache alleine ist praktisch wertlos, man benötigt
  - **Programmierwerkzeuge**, z.B. Compiler
  - **Akzeptanz** bei den Benutzern (u.a. wegen *legacy code*)

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

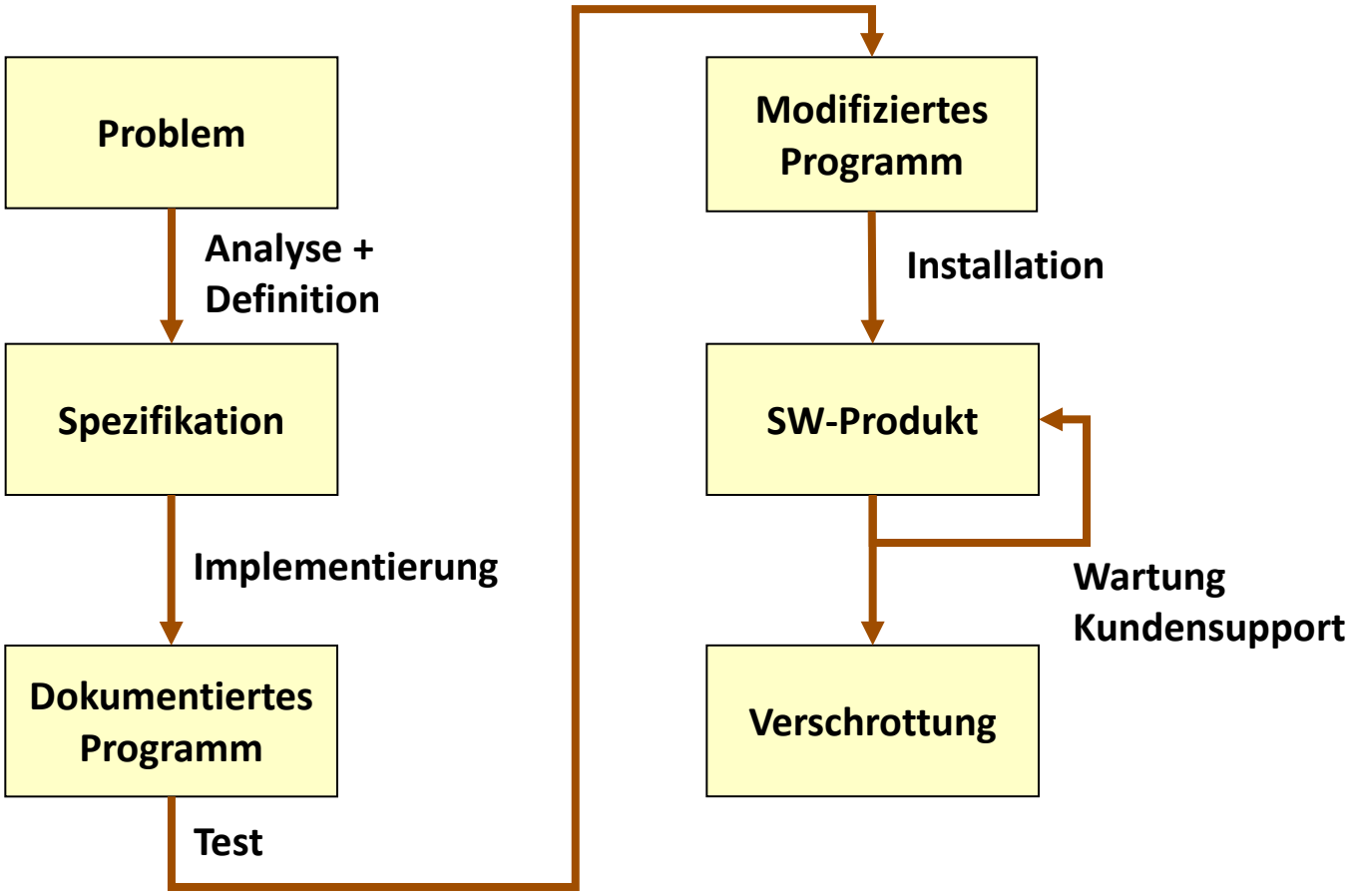
Wahl der Programmiersprache

- Einteilung:
  - **Imperative Sprachen**: konkrete Beschreibung eines Algorithmus als Folge von Anweisungen (Beispiel: C, Pascal)
  - **Deklarative Sprachen**: Beschreibung des erwünschten Programmverhaltens, konkrete Umsetzung erfolgt teilweise durch den Compiler und Laufzeitsystem (Beispiel: LISP, Prolog)
  - **Objektorientierte Sprachen**: Zusammenfassung von Algorithmen und Daten zu Objekten, dadurch einfachere Programmwartung (Beispiel: C++, Java)
  - Einteilung nicht disjunkt, z.B. C $\subset$ C++
- Für GGI1: **C und C++** aufgrund großer praktischer Relevanz
- Nicht alle **Sprachdetails** werden vorgestellt
- Programmieren lernt man nur durch **Übung**!

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Software Life Cycle

- GGI1 stellt elementare Grundlagen der SW-Entwicklung bereit
- Typischer SW Life Cycle:



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

1. Einleitung

1.1 Thematische Einordnung
1.2 Hardware und Software
1.3 Programmiersprachen
1.4 Algorithmen und Programme

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Grundbegriffe aus GGI1

- **Algorithmus:**
  - Abstrakte Handlungsanweisung, um ein Problem zu lösen
  - Ausgabe als Funktion der Eingabe
  - In endlicher Zeit, eindeutig und deterministisch
  - Kann in natürlicher Sprache formuliert sein
- **Programm:**
  - Codierung eines Algorithmus in Programmiersprache
- **Datenstruktur:**
  - Mathematisches Objekt zur Speicherung von Daten
  - Auf bestimmte Weise angeordnet und zusammengefasst
  - Sinnvoll nur im Zusammenhang mit Operationen auf den Daten
  - Beispiele: Arrays, Listen, Stacks, Bäume, Graphen, ...

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Vom Problem zum Programm

- Für jedes (per Computer lösbare) Problem gibt es **unendlich viele** Algorithmen
- Für jeden Algorithmus existieren **unendlich viele** Implementierungen in einer Programmiersprache
- **Anforderung:** Finde den besten Algorithmus und die beste Implementierung hierfür; Kriterien:
  - Korrektheit
  - Effizienz (Laufzeit und/oder Speicherverbrauch)
  - Eleganz, Programmierstil
- **Korrektheit** eines Algorithmus lässt sich (mathematisch) beweisen, Korrektheit eines Programms oft nicht
- Zur Bewertung der Effizienz von Algorithmen dient die **Komplexitätsanalyse**
- Guter **Programmierstil** muss praktisch erlernt werden

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Grenzen der Programmierung

- Wann ist ein Problem per Computer lösbar?
  - Problemlösung muss **mathematisch exakt** als Algorithmus beschreibbar sein
  - Bsp.: Sortiere eine Menge von Namen in lexikographischer Reihenfolge
- Jedoch: Viele wichtige Probleme sind **prinzipiell lösbar**, aber in der Praxis zu komplex
  - Bsp.: Primfaktorzerlegung sehr großer Zahlen
  - Bsp.: Optimale Routenplanung für Spediteure (**Traveling Salesman Problem**)
  - Auch schnellere Rechner helfen **nicht** weiter
- Andere Probleme sind **prinzipiell unlösbar**
  - Bsp.: Bestimme, ob ein Programm nach endlicher Zeit auf gegebener Eingabe anhält (**Halteproblem**)

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Inhalt der Vorlesung

1. Einleitung
2. Einführendes Beispiel
3. Elementare Datentypen und Anweisungen
4. Funktionen und Programmaufbau
5. Datentypen
6. Analyse von Algorithmen
7. Sortieralgorithmen
8. Lineare Datentypen
9. Bäume
10. Graphen
11. Techniken zum Algorithmenentwurf

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

2. Einführendes Beispiel

2.1 Temperaturumrechnung

2.2 Analyse des Beispiels

2.3 Ausführung und Fehlersuche

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Einführendes C-Programmbeispiel

- Umwandlung von Grad Fahrenheit in Grad Celsius
- $\frac{y}{°C} = (5/9) \times \left(\frac{x}{°F} - 32\right)$

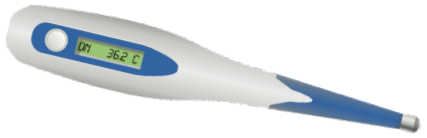
```
#include <stdio.h>

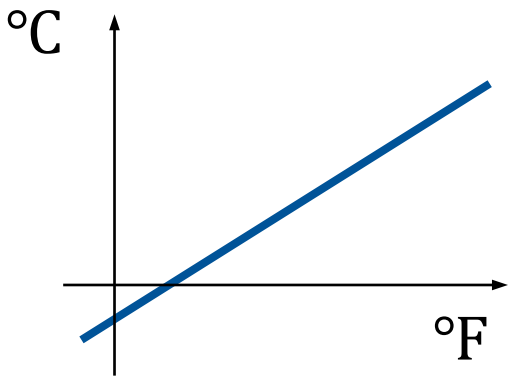
/* Umwandlung von Fahrenheit in Celsius
    fuer fahr = 0, 20, …, 300   */
int main()
{
    int fahr, celsius;
    int lower, upper, step;

    lower = 0; // untere Grenze der Temperaturtabelle
    upper = 300; // obere Grenze
    step  = 20; // Schrittweite

    fahr  = lower;
    while (fahr <= upper)  {
        celsius = 5  *  (fahr-32) / 9;
        printf("%d\t%d\n", fahr, celsius);
        fahr = fahr + step;
    }

    return 0;
}
```





© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Libraries

- Von vielen Programmen häufig wiederverwendeter Code wird in Bibliotheken (Libraries) abgelegt
- Library-Code wird nur einmal compiliert und kann dann von allen Programmen eingebunden werden
- Dies erfolgt nach der Assemblierung durch den Linker
- C besitzt insges. 29 Standard-Libraries, z.B.:
  - stdio: Ein- und Ausgabe
  - math: Mathematische Funktionen
  - string: String-Manipulation
- Durch Einbinden einer Header-Datei (z.B. „stdio.h“) werden die Library-Funktionen dem benutzenden Programm bekannt gemacht
- Programmierer können (und sollten) auch eigene, benutzerspezifische Libraries anlegen

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Standardbibliothek C++ - Beispiele

- **Zeichenketten und Streams**
  - string, iomanip, iostream, cstdio
- **Abtrakte Datentypen / Container**
  - vector, array, (forward_)list, queue, stack
  - (unordered_)set, (unordered_) map
  - iteratoren
- **Smart Pointer**
  - unique_prt
  - shared_prt
  - weak_ptr
- **Algorithmen**
  - for_each, sort, copy, find & search, transform
- **Funktoren**
  - multiplies, negate, …

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

2. Einführendes Beispiel

2.1 Temperaturumrechnung

2.2 Analyse des Beispiels

2.3 Ausführung und Fehlersuche

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Analyse des C-Programms

- **Einbinden** der Standard-Library zur Ein/Ausgabe
  - `#include <stdio.h>`
  - Ermöglicht Kommunikation des Programms mit der Außenwelt (z.B. Benutzer)
  - Siehe C-Präprozessor
- **Kommentare**
  - `/* Umwandlung … */`
  - `// untere Grenze …`
  - Erläuterung der Programmfunktion für Entwickler
  - Text innerhalb `/* … */` und in einer Zeile nach `//` wird vom Compiler ignoriert
- **Hauptfunktion** `main`
  - `int main() { … }`
  - Alle Anweisungen innerhalb `{ … }` gehören zu `main`
  - `main` wird nach Programmstart immer direkt aufgerufen
  - Im Allg. ruft `main` dann weitere Funktionen auf

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Analyse des C-Programms

- **Variablen-Vereinbarungen**
  - `int fahr, celsius;`
  - Variablen sind benannte Speicherplätze für Werte
  - Können lesend und schreibend zugegriffen werden
  - Variablen müssen vor Verwendung vereinbart werden
  - Hier: Verwendung von ganzzahligen (integer) Variablen
- **Zuweisungen**
  - `lower = 0;`
  - `upper = 300;`
  - Schreiben von Werten in die Variablen
- **Schleife**
  - `while (fahr <= upper) { /* ... */ }`
  - `while` ist (wie auch `int`) ein **C-Schlüsselwort**
  - Code innerhalb `{ … }` wird solange wiederholt, bis Bedingung nicht mehr erfüllt

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Analyse des C-Programms

- **Umrechnung**
  - `celsius = 5 * (fahr-32) / 9;`
  - Rechenoperationen auf Variablen
- **Ausgabe**
  - `printf("%d\t%d\n", fahr, celsius);`
  - Formatierte Ausgabe:
    - `fahr` als Integer-Zahl (erstes `%d` als Platzhalter)
    - Tabulator `\t`
    - `celsius` als Integer-Zahl (zweites `%d` als Platzhalter)
    - Zeilenumbruch `\n`
- Vorbereitung des **nächsten Schleifendurchlaufs**
  - `fahr = fahr + step;`
  - `fahr` wird um `step` erhöht, bis `upper` überschritten

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

- **Formatierte Ausgabe**

Classic C++

```
#include <iostream>  // std::cout, std::right, std::endl
#include <iomanip>   // std::setw

int main() {
    int val = 65;

    std::cout << "Formatted output using C++" << std::endl;
    std::cout << "-start-" << std::endl;
    std::cout << std::right;  // right-adjusted (manipulator)
    std::cout << "'" << std::setw(10) << val << "'" << std::endl;
    std::cout << "-end-" << std::endl;

    return 0;
}
```

C++11 oder c

```
#include <cstdio>  // printf; for c library: <stdio.h>, no "using ..."
using namespace std;

int main() {
    int val = 65;

    printf("Now using C++11:\n");
    printf("-start-\n'%10d'\n-end-\n", val);

  return 0;
}
```


© T. Gemmeke, IDS, RWTH

- `Using` und namespace

```
#include <string>
#include <iostream>

int main( void ) {

    std::string msg = "Hello world!";

    std::cout << msg << std::endl;

    return 0;
}
```

```
#include <string>
#include <iostream>

using std::cout, std::endl, std::string;
// using namespace std;

int main( void ) {

    string msg = "Hello world!";

    cout << msg << endl;

    return 0;
}
```

std::cout → cout

© T. Gemmeke, IDS, RWTH

C-Präprozessor

- C-Präprozessor (cpp) läuft **vor dem Compiler** über den Quellcode (C-nach-C-Übersetzung)
- cpp-Anweisungen beginnen mit Zeichen #
- cpp-Verwendung erhöht **Lesbarkeit** der Programme
- Vor allem folgende Anwendungen:
  - **Symbolische Definitionen** (Konvention: Großbuchstaben)
    - `#define UPPER 300`
    - Alle Vorkommen von `UPPER` werden durch `300` ersetzt
  - **Makros**
    - `#define MAX(a,b) (a > b) ? a : b`
    - Alle Vorkommen von `MAX(..., ...)` werden ersetzt
  - **Einbinden** von Standard-Libraries (und allgemein Header-Dateien mit Deklarationen)
    - `#include <stdio.h>`
    - Datei `stdio.h` wird an dieser Stelle ins Programm eingefügt

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

2. Einführendes Beispiel

2.1 Temperaturumrechnung

2.2 Analyse des Beispiels

2.3 Ausführung und Fehlersuche

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Übersetzung und Ausgabe

- Programm in **Textdatei** „temp.c“ gespeichert
- Aufruf des **C-Compilers** (z.B. aus Linux shell)
  - `cc temp.c` (je nach Compiler)
  - **Präprozessor** und **Assembler** werden implizit aufgerufen
  - **Zielprogramm** „a.out“ wird erzeugt
  - **Ausführung** von „a.out“ generiert die gezeigte Ausgabe

Ausgabe

```
0     -17
20    -6
40    4
60    15
80    26
100   37
120   48
140   60
160   71
180   82
200   93
220   104
240   115
260   126
280   137
300   148
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Fehlerhafte Programme

- **Möglichkeit 1**: Programm entspricht nicht dem C-Standard
- Derartige Fehler werden vom Compiler (oder auch Linker) bemerkt und dem Programmierer mitgeteilt

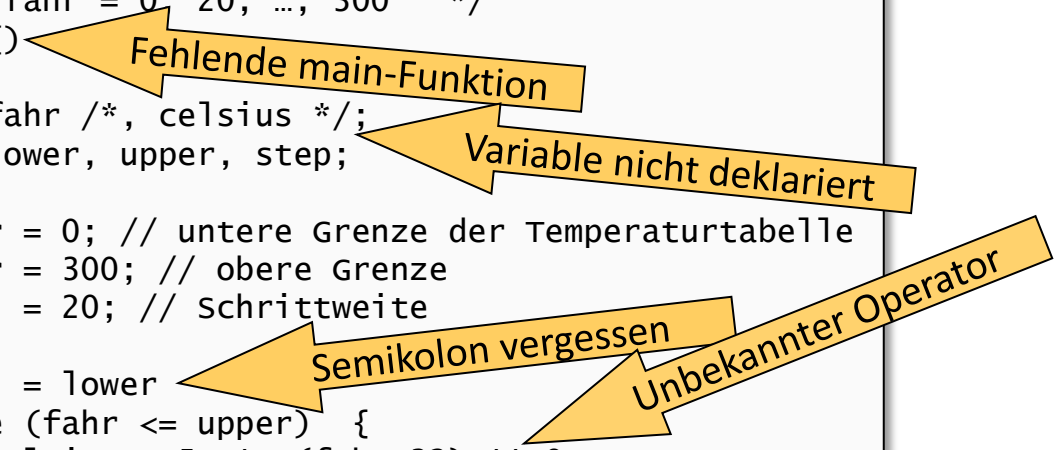
```
#include <stdio.h>

/* Umwandlung von Fahrenheit in Celsius
    fuer fahr = 0, 20, …, 300   */
int mein()
{
    int fahr /*, celsius */;
    int lower, upper, step;

    lower = 0; // untere Grenze der Temperaturtabelle
    upper = 300; // obere Grenze
    step  = 20; // Schrittweite

    fahr  = lower
    while (fahr <= upper)  {
        celsius = 5  *  (fahr-32) ** 9;
        printf("%d\t%d\n", fahr, celsius);
        fahr = fahr + step;
    }

    return 0;
}
```



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Fehlerhafte Programme

- **Möglichkeit 2**: Programm ist gültig (d.h. compilierbar), führt aber nicht die gewünschte Funktion aus (es hat „Bugs“)
- Wesentlich schwieriger zu finden und zu beheben!

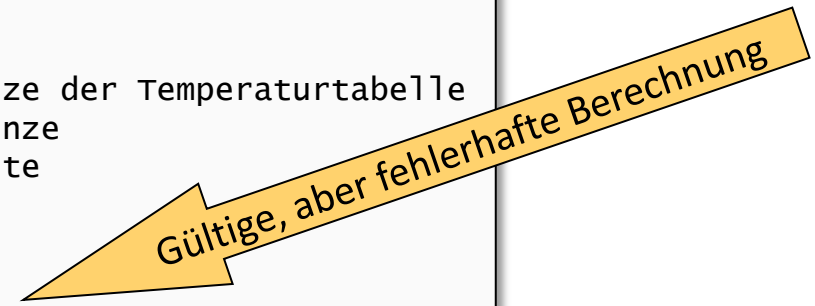
```
#include <stdio.h>

/* Umwandlung von Fahrenheit in Celsius
    fuer fahr = 0, 20, …, 300   */
int main()
{
    int fahr, celsius;
    int lower, upper, step;

    lower = 0; // untere Grenze der Temperaturtabelle
    upper = 300; // obere Grenze
    step  = 20; // Schrittweite

    fahr  = lower;
    while (fahr <= upper)  {
        celsius = 5  *  (fahr+32) / 9;
        printf("%d\t%d\n", fahr, celsius);
        fahr = fahr + step;
    }

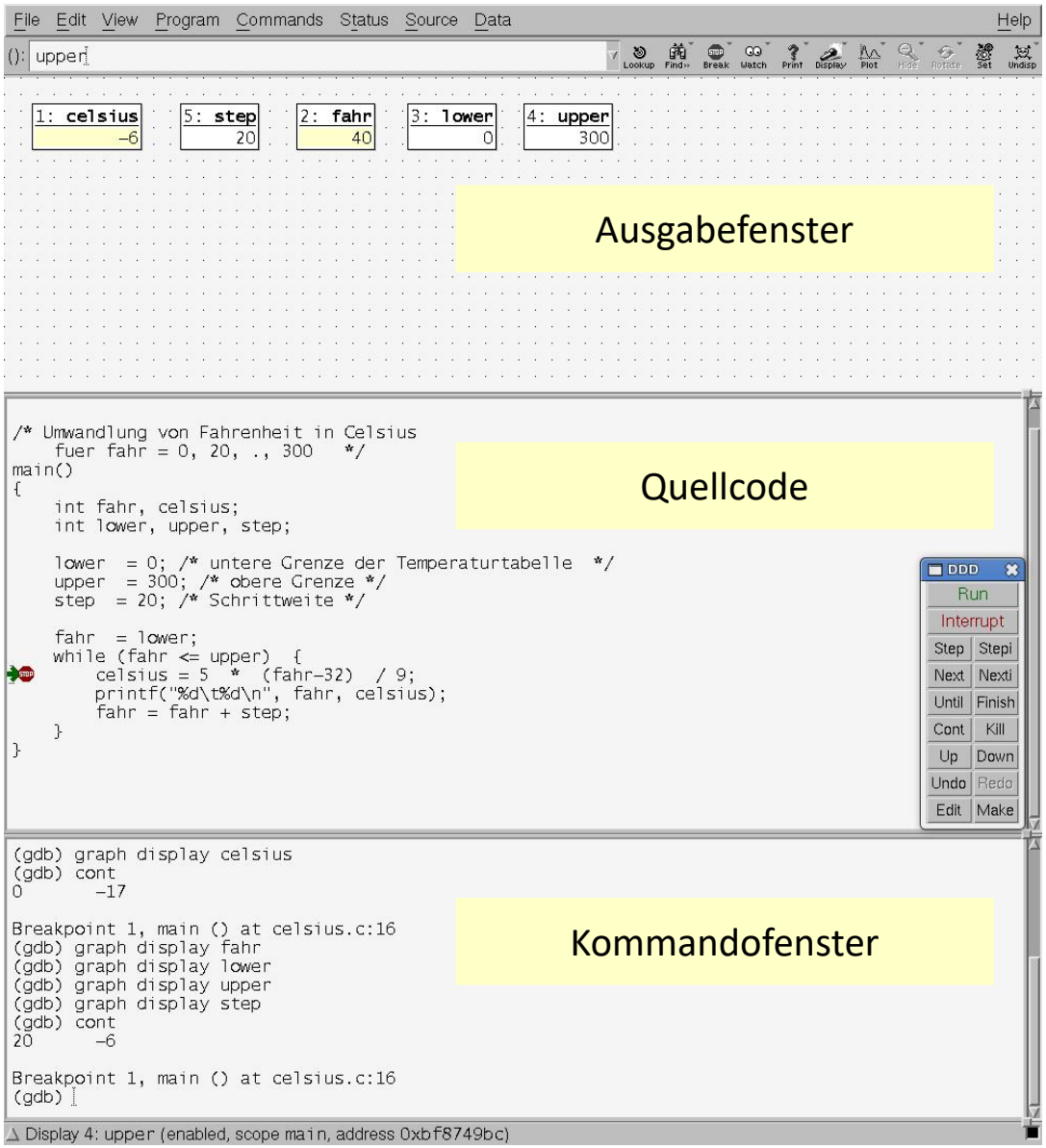
    return 0;
}
```



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Fehlerhafte Programme

- Programm unter Kontrolle eines **Debuggers** ausführen
- Ermöglicht **Einzelschrittmodus**, Beobachtung von **Variablen** u.v.m.
- Hilfreich vor allem bei **Endlosschleifen** und **Programmabstürzen**
- Debugger kann Programmverhalten beeinflussen („**Heisenbugs**“)



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

3. Elementare Datentypen und Anweisungen

3.1 Einführung
3.2 Variablen und Konstanten
3.3 Zeiger
3.4 Arithmetische und logische Operationen und Zuweisungen
3.5 Kontrollanweisungen

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Datentypen - Einführung

Def.: Ein **Datentyp** (**DT**) ist eine Menge von Werten und eine Sammlung von Operationen auf diesen Werten.

- Beispiel: **natürliche Zahlen** 1, 2, 3, ...
  - Operationen: +, -, *, /, ...
- Beispiel: **Zeichenketten** (Strings) über Alphabet A-Z
  - Operationen:
    - Bestimme Stringlänge
    - Ordne Strings alphabetisch an
    - ...
- Beispiel: Menge von **Personen-Stammdaten** (Name, Geburtsdatum, Adresse, ...)
  - Operationen:
    - Finde Teilmenge aller Personen älter als 50 Jahre
    - Finde Teilmenge aller Personen, die in Aachen wohnen
    - ...

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Datentypen - Einführung

Def.: Ein **abstrakter Datentyp** (**ADT**) ist ein Datentyp der nur über eine Schnittstelle zugänglich ist.

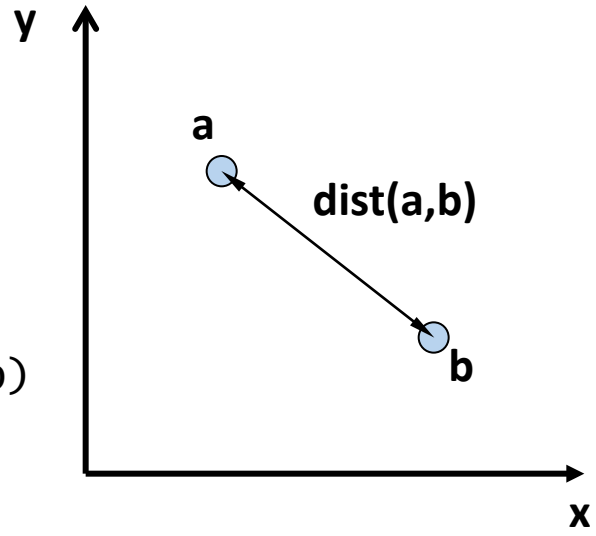
- Funktion vs. Implementierung des Datentyps
- Nur Funktion nach außen von Interesse
- Implementierung kann sich evtl. ändern

- Beispiel: DT „Punkt“ in C
- Koordinaten:

```
struct point { float x; float y; }
```

- Operation „Euklidischer Abstand“:

```
float dist(struct point a, struct point b)
{
  float dx, dy;
  dx = a.x – b.x;
  dy = a.y – b.y;
  return sqrt(dx*dx + dy*dy);
}
```



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Datentypen - Einführung

- In der C-Version sind Daten (point) und Operationen (dist) **getrennt**
- Programme könnten (und i.d.R.: werden) ohne „dist“ auch **direkt** auf die x- und y-Koordinaten zugreifen
- Ändert sich die interne Implementierung von point, so wäre evtl. ein Großteil des **übrigen Programmcodes** betroffen
- **ADT-Version** in C++: (siehe Kapitel 5)

```
class point {
  private:      // interne Repräsentation nicht nach außen sichtbar
    float x, y; // oder andere interne Repräsentation
  public:       // Schnittstelle für Benutzer von „point“
    float dist(point);
    /* ... */
};
```

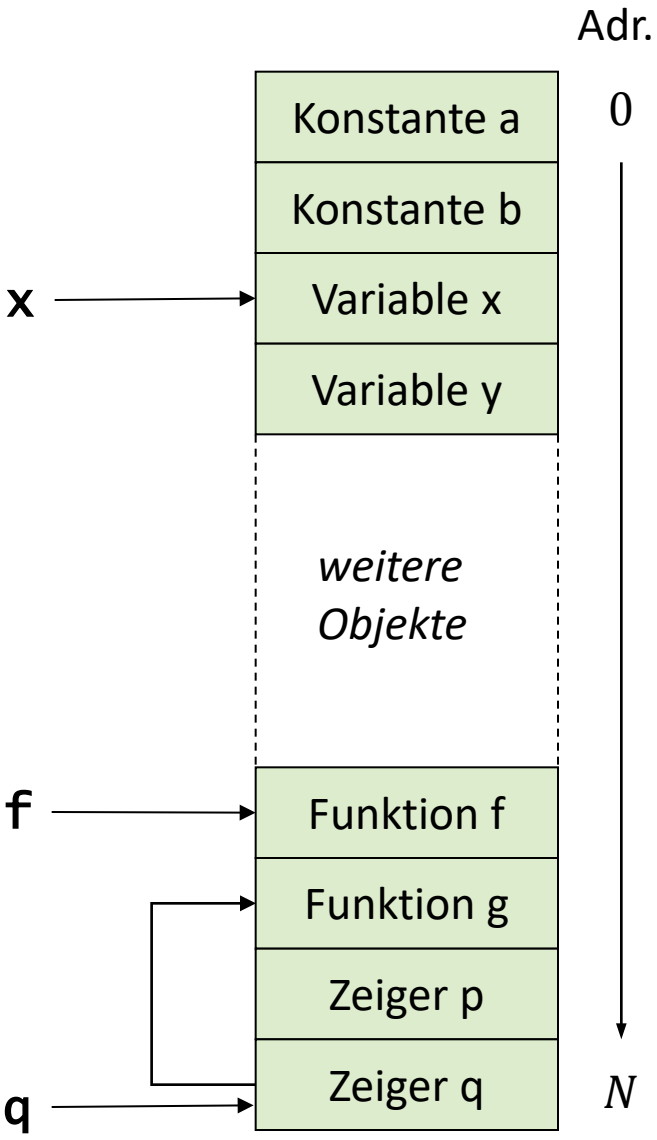
- Daten und Operationen per Konstruktion (class) **vereinigt**
- Interne Daten nur über Schnittstelle verfügbar!

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Objekte in C

- Variablen, Konstanten und Programmcode (Funktionen) werden im **Speicher** abgelegt
- Ein **Objekt** ist ein benannter Speicherbereich
- Auf Objekte wird direkt (per **Namen**) oder indirekt (per **Zeiger**) zugegriffen; Zeiger (symbolische Adressen) sind selbst Objekte
- Der **benötigte Speicherplatz** eines Objektes sowie die darauf **zulässigen Operationen** hängen vom Typ ab
- Im Speicher befinden sich nur **Binärzahlen**; Interpretation ist den Operationen überlassen

Vereinfachtes Speichermodell



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Basis-Datentypen in C

- `char`: **Zeichen** (character) = 1 Byte (8 Bits)
- `int`: **ganze Zahl** (integer)
  - `short int`: kleinerer Wertebereich (mind. 16 Bit)
  - `long int`: größerer Wertebereich (mind. 32 Bit)
- `float`: **Fließkommazahl** (floating point)
  - Aufteilung in Exponenten und Mantisse
  - z.B. „12.34E10“ = $12{,}34 \cdot 10^{10}$
  - `double`: größerer Wertebereich
  - `long double`: noch größerer Wertebereich
- char und ganze Zahlen können **vorzeichenbehaftet** (Standardfall) oder **vorzeichenlos** sein:
  - `signed short int` $\in [-2^{16-1}, 2^{16-1}-1] = [-32768, 32767]$
  - `unsigned short int` $\in [0, 2^{16}-1] = [0, 65535]$

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

3. Elementare Datentypen und Anweisungen

3.1 Einführung
3.2 Variablen und Konstanten
3.3 Zeiger
3.4 Arithmetische und logische Operationen und Zuweisungen
3.5 Kontrollanweisungen

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Variablen

- Variablen
  - speichern Werte eines bestimmten **Datentyps**
  - müssen vor Verwendung **vereinbart** werden
    - z.B. `float a, b; char x, y; double e, f;`
  - besitzen einen eindeutigen **Namen**
  - Name sollte über **Verwendung** Aufschluss geben!
    - z.B. `int counter;` ist besser als `int c;`
- **Schlüsselwörter** sind als Namen nicht erlaubt
  - z.B. `for, while, goto, int, float, ...`
    sind in C/C++ reserviert
- Je nach Compiler:
  - Es sind nur die ersten 31 Zeichen **signifikant**
  - z.B. `int einLangerVariablenName1234567890, einLangerVariablenName1234567891` gelten als gleich

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Variablen

- Aufbau von Variablennamen (Identifier-Syntax)
- Kompakte Beschreibung mit **regulären Ausdrücken**
  - *letter* = "a" | "b" | … | "z" | "A" | "B" | … | "Z" | "_"
    - ***letter*** = ein Buchstabe oder „_“
  - *digit* = "0" | … | "9"
    - ***digit*** = eine Dezimalziffer
  - *identifier* = *letter* { *letter* | *digit* }
    - ***identifier*** = ein *letter*, gefolgt von beliebig vielen Elementen (auch 0) *letter* oder *digit*

- Z.B. zulässig:
  - `int x, xX, y_, a_42;`
- Z.B. unzulässig:
  - `int 0z, 1234_x, return;`

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Variablen-Vereinbarung

- Vereinbarung vor erster Verwendung der Variable
  - Deklaration ermöglicht **Korrektheitsprüfungen** durch Compiler
  - Definition allokiert benötigten **Speicherplatz**

```
int main(void)
{
   /* Variablen-
      Vereinbarungen */

   /* ... */

   /* Anweisungen mit
      Variablen-
      Verwendung */

   return 0;
}
```

- **Allgemeine Form** der Vereinbarung:
  - *Typ* ID (, ID)* ;
  - *Typ* darf sein:
    - Basis-DT (Schlüsselwort): `char, int, float, ...`
    - Zusammengesetzter DT (Zeiger, Array, Struct, ...)
    - Benutzerdefinierter DT
      - `typedef unsigned long int UL;`
      - Macht `UL` als Abkürzung für `unsigned long int` bekannt
      - Hilfreich vor allem bei zusammengesetzten DT

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Konstanten

- Falls Werte sich innerhalb eines Programms nie ändern, so werden diese als **Konstanten** statt Variablen dargestellt
- Bessere **Lesbarkeit**, mehr **Optimierungsmöglichkeiten**
- Konstanten benötigen keine explizite Definition, da ihr **Typ implizit** ist; Beispiele:

  `12345`: ganze Zahl (int)

  `12345U`: vorzeichenlose ganze Zahl (unsigned int)

  `0xAFFE`: ganze Zahl in Hexcode (int)

  `123.45`: Fließkommazahl (double)

  `123E-2`: Fließkommazahl (double)

  `'a'`: Zeichen (char)

  `"String"`: Zeichenkette (Array aus char, s. später)

int-Konstante als regulärer Ausdruck: "0" | (["-"], dnz, { d })

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

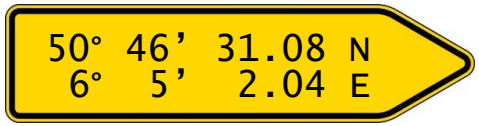
Weitere Formen von Konstanten

- **Symbolische Konstanten** im Präprozessor
  - `#define MAXCOUNT 1000`
  - Alle Vorkommen von `MAXCOUNT` im Programm werden durch `1000` ersetzt
  - Bessere Lesbarkeit und Wartbarkeit des Codes
- **Konstante Ausdrücke**
  - `MAXCOUNT + 1, MAXCOUNT * 2, ...`
  - Werden vom Compiler vorab ausgerechnet, nicht zur Programm-Laufzeit (*compile time*-Konstante)
- **Konstante Variablen**
  - `const int a = 100;` (`const` ist ein *type qualifier*)
  - Variable `a` darf kein neuer Wert zugewiesen werden
  - Effekt: Ablage von `a` im *read only*-Speicher (ROM)

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

3. Elementare Datentypen und Anweisungen

3.1 Einführung
3.2 Variablen und Konstanten
3.3 Zeiger
3.4 Arithmetische und logische Operationen und Zuweisungen
3.5 Kontrollanweisungen





© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Zeiger

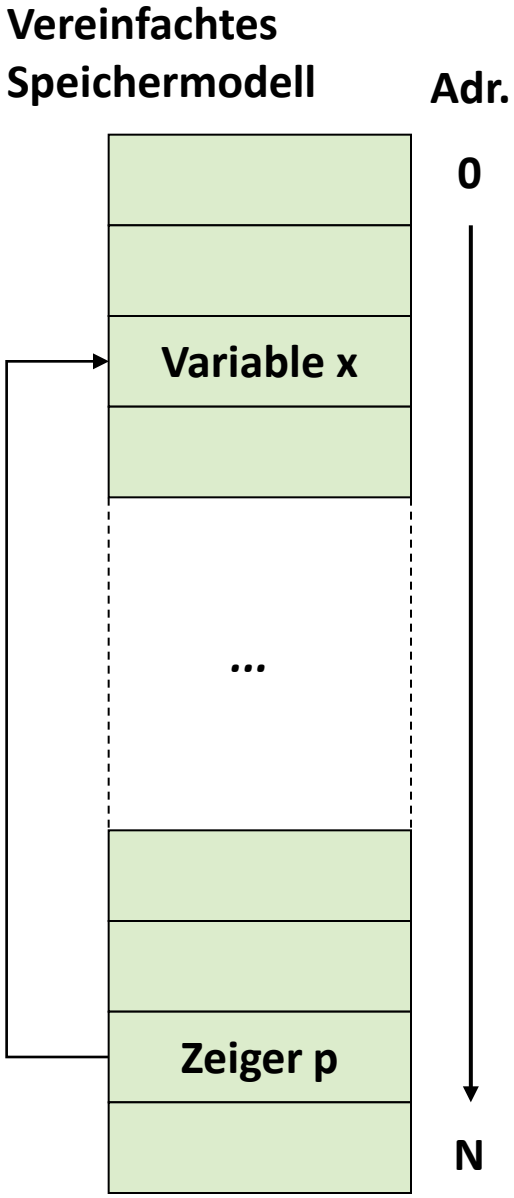
Def.: Ein **Zeiger** (engl. *pointer*) ist eine Variable, die die Speicheradresse einer Variablen enthält.

- Zeiger gehören zu den **wichtigsten Konzepten** in C/C++ zur Definition komplexer und effizienter Datentypen
- Zeiger gehören (leider) auch zu den **häufigsten Fehlerquellen** bei der Programmierung
- Bsp.:
  - Der spezielle Zeigerwert `NULL` (0) wird oft verwendet, um einen „leeren“ Zeiger anzuzeigen (`#define NULL 0`)
  - Der Versuch, einen Wert an Adresse 0 zu schreiben, führt in aller Regel zum **Programmabsturz**
  - Enthält ein Zeiger einen sonstigen illegalen Wert, so kann sogar der **Programmcode** selbst überschrieben werden

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Zeiger

- Alle **Programm-Objekte** liegen im Speicher (s.o.)
- Jedes Objekt besitzt eine **Adresse** im Speicher (wie eine Hausnummer)
- Häufig werden Objekte **per Namen** zugegriffen (z.B. Variable „x“)
- Oft ist auch der **indirekte Zugriff** auf ein Objekt über seine Adresse sinnvoll
- Objekt-Adressen werden in **Zeigern** gespeichert
- Zeiger sind selbst Objekte mit Adressen, daher sind auch **Zeiger auf Zeiger** möglich



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Zeigervereinbarung und -operationen

- Zeiger werden mittels `*` deklariert
- `Typ * ID;` vereinbart einen Zeiger auf ein Objekt vom Typ `Typ`
- Bsp.: `int * p; int ** q;`
- Symbol `*` gehört syntaktisch zu `ID`, nicht zum Typ, z.B.

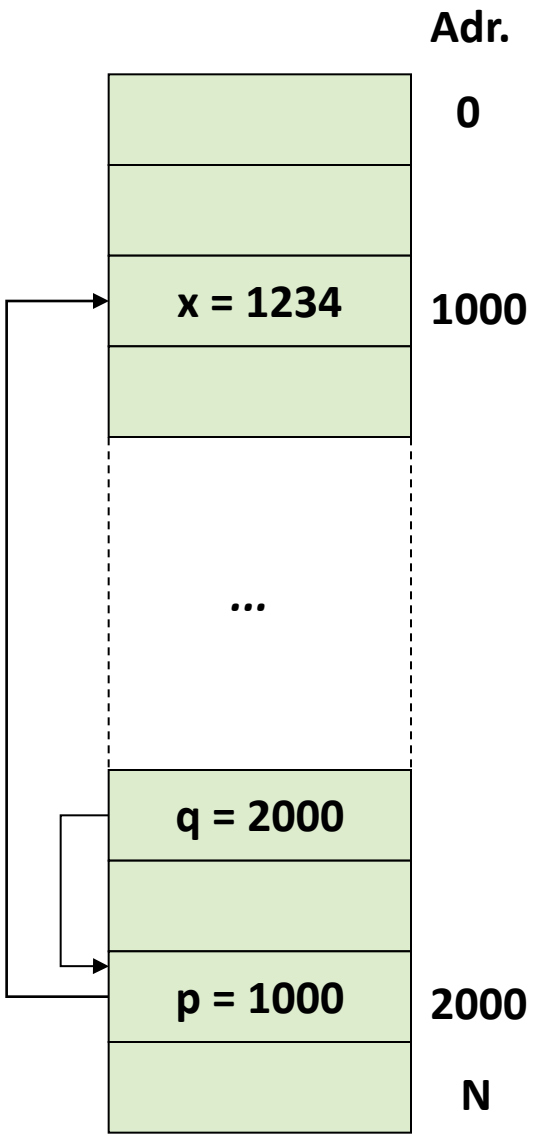
  ```
  int * p, q;
  ```

  vereinbart einen `int`-Zeiger `p` und eine `int`-Variable `q`, aber

  ```
  int *p,*q;
  ```

  vereinbart zwei `int`-Zeiger
- Äquivalent wäre:

  ```
  typedef int* intptr;
  intptr p, q;
  ```



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Zeigerdeklaration und -operationen

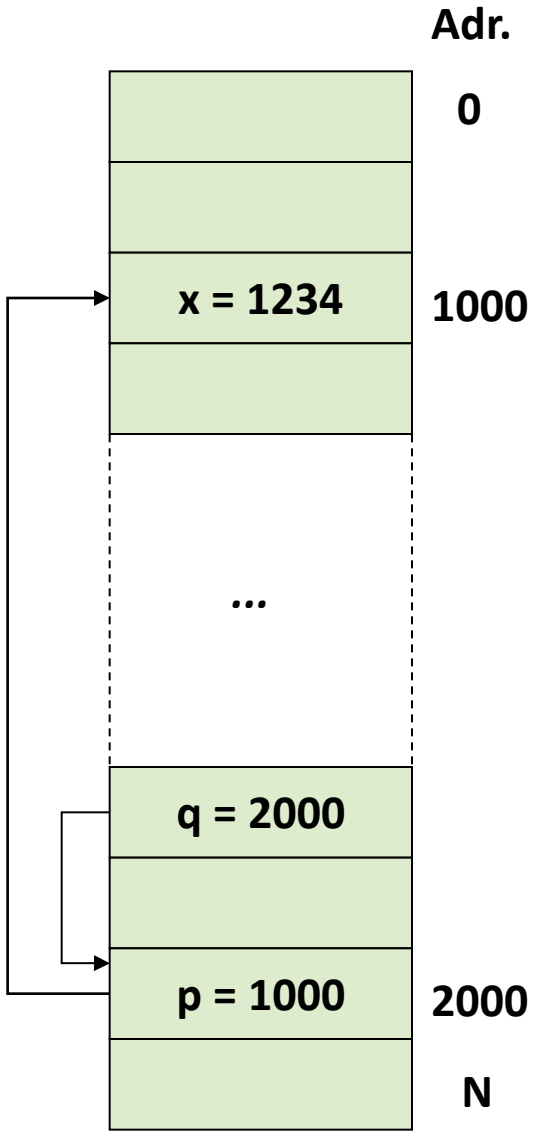
- **Annahme**: Adresse von Objekt `x` in Zeiger `p` gespeichert
- Wie erfolgt der Zugriff auf `x`?
  - `*`-Operator
  - Ausdruck `*pointer` liefert Objekt `x`
- **Beispiel**:

  `*p` liefert den Wert 1234 (`int`)

  `*q` liefert den Wert 1000 (`intptr`)
- **Schreibzugriff** per `*`:

  `*p = 4321;` überschreibt `x` mit 4321

  `**q = 5678;` überschreibt `x` mit 5678
- Vergleiche Syntax der **Vereinbarung**
  - `int *p;`
  - **implizite** Deklaration von `p` als Zeiger



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Adressoperator

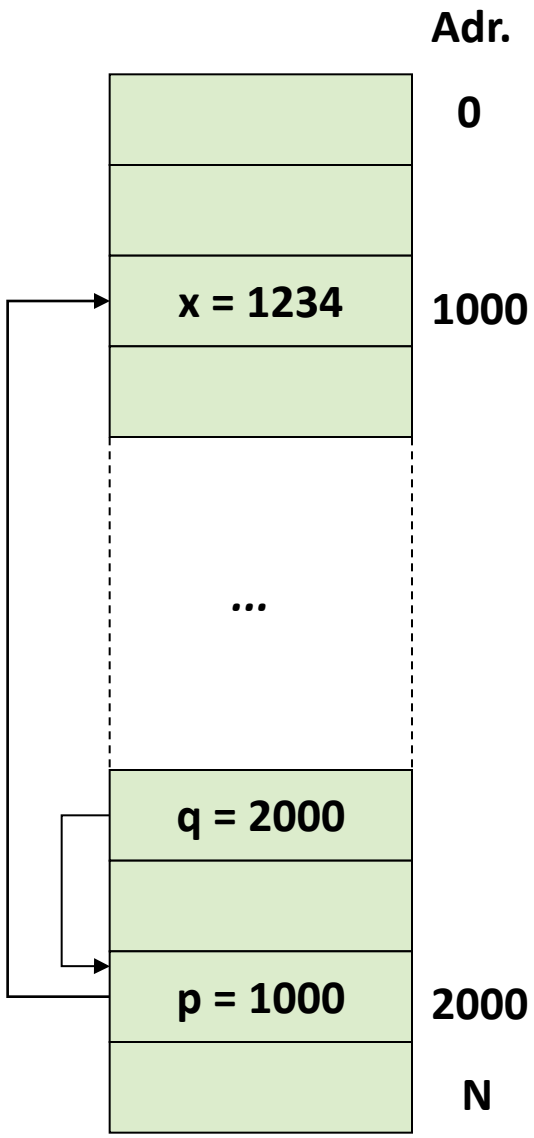
- Zeiger auf x wird auch als **Referenz** auf x bezeichnet
- *-Operator heißt auch **Dereferenzierung** (*indirection*)
- Erzeugung von Referenzen durch den **Adressoperator** &

```
p = &x;
```

  - An p wird die Adresse von x zugewiesen (p „zeigt“ dann auf x)

```
q = &p;
```

  - An q wird die Adresse von p zugewiesen
- & und * sind invers zueinander:
  - *(&x) = x
- & nur auf **Objekte** anzuwenden, * nur auf **Zeiger**



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Type cast mit Zeigern

- Zeiger sind im Prinzip **identisch** mit (positiven) Integer-Zahlen
- Daher können Zeiger und int ineinander **umgewandelt** werden

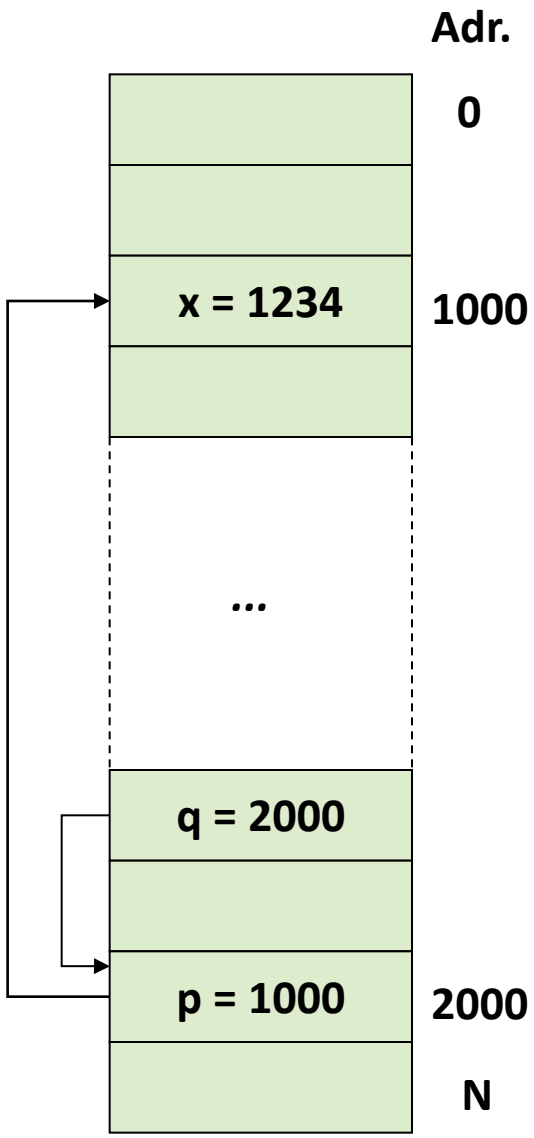
```
p = (int*)x;
x = *p;
```

  - lädt `x` mit dem Inhalt von Speicherstelle 1234

```
x = (int)q;
```

  - lädt `x` mit 2000

- Diese Eigenschaft von C/C++ bringt große **Flexibilität**, birgt jedoch auch **Gefahren**, z.B. möglicher Zugriff auf geschützte Speicherbereiche



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Wozu dienen Zeiger?

- Allgemein: falls mittels eines **einzigen Objektes** (Zeiger) im Programmverlauf auf **verschiedene andere Objekte** zugegriffen werden soll
- Oder falls (in der maschinennahen Programmierung) Zugriffe auf **vorgegebene feste Speicherstellen** notwendig sind
- Vor allem sind Zeiger sinnvoll bei **dynamischen** (d.h. zur Laufzeit erzeugten) Datenstrukturen
- Datenobjekte, deren Größe während der Programmentwicklung noch nicht feststeht und die daher **nicht statisch deklariert** werden können
- Hierfür kann dynamisch (d.h. während der Programmlaufzeit) Speicher angefordert und wieder freigegeben werden
- Der Speicherbereich hierfür heißt **Heap** (Halde, Haufen)

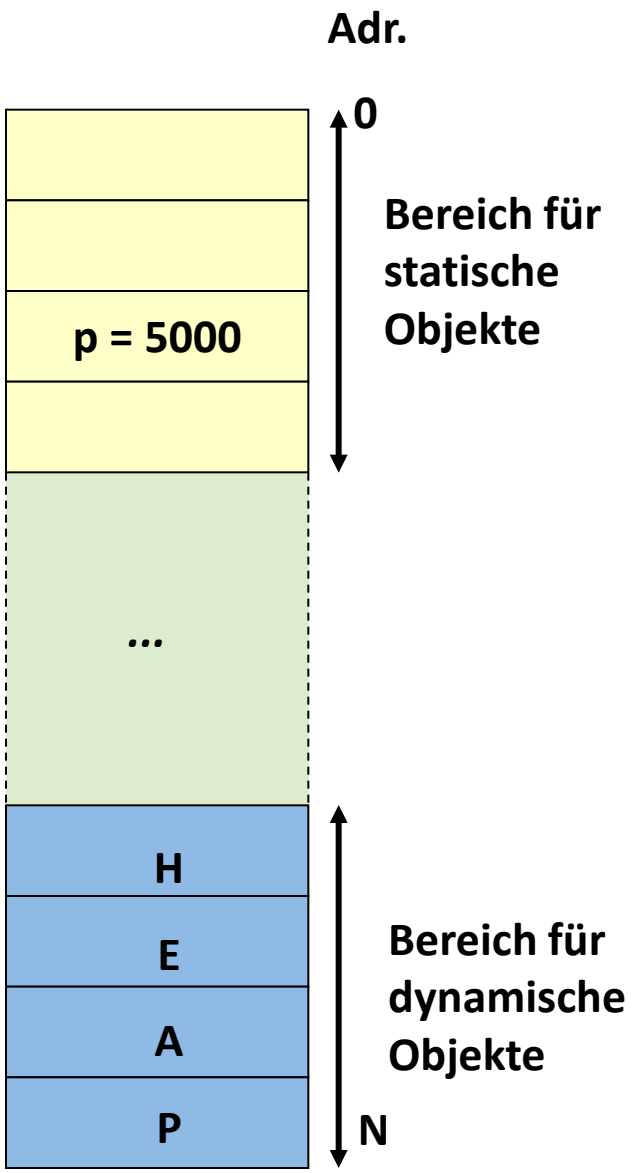
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Heap-Speicher

- Programmcode und Variablen im **statischen** Speicherbereich
- Zugriff auf den **Heap** wie folgt:
- Anforderung von Heap-Speicher mittels Funktion `malloc` (*memory allocation*)

```
int* p = (int*) malloc(1000);
```

  - p zeigt dann auf Anfang von freiem 1000-Byte Heap-Block
  - Adresse eines freien Blockes wird von `malloc` **automatisch** ermittelt (z.B. 5000)



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Heap-Speicher

- Adressen **5000-5999** durch `malloc` reserviert

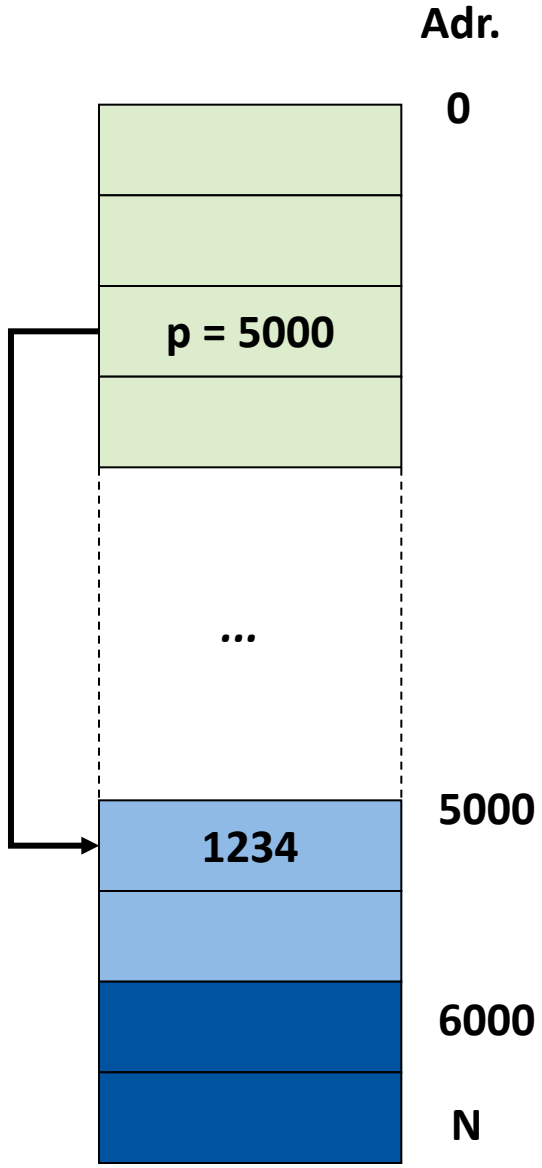
  `*p = 1234;`

  weist 1234 an erste Speicherzelle im Block zu

- Da die an `malloc` übergebene **Blockgröße** variabel ist, kann Heap-Speicher immer **passend** angefordert werden

- Nach Verwendung sollte der Heap-Block auch wieder **freigegeben** werden, um den Speicherplatz für andere Zwecke wiederverwenden zu können:

  `free(p); // Block 5000-5999 freigeben`



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Dynamische Speicherallokierung

```c
#include <stdlib.h>
#include <stdio.h>

int main( void ) {
    int* p;
    printf("Any memory         @ %p: %d_%d_%d\n", p, p[0], p[1], p[2]);
    p = (int*) malloc(1000);
    printf("Allocated memory   @ %p: %d_%d_%d\n", p, p[0], p[1], p[2]);
    p[0] = 4;
    p[1] = 7;
    p[2] = 11;
    printf("Initialized memory @ %p: %d_%d_%d\n", p, p[0], p[1], p[2]);
    free(p);
    printf("Deallocated memory @ %p: %d_%d_%d\n", p, p[0], p[1], p[2]);

    return 0;
}
```

```c
#include <cstdio>
#include <memory>
using namespace std;
int main( void ) {
    int* p;
    printf("Any memory         @ %p: %d_%d_%d\n", p, p[0], p[1], p[2]);
    p = new int[250];
    printf("Allocated memory   @ %p: %d_%d_%d\n", p, p[0], p[1], p[2]);
    p[0] = 4;
    p[1] = 7;
    p[2] = 11;
    printf("Initialized memory @ %p: %d_%d_%d\n", p, p[0], p[1], p[2]);
    delete [] p;
    printf("Deallocated memory @ %p: %d_%d_%d\n", p, p[0], p[1], p[2]);

    return 0;
}
```


© T. Gemmeke, IDS, RWTH

Smart Pointer

- Motivation

```
#include <iostream>
#include <memory>
#include <sys/resource.h>
using namespace std;

// get memory footprint
long get_mem() {
    struct rusage is;
    getrusage(RUSAGE_SELF, &is);
    return is.ru_maxrss; // in KB
}

void subA() {
    int a; int* pa = &a;
}

void subB() {
    int* pa = new int;
    // delete pa; // free memory space, no automatic deallocation
}

void subC() { unique_ptr<int> pa = make_unique<int>(); }

int main( void ) {
    long mem_base = get_mem();
    cout << "Memory base usage: " << mem_base << endl;

    for( long i=0; i<10000000; i++ ) {
        subA();
        if( i%1000000 == 0) cout << "Addt'l allocation: " << get_mem() - mem_base << endl;
    }
    return 0;
}
```


© T. Gemmeke, IDS, RWTH

- Unique Pointer

```
#include <iostream>
#include <memory>
using namespace std;

unique_ptr<string> subC() {
    unique_ptr<string> p = make_unique<string>("unique");
    return p;
}

int main( void ) {
    unique_ptr<string> pu = subC();
    unique_ptr<string> pu2;

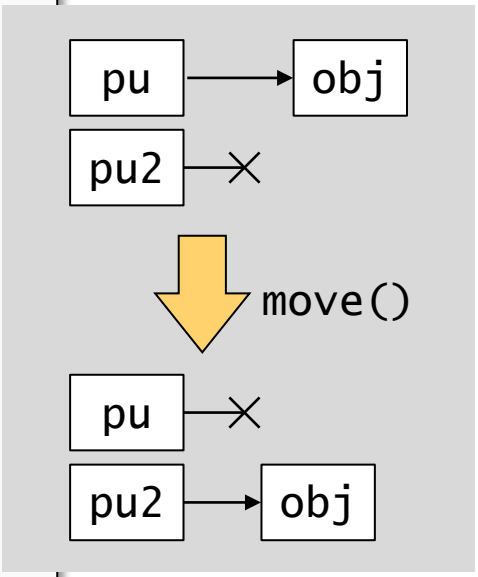
    if( pu != nullptr ) cout << "Unique pointer pu: " << *pu << endl;
    if( pu2 != nullptr ) cout << "Unique pointer pu2: " << *pu2 << endl;

    cout << "Moving ownership from pu to pu2 ..." << endl;
    pu2 = move(pu);

    if( pu ) cout << "Unique pointer pu: " << *pu << endl;
    if( pu2 ) cout << "Unique pointer pu2: " << *pu2 << endl;

    return 0;
}
```

Anmerkung:
Unique pointers können als Rückgabewert verwendet werden, falls der aktuelle Besitzer nur temporäre Gültigkeit im lokalen Kontext hat.



Im 2. Beispiel ist die Überprüfung auf Gültigkeit vereinfacht.

© T. Gemmeke, IDS, RWTH

- Shared Pointer

Wir nutzen call-by-reference, um eine Erhöhung der Pointerzahl zu vermeiden.

```
#include <iostream>
#include <memory>
using namespace std;

void show( shared_ptr<string>& p0, shared_ptr<string>& p1 ) {
    if( p0 != nullptr ) cout << "Shared pointer p0: '" << *p0 << "'" << endl;
    if( p1 != nullptr ) cout << "Shared pointer p1: '" << *p1 << "'" << endl;
    cout << " Ptr of p0 use count: " << p0.use_count() << endl;
    cout << " Ptr of p1 use count: " << p1.use_count() << endl << endl;
}



int main( void )
{
    shared_ptr<string> ptr0 = make_shared<string>("shared");
    shared_ptr<string> ptr1;
    show(ptr0, ptr1);

    cout << "Duplicating reference to object ..." << endl;
    ptr1 = ptr0;
    show(ptr0, ptr1);

    ptr0 = make_shared<string>("something new");
    show(ptr0, ptr1);

    return 0;
}
```


© T. Gemmeke, IDS, RWTH

- Weak Pointer

Einen *weak pointer* zu de-referenzieren ist etwas kompliziert. Es benötigt eine temporäre Kopie der Referenz. Diese wird durch die Methode `lock()` erreicht.

```
#include <iostream>
#include <memory>
using namespace std;

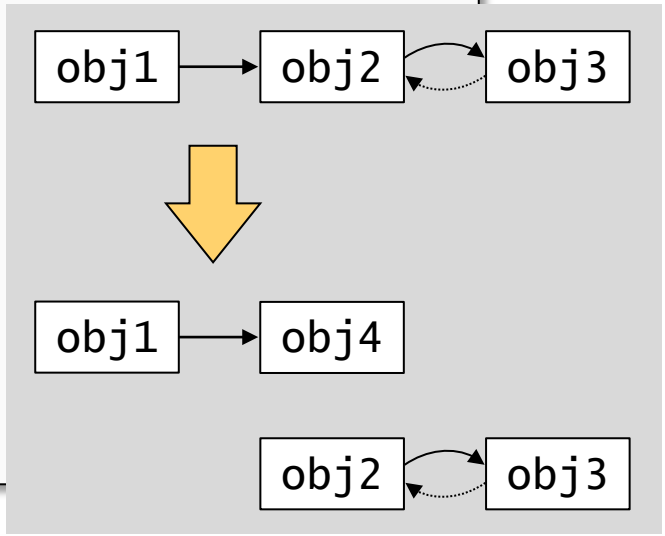
void show( shared_ptr<string>& p0, weak_ptr<string>& p1 ) {
    if( p0 != nullptr ) cout << "Shared pointer p0: '" << *p0 << "'" << endl;
    if( !p1.expired() ) cout << "Weak pointer p1: '" << *(p1.lock()) << "'" << endl;
    cout << " Ptr of p0 use count: " << p0.use_count() << endl;
    cout << " Ptr of p1 use count: " << p1.use_count() << endl << endl;
}


int main( void )
{
    shared_ptr<string> ptr0 = make_shared<string>("shared");
    weak_ptr<string> ptr1;
    show(ptr0, ptr1);

    cout << "Duplicating reference to object ..." << endl;
    ptr1 = ptr0;
    show(ptr0, ptr1);

    ptr0 = make_shared<string>("something new");
    show(ptr0, ptr1);

    return 0;
}
```



© T. Gemmeke, IDS, RWTH

3. Elementare Datentypen und Anweisungen

3.1 Einführung
3.2 Variablen und Konstanten
3.3 Zeiger
3.4 Arithmetische und logische Operationen und Zuweisungen
3.5 Kontrollanweisungen

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Arithmetische und logische Operationen

- Auf Variablen und Konstanten können verschiedene **Rechenoperationen** angewendet werden
  - `+, -, *, /`
  - `%` (modulo, nicht bei float-Werten)
- Multiplikative Operationen haben **Vorrang**
  - Der **Ausdruck** `a*b+c/d` entspricht `(a*b) + (c/d)`
- Diese primitiven Rechenoperationen sind in der Programmiersprache „eingebaut“
- Komplexere Operationen sind in **Libraries** verfügbar oder werden vom **Programmierer definiert**
- Z.B. Standard-Library `math.h`
  - `sin, cos, tan, pow` (*power*), `sqrt` (*square root*), ...
- **Ergebnisse** der Operationen haben einen bestimmten DT, der sich aus dem Operator und den Argumenttypen ergibt

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Zuweisungen

- **Allgemeine Form:**
  - *Ziel* = *Ausdruck*
  - = ist der Zuweisungsoperator
  - *Ausdruck* wird berechnet, Wert wird in *Ziel* gespeichert
- **Kurzformen:**
  - ++, --: Increment, Decrement

    x++ entspricht x = x+1
  - +=, *=, ...

    x += y entspricht x = x+y

```
int main()
{
  int a,b;
  int c,d;

  a = 1;
  b = 2;

  c = a+b;
  d = c+1;
  d = d+1;
  printf("%d", d);

  return 0;
}
```

```
#define a 1
#define b 2

int main()
{
  int c,d;

  c = a+b;
  d = c+1;
  d++;
  printf("%d", d);

  return 0;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Arithmetische und logische Operationen

- **Vergleichsoperationen**

  `>, >=, <, <=`

  `==` (Gleichheit), `!=` (Ungleichheit)

- Ergebnis einer Vergleichsoperation ist stets ein **Boolescher Wert** (*true* oder *false*)
- Boolescher DT in C: `_Bool`
  - Äquivalenzen aus *stdbool.h*:
    - `bool  := _Bool`
    - `false := 0`
    - `true  := 1`
- **Schiebeoperationen** (shift)

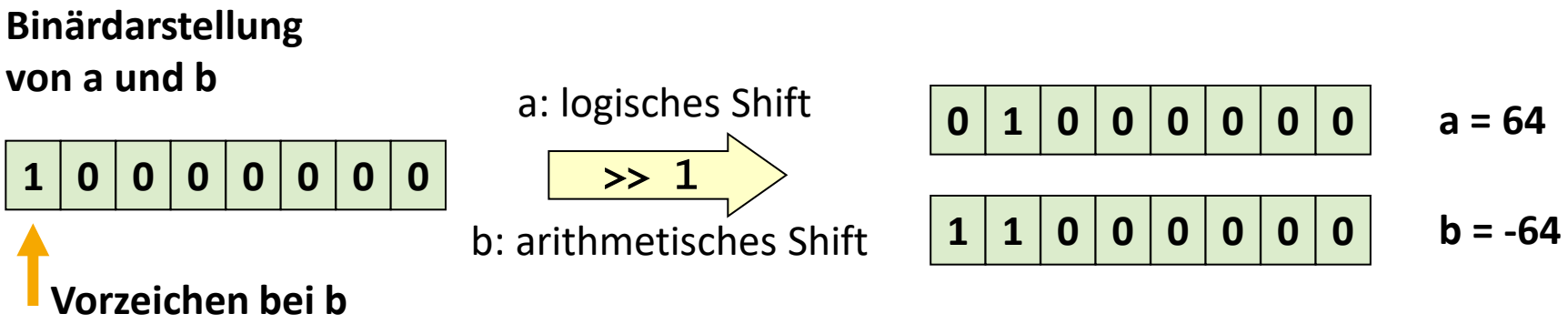
  `>>` Rechtsshift

  `<<` Linksshift

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Arithmetische und logische Operationen

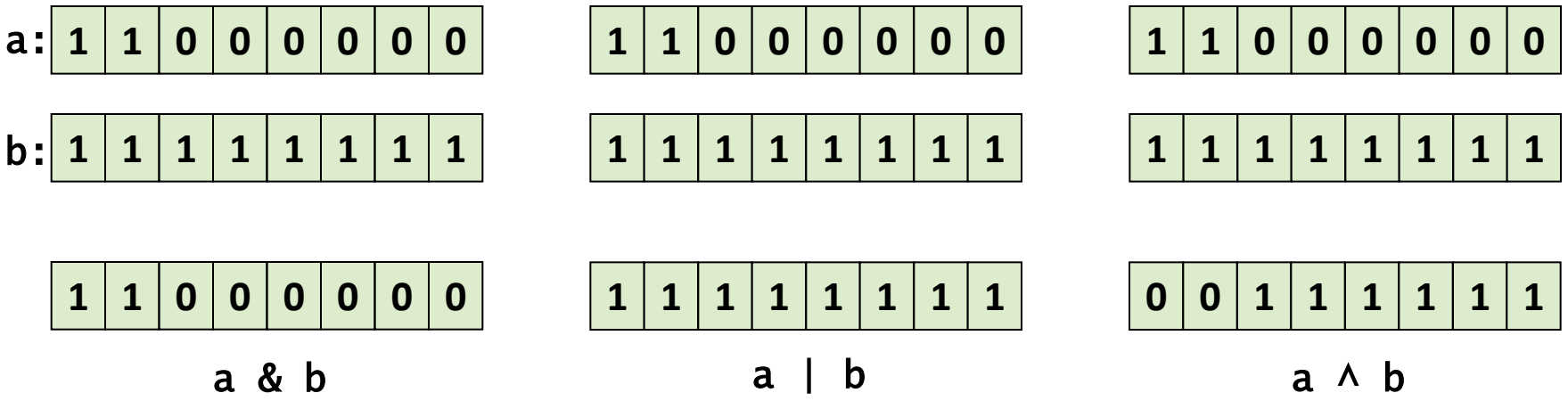
- Shift-Operationen nützlich für
  - Zugriff auf einzelne Bits eines Wertes
  - Vereinfachung von Rechenoperationen, z.B.
    - `x >> 1` entspricht `x/2`
    - `x << 2` entspricht `x*4`
- Rechtsshift abhängig vom DT des ersten Arguments:
  - **Logisches** Shift: Auffüllen links mit 0
  - **Arithmetisches** Shift: Auffüllen links mit Vorzeichen
- Beispiel: `unsigned char a = 128; signed char b = -128;`



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Arithmetische und logische Operationen

- &: bitweise **UND-Verknüpfung**
- |: bitweise **ODER-Verknüpfung**
- ^: bitweise **Exklusiv-ODER-Verknüpfung** (XOR, „entweder...oder“)
- Prüfen, Setzen oder Löschen **einzelner Bits** von Werten
- Bsp.:
  - `signed char a = -64, b = 0xFF; // oder b = -1;`



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Typumwandlung (type cast)

- Beispiel:
  - `int a = 10, b;  float pi = 3.14;`
  - `b = pi + a;`
- Welche Typen werden verwendet?
  - `a` wird in intern in `float` umgewandelt (10.0 = 1E1)
  - `pi + a` ergibt `float`-Resultat (13.14)
  - Resultat wird in `int` zurückverwandelt (13)
- In C/C++ gibt es umfangreiche ***type cast*-Regeln** für alle Kombinationen der primitiven DT
- Viele Einzelheiten der Typumwandlung sind jedoch ***compilerabhängig*** (und damit eine „beliebte“ Fehlerquelle) !
- ***Expliziter*** *type cast* durch Programmierer:
  - *Ziel* = (*Typ*) *Ausdruck*;

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

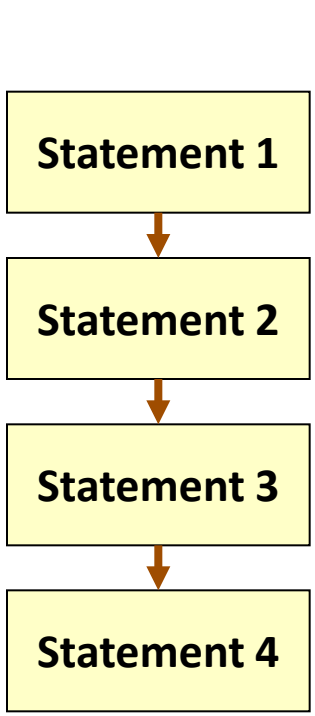
3. Elementare Datentypen und Anweisungen

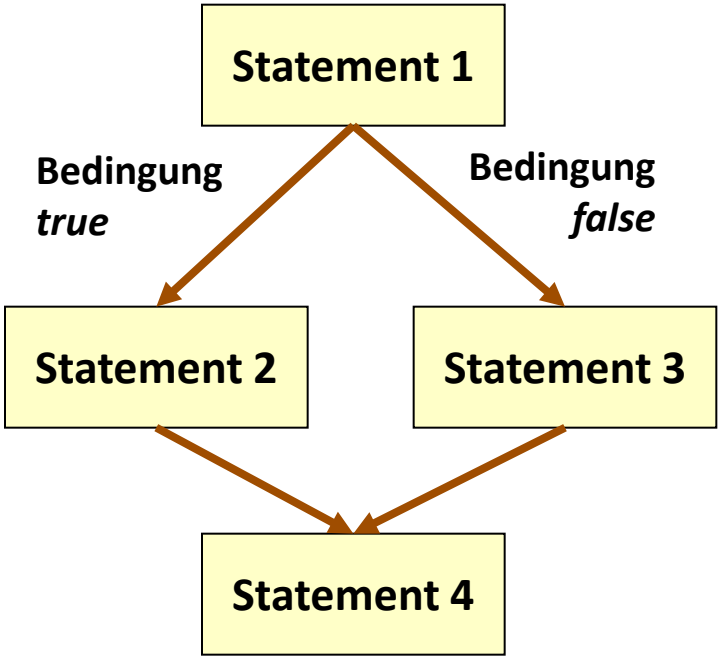
3.1 Einführung
3.2 Variablen und Konstanten
3.3 Zeiger
3.4 Arithmetische und logische Operationen und Zuweisungen
3.5 Kontrollanweisungen

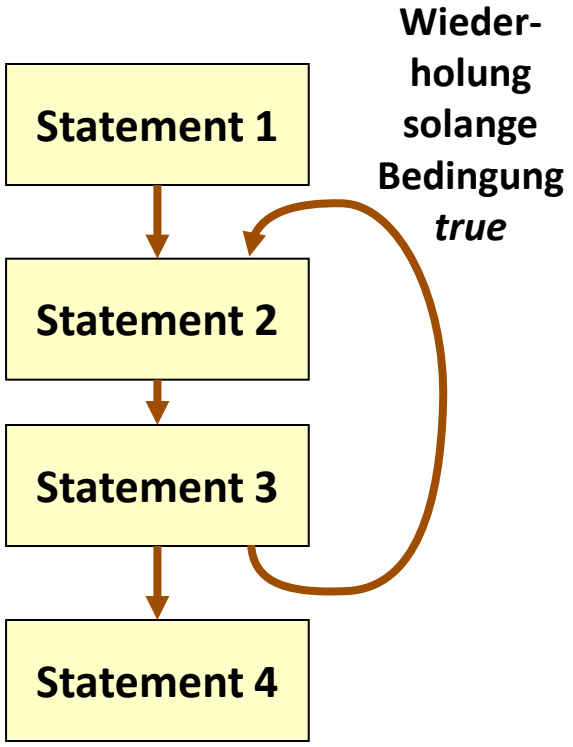
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Kontrollanweisungen

- Bisher: Alle Anweisungen (*statements*) werden **sequentiell** abgearbeitet
- Kontrollanweisungen ermöglichen
  - **bedingte Ausführung** von Code mittels Verzweigungen
  - **wiederholte Ausführung** von Code mittels Schleifen







© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Verzweigungen: if-then-else

- Beispiel: Maximumsberechnung
- Allgemeine Formen:
  - `if ( expression ) { statement* }`
  - `if ( expression ) { statement* } else { statement* }`
- Bedingung muss *true* oder *false* ergeben (0 ≙ false)
- „Beliebter“ Fehler bei Gleichheitstest:
  - `if (a = b)` statt `if (a == b)`

```
#include <stdio.h>

int main(void)
{
  int a, b, max;

  // Einlesen von a und b
  scanf("%d %d", &a, &b);

  if (a > b) {
    max = a;
  } else // d.h. a <= b
    max = b;
  }

  printf("max = %d", max);

  return 0;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Verzweigungen: if-then-else

- Komplexere Bedingungen können durch **logische Verknüpfungen** geprüft werden
  - ODER-Operator ||:
    mindestens eine der Teilbedingungen muss *true* sein
  - UND-Operator &&:
    alle Teilbedingungen müssen *true* sein

```
int a, b, c, x, y;
/* ... */
if (a > 10) {
  if (b >= 20) {
    if (c == 30) {
      x++;
    }
  }
}

if (a < 40) {
  y--;
} else if (b <= 50) {
  y--;
} else if (c != 60) {
  y--;
}
```

```
int a, b, c, x, y;
/* ... */
if ((a > 10) && (b >= 20) && (c == 30)) {
  x++;
}

if ((a < 40) || (b <= 50) || (c != 60)) {
  y--;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Dangling else-Problem

- Betrachte folgenden Code:

```
if (a > 100) if (b > 50) x++; else x--;
```

- **Zwei Möglichkeiten** der Interpretation:
  - Wird `x--` ausgeführt, falls `(a <= 100)` ?

    (1) `if (a > 100) if (b > 50) x++; else x--;`
  - Oder falls `(a > 100) && (b <= 50)` ?

    (2) `if (a > 100) if (b > 50) x++; else x--;`
- **Konvention**: `else` gehört immer zum innersten freien `if`, d.h. Version (2) ist richtig
- **Besser**: Vermeidung von Fehlern durch expliziten Gebrauch der **Blockstruktur** `{ statement* }` in der Programmierung
  - `if (a > 100) { if (b > 50) x++; else x--; }`

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Verzweigungen: goto

- Verzweigung auch über **Labels** und `goto`**-Statements** möglich
- Ein Label ist eine symbolische **Sprungmarke** im Code
- Syntax:
  - *ID*: *statement* (*ID* ist Bezeichner des Labels)
  - `goto` *ID*; (Sprung nach Position *ID*)

```
    if (a > 10) {
      goto L1;
    }

L2: if (b < 40) {
      goto L3;
    }

L4: goto L5;

L1: x++;
    goto L2;

L3: y--;
    goto L4;

L5: ;
```

- Labels und gotos
  - beeinträchtigen die **Lesbarkeit** des Programms
  - sind ein „historisches Relikt“ in Programmiersprachen
  - sind nicht notwendig und sollten nur bei **zwingenden Gründen** eingesetzt werden

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Verzweigungen: `switch`

- Häufiger Fall: **Fallunterscheidung** über eine bestimmte Variable mit jeweils konstantem Vergleichswert
- Normale Implementierung: **Kette** von if-then-else Statements
- Gültig, aber umständlich
- Bsp.: Umrechung von römischen Ziffern:

```
char digit, int value;

/* ... */

if      (digit == 'I') value = 1;
else if (digit == 'V') value = 5;
else if (digit == 'X') value = 10;
else if (digit == 'L') value = 50;
else if (digit == 'C') value = 100;
else if (digit == 'D') value = 500;
else if (digit == 'M') value = 1000;
else printf("Keine gültige Ziffer!");
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Verzweigungen: switch

- **Kompaktere** Implementierung: switch-Statement
- Allgemeine Form:

```
switch (Ausdruck)
{
    case Konstante1:
      statement*
    /* … */
    case Konstantem:
      statement*
    default: statement*
}
```

```
char digit, int value;

/* ... */

switch (digit)
{
  case 'I': value = 1;    break;
  case 'V': value = 5;    break;
  case 'X': value = 10;   break;
  case 'L': value = 50;   break;
  case 'C': value = 100;  break;
  case 'D': value = 500;  break;
  case 'M': value = 1000; break;
  default:
    printf("Keine gültige Ziffer!");
    break;
}
```

- **break**: sofortiges Verlassen des ganzen switch-Statements
- **default**: Code im „sonst“-Fall (optional)

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Schleifen

- Wiederholte Ausführung einer Anweisungsfolge
- Bsp.: Berechnung der Fakultätsfunktion

$$n! := 1 \times 2 \times \cdots \times n$$

- Ohne Schleife: Code muss bei Änderung von n modifiziert werden

```
#define N 10

  /* ... */

  int fact, count;

  count = 0;
  fact = 1;

  count++;
  fact *= count;

  /* ... N-fache Wiederholung */

  count++;
  fact *= count;
  printf("factorial(%d) = %d", N, fact);
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Schleifen

- Flexiblere Variante
- Code zwischen `L1` und `L2` wird wiederholt bis Zähler größer als `N`
- Umdefinieren von `N` (`#define`) reicht aus
- Jedoch: **schlechter Programmierstil**

(gotos sollten nicht verwendet werden!)

```
#define N 10
    /* ... */

    int fact, count;

    count = 0;
    fact = 1;

L1: if (count >= N) goto L2;
    count++;
    fact *= count;
    goto L1;

L2: printf("factorial(%d) = %d",
      N, fact);
```

- C/C++ bietet drei Formen von **strukturierten Schleifen**
  - `for`, `while`, `do`

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

for-Schleifen

- Allgemeine Form:
  `for (expr1; expr2; expr3) { statement* }`

- `expr1:`
  - Wird vor Schleifenbeginn einmal ausgeführt
    → Initialisierung

```
#define N 10

  /* ... */

  int fact, count;

  fact = 1;
  for (count = 1; count <= N; count++) {
    fact *= count;
  }
  printf("factorial(%d) = %d", N, fact);
```

- `expr2:`
  - Bedingung für jeden Durchlauf (jede Iteration)

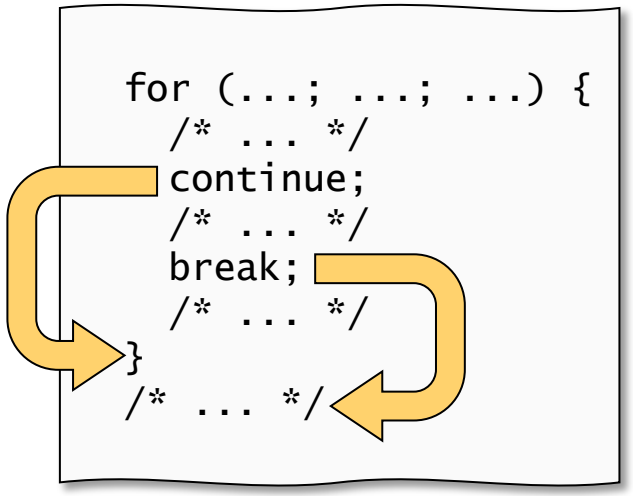
- `expr3:`
  - Wird nach `statement*` ausgeführt

- Nach `expr3` wird an den Schleifenanfang **zurückgesprungen**

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

for-Schleifen

- Warum ist `count = 1` ein **Ausdruck**?
  - In C sind auch Zuweisungen Ausdrücke
  - Das Ergebnis von „*Ziel* = *Ausdruck*“ ist per Definition „*Ziel*“
- Der Wert von `expr1` wird bei `for`-Schleifen jedoch verworfen, nur der **Seiteneffekt** einer Zuweisung ist relevant
- Zwei spezielle Statements für Ausnahmefälle in Schleifen erlaubt
  - `break`: verlässt Schleife komplett
  - `continue`: verlässt aktuelle Iteration
- Wie `goto` mit Vorsicht zu gebrauchen!



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

while-Schleifen

- for-Schleifen werden vor allem für eine **feste Anzahl** von Iterationen verwendet
- Soll die Schleife ausgeführt werden, solange eine bestimmte **Bedingung** gilt, so ist eine `while`-Schleife besser
- Allgemeine Form:

```
while (expression) {
  statement*
}
```

- `expression` wird ausgewertet
  - Falls *nicht null*, so wird die Schleife betreten
  - Nach `statement*` wird an den Anfang zurückgesprungen, und `expression` wird erneut geprüft

```
#define N 10

  /* ... */

  int fact, count;

  fact = 1; count = 1;

  while(count <= N) {
    fact *= count;
    count++;
  }

  printf("factorial(%d) = %d",
    N, fact);
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

do-Schleifen

- Ebenfalls für Schleifen mit **unbekannter Anzahl** von Iterationen
- **Unterschied** zu `while`: Mindestens eine Iteration findet statt
- Allgemeine Form:

```
do { statement* } while
  (expression)
```

- `statement*` wird ausgeführt, und `expression` wird ausgewertet
- Falls *true*, so wird an den Anfang zurückgesprungen
- Bsp.: **Mindestens eine** Benutzereingabe wird erwartet

```
char c;
/* ... */

do {
  scanf("%c",&c);
  /* Berechnung auf c */
} while (c != 'q'); // quit?
```

- `break, continue` können bei `while`- und do-Schleifen ebenfalls verwendet werden

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Äquivalenz von `for, while, do`

- `for` würde als **einziges** Schleifenkonstrukt ausreichen
- Jede Schleifenform lässt sich in die anderen **umsetzen**, z.B.

```
while (Ausdruck) { statement* }
```

entspricht

```
for (;Ausdruck;) { statement* }
```

```
for (expr1; expr2; expr3) { statement* }
```

entspricht

```
expr1;
while (expr2) { statement* expr3; }
```

- Jedoch ist aus Gründen der **Programm-Lesbarkeit** die jeweils am besten geeignete Variante zu bevorzugen
- Der Einsatz von **strukturierten Schleifen** (`for, while, do`) und der **Verzicht auf Sprünge** (`goto`) führt beim Übersetzen auch oft zu besserem Maschinencode

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Endlosschleifen

- Falls die Abbruchbedingung einer Schleife nie erfüllt ist, **terminiert** das Programm nicht
- Compiler kann dies i.A. **nicht bemerken**
- Bei Ausführung: Ab einem gewissen Zeitpunkt **kein Programmfortschritt** mehr zu beobachten
- **Abhilfe**:
  - Programmabbruch
  - Erneutes Starten unter Debugger-Kontrolle

```
while ((a > 10) || (b > 20)) {
   if ( /* ... */ ) {
     /* ... */
     a = 100;
     /* ... */
   } else {
     /* ... */
     b = 30;
     /* ... */
   }
   /* danach Bedingung
      stets erfüllt */
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

4. Funktionen und Programmaufbau

4.1 Funktionen

4.2 Parameterübergabe

4.3 Polymorhpie

4.4 Templates

4.5 Lambda Funktionen

4.6 Module und Programme

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Funktionen und Programmaufbau

Def.: Das **kartesische Produkt** $A \times B$ zweier Mengen A und B ist definiert durch

$$A \times B = \{(a, b) \mid a \in A, b \in B\}$$

Def.: Eine (binäre) **Relation** R ist eine Teilmenge des kartesischen Produktes zweier Mengen: $R \subseteq A \times B$

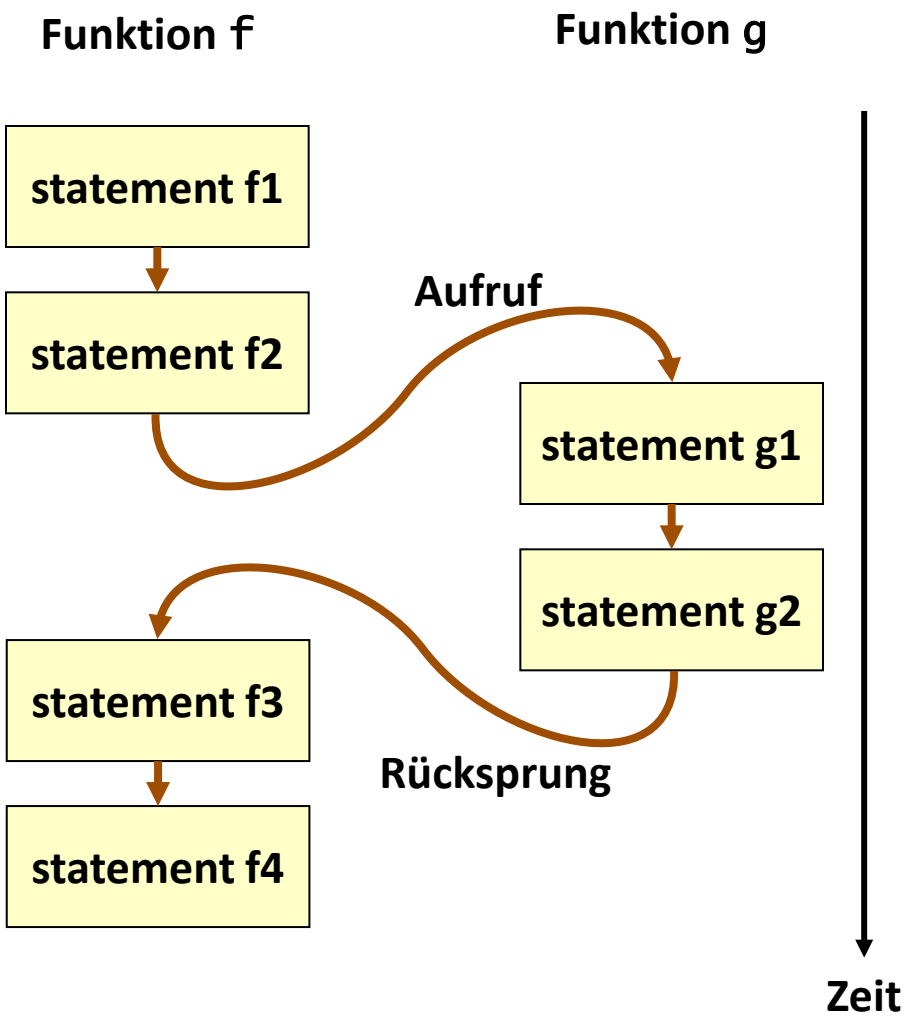
Def.: Eine **Funktion** $F : A \rightarrow B$ ist eine rechtseindeutige Relation, d.h. jedem a $\in A$ ist höchstens ein $b \in B$ zugeordnet.

- In **Programmiersprachen**: Eine Funktion (oder: Prozedur, Subroutine) ist ein abgegrenztes **Teilprogramm**, welches aus einer Menge von **Eingabeparametern** eine fest definierte **Ausgabe** erzeugt
- **Verwendung**: Einkapselung von häufig benötigten Teilprogrammen, Aufruf aus anderen Teilprogrammen

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Modell der Funktionsabarbeitung

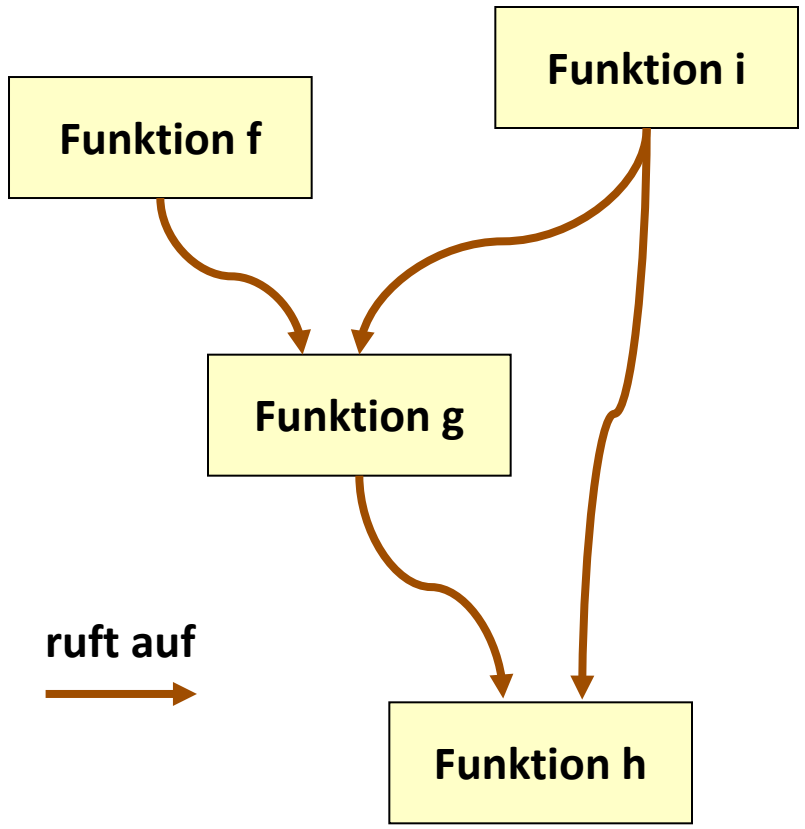
- **Zu jedem Zeitpunkt** befindet sich das Programm innerhalb einer bestimmten Funktion `f`
- Zu Beginn ist dies die Funktion `main` (s.o.)
- Ausführung von `f` kann durch **Aufruf** einer anderen Funktion `g` jederzeit **unterbrochen** werden
- `f` übergibt **Parameter** an `g`
- `g` berechnet einen **Funktionswert** und springt an **Aufrufstelle** in `f` zurück



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Modell der Funktionsabarbeitung

- Während der Ausführung kann g ihrerseits **weitere Funktionen** aufrufen (verschachtelte Funktionsaufrufe)
- Ebenso können nach Ausführung von f auch **andere Funktionen** auf g zurückgreifen
- **Vorteil**: Programmcode für mehrfach verwendete Routinen muss nur einmal vorhanden sein
  - bessere **Wartbarkeit**
  - geringere **Codegröße**



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Funktionen

- Funktionen besitzen eine **Deklaration** und eine **Definition**
- **Deklaration**: Name, Definitionsbereich, Wertebereich (d.h. **Interface** der Funktion nach außen)
- **Definition**: Programmcode zur **Implementierung** der gewünschten Funktionalität
- Erfolgt die Definition vor der ersten Verwendung einer Funktion, so darf die Deklaration **entfallen**
- **Allgemeine Form der Deklaration**:
  - *Typ* `ID (`*Parameter-Deklaration*`*);`
  - Parameter-Deklaration syntaktisch wie Variablendeklaration
  - Jedoch: Parameter-ID kann auch entfallen
  - `float max(float x, float y);`
    oder
    `float max(float, float);`
  - Deklariert Funktion $\max: \text{float} \times \text{float} \rightarrow \text{float}$

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Funktionsdefinitionen

- **Allgemeine Form**:
  - *Typ* `ID` `(`*Parameter-Deklaration**`)` `{` *statement** `}`
- Interface-Definition **muss** vorheriger Deklaration entsprechen (falls vorhanden)
- **Funktionsrumpf** innerhalb `{ /* ... */ }` kann beliebige Statements enthalten
- Verlassen einer Funktion durch `return`**-Statement**
  - `return` *Ausdruck*;
  - Typ von *Ausdruck* muss dem **Wertebereich** der Funktion entsprechen (falls nötig: *type cast*)
  - Funktionen können **nicht** per `goto` verlassen werden (Labels nur gültig **innerhalb** einer Funktion)

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Funktionsaufrufe

- Kommunikation zwischen **aufrufender** (*caller*) und **aufgerufener** (*callee*) Funktion:

```
void f(void) { int x; /* ... ruft g auf */ };
int g(int a, int b) { return a+b; }
```

- `a` und `b` sind die **formalen Parameter** der Funktion g gemäß Deklaration
- Aufruf von `g` durch `f` per Namen und Übergabe **der aktuellen Parameter**, z.B.
  - `x = g(1,2);`
  - In `g` werden die „Platzhalter“ `a` und `b` durch 1 bzw. 2 ersetzt, `g` wird ausgeführt und per `return` beendet
  - Der Ausdruck `g(1,2)` liefert dann den **Rückgabewert** 3; dieser kann z.B. an eine Variable (hier: `x`) zur Weiterverarbeitung zugewiesen werden

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

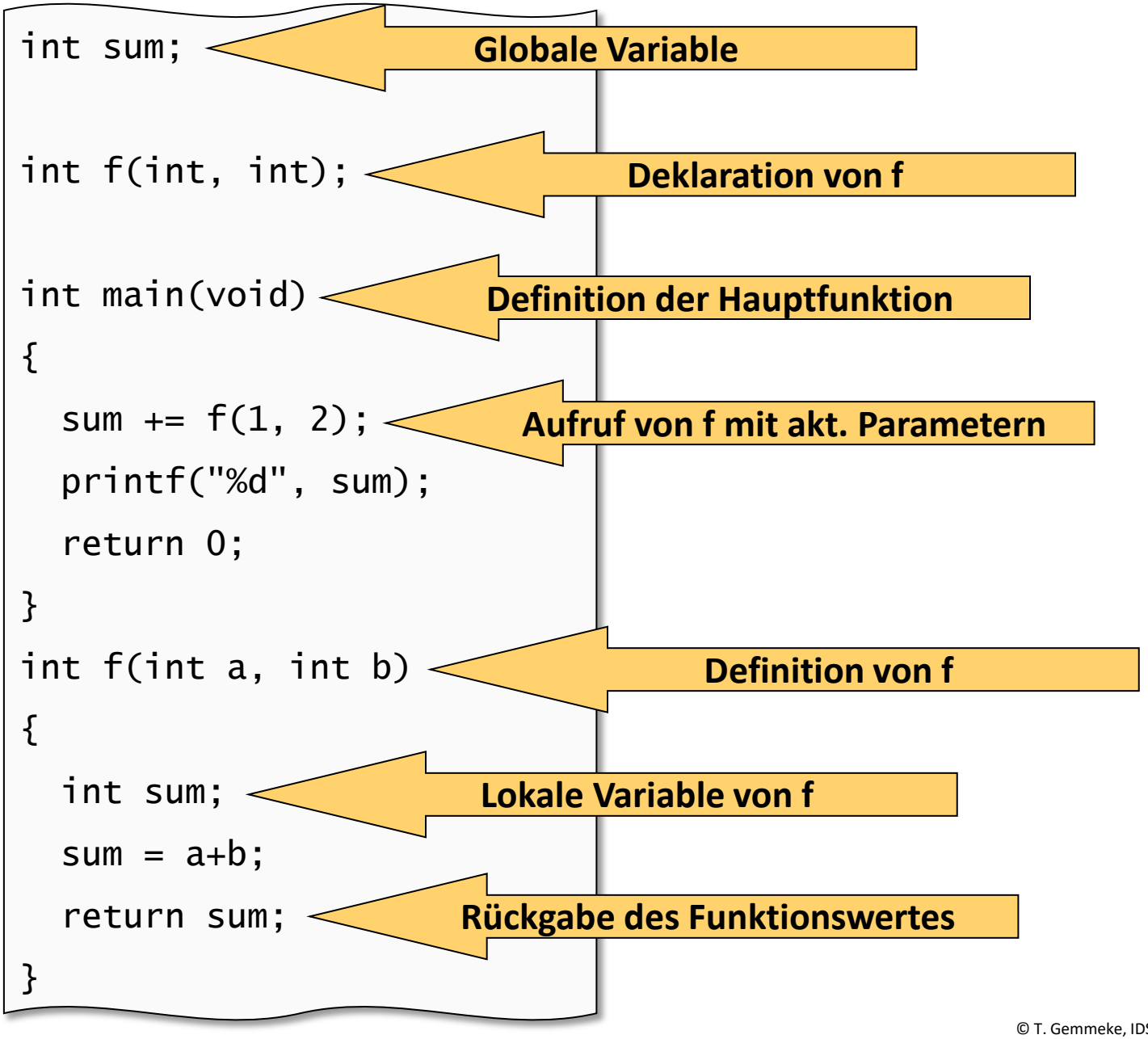
Lokale und globale Variablen

- Zwei Arten von Variablen:
  - **Globale Variablen** sind im gesamten Programm **gültig** (d.h. können zugegriffen werden)
  - **Lokale Variablen** sind nur in einer bestimmten Funktion gültig (ebenso wie **formale Funktionsparameter**)
  - Lokale Variablen können globale Variablen gleichen Namens **verdecken**
- Globale Variablen sollten zur besseren Programm-Wartbarkeit **möglichst selten** verwendet werden (sonst Gefahr von unerwünschten Seiteneffekten)

```
int a=2, b=2, i=10;
int f(int a, int b)
{
  int i;
  i = a+b;
  return i;
}
int main(void)
{
  /* ... */
  i -= f(a, b);
  return 0;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Funktionen auf einen Blick



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel

- Berechnung einer **Kreisfläche** als Funktion des Radius

  $A = \pi * r^2$

- Definition von π als **Konstante**
- **Einkapselung** der Flächenberechnung in Funktion `area`
- Hauptfunktion `main`:
  Rückgabetyp immer `int`
  hier keine Eingangsparameter: `void`
- Fortlaufende Ein/Ausgabe von Radius und Fläche

```
#define PI 3.1415926

float area(float r)
{
  float a;
  a = PI * r * r;
  return a;
}

int main(void)
{
  float rad, A;
  while (1) {
    scanf("%f", &rad);
    A = area(rad);
    printf("%f \n", A);
  }
  return 0;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

4. Funktionen und Programmaufbau

4.1 Funktionen

4.2 Parameterübergabe

4.3 Polymorhpie

4.4 Templates

4.5 Lambda Funktionen

4.6 Module und Programme

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Wert- und Referenzparameter

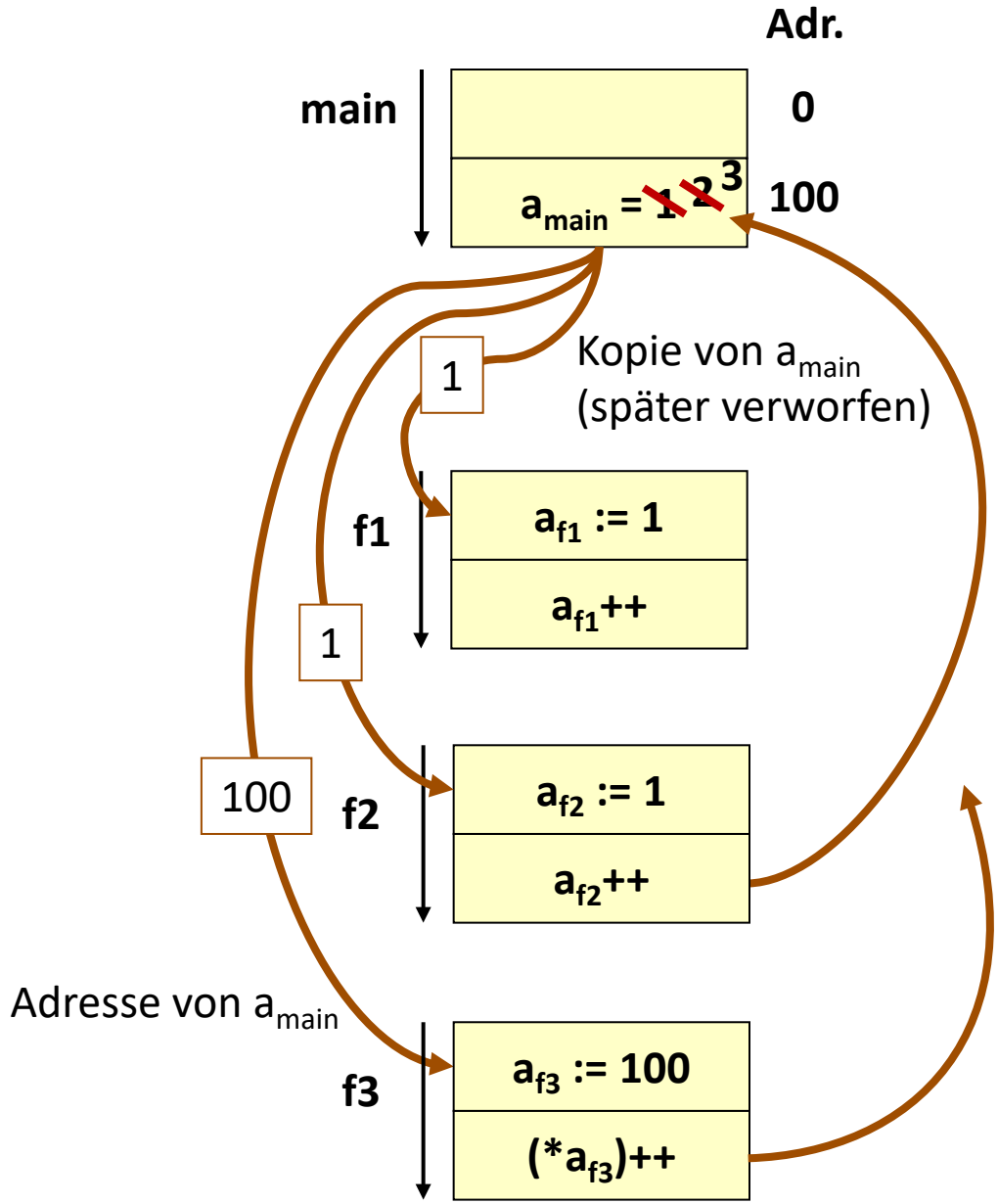
- Bei Übergabe eines Wertes w an eine Funktion f wird dem formalen Parameter von f eine **lokale Kopie** von w zugewiesen (*call by value*)
- f arbeitet nur mit der Kopie, kann w **nicht modifizieren**
- Falls Modifikation von w erwünscht: Übergabe als **Referenzparameter** (*call by reference*)
  - in C nur *call-by-value* (f1)
  - in C++: Operator & (f2)
  - **Nachbildung** in C: *call by value* mit Zeiger auf w (f3)

```
void f1(int a) { a++; }
void f2(int& a) { a++; }
void f3(int* a) { (*a)++; }

int main(void)
{
  int a;
  a = 1;
  f1(a);  // a = 1
  f2(a);  // a = 2
  f3(&a); // a = 3
  return 0;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Wert- und Referenzparameter




```
void f1(int a) { a++; }
void f2(int& a) { a++; }
void f3(int* a) { (*a)++; }

int main(void)
{
  int a;
  a = 1;
  f1(a);  // a = 1
  f2(a);  // a = 2
  f3(&a); // a = 3
  return 0;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Funktionen mit mehreren Rückgabewerten

- **Beispiel**: komplexe Zahlen
  c = (a,b)
  Realteil a, Imaginärteil b
- **Produkt** zweier komplexer Zahlen $c_1 = (a_1,b_1)$, $c_2 = (a_2,b_2)$:
  $c_3 = (a_1 * a_2 - b_1 * b_2,$
  $a_1 * b_2 + a_2 * b_1)$
- Komplexe Zahlen sind **kein Basis-Datentyp** in C
- **Naive Lösung**: zwei Funktionen für Real- und Imaginärteil

```
float prd_real(float a1, float a2,
        float b1, float b2)
{
  return a1*a2-b1*b2;
}

float prd_imag(float a1, float a2,
        float b1, float b2)
{
  return a1*b2+a2*b1;
}

int main(void)
{
  float c_real, c_imag;
  c_real = prd_real( /* ... */ );
  c_imag = prd_imag( /* ... */ );
  return 0;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Funktionen mit mehreren Rückgabewerten

- Definition des Datentyps `complex` als `struct`
- structs werden als **Wertparameter** übergeben
- D.h. evtl. **kopieren** großer Speicherblöcke (langsam)
- Übergabe als **Referenzparameter** (per Zeiger) evtl. besser
- **Arrays** (s.u.) werden immer als Referenzparameter übergeben
- Arrays als **Rückgabewert** unzulässig (warum?)

```
typedef struct c {
 float a, b; } complex;

complex prod(complex c1, complex c2)
{
  complex c3;
  c3.a = c1.a*c2.a-c1.b*c2.b;
  c3.b = c1.a*c2.b+c2.a*c1.b;
  return c3;
}

int main(void)
{
  complex c1, c2, c3;
  c3 = prod(c1, c2);
  return 0;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

4. Funktionen und Programmaufbau

4.1 Funktionen
4.2 Parameterübergabe
4.3 Polymorhpie
4.4 Templates
4.5 Lambda Funktionen
4.6 Module und Programme

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

- **Auswahl der Funktionsdefinition in Abhängigkeit der Operanden**

```
#include <stdio.h>
#include <math.h>

// most of the time we compute square root
   float root(float x) {
      return pow(x,1/2.0);
   }

// ... but we can do any root
   float root(float x, float y) {
      return pow(x,1/y);
   }

int main( void )
{
    float x1 = 4, x2 = 27;

    printf("Square root of %.1f = %.3f\n", x1, root(x1));
    printf("Cubic root of %.1f = %.3f\n", x2, root(x2,3));

    return 0;
}
```

in c++:

```
... Program finished with exit code 0
```

Statische Polymorphie (compile-time polymorphism)
Auswahl der Methode während Kompilierung
*Hier: entscheidendes Merkmal - **Anzahl der Operanden***

(Zur Vereinfachung sind hier ‘stdio.h’ und ‘math.h’ eingebunden. In C++ sollten die Bibliotheken ‘cstdio’ und ‘cmath’ verwendet werden.)

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Polymorphie

- **Auswahl der Funktionsdefinition in Abhängigkeit der Operanden**

```
#include <stdio.h>

   #define TEST_INT
   #define TEST_FLOAT

#ifdef TEST_INT
int mult(int a, int b) {
    printf("Integer: %d x %d = %d\n", a, b, a*b);
    return a*b;
}
#endif

#ifdef TEST_FLOAT
float mult(float a, float b) {
    printf("Float: %.2f x %.2f = %.2f\n", a, b, a*b);
    return a*b;
}
#endif

int main( void )
{
    int i_a = 3, i_b = 4;
    float f_a = 3.1, f_b = 4.5;

    mult(i_a,i_b);
    mult(f_a,f_b);

    return 0;
}
```

Statische Polymorphie (compile-time polymorphism)
Auswahl der Methode während Kompilierung
*Hier: entscheidendes Merkmal - **Typ***

(dynamische Polymorphie folgt später ...)

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Funktionen als Parameter

- Funktionen sind keine Variablen, besitzen aber eine **Speicheradresse**
- Daher gibt es **Zeiger auf Funktionen**, diese können als **Parameter** übergeben und „aufgerufen“ werden
- Sinnvoll, falls **Aufrufstruktur** der Funktionen nicht statisch festliegen soll
- Bsp.:
  - Funktion `f`:

    ```
    int f(float, float);
    ```

  - Zeiger auf `f`:

    ```
    int (*p)(float,float);
    p = &f;
    ```

```
#include <stdio.h>
int add(int a, int b)
{
  return a+b;
}

int sub(int a, int b)
{
  return a-b;
}

int op(int (*f)(int,int), int a, int b)
{
  return f(a, b);
}

int main(void)
{
  char c;
  int result;
  scanf("%c", &c);
  if (c == 'a')
    result = op(add, 1, 2);
  else if (c == 's')
    result = op(sub, 1, 2);
  printf("%d\n", result);
  return 0;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

4. Funktionen und Programmaufbau

4.1 Funktionen
4.2 Parameterübergabe
4.3 Polymorhpie
4.4 Templates
4.5 Lambda Funktionen
4.6 Module und Programme

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

- **Typ als Variable**

Bisher:

```
int mult(int a, int b) {
    printf("Integer: %d x %d = %d\n", a, b, a*b);
    return a*b;

float mult(float a, float b) {
    printf("Float: %.2f x %.2f = %.2f\n", a, b, a*b);
    return a*b;
}
```

```
#include <cstdio>
#include <iostream>

template <typename T>
T mult(T a, T b) {
    std::string tname;
    switch( typeid(a).name()[0] ) {
        case 'i': tname = "Integer"; break;
        case 'f': tname = "Float"; break;
    }
    std::cout << tname << ": " << a << " x " << b << " = " << a*b << std::endl;
    return a*b;
}

int main( void )
{
    int i_a = 3, i_b = 4;
    float f_a = 3.1, f_b = 4.5;

    mult<int>(i_a,i_b);
    mult<float>(f_a,f_b);

    return 0;
}
```

Entsprechend können auch template-Klassen vereinbart werden.

© T. Gemmeke, IDS, RWTH

4. Funktionen und Programmaufbau

4.1 Funktionen
4.2 Parameterübergabe
4.3 Polymorhpie
4.4 Templates
4.5 Lambda Funktionen
4.6 Module und Programme

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Lambda Funktion

Anwendung von *lambda*-Funktionen
- Lokale und einmalige Nutzung
- Kein spezifischer Name
- Kleiner Funktionsumfang (‚body')

```
#include <iostream>
#include <string>
using namespace std;

// Regular Function
void report_age1(int val) {
    cout << "My age is " << val << endl;
}


int main( void )
{
    int age = 23;

    // Lambda Function
    auto report_age2 = [](int val) ->  string { return "My age is " + to_string(val); };
    auto report_age3 = [age]() -> string { return "My age is " + to_string(age); };

    report_age1(19);
    cout << report_age2(21) << endl;
    cout << report_age3() << endl;

    return 0;
}
```

Falls die lokale Variable in der *lambda*-Funktion geändert werden soll, ist ein ein call-by-reference erforderlich. Hier wäre dies:

“... [&age] ...”.

© T. Gemmeke, IDS, RWTH

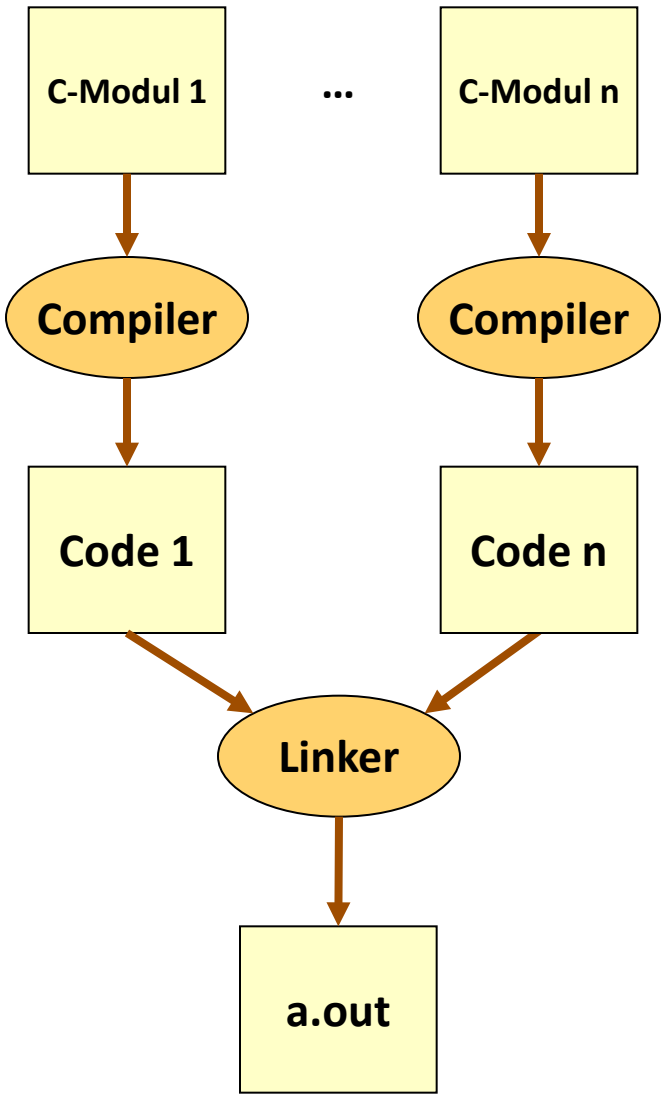
4. Funktionen und Programmaufbau

4.1 Funktionen
4.2 Parameterübergabe
4.3 Polymorhpie
4.4 Templates
4.5 Lambda Funktionen
4.6 Module und Programme

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Module und Programme

- C-Compiler übersetzt immer **ein einzelnes** C-Modul
- **Modul** = globale Variablen und Funktionen
- Entwicklung von Modulen kann **parallel** erfolgen
- Modul (z.B. Library) muss nicht unbedingt **main-Funktion** enthalten
- Komplettes **C-Programm** wird aus einzelnen Modulen zusammengesetzt (durch den Linker)
- C-Programm muss **genau eine** main-Funktion enthalten
- **Globale Variablen** und **Funktionen** gleichen Namens dürfen nur einmal vorkommen



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Module und Programme

- Zweckmäßig: **Trennung** von Deklarationen und Definitionen in zwei Dateien (meist: *.h und *.c)
- Deklarationen im **Header File**
- Header File kann von allen C-Modulen **eingebunden** werden

lib.h

```
/* Library
 mit math. Funktionen */
float sqrt( /* ... */ );
float sin( /* ... */ );
float cos( /* ... */ );
```

main.c

```
#include "lib.h"
int main(void)
{
  /* ... */
  a = sqrt( /* ... */ );
  b = sin( /* ... */ );
  c = cos( /* ... */ );
  return 0;
}
```

lib.c

```
/* Implementierung
 der math. Funktionen */
float sqrt( /* ... */ )
  { /* ... */ }
float sin( /* ... */ )
  { /* ... */ }
float cos( /* ... */ )
  { /* ... */ }
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

5. Datentypen

5.1 Arrays

5.2 Strukturen

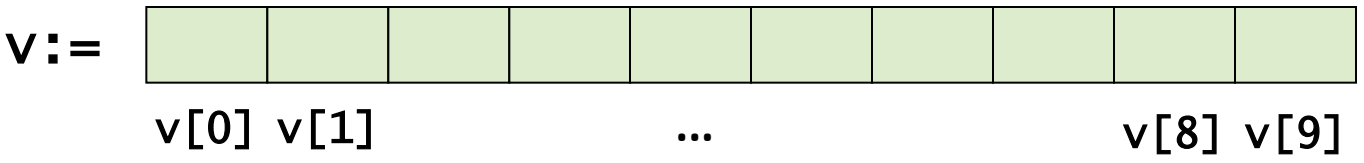
5.3 Abstrakte Datentypen

5.4 Klassen

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Arrays

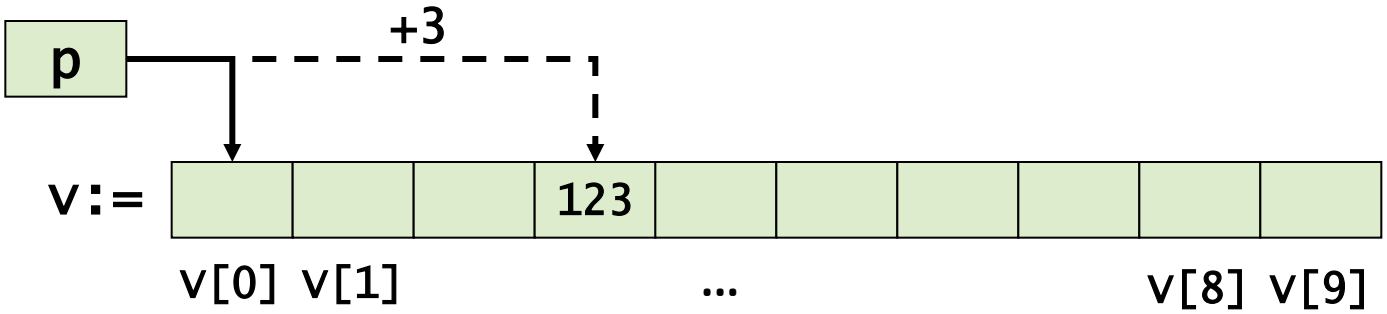
- **Arrays** (oder: Felder, Vektoren) definieren zusammengesetzte Datentypen, deren Elemente durch einen **Index** zugegriffen werden
- Vektor (**mathematisch**):
  - $V = (v_1, \ldots, v_n)$; v_i bezeichnet i-tes Element von V
- Vektor/Array-Deklaration in C/C++:
  - *Typ* `ID [`*int-Konstante*`];`
  - Z.B. `int v[10];`
    - deklariert 10-elementigen Vektor V
    - Komponenten sind vom Typ int
    - Zugriff auf i-tes Element mittels Operator `[...]`, z.B. `v[3]`
    - Der Ausdruck `3` ist der **Index** des Elementes
    - **Index-Numerierung beginnt immer bei 0**



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Arrays und Zeiger

- Benachbarte Arrayelemente befinden sich stets auch **benachbart** im Speicher
- Daher bei `v[i]`:
  - Index `i` bezeichnet den Abstand (*offset*) des i-ten Elementes von der **Basisadresse** `v` (= `&v[0]`)
  - Operator „[...]“ entspricht **Addition** von Array-Basisadresse und Index
  - `v[i] = *(v + i) = *(i + v) = i[v]`
  - `p = &v[0];` oder `p = v;` lässt `p` auf Anfang von `v` zeigen
  - Die folgenden 3 Ausdrücke sind äquivalent:
    `v[3] = 123;` `p[3] = 123;` und `*(p+3) = 123;`



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Arrays und Schleifen

- Vorteil von Arrays: **Menge „gleichartiger“ Objekte** kann mit einem Namen und variablem Index zugegriffen werden
- **Meist**: in Schleifen, damit Vermeidung von Code-Duplizierung
- Beispielprogramme zur Initialisierung von Array A sind äquivalent
- **Post-Inkrement:**

  `*p++ = ... ;`

  ist zu lesen wie

  `*p = ... ; p++;`

```
int i, a[10];

for(i=0; i<10; i++) {
  a[i] = i;
}
```

```
int *p, i, a[10];

p = &a[0];
for(i=0; i<10; i++) {
  *p++ = i;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Arrays und Schleifen

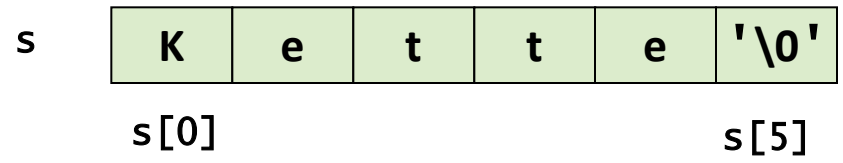
- **Array-Basisadressen** verhalten sich wie Zeiger (auf das erste Array-Element)
- Nach Deklaration `int a[10];` ist der Ausdruck `a` **identisch** mit dem Ausdruck `&a[0]`
- Grund: `a = a + 0 = &(*(a + 0)) = &a[0]`
- Achtung: Arrays können **nicht** mit einem einzigen Befehl kopiert werden

```
int a[10], b[10], i;
// unzulässig:
a = b;      // entspricht &a[0] = &b[0]
// stattdessen:
for (i=0; i<10; i++) {  // elementweises Kopieren
  a[i] = b[i];
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel: Ermittlung einer Stringlänge

- `String` = Array mit char-Elementen
- Element-Wert 0 (oder `'\0'` als char-Konstante) kennzeichnet **String-Ende**
- Daher: char-Arrays immer **ein Byte länger** als „Nutzlast“-String
- Beginnend bei 0:
  Zähler **inkrementieren**, bis
  `'\0'` -Zeichen erreicht
- Bsp.: `char s[6] = "Kette";`




```
#define STR "teststring"

int main(void)
{
  char* s = STR;
  int i, count = 0;

  for(i=0; s[i] != 0; i++) {
    count++;
  }

  printf("Länge = %d", count);
}
```

```
  /* ... */

  count = 0;
  while (s[count]) {
    count++;
  }

  /* ... */
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Schleifen in C++ mit Iteratoren

```
#include <iostream>
#include <algorithm>
#include <vector>
#include <string>
using namespace std;

void show(vector<int> c, string str) {cout<<str<<": "; for(auto e: c) cout<<e<<' ';cout<<endl<<endl;}

int main( void )
{
    vector<int> v = {1, 2, 2, 5, 6, 7};
    show(v,"Ausgangslage");

    // 1. Loop: using a copy of the value
    for(auto elm : v) { elm++; }
    show(v,"1. Loop");
    // 2. Loop: using the reference
    for(auto& elm : v) { elm++; }
    show(v,"2. Loop");
    // 3. Loop: using the reference and const
    // for(const auto& elm : v) { elm++; }
    // -> error: increment of read-only reference ‘elm’
    // 4. Loop: using for_each
    for_each(v.begin(), v.end(), [](int elm) { elm++; });
    show(v,"4. Loop");
    // 5. Loop: using classic for-loop and an iterator of a container
    for(auto it = v.begin(); it != v.end(); ++it) { (*it)++; };
    show(v,"5. Loop");

    return 0;
}
```

`for_each` benötigt ein *unary function object*

`auto` sucht den passen Typ automatisch aus.
Hier ist es: `std::vector<int>::iterator`

© T. Gemmeke, IDS, RWTH

Mehrdimensionale Arrays

Stunde	Mo	Di	Mi	Do	Fr
1	Mathe	Politik	Bio	Mathe	Politik
2	Mathe	Kunst	Deutsch	Mathe	Kunst
3	Deutsch	Informatik	Sport	Deutsch	Informatik
4	Englisch	Informatik	Sport	Englisch	Informatik
5	Sport	Mathe	Physik	Sport	Mathe
6	Sport	Mathe	Physik	Sport	Mathe

- Beispiel: Stundenplan
- 2-dimensionales Array (**Matrix**) als Datenstruktur
- Elemente vom Typ `String`
- Indizes:
  - Stunden (`int`)
  - Wochentage Mo-Fr (?)
  - `enum tag_typ {Mo, Di, Mi, Do, Fr}`
  - **Aufzählungstyp**: deklariert `Mo, ..., Fr` als symbolische Konstanten
  - **Intern**: `Mo = 0, Di = 1, ..., Fr = 4`

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Mehrdimensionale Arrays

```
#define STUNDEN 6
#define TAGE 5

#define MA "Mathe"
#define DE "Deutsch"
#define SP "Sport"
/* ... */

enum tag_typ {Mo, Di, Mi, Do, Fr};

int main (void)
{
  char* plan[STUNDEN][TAGE];
  int tag, stunde;

  /* ... */
```

```
  /* ... */
  plan[0][Mo] = MA;
  /* ... */
  plan[5][Mo] = SP;
  /* ... */
  plan[5][Fr] = MA;

  printf("Stundenplanabfrage:\n");
  scanf("%d",&tag);
  scanf("%d",&stunde);

  if ((tag > Fr) || (stunde > STUNDEN))
    printf("Ungültig");
  else
    printf("Fach = %s", plan[stunde-1][tag];

  return 0;
}
```

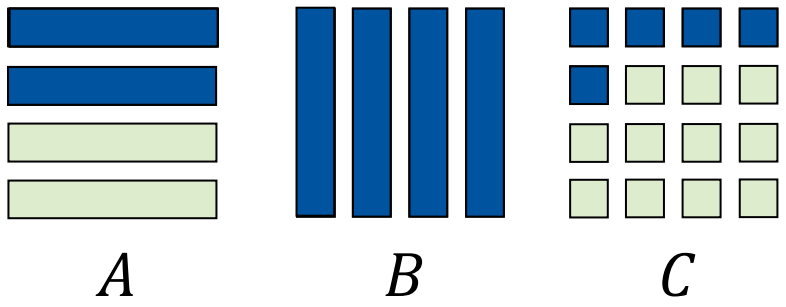
Stunde	Mo	Di	Mi	Do	Fr
1	Mathe	Politik	Bio	Mathe	Politik
2	Mathe	Kunst	Deutsch	Mathe	Kunst
3	Deutsch	Informatik	Sport	Deutsch	Informatik
4	Englisch	Informatik	Sport	Englisch	Informatik
5	Sport	Mathe	Physik	Sport	Mathe
6	Sport	Mathe	Physik	Sport	Mathe

```
Stundenplanabfrage:
> 2
> 4
Fach = Sport
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel: Matrixmultiplikation

- $(N \times N)$-Matrizen:
  $A = (a_{ij}), B = (b_{ij})$
- **Produkt** $C = (c_{ij})$ ergibt sich aus:
  $c_{ij} = \sum_{k=1}^{N} a_{ik} \cdot b_{kj}$
- **Geschachtelte** Schleifen
  - Iteration über Zeilen $i = 1 \ldots N$
  - Für jedes i: Iteration über Spalten $j = 1 \ldots N$
  - Für jedes (i, j):
    Bilde Skalarprodukt von A-Zeile i und B-Spalte j
- C/C++ gestattet eine beliebige Anzahl von **Dimensionen** für Arrays
- Elemente liegen stets benachbart im Speicher




```
typedef int matrix[N][N];
matrix a, b, c;
for (i=0; i<N; i++)
  for (j=0; j<N; j++) {
    c[i][j] = 0;
    for (k=0; k<N; k++)
      c[i][j] += a[i][k]*b[k][j];
  }
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

5. Datentypen

5.1 Arrays

5.2 Strukturen

5.3 Abstrakte Datentypen

5.4 Klassen

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Strukturen

- **Struktur** (oder: *struct, record*): Zusammenfassung einer Gruppe von Variablen zu einer **Einheit**
- **Unterschied** zum Array: Adressierung innerhalb einer Struktur per Namen, nicht per Index
- Bsp. (s.o.): geometrischer Punkt

```
struct point { float x, y; };
```

- Allgemeine Form einer struct-Deklaration:
  - `struct` *`ID`* `{` *`Variablendeklaration*`* `}`
  - Beliebige Variablendeklarationen (auch structs)
  - *ID* heißt *structure tag*
- struct-Deklaration oft in Verbindung mit typedef:

```
struct point { float x, y; } p1,p2;  oder
typedef struct point { float x, y; } Point;
Point p1, p2;
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Operationen auf Strukturen

```
typedef struct person
   { char name[100], vorname[100];
     struct datum { int tag,monat,jahr; } gebdatum;
   } Person;
Person p1, p2, *p_ptr;
```

- **Kopieren:**

```
p1 = p2;  /* kopiert alle Komponenten auf einmal */
```

- **Adressberechnung:**

```
p_ptr = &p1;
```

- **Komponentenzugriff direkt** (Operator „.“):

```
p2.gebdatum.jahr = 1950;
```

- **Komponentenzugriff per Zeiger** (Operator „->“):

```
p_ptr->gebdatum.monat = 12;
  // "p->" entspricht "(*p)."
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Array von structs

- Mit Arrays und structs lassen sich beliebige **komplexe Datenstrukturen** aufbauen
- Bsp.: **Bundesliga-Tabelle**
- **Übung**: Neuberechnung nach Spieltag-Ergebniseingabe

		SP	S	U	N	T	DIFF	PKT
1	Bayern München	3	3	0	0	10:0	10	9
2	Arminia Bielefeld	3	2	1	0	7:3	4	7
3	VfL Bochum	3	2	1	0	6:4	2	7
4	Eintracht Frankfurt	3	2	1	0	4:2	2	7
5	Hamburger SV	3	2	0	1	3:2	1	6
6	FC Schalke 04	3	1	2	0	7:4	3	5
7	Bayer Leverkusen	3	1	1	1	3:1	2	4
8	VfL Wolfsburg	3	1	1	1	5:5	0	4
9	VfB Stuttgart	3	1	1	1	4:5	-1	4
10	Werder Bremen	3	1	1	1	3:6	-3	4
11	MSV Duisburg	3	1	0	2	4:5	-1	3
12	Hertha BSC	3	1	0	2	3:4	-1	3
13	Borussia Dortmund	3	1	0	2	5:7	-2	3
14	Karlsruher SC	3	1	0	2	3:5	-2	3
15	1. FC Nürnberg	3	1	0	2	2:4	-2	3
16	Hannover 96	3	1	0	2	2:5	-3	3
17	Energie Cottbus	3	0	1	2	1:5	-4	1
18	Hansa Rostock	3	0	0	3	1:6	-5	0

```
typedef struct team {
 char *name;
 int SP,S,U,N,T1,T2,DIFF,PKT;
} Team;

Team liga[18]; /* 0...17 */
/* ... */
liga[5].name = "FC Schalke 04";
liga[13].DIFF = liga[13].T1-liga[13].T2;
/* ... */
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

5. Datentypen

5.1 Arrays

5.2 Strukturen

5.3 Abstrakte Datentypen

5.4 Klassen

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Abstrakte Datentypen

Def.: Ein **abstrakter Datentyp** (**ADT**) ist ein Datentyp (eine Menge von Werten und eine Sammlung von Operationen auf diesen Werten), der nur über eine Schnittstelle zugänglich ist.

[Sedgewick, Algorithmen in C++]

- Menge `set`

```
#include <iostream>
#include <set>
#include <algorithm> // required for set_union
using namespace std;

int main( void )
{
    set<int> s = {1, 2, 2};
    set<int> t = {7, 2, 4, 1};

    set<int> r;
    set_union(s.begin(), s.end(), t.begin(), t.end(), inserter(r, r.begin()));

    // print the contents of the sets
    cout << "s: "; for(auto elm : s) cout << elm << ' '; cout << endl;
    cout << "t: "; for(auto elm : t) cout << elm << ' '; cout << endl;
    cout << "r: "; for(auto elm : r) cout << elm << ' '; cout << endl;

    return 0;
}
```

Weitere Methoden in dieser Gruppe:

- `insert`
- `erase`
- `merge`
- `set_difference` (C++20)
- `set_intersection` (C++20)

© T. Gemmeke, IDS, RWTH

5. Datentypen

5.1 Arrays

5.2 Strukturen

5.3 Abstrakte Datentypen

5.4 Klassen

i (De)Konstruktor

ii Public vs. private

iii Vererbung

iv Abstrakte Klassen

v Überladen von Operatoren

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

- **Klassen und Methoden**

```
#include <string>
#include <iostream>
using namespace std;

class Member {
    public:
        string status = "public";
};

int main( void ) {

    Member someone;

    cout << someone.status << endl;

    return 0;
}
```


© T. Gemmeke, IDS, RWTH

5. Datentypen

5.1 Arrays

5.2 Strukturen

5.3 Abstrakte Datentypen

5.4 Klassen

i (De)Konstruktor

ii Public vs. private

iii Vererbung

iv Abstrakte Klassen

v Überladen von Operatoren

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

- (De)Constructor

```
#include <string>
#include <iostream>
using namespace std;

class Member {
    public:
        string status;
        // constructor
        Member() { status = "starter"; }
        Member( string val ) : status(val) {}
        // (de)constructor
        ~Member() { cout << "... a member left the pack." << endl; }
        void show_status( void ) {
            cout << "Membership status: " << status << endl;
        }
};
void sub( void ) {
    cout << "> Entering function 'sub'" << endl;
    Member pers("gold");
    pers.show_status();
    cout << "> About to leave 'sub'" << endl;
}
int main( void ) {
    cout << "> Starting in 'main'" << endl;
    Member someone;
    someone.show_status();
    sub();
    cout << "> About to leave 'main'" << endl;
    return 0;
}
```


© T. Gemmeke, IDS, RWTH

5. Datentypen

5.1 Arrays

5.2 Strukturen

5.3 Abstrakte Datentypen

5.4 Klassen

i (De)Konstruktor

ii Public vs. private

iii Vererbung

iv Abstrakte Klassen

v Überladen von Operatoren

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

- **Klassenelemente: public vs. private**

```
#include <string>
#include <iostream>
using namespace std;

class Member {
    public:
        string statusA = "public"; // public Variable
    private:
        string statusB = "secret"; // private Variable
};

int main( void ) {

    Member someone;

    cout << someone.statusA << endl;
    cout << someone.statusB << endl;

    return 0;
}
```

error: ‘std::string Member::statusB’ is private within this context

© T. Gemmeke, IDS, RWTH

- **Klassenelemente: public vs. private**

```
#include <string>
#include <iostream>
using namespace std;

class Member {
    private:
        string status = "unknown"; // Private Variable

    public:
        // public methods
        void show_status( void ) {
            cout << "Status: " << status << endl;
        }
        void update_status( string new_status ) {
            status = new_status;
        }
};

int main( void ) {

    Member someone;

    someone.show_status();
    someone.update_status("gold");
    someone.show_status();

    return 0;
}
```

Private Variablen werden mit sogenannten *get* und *set* Methoden verarbeitet.

© T. Gemmeke, IDS, RWTH

5. Datentypen

5.1 Arrays

5.2 Strukturen

5.3 Abstrakte Datentypen

5.4 Klassen

i (De)Konstruktor

ii Public vs. private

iii Vererbung

iv Abstrakte Klassen

v Überladen von Operatoren

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

- **Vererbung**

Weiterführung des Beispiels:
Die Funktion **`main`** auf der vorrangegangen Folie wird durch den Code auf dieser Folie ersetzt.

```
...

class Student : public Member {
    public:
        Student() { // constructor
            Member::update_status("Student");
        }
};

int main( void ) {
    Member someone;
    Student pers;

    someone.show_status();
    pers.show_status();

    return 0;
}
```


© T. Gemmeke, IDS, RWTH

- **Vererbung**

... und noch einen Schritt ...

```
...

class Prof : public Member {
    public:
        Prof() { // constructor
            Member::update_status("Professor");
        }
};

int main( void ) {
    Member someone;
    Student persA;
    Prof persB;

    someone.show_status();
    persA.show_status();
    persB.show_status();

    return 0;
}
```


© T. Gemmeke, IDS, RWTH

- **Dynamische Auswahl der Methode**

Dynamische Polymorphie (run-time polymorphism)
Auswahl der Methode während der Ausführung.
Hier: entscheidendes Merkmal:
Zugehörigkeit zu einer (Sub)Klasse

```
#include <string>
#include <iostream>
using namespace std;

class Being {
    public:
       string name;
        Being() {}
        Being( string str )  { name = str; }
        virtual void show( void ) { cout << "'" << name << "' is an unkown being." << endl << endl; }
};

class Insect : public Being {
    public:
        Insect( string str )  { name = str; }
        void show( void ) override { cout << "The '" << name << "' is an insect." << endl << endl; }
};

class Mammal : public Being {
    public:
        Mammal( string str )  { name = str; }
        void show( void ) override { cout << "The '" << name << "' is a mammal." << endl << endl; }
};

int main( void )
{
    Being* b; // pointer to the base class
    string in;

    while( 1 ) {
        cout << "Enter the type of being (i: insect, m: mammal): ";
        cin >> in;
        cout << endl;

       if( in[0] == 'i' ) b = new Insect("ant");
       else if( in[0] == 'm' ) b = new Mammal("cat");
       else b = new Being(in);

       b->show(); // method show is dynamically selected
    }
    return 0;
}
```


© T. Gemmeke, IDS, RWTH

5. Datentypen

5.1 Arrays

5.2 Strukturen

5.3 Abstrakte Datentypen

5.4 Klassen

i (De)Konstruktor

ii Public vs. private

iii Vererbung

iv Abstrakte Klassen

v Überladen von Operatoren

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

- **Beispiel: Schnittstellendefinition**

```
#include <string>
#include <iostream>
using namespace std;

class Being {
    public:
       string name;
        Being() {}
        Being( string str )  { name = str; }
        virtual void show() = 0; // Definition einer rein virtuellen Methode
};

class Insect : public Being {
    public:
        Insect( string str )  { name = str; }
        void show() { cout << "The '" << name << "' is an insect." << endl << endl; }
};

class Mammal : public Being {
    public:
        Mammal( string str )  { name = str; }
        void show() { cout << "The '" << name << "' is a mammal." << endl << endl; }
};

int main( void )
{
    Being* b; // pointer to the base class
    string in;

    while( 1 ) {
        cout << "Enter the type of being (i: insect, m: mammal): ";
        cin >> in;
        cout << endl;

        if( in[0] == 'i' ) b = new Insect("ant");
        else b = new Mammal(in);

       b->show();
    }
    return 0;
}
```

Welcher Programmierfehler ist hier versteckt?

© T. Gemmeke, IDS, RWTH

5. Datentypen

5.1 Arrays

5.2 Strukturen

5.3 Abstrakte Datentypen

5.4 Klassen

- i (De)Konstruktor
- ii Public vs. private
- iii Vererbung
- iv Abstrakte Klassen
- **v Überladen von Operatoren**

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Überladen von Operatoren

- 1 + 1 = …?!

```
#include <iostream>
#include <vector>
using namespace std;

class myInt {
    public:
        int x;
        myInt() { x = 0; };
        myInt( int val ) : x(val){};
        myInt& operator+(const myInt& val) {
            myInt* o = new myInt(this->x + val.x + 1);
            return *o;
        }
        myInt& operator=(const myInt& rhs) {
            x = rhs.x;
            return *this;
        }
        friend ostream& operator<<(ostream& os, const myInt& val) {
            os << val.x;
            return os;
        }
};

int main( void )
{
    myInt a = 1, b = 1;

    cout << "a = " << a << "; b = " << b << "  =>  a+b = " << a+b << endl;

    return 0;
}
```

Quiz: Was ist das Ergebnis von 1+1? 0, 1, 2, oder 3?

© T. Gemmeke, IDS, RWTH

Überladen von Operatoren

- Vereinigungsmenge (‚union‘) zweier Vektoren mit dem ‘+’-Operator

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

vector<int>& operator+(const vector<int>& a, const vector<int>& b) {
    vector<int>* res = new vector<int>();
    res->insert(res->begin(), a.begin(), a.end());
    res->insert(res->end(), b.begin(), b.end());
    // Let's have the vector sorted
    sort(res->begin(), res->end());
    // ... and remove all duplicate.
    // Note: the vector has to be sorted for unique to work
    // Note: unique does not shrink the vector length, so we add a call to erase
    res->erase( unique( res->begin(), res->end() ), res->end() );
    return *res;
}

int main( void )
{
    vector<int> s = {1, 2, 2};
    vector<int> t = {4, 5, 6};

    vector<int> r = s+t;

    // fancy: for(const auto& elm : ...
    for(auto elm : r) cout << elm << ' ';
    cout << endl;

    return 0;
}
```

Anmerkung: der ADT `set` bietet diese Funktionalität ...

© T. Gemmeke, IDS, RWTH

Überladen von Operatoren

- **Stream**

```
#include <string>
#include <iostream>
using namespace std;

class Graphic {
    public:
        string name;
        Graphic() { name = "generic object"; }
};

ostream& operator<<(ostream& os, const Graphic& gr) {
            os << gr.name;
            return os;
}

int main( void ) {

    Graphic a;

    cout << "This is a '" << a << "'." << endl;

    return 0;
}
```


© T. Gemmeke, IDS, RWTH

Überladen von Operatoren

- **Stream**

```
class Point : public Graphic {
    public:
        int x0, y0;
        Point( int a, int b ) : x0(a), y0(b) { name = "Point"; }
};
class Line : public Graphic {
    public:
        int x0, y0, x1, y1;
        Line( int a, int b, int c, int d) : x0(a), y0(b), x1(c), y1(d) { name = "Line"; }
};

ostream& operator<<(ostream& os, const Point& gr) {
            os << gr.name << " at (" << gr.x0 << ", " << gr.y0 << ")";
            return os;
}
ostream& operator<<(ostream& os, const Line& gr) {
            os << gr.name << " from (" << gr.x0 << ", " << gr.y0 << ")";
            os << " -> (" << gr.x1 << ", " << gr.y1 << ")";
            return os;
}

int main( void ) {
    Graphic a;
    Point b(1,2);
    Line c(3,4,5,6);

    cout << "This is a '" << a << "'." << endl;
    cout << "This is a '" << b << "'." << endl;
    cout << "This is a '" << c << "'." << endl;

    return 0;
}
```


© T. Gemmeke, IDS, RWTH

Überladen von Operatoren

- Functor

Funktoren werden häufig in der Methode `transform` genutzt. Diese wendet den Funktor auf alle Elemente aus einem vorgegebenen Bereich an.

```
#include <iostream>
using namespace std;

class Modulo {
    private:
        long mod;
    public:
        Modulo(long val) : mod(val) {}

        long operator()(long x) {
            return x%mod;
        }

        friend ostream& operator<<(ostream& os, const Modulo& x) {
            os << x.mod;
            return os;
        }
};

int main( void )
{
    Modulo m(5);
    cout << "129 mod " << m << " = " << m(129) << endl;
    cout  << "127 mod " << m << " = " << m(127) << endl;

    return 0;
}
```


© T. Gemmeke, IDS, RWTH

Überladen von Operatoren

- Beispiel: Functor *translate*

Functors hat die besondere Eigenschaft zustandsorientiert (‚stateful‘) zu sein.

```
#include <iostream>
#include <algorithm>
#include <string>
#include <vector>
using namespace std;

class translate {
    private:
        vector<string> ge = {"eins","zwei","drei","vier"};
        vector<string> en = {"one","two","three","four"};
        vector<string> fr = {"un","deux","trois","quatre"};
        vector<string>* source;
        vector<string>* target;
    public:
        translate(string str) { // constructor
            switch( str[0] ) {
                case 'G': source = &ge; break;
                case 'E': source = &en; break;
                case 'F': source = &fr; break;
            }
            switch( str[2] ) {
                case 'G': target = &ge; break;
                case 'E': target = &en; break;
                case 'F': target = &fr; break;
            }
        }
        string operator() (string word) { // overloading of function call operator ()
            string res = word;
            auto it = find(source->begin(), source->end(), word);
            size_t i = distance(source->begin(), it);
            if( i<source->size() && i<target->size() ) res = (*target)[i];
            return res;
        }
};

int main( void ) {
    vector<string> number = {"vier", "eins", "zwei", "four", "trois"};
    for( auto elm : number ) cout << ' ' << elm; cout << endl;

    transform(number.begin(), number.end(), number.begin(), translate("GeEn"));
    for( auto elm : number ) cout << ' ' << elm; cout << endl;

    return 0;
}
```

`find` ersetzt klassische Suche im ADT `array`:

```
size_t i=0;
while( i<source->size() && word != (*source)[i] ) i++;
```


© T. Gemmeke, IDS, RWTH

6. Analyse von Algorithmen

6.1 Laufzeitanalyse

6.2 $\mathcal{O}$-Notation für Wachstumsordnungen

6.3 Beispiel: Exponentialfunktion

6.4 Beispiel: Suchen in Arrays

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Analyse von Algorithmen

- Für jedes Problem gibt es **viele mögliche** Algorithmen und Implementierungen
- Häufig gesucht: **effizienteste/schnellste** Implementierung
- Wie wird diese bestimmt?
  - Algorithmenentwicklung ist ein **kreativer** Prozess
  - **Analyse** eines gegebenen Algorithmus mittels **Wachstumsordnungen**
- Bsp.: Fakultätsfunktion

Factorial: $n! = 1 \cdot 2 \cdot \ldots \cdot n = \prod_{i=1}^{n} i$

```
#define n 10

/* ... */

  int fact, count;

  fact = 1;

  for (count = 1; count <= n; count++) {
    fact *= count;
  }

  printf("factorial(%d) = %d", n, fact);
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Analyse von Algorithmen

- Beispiel-Ablauf für $n = 5$
- Welche **Laufzeit** ergibt sich?
  - 19 Einzelschritte (= Statements)
- **Probleme**:
  - Welche Laufzeit hat ein Schritt?
  - Z.B. ist „++“ teurer als „*=“ ?
  - **Rechnerabhängig**!
- Daher: **abstrakte** Analyse
  - Wie viele Einzelschritte werden ausgeführt?
  - Unabhängig von Rechnergeschwindigkeit (MIPS) und Kosten der Einzelschritte

count	
	fact = 1
1	count = 1
	count <= n?
	fact *= count
2	count++
	count <= n?
	fact *= count
3	count++
	count <= n?
	fact *= count
4	count++
	count <= n?
	fact *= count
5	count++
	count <= n?
	fact *= count
6	count++
	count <= n?
	printf(...)

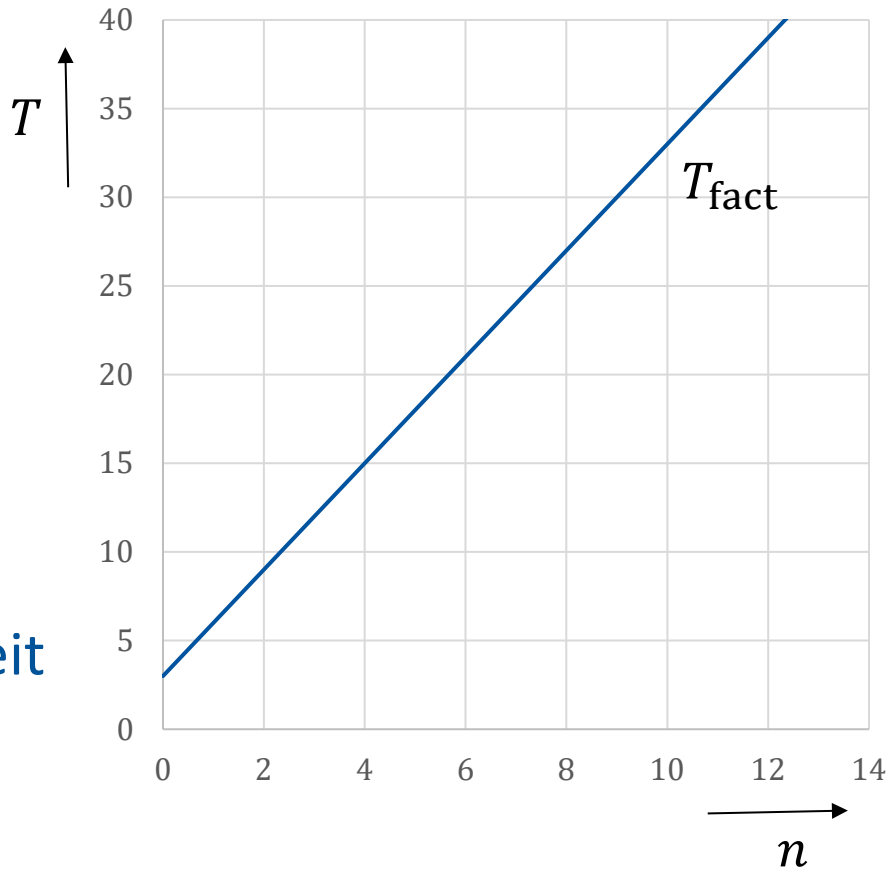
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Analyse von Algorithmen

- Einzelschritte bei Funktion `fact`:
  - **Initialisierung** von `fact` und `count`: 2 Schritte
  - Pro **Schleifeniteration**: 3 Schritte (<=, *=, ++)
  - **Schleifenende**: 1 Schritt
- Für beliebiges n:

$$T_{\text{fact}}(n) = 2 + 3 \cdot n + 1 = 3n + 3$$

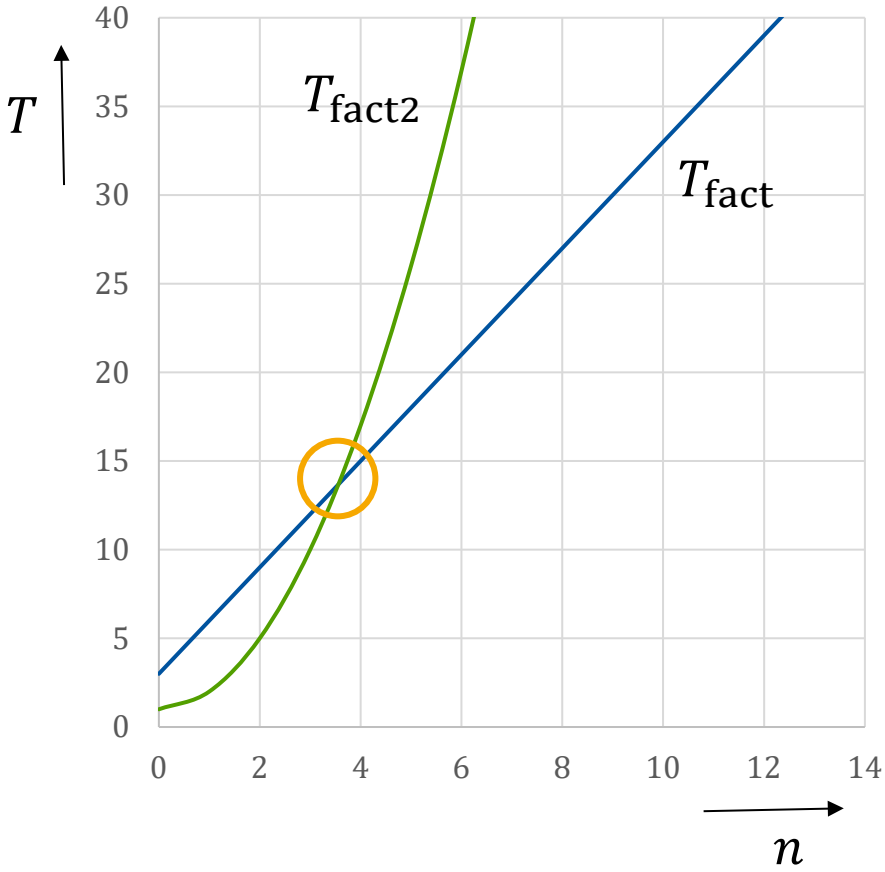
- **Lineare Laufzeit** in Abhängigkeit der Eingabegröße n



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Analyse von Algorithmen

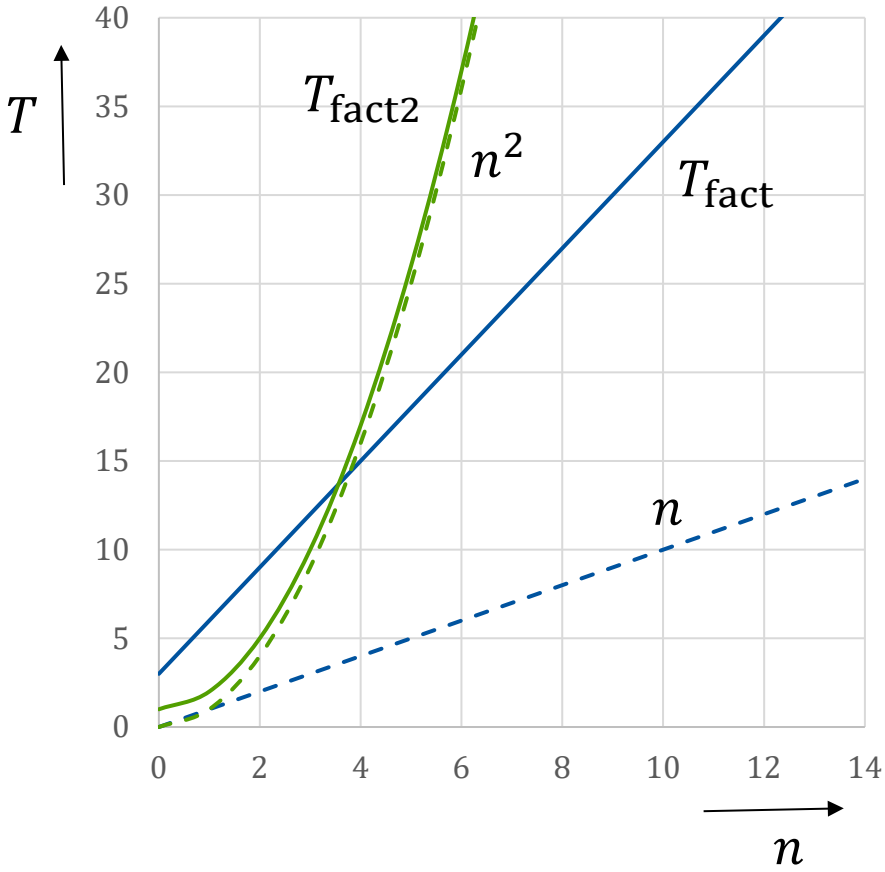
- **Annahme**: ein anderer Algorithmus `fact2` hat Laufzeit
  $$T_{\text{fact2}}(n) = n^2 + 1$$
- `fact2` ist schneller für
  $$n \leq 3$$
- Ab $n = 4$ benötigt `fact2` stets höhere Laufzeit
- Differenz steigt mit n
- **Ergebnis**: `fact2` ist für fast alle n langsamer als `fact` (d.h. die **asymptotische** Laufzeit von `fact2` ist größer als von `fact`)



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Analyse von Algorithmen

- **Konstante** Faktoren und **Konstanten** vernachlässigen!
- Analyse unabhängig von **Rechnerdetails**
- Frage: wie **wächst die Laufzeit** bei Erhöhung der Eingabegröße n?
- **Näherung**:
  $T_{\text{fact}}(n) \approx n$
  $T_{\text{fact2}}(n) \approx n^2$
- Beeinflusst den **Vergleich** der Algorithmen nur unwesentlich



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

6. Analyse von Algorithmen

6.1 Laufzeitanalyse

6.2 $\mathcal{O}$-Notation für Wachstumsordnungen

6.3 Beispiel: Exponentialfunktion

6.4 Beispiel: Suchen in Arrays

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

$\mathcal{O}$-Notation

- $\mathcal{O}$-Notation ($\mathcal{O}$ = *order*) ist das gebräuchlichste Hilfsmittel zur **Wachstumsanalyse** von Algorithmen
- $\mathbb{N}$ und $\mathbb{R}$ seien die Menge der natürlichen bzw. reellen Zahlen

Def.: Sind $f : \mathbb{N} \to \mathbb{N}$ und $g : \mathbb{N} \to \mathbb{N}$ zwei Funktionen, so ist

$$f \in \mathcal{O}(g), \text{ falls}$$
$$\exists\, n_0 \in \mathbb{N}, \exists\, c \in \mathbb{R}, \text{ so dass}$$
$$\forall\, n \geq n_0 \colon f(n) \leq c \cdot g(n)$$

- $f \in \mathcal{O}(g)$ heißt:
  „f wächst nicht schneller als g“
- $\mathcal{O}$-Notation **eliminiert Konstanten** aus der Laufzeitberechnung
- Z.B.: $f(n) = 10n^3 + 20n^2 + 16 \in \mathcal{O}(n^3)$, da $f(n) \leq 11n^3$ (d.h. $c = 11$) sicherlich für alle $n \geq 30$

n	$f(n)$	$c \cdot g(n)$
0	16	0
10	12016	11000
20	88016	88000
30	288016	297000
40	672016	704000
50	1300016	1375000
60	2232016	2376000
70	3528016	3773000
80	5248016	5632000
90	7452016	8019000
100	10200016	11000000

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

$\mathcal{O}$-Notation

- $\mathcal{O}$-Notation gibt Wachstumsordnung nur **näherungsweise** an
- Nach **Definition** gilt z.B. auch

$$10^{20}\, n^2 \in \mathcal{O}(1/100 \cdot n^3)$$

- Jedoch gilt $10^{20}\, n^2 \leq 1/100\, n^3$ erst ab $n_0 = 10^{22}$
- Eingabegröße von 10^{22} **unrealistisch**, daher wäre die obige Wachstumsordnung wenig sinnvoll
- In der Praxis kommen jedoch eher **kleine konstante** Faktoren vor, daher ermöglicht die O-Notation
  - Die ungefähre **Abschätzung** der Laufzeit eines Algorithmus
  - Den näherungsweisen **Vergleich** der Laufzeit verschiedener Algorithmen
- Vor allem: sinnvoll **vor** der detaillierten Implementierung

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

$\mathcal{O}$-Notation

- Mittels $\mathcal{O}$-Notation können auch **andere Maße** für die Qualität eines Algorithmus im Vergleich zu anderen abgeschätzt werden, z.B. **Speicherbedarf**
- Außerdem kann die $\mathcal{O}$-Notation aussagen, ob sich die Suche nach einem **besseren Algorithmus** für ein Problem lohnt:
  - Für viele praktisch relevante Probleme ist bekannt, dass ihre optimale Lösung **mindestens Zeit $\mathcal{O}(2^n)$** benötigt
  - Daher ist die Suche nach einem effizienten Algorithmus **Zeitverschwendung**
  - Für andere Probleme sind bessere **untere Schranken** bekannt
  - Z.B. $\mathcal{O}(n \log n)$ für das **Sortieren** von n Zahlen
  - Solche Algorithmen **wurden bereits gefunden**, daher lohnt die Suche nach besseren Verfahren kaum

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Rechnen mit $\mathcal{O}$-Notationen

- Häufig wird die **Gesamtlaufzeit** eines Algorithmus als mathematischer Ausdruck der **Laufzeiten von Teilalgorithmen** bestimmt
- **Rechenregeln**:
  - „$\mathcal{O}$“ im Ausdruck vernachlässigen
  - Umformen des Ausdrucks wie gewohnt
  - Nur den führenden Term (höchster Exponent) behalten
- Z.B. $g(n) = 2n \cdot \mathcal{O}(n) \in \mathcal{O}(n^2)$
- $\mathcal{O}(n)$ repräsentiert alle Funktionen f mit **linearem Wachstum**, d.h.

$$f(n) = c_1 n + c_2 \quad \text{mit } c_1, c_2 \in \mathbb{R}$$

$$g(n) = 2n \cdot (c_1 n + c_2) = 2c_1 n^2 + 2c_2 n$$

$$c_3 := 2c_1$$

$$c_4 := 2c_2$$

$$\Longleftrightarrow c_3 n^2 + c_4 n \quad \text{mit } c_3, c_4 \in \mathbb{R}$$

$$\in \mathcal{O}(n^2) + \mathcal{O}(n) \in \mathcal{O}(n^2) \text{ gemäß Definition}$$

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

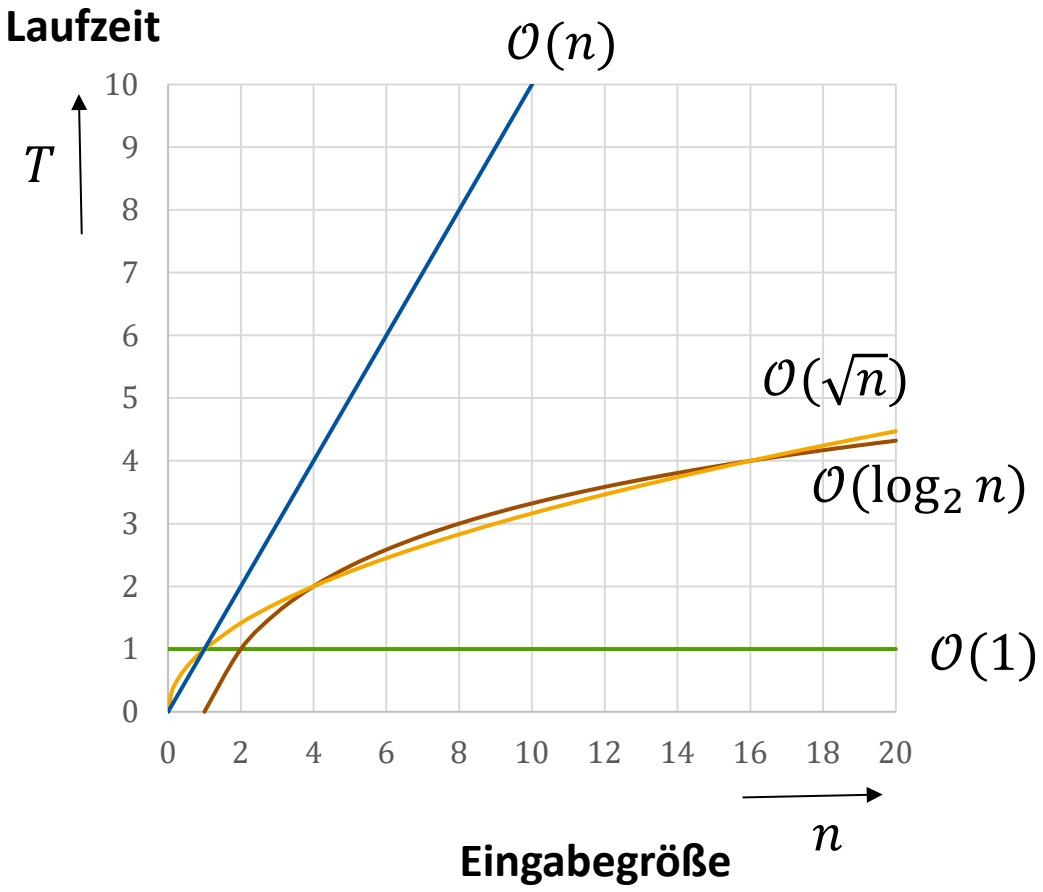
Rechnen mit $\mathcal{O}$-Notationen

- Komplexeres Beispiel:

$$
\begin{aligned}
&\big(\boldsymbol{n + \mathcal{O}(1)}\big)\big(\boldsymbol{n + \mathcal{O}(\log n) + \mathcal{O}(1)}\big) \\
&= n^2 + n \cdot \mathcal{O}(\log n) + n \cdot \mathcal{O}(1) + \\
&\quad \mathcal{O}(1) \cdot n + \mathcal{O}(1) \cdot \mathcal{O}(\log n) + \mathcal{O}(1) \cdot \mathcal{O}(1) \\
&= n^2 + \mathcal{O}(n \log n) + \mathcal{O}(n \cdot 1) + \mathcal{O}(1 \cdot n) + \mathcal{O}(1 \cdot \log n) + \mathcal{O}(1 \cdot 1) \\
&= \mathcal{O}(n^2) + \mathcal{O}(n \log n) + \mathcal{O}(n) + \mathcal{O}(n) + \mathcal{O}(\log n) + \mathcal{O}(1) \\
&= \mathcal{O}(n^2) + \mathcal{O}(n \log n) + \mathcal{O}(2 \cdot n) + \mathcal{O}(\log n) + \mathcal{O}(1) \\
&= \mathcal{O}(n^2) + \mathcal{O}(n \log n) + \mathcal{O}(n) + \mathcal{O}(\log n) + \mathcal{O}(1) \\
&= \mathcal{O}(n^2 + n \log n + n + \log n + 1) \\
&= \boldsymbol{\mathcal{O}\big(n^2\big)}
\end{aligned}
$$

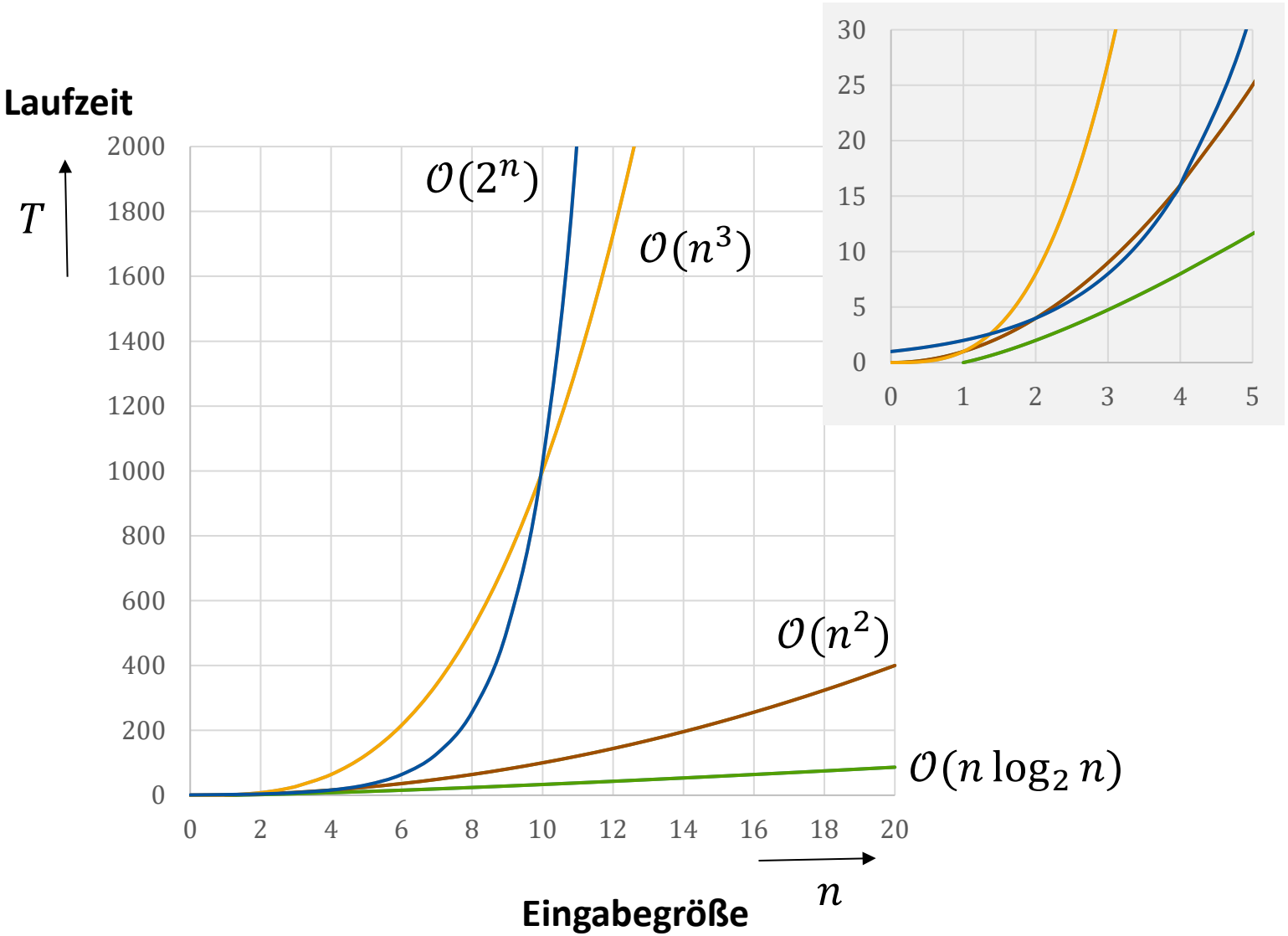
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Wichtige Wachstumsordnungen



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Wichtige Wachstumsordnungen



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Wichtige Wachstumsordnungen

- Annahme: Rechner kann **1000 Operationen pro Sekunde** ausführen
- Welche **Eingabegröße** kann in Zeit t bearbeitet werden ?
- Kleine konstante Faktoren spielen nur **geringe Rolle**!

$T(n)$	$t = 1\text{sec}$ (1000 Op)	$t = 1\ \text{min}$ (60.000 Op)	$t = 1\text{h}$ (3.600.000 Op)
$\log_2 n$	2^{1000}	$2^{60.000}$	$2^{3.600.000}$
$\sqrt{n}$	1.000.000	3.600.000.000	12.960.000.000.000
n	1000	60.000	3.600.000
$n \log_2 n$	140	4.895	204.094
n^2	31	244	1.897
n^3	10	39	153
2^n	9	15	21

Wichtige Wachstumsordnungen

- Was ändert sich, wenn der Rechner **10x schneller** wird?
- Betrachte max. Eingabegröße m, die in vorgegebener Zeit bearbeitet werden kann
- Bei **exponentieller Laufzeit** $\mathcal{O}(2^n)$ lässt auch eine **Vertausendfachung** der Rechenleistung die max. Eingabegröße nur um ≈ 10 wachsen (da $\log_2 1000 \approx 10$)

$T(n)$	vorher	nachher	
$\log_2 n$	m	m^{10}	$\log m^{10} = 10 \log m$
$\sqrt{n}$	m	$100\,m$	$10 = \sqrt{100}$
n	m	$10\,m$	
$n \log_2 n$	m	$\approx 8\,m$	Faktor hängt von m ab
n^2	m	$3{,}16\,m$	$10 \approx 3{,}16^2$
n^3	m	$2{,}15\,m$	$10 \approx 2{,}15^3$
2^n	m	$m + 3{,}3$	$10 \approx 2^{3{,}3}$

6. Analyse von Algorithmen

6.1 Laufzeitanalyse

6.2 $\mathcal{O}$-Notation für Wachstumsordnungen

6.3 Beispiel: Exponentialfunktion

6.4 Beispiel: Suchen in Arrays

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel: Exponentialfunktion

- Berechne b^n für $b \in \mathbb{R}$ und $n \in \mathbb{N}$
- Es gilt:
  - $b^0 = 1$, und für $n > 0$:
  - $b^n = b \cdot b^{n-1}$
- **Rekursiver** Algorithmus (d.h. Algorithmus ruft sich selbst auf)
- Z.B. pow(10,5)
  - = 10*pow(10,4)
  - = 10*10*pow(10,3)
  - = 10*10*10*pow(10,2)
  - = 10*10*10*10*pow(10,1)
  - = 10*10*10*10*10*pow(10,0)
  - = 10*10*10*10*10*1
  - = 100.000

```
float pow(float b, int n)
{
  if (n == 0) {
    return 1;
  } else {
    return b * pow(b, n-1);
  }
}

int main(void)
{
  float b, int n;
  scanf("%f,%d", &b, &n);
  printf("%f", pow(b, n));

  return 0;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel: Exponentialfunktion

- **Laufzeitanalyse:**
  - Funktion pow wird aufgerufen für n, n–1, n–2, …, 1, 0, d.h. $(n+1)$-mal und unabhängig von Eingabe b
  - Schritte innerhalb jedes pow-Aufrufs:
    - 2 Schritte (falls n = 0) oder
    - 5 Schritte
  - Gesamt:
    $5n + 2 = \mathcal{O}(n) + \mathcal{O}(1) = \mathcal{O}(n)$
- **Lineare** Laufzeit in n
- Geht es auch schneller?

```
float pow(float b, int n)
{
  if (n == 0) {
    return 1;
  } else {
    return b * pow(b, n-1);
  }
}

int main(void)
{
  float b, int n;
  scanf("%f,%d", &b, &n);
  printf("%f", pow(b, n));

  return 0;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel: Exponentialfunktion

- n **gerade**: $b^n = (b^{n/2})^2$
- n **ungerade**: $b^n = b * b^{n-1}$

fastpow(10,5)
= 10*fastpow(10,4)
= 10*fastpow(10,2)^2
= 10*(fastpow(10,1)^2)^2
= 10*((10*fastpow(10,0))^2)^2
= 10*((10*1)^2)^2
= 10*(10^2)^2
= 10*100^2
= 100.000

- Asymptotisch schneller als pow?

```
float sqr(float x)
{
  return x*x;
}
float fastpow(float b, int n)
{
  if (n == 0) {
    return 1;
  } else if (n % 2 == 0) {
    // gerade!
    return sqr(fastpow(b, n/2));
  } else { // n ungerade
    return b * fastpow(b,n-1);
  }
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel: Exponentialfunktion

- Jeder Aufruf von `fastpow` benötigt $\mathcal{O}(1)$ Schritte
  - `n = 0`: sofortiges `return`
  - n gerade: „/“ und „*“
  - n ungerade: „*“ und „-“
- Anzahl Aufrufe von fastpow:
  - **n gerade**: n wird halbiert, danach gerade oder ungerade
  - **n ungerade**: n−1 ist gerade
- **Spätestens** alle 2 Aufrufe wird n **halbiert**
- Max. $2 \cdot \mathcal{O}(\log n)$ Aufrufe
- **Gesamtlaufzeit**:
  $\mathcal{O}(1) \cdot \mathcal{O}(\log n) = \mathcal{O}(\log n)$
  (hier: $\log = \log_2$)

```
float fastpow(float b, int n)
{
  if (n == 0) {
    return 1;
  } else if (n % 2 == 0) {
    // gerade!
    return sqr(fastpow(b, n/2));
  } else { // n ungerade
    return b * fastpow(b,n-1);
  }
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

6. Analyse von Algorithmen

6.1 Laufzeitanalyse

6.2 $\mathcal{O}$-Notation für Wachstumsordnungen

6.3 Beispiel: Exponentialfunktion

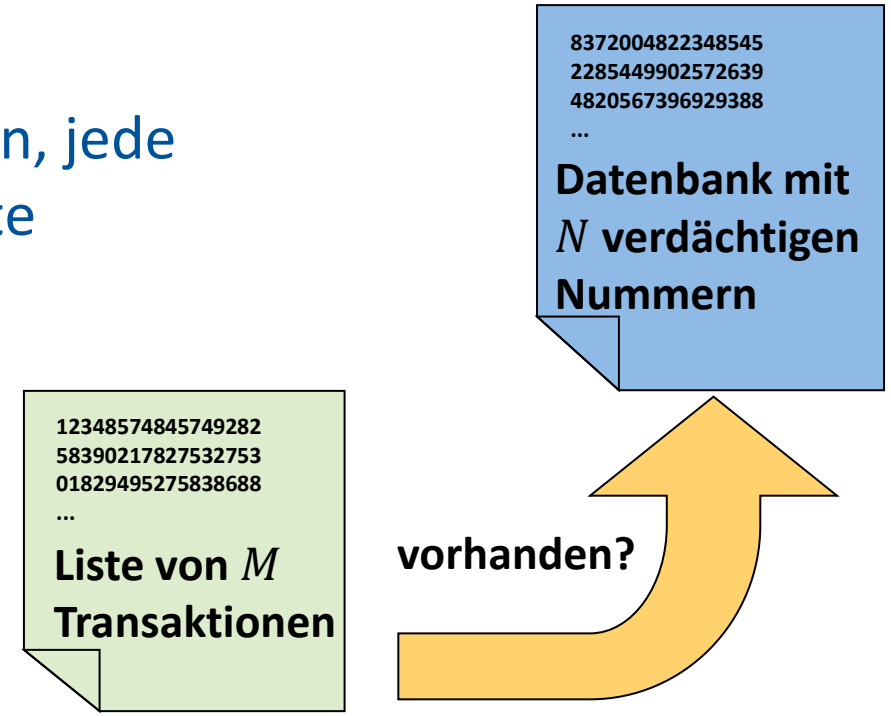
6.4 Beispiel: Suchen in Arrays

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel

- **Problemstellung**:
  Kreditkartenunternehmen möchte prüfen, ob Transaktionen mit gestohlenen Karten durchgeführt wurden
- **Modell**:
  - N „verdächtige“ Kartennummern sind gespeichert
  - M Transaktionen sind zu prüfen, jede bezieht sich auf eine bestimmte Kartennummer
  - $N \approx 10^6$
  - $M \approx 10^9$
- Anwendung ist **zeitkritisch**!





© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Lineare Suche

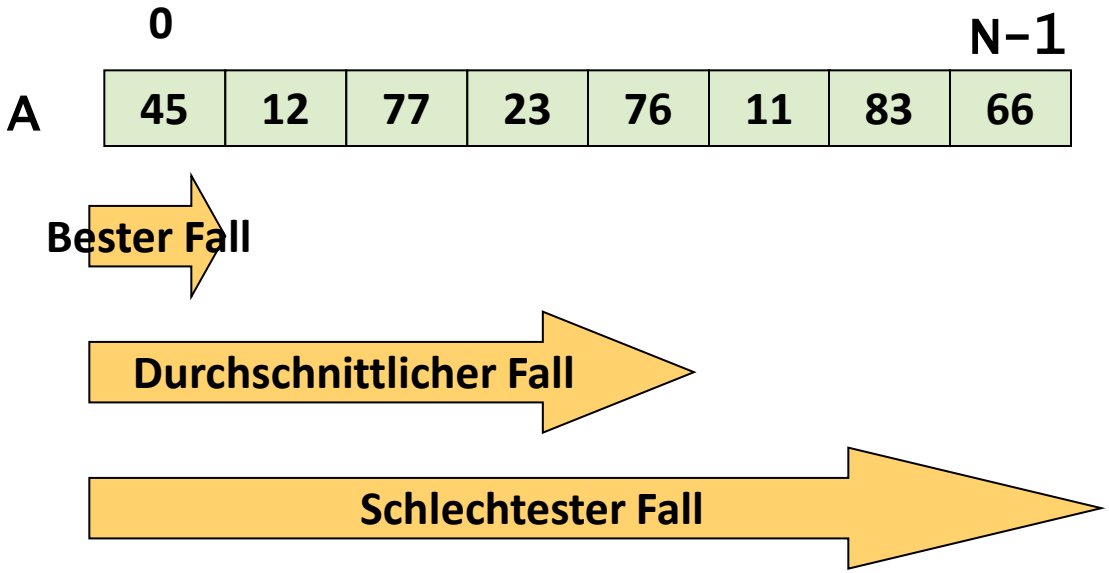
- Verdächtige Nummern in **Array** A in beliebiger Reihenfolge gespeichert
- Für jede Transaktionsnummer m: prüfe ob **Suchschlüssel** (key) m in Array A enthalten ist
  - **Ja**: Index zurückgeben
  - **Nein**: Speziellen Wert zurückgeben (NOT_FOUND)
  - Aufruf aus main: search(A,m,0,N-1);

```
#define NOT_FOUND -1
int search(int A[], int key, int l, int r)
{
  int i;
  for (i = l; i <= r; i++) {
    if (key == A[i]) return i;
  }
  return NOT_FOUND;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Laufzeitanalyse

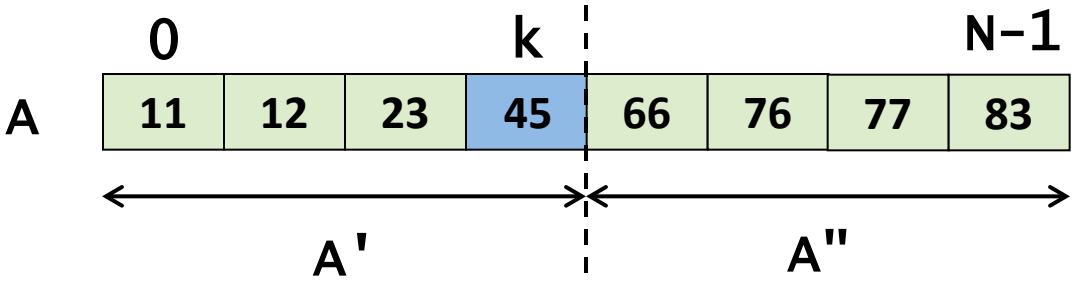
- **Bester** Fall: key = 45, sofort gefunden, $\mathcal{O}(1)$ Schritte
- **Durchschnittlicher** Fall: z.B. key = 76, $\mathcal{O}(N/2)$ Schritte
- **Schlechtester** Fall: z.B. key = 99, $\mathcal{O}(N)$ Schritte
- Wegen $\mathcal{O}(N/2) = \frac{1}{2} \cdot \mathcal{O}(N) = \mathcal{O}(N)$:
  Laufzeit schlimmstenfalls **linear**, aber auch im Durchschnitt
- **Gesamtlaufzeit** für alle Transaktionen: $\mathcal{O}(N \cdot M)$



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Binäre Suche

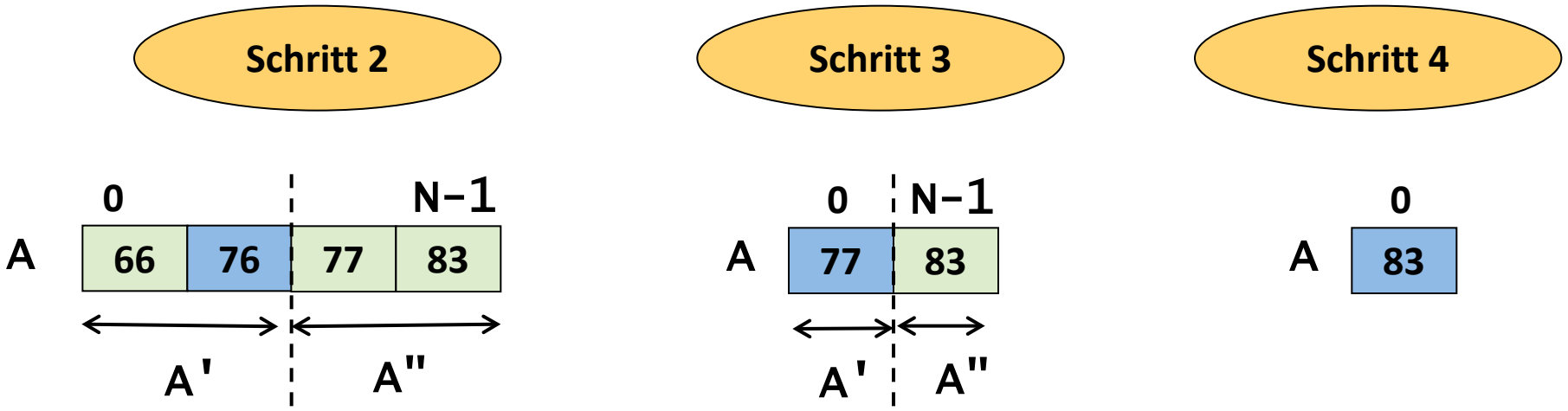
- Annahme: Array A ist **aufsteigend sortiert**
- Schnelle Prüfung, ob key (falls vorhanden) in **linker** oder **rechter** Hälfte des Arrays liegen müsste:
  - Sei $k = N/2 - 1$ falls N gerade, sonst $(N-1)/2$
  - Betrachte „**mittleres**“ Arrayelement A[k]
  - key <= A[k]: key in **linker** Hälfte → A' := A[0...k]
  - key > A[k]: key in **rechter** Hälfte → A" := A[k+1...N-1]
- Dann: suche key in Array **halber Größe** (A' oder A")
- Z.B. **key = 83**, k = 8/2-1 = 3:
  - key > A[3] = 45, also key in A" suchen



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Binäre Suche

- **Vorheriges** Teilarray `A'` nicht mehr relevant
- Ausnutzen, dass Array `A"` **ebenfalls sortiert** ist
- Setze `A = A"`, wende o.g. Verfahren **erneut** an
- `k = 4/2-1 = 1`
- `key > A[k] = 76`, daher in `A"` (neu) weitersuchen
- **Fortsetzen**, bis key gefunden, oder Array nicht weiter teilbar



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Binäre Suche

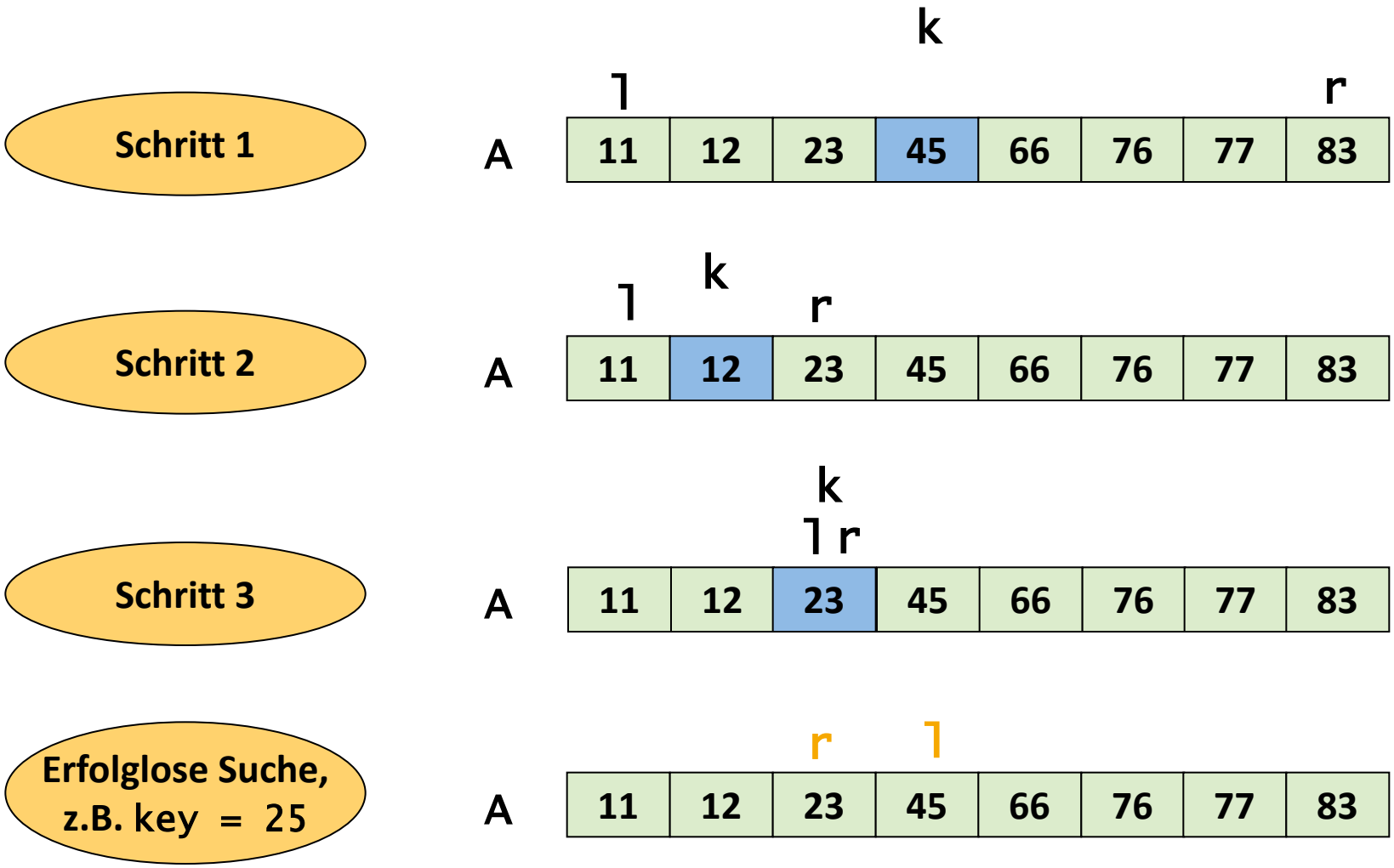
- Implementierung: Array `A` nicht tatsächlich in `A'` und `A''` teilen, sondern nur Index-Zeiger `l` und `r` umsetzen
- Aufruf: `binsearch(A,m,0,N-1);`

```
#define NOT_FOUND -1
int binsearch(int A[], int key, int l, int r)
{
  int k;
  while (r >= l) {
    k = (l+r)/2;
    if (key == A[k]) return k;
    if (key < A[k]) r = k-1;
    else l = k+1;
  }
  return NOT_FOUND;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel

- Suche nach key = 23:



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Laufzeitanalyse

- **Bester** Fall: `key == A[k]` im 1. Schritt: $\mathcal{O}(1)$
- **Schlechtester** Fall: `key` erst nach t Teilungsschritten gefunden bzw. gar nicht gefunden
  - Wie oft kann N-elementiges Array halbiert werden?
  - $\mathcal{O}(\log N)$ mal (vgl. Bsp. Exponentialfunktion)
- **Durchschnittlicher** Fall: Ebenfalls $\mathcal{O}(\log N)$
- Gesamtlaufzeit für M Transaktionen: $\mathcal{O}(M \cdot \log N)$
- Abschätzung der **absoluten** Laufzeit:
  - Annahme: Rechner mit 10^9 Operationen pro Sekunde
  - $\mathcal{O}(M \cdot N) \approx 10^{15}/10^9\,\text{sec} \approx 278\,\text{h} \approx 12\,\text{Tage}$
  - $\mathcal{O}(M \cdot \log N) \approx 10^9 \cdot \frac{20}{10^9}\,\text{sec} = 20\,\text{sec}$
  - **Zusatzaufwand**: Sortierung des Arrays `A`, jedoch hierdurch keine signifikante Zunahme der Laufzeit (< 1 sec)

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

7. Sortieralgorithmen

7.1 Einfache Verfahren (Selection/Insertion/Bubblesort)

7.2 Quicksort

7.3 Heapsort

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Warum Betrachtung von Sortieralgorithmen?

- Sortierung von Datensätzen wird als **Subroutine** in sehr vielen Anwendungen benötigt (s. Beispiel „Suchen in Arrays“)
- Einfachster Fall: Sortieren eines **Arrays von Zahlen** (z.B. `int`)
- Allg. Fall: Sortieren eines **Arrays von Datensätzen** vom Typ `struct`
- Beispiel:

```
typedef struct person {
  char name[100], vorname[100];
  struct datum { int tag, monat, jahr; } gebdatum;
} Person;
```

- Sortierung nach Namen, Vornamen oder Geburtsdatum?
- Erfordert Definition eines **Schlüssels** (key), d.h. `struct`-Komponente, gemäß der sortiert werden soll
- Unterschiedliche Vergleichsfunktionen erforderlich

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Definition des Sortierproblems

- **Gegeben**: Folge $S = (s_1, \ldots, s_N)$ von **Datensätzen** (`struct`)
- Jeder Datensatz s_i besitzt einen Eintrag key eines mittels einer Ordnungsrelation **linear geordneten Datentyps**
- Z.B. **Zahlen**: „≤“-Relation, **Buchstaben**: alphabetische Reihenfolge, **Geburtsdatum**: 3-stufiger Vergleich Jahr, Monat, Tag
- Gesucht: **Permutation** (Anordnung) $S' = (s'_1, \ldots, s'_N)$ der Elemente von S, so dass

$$s'_1.key \leq s'_2.key \leq \cdots \leq s'_N.key$$

- Gleiche Schlüssel dürfen auch **mehrfach** vorkommen
- Ein Sortieralgorithmus, der die ursprüngliche Reihenfolge bei gleichem Schlüssel beibehält, heißt **stabil**
- Sprachelemente in c:

  Definition der Folge als Array: `Person s[N];`

  Auslesen des Schlüssels von Datensatz i: `s[i].key`

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

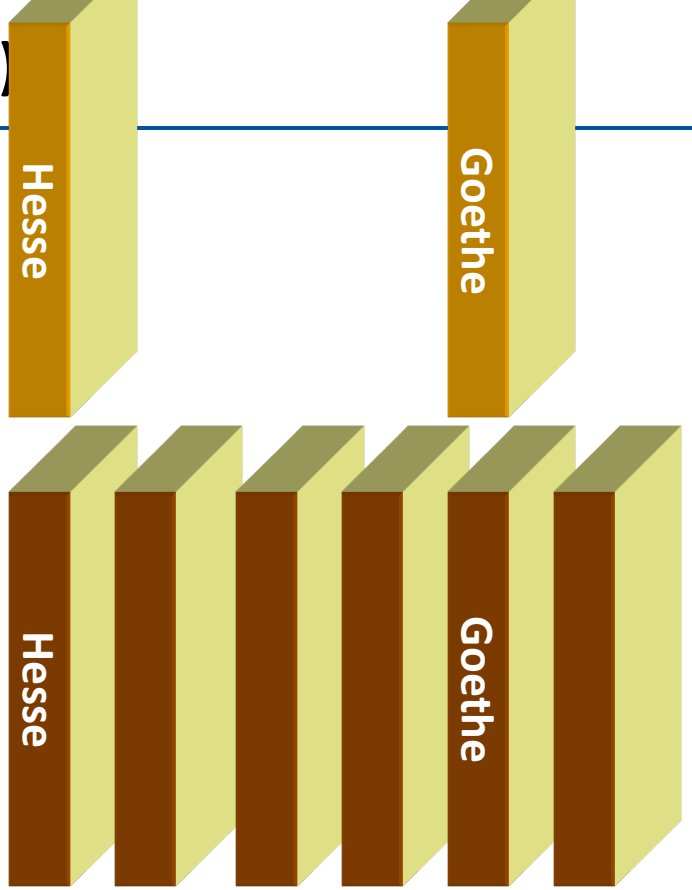
Analyse von Sortieralgorithmen

- Wie üblich: Verwendung der $\mathcal{O}$-**Notation** zur Abstraktion von konstanten Faktoren und Rechnerdetails
- Elementare Operationen von Sortieralgorithmen:
  - **Vergleich** zweier Schlüssel
  - **Vertauschen** von Daten in Folge S (Array oder File)
- Daher Abschätzung der **Laufzeit** in diesen Größen
- **Reale Laufzeit**: abhängig von der Komplexität dieser Operationen
- Bsp.: Sortieren eines **Bücherregals**, Vergleich der Autorennamen aufwendiger als Vertauschen
- Bsp.: Sortieren einer **Datenbank** mit großen Datensätzen in einem File, Umkopieren auf Festplatte oder Magnetband aufwendiger als Key-Vergleich
- Hier: nur Betrachtung der **Vergleichsoperationen**

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Sortieren durch Auswählen (Selection Sort)

- Manuelles **alphabetisches Sortieren** eines Bücherregals mit N Büchern:
  - **Schlüssel**: Autorenname
  - Durchlaufen der Bücher von links nach rechts
  - **Merken** des jeweils „kleinsten“ bisher gefundenen Buches und dessen **Position** p im Regal
  - Zum Schluss: „Kleinstes“ Buch **ganz links** hinstellen, dort bisher stehendes Buch (vorläufig) an Position p stellen
  - Verfahren **fortsetzen** mit $p = 2$



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

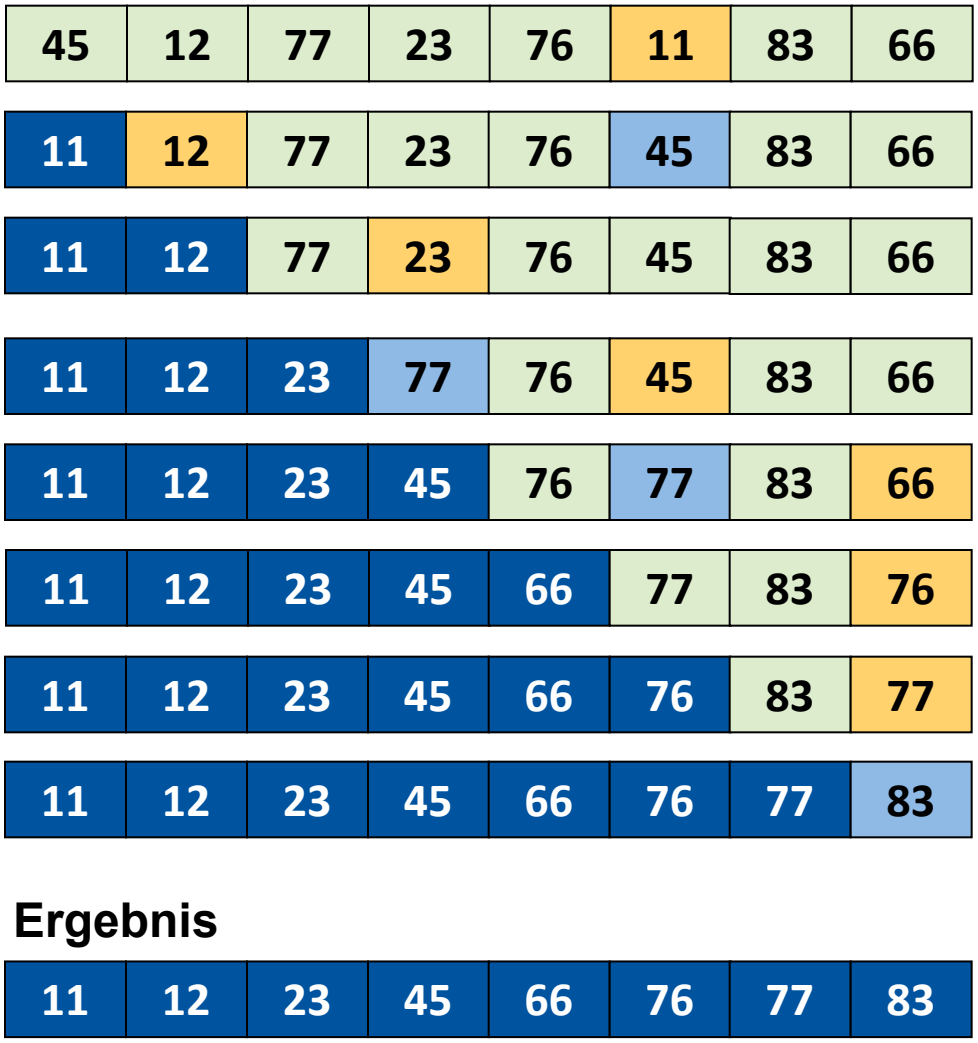
Algorithmus im Detail

- **Vereinfachung**: Sortieren von Arrays vom Typ `int`, d.h. Vergleichsoperation auf Zahlen, z.B. `A[i] < A[j]`
- **Real**: Hilfsfunktion „less“ auf Sortierschlüsseln, jeweils Vergleich `less(A[i].key, A[j].key)`

```
void selection_sort(int A[], int l, int r)
{
  int i, j, min;
  for (i = l; i < r; i++) {
    min = i;
    for (j = i+1; j <= r; j++) {
      if (A[j] < A[min]) min = j;
    }
    exchange(A[i], A[min]); // vertauschen
  }
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel



noch unsortiert

aktuell min. Schlüssel

vertauscht

bereits sortiert

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Laufzeitanalyse

```
void selection_sort(int A[], int l, int r)
{
  int i, j, min;
  for (i = l; i < r; i++) {
    min = i;
    for (j = i+1; j <= r; j++) {
      if (A[j] < A[min]) min = j;
    }
    exchange(A[i], A[min]);
  }
}
```

N-1 Iterationen

N-1, N-2, …, 1 Iterationen

$\mathcal{O}(1)$ Operationen

$\mathcal{O}(1)$ Operationen

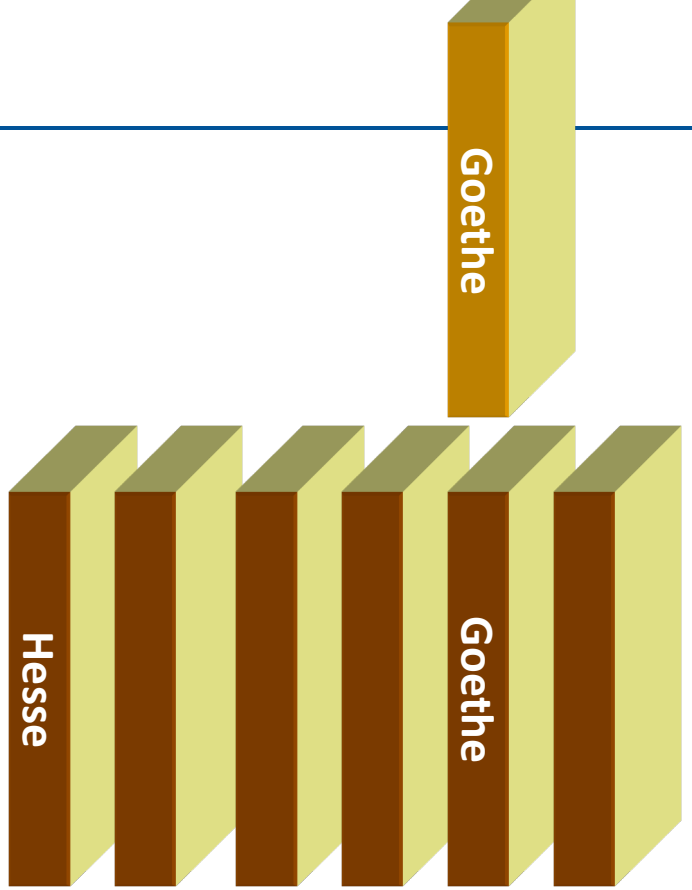
- Aufruf: `selection_sort(A,0,N-1);`

$$(\mathbf{1} + \mathbf{2} + \ \ldots + \boldsymbol{N} - \mathbf{2} + \ \boldsymbol{N} - \mathbf{1}) \cdot \mathcal{O}(1) + (N-1) \cdot \mathcal{O}(1)$$
$$= \ \boldsymbol{N} \cdot (\boldsymbol{N} - \mathbf{1})/\mathbf{2} \cdot \mathcal{O}(1) + \mathcal{O}(N) = \ \mathcal{O}(N^2) + \mathcal{O}(N) = \mathcal{O}(N^2)$$

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Sortieren durch Einfügen (Insertion Sort)

- Manuelles **alphabetisches Sortieren** eines Bücherregals mit N Büchern:
  - **Schlüssel**: Autorenname
  - Durchlaufen der Bücher von links nach rechts ($i = 1 \ldots N$)
  - Betrachtung des Buchs an Position $\boldsymbol{p = i}$
  - Buch Nr. i **entnehmen**
  - Bücher **links** von $p = i$ (d.h. $p < i$) nach rechts rücken, bis korrekte **Einfügeposition** für Buch i gefunden
  - Verfahren **fortsetzen** mit $i = i + 1$



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

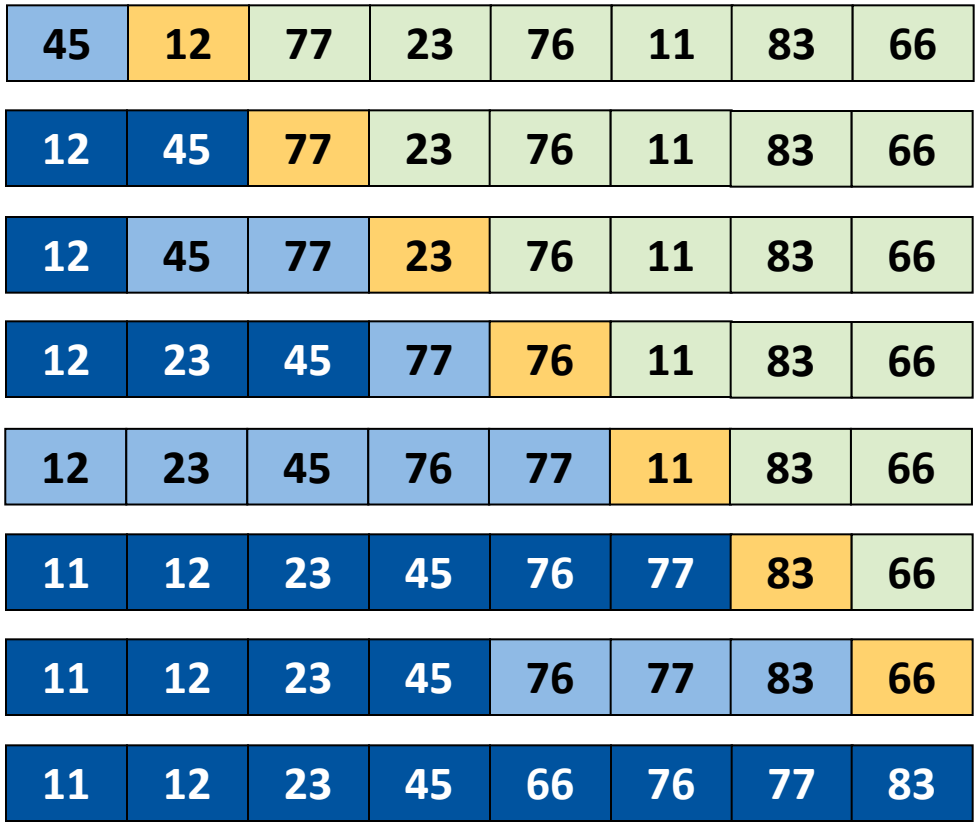
Algorithmus im Detail

- Betrachtung aller Bücher von links nach rechts
- **Erstes Buch** braucht nicht betrachtet zu werden, da kein linker Nachbar (d.h. zu Beginn `i = l+1`)

```
void insertion_sort(int A[], int l, int r)
{
  int i, j, v;
  for (i = l+1; i <= r; i++) {
    v = A[i];
    for (j = i-1; j >= l; j--) {
      if (v < A[j]) A[j+1] = A[j];
      else break;
    }
    A[j+1] = v;
  }
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel



noch unsortiert

aktuelles Element

bereits sortiert,
aber zu verschieben

bereits sortiert

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Laufzeitanalyse

```
void insertion_sort(int A[], int l, int r)
{
  int i, j, v;
  for (i = l+1; i <= r; i++) {
    v = A[i];
    for (j = i-1; j >= l; j--) {
      if (v < A[j]) A[j+1] = A[j];
      else break;
    }
    A[j+1] = v;
  }
}
```

N-1 Iterationen

Max. 1,2,…,N-2,N-1 Iterationen

O(1) Operationen

O(1) Operationen

- Aufruf: insertion_sort(A,0,N-1);

$$(\mathbf{1} + \mathbf{2} + \ \ldots + \boldsymbol{N} - \mathbf{2} + \boldsymbol{N} - \mathbf{1}) \cdot \mathcal{O}(1) + \ (N-1) \cdot \mathcal{O}(1)$$

$$= \ \boldsymbol{N} \cdot (\boldsymbol{N} - \mathbf{1})/\mathbf{2} \cdot \mathcal{O}(1) \ + \mathcal{O}(N) \ = \mathcal{O}(N^2) \ + \mathcal{O}(N) \ = \mathcal{O}(N^2)$$

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Bubblesort

- Idee: Elemente mit kleinem Schlüssel sind „**leicht**“ und steigen wie Blasen im Array auf (d.h. wandern nach links)
- Aufsteigen durch fortgesetztes **paarweises Vertauschen** der Elemente von rechts nach links implementiert



```
void bubble_sort(int A[], int l, int r)
{
  int i, j;
  for (i = l; i < r; i++) {
    for (j = r; j > i; j--) {
      if (A[j-1] > A[j])
        exchange(A[j-1],A[j]);
    }
  }
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel (1. Durchlauf der äußeren Schleife)

45	12	77	23	76	11	83	66
45	12	77	23	76	11	66	83
45	12	77	23	11	76	66	83
45	12	77	11	23	76	66	83
45	12	11	77	23	76	66	83
45	11	12	77	23	76	66	83
11	45	12	77	23	76	66	83

noch unsortiert

aktuelle linke Grenze

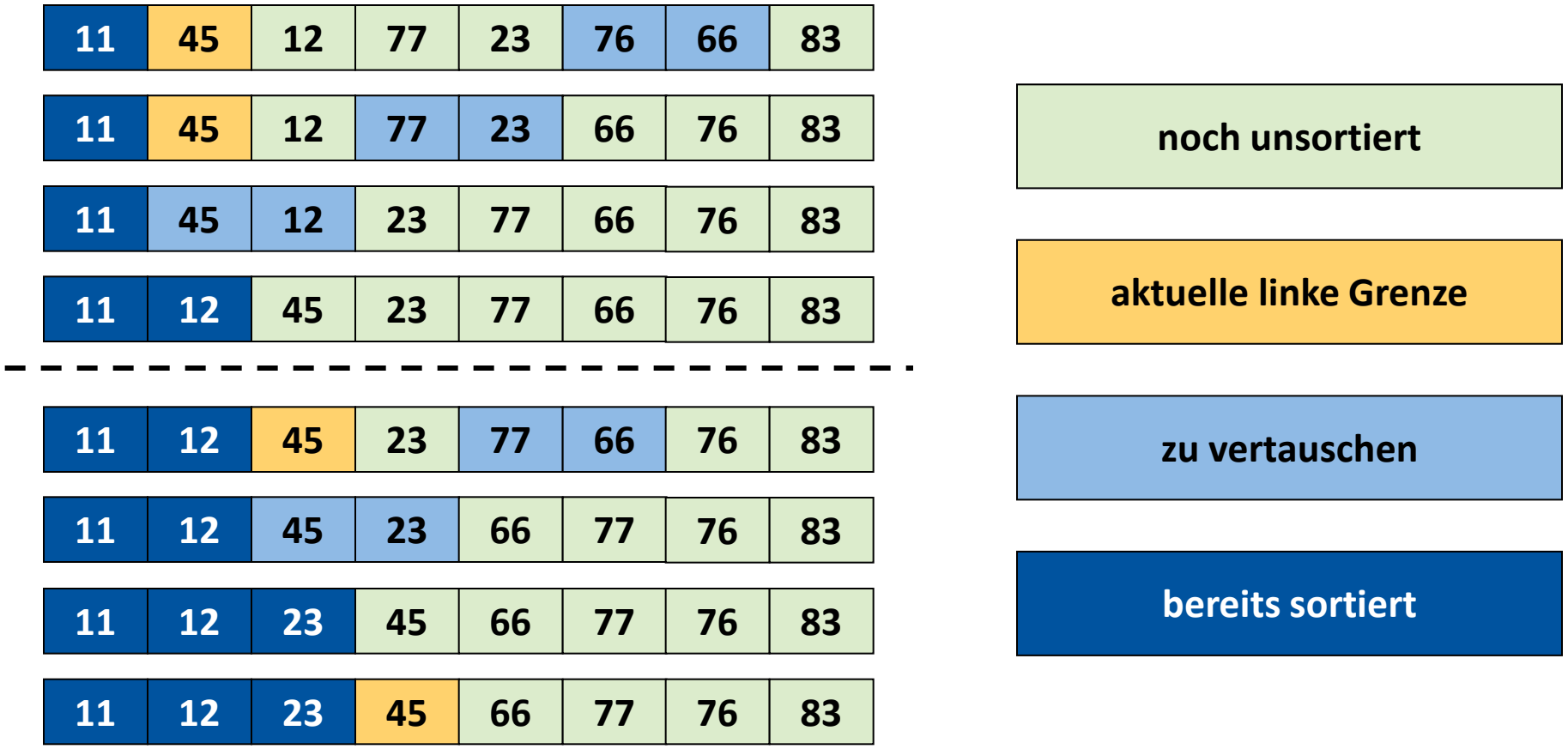
zu vertauschen

bereits sortiert

- Kleinstes Element (11) am Ende des 1. Durchlaufs an der linken Position
- Andere relativ „leichte“ Elemente (z.B. 66) sind bereits teilweise nach links gerückt, schwere nach rechts (45, 77)

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel (2. und 3. Durchlauf der äußeren Schleife)



- **Eigenschaft**: nach i-tem Durchlauf ist i-tes kleinstes Element an seiner endgültigen Position
- Andere Elemente wurden weiter **vorsortiert**

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Laufzeitanalyse

```
void bubble_sort(int A[], int l, int r)
{
  int i, j;
  for (i = l; i < r; i++) {
    for (j = r; j > i; j--) {
      if (A[j-1] > A[j])
        exchange(A[j-1],A[j]);
    }
  }
}
```

$N-1$ Iterationen

$N-1, N-2, \ldots, 1$ Iterationen

$\mathcal{O}(1)$ Operationen

$$(1 + 2 + \ldots + N - 2 + N - 1) \cdot \mathcal{O}(1)$$
$$= N \cdot (N-1)/2 \cdot \mathcal{O}(1) = \mathcal{O}(N^2)$$

- Selection Sort, Insertion Sort und Bubblesort haben asymptotisch alle **quadratische Laufzeit** in der **Eingabegröße** N und sind (bis auf konstante Faktoren) gleichwertig

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

7. Sortieralgorithmen

7.1 Einfache Verfahren (Selection/Insertion/Bubblesort)

7.2 Quicksort

7.3 Heapsort

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Divide-and-conquer

- „Divide et impera“, „Teile und herrsche“
- Verbreitetes Prinzip beim **Algorithmenentwurf**
- D.h. selbst kein Algorithmus, nur ein „**Kochrezept**“ bei der Entwicklung
- Divide-and-conquer-Idee:
  - Mehrere kleine Probleme sind insgesamt **leichter zu lösen** als ein großes
  - **Teile** großes Problem in mehrere kleine, löse diese, und berechne daraus die **Gesamtlösung**
- Hier:
  - Zerlege zu sortierendes Array A in zwei Teile A' und A'', die **unabhängig voneinander** sortiert werden können
  - Sortiere A' und A'' getrennt
  - Zusammenfügen (**Konkatenation**) der sortierten Arrays A' und A'' ergibt sortiertes Gesamtarray A

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

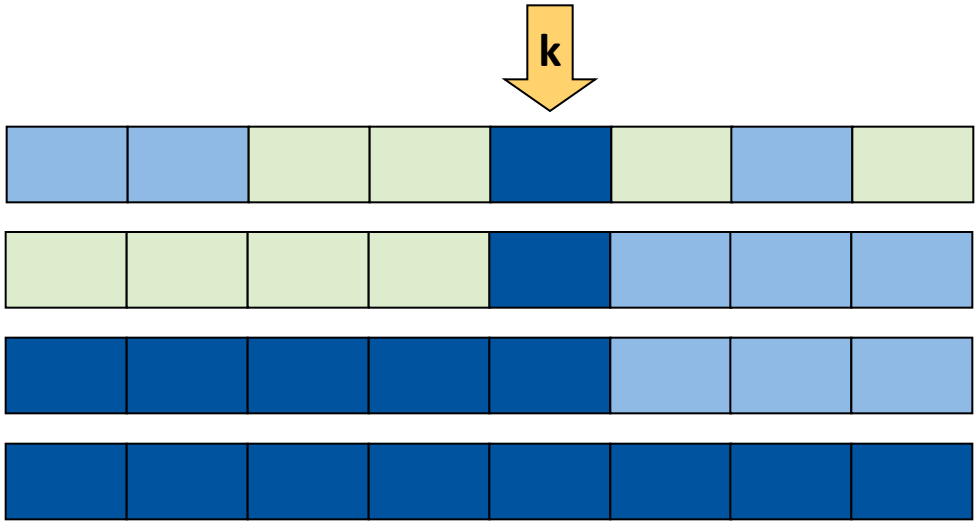
Quicksort-Algorithmus

- Für Array `A[l...r]` wähle ein **bestimmtes `A[k]`** aus
- A[k] ist das **Trennelement**
- Ordne Array so um, dass:
  - `A' = A[l...k-1]` hat **nur Elemente ≤ `A[k]`**
  - `A'' = A[k+1...r]` hat nur **Elemente ≥ `A[k]`**
- Dies wird realisiert, indem bzgl. des Trennelementes „falsche“ Elemente in `A'` und `A''` **paarweise vertauscht** werden
- Achtung: Arrays `A'`, `A''` sind danach i.a. **immer noch unsortiert**
- Sortiere die Teilarrays `A'` und `A''` mittels erneutem Aufruf des Quicksort-Algorithmus (**Rekursion**)
- Anschließend ist **`A = A' A[k] A''`** ein sortiertes Gesamtarray

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Algorithmus im Detail

```
void quick_sort(int A[], int l, int r)
{
  int k;
  if (r <= l) return;
  k = partition(A,l,r);
  quick_sort(A,l,k-1);
  quick_sort(A,k+1,r);
}
```



>= A[k]

<= A[k]

fertig sortiert

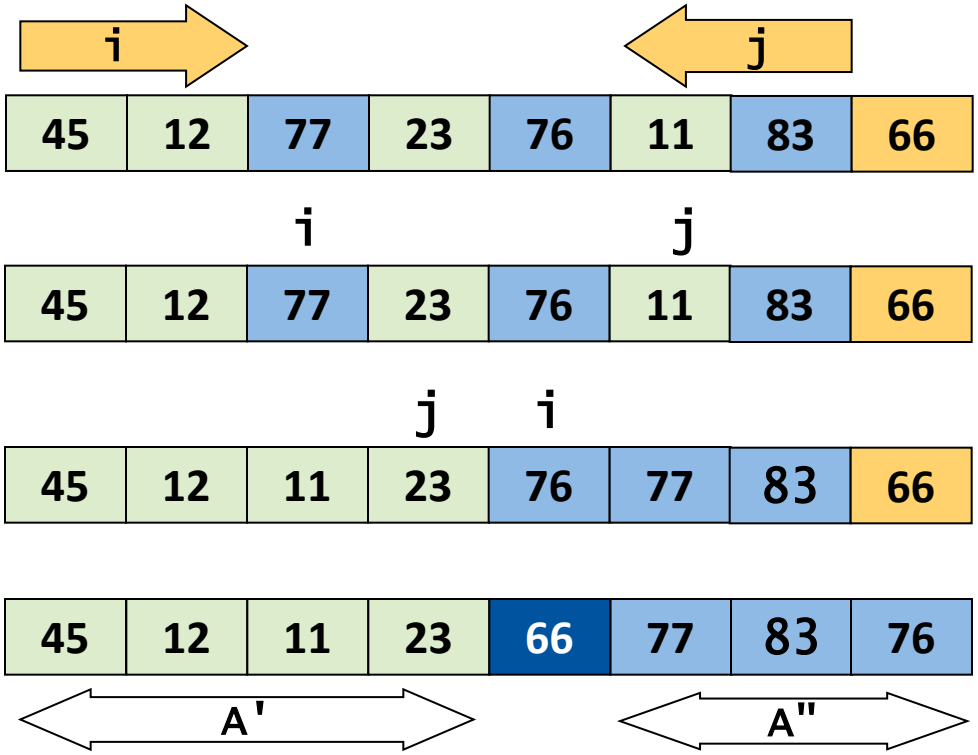
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Algorithmus im Detail

```
int partition(int A[], int l, int r)
{
  int i, j, k, v;
  k = r; v = A[k]; // willkürliches Trennelement
  i = l; // starte am linken Rand
  j = r-1; // starte am rechten Rand – 1
  while (1) { // durchlaufen bis Abbruch
    while (i < r  && A[i] <= v) i++;
    while (j >= l && A[j] >= v) j--;
    if (i >= j) // aneinander vorbeigelaufen ?
      break; // Abbruch der while-Schleife
    else
      exchange(A[i],A[j]);
  }
  exchange(A[i],A[k]);
  return i;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel (1. Durchlauf von partition)



Trennelement

>= A[k]

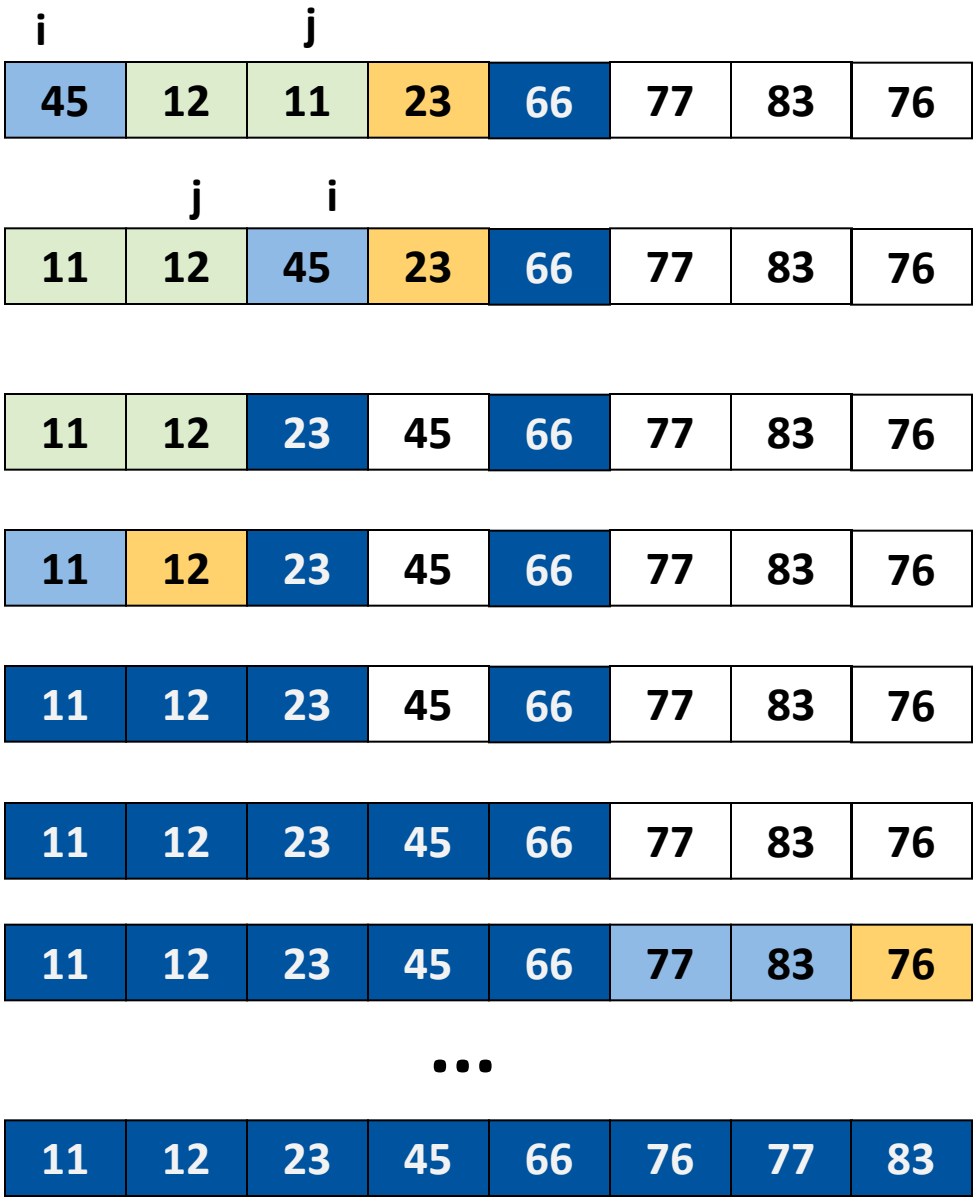
<= A[k]

fertig sortiert

- Trennelement ist 66
- 11 und 77 werden vertauscht
- `i` und `j` laufen dann aneinander vorbei
- 66 und 76 werden vertauscht
- Anschließend Sortieren von `A'` und `A"`

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel (Forts.)



Trennelement

>= A[k]

<= A[k]

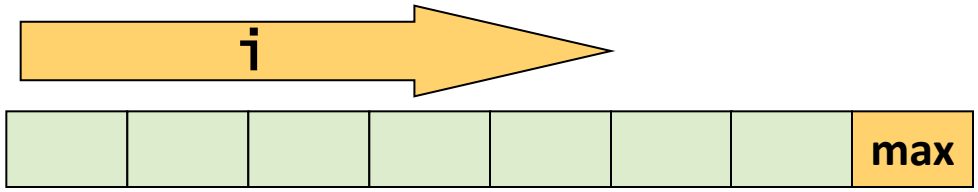
fertig sortiert

noch nicht betrachtet

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Laufzeitanalyse

- **Schlechtester Fall**: zufällig gewähltes Trennelement ist das **Maximum** des Arrays
- Z.B. bei **vorsortiertem** Array
- `i` durchläuft `N-1` Elemente, d.h. $\mathcal{O}(N-1)$ **Vergleiche**
- Jeder Teilungsschritt **spaltet genau ein Element** ab und hinterlässt ein um ein Element kleineres Teilarray `A'`
- Ist `A'` ebenfalls schon sortiert, so **wiederholt** sich dieser Effekt
  - Trennelement ist Maximum von `A'`
  - `i` durchläuft `N-2` Elemente
  - usw.
- **Analoger Effekt** bei Trennelement ganz links und Array-Min.



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Laufzeitanalyse

- Anzahl Vergleiche im **worst case**:

  $\mathcal{O}(N-1) + \mathcal{O}(N-2) + \ \ldots\ + 1 = \mathcal{O}(N^2)$

- Jedoch wesentlich besser im **best case**:
  - T_N := Zeit, um Array der Länge N zu sortieren
  - Pro Teilungsschritt durchlaufen i und j nicht mehr als N Elemente, d.h. es gibt $\mathcal{O}(N)$ Vergleiche
  - Im besten Fall entstehen **jeweils 2 Teilarrays gleicher Größe**

$$
\begin{aligned}
T_N &= 2 \cdot T_{N/2} + \mathcal{O}(N) \\
&= 2 \cdot \left(2 \cdot T_{N/4} + \mathcal{O}(N/2)\right) + \mathcal{O}(N) \\
&= 4 \cdot T_{N/4} + 2 \cdot \mathcal{O}(N/2) + \mathcal{O}(N) \\
&= 4 \cdot T_{N/4} + \mathcal{O}(N) + \mathcal{O}(N) && \log N \text{ Teilungsschritte, } T_1 = \mathcal{O}(1) \\
&= \cdots = \mathcal{O}(N) + \cdots + \mathcal{O}(N) && \log N \text{ mal} \\
&= \log N \cdot \mathcal{O}(N) \\
&= \mathcal{O}(N \log N)
\end{aligned}
$$

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Laufzeitanalyse

- Durch **einfache Techniken** kann man den *worst case* meist ausschließen
- Z.B. **Median von 3 Elementen** als Trennelement verwenden
- Man kann zeigen: Quicksort hat auch im **durchschnittlichen Fall** (*average case*) eine Laufzeit von $\mathcal{O}(N \log N)$
- D.h. Quicksort ist fast immer **effizient** und darüber hinaus recht **einfach** zu implementieren
- Z.B. N = 1.000.000, 10^9 Operationen pro sec
  - $\mathcal{O}(N \log N)$: Laufzeit < 1 sec
  - $\mathcal{O}(N^2)$: Laufzeit ≈ 17 min
- Daher: Quicksort ist das Sortierverfahren mit der **größten Verbreitung**
- Z.B. Bestandteil jeder C-Standard-Library (**qsort-Funktion**)

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

7. Sortieralgorithmen

7.1 Einfache Verfahren (Selection/Insertion/Bubblesort)

7.2 Quicksort

7.3 Heapsort

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Heapsort

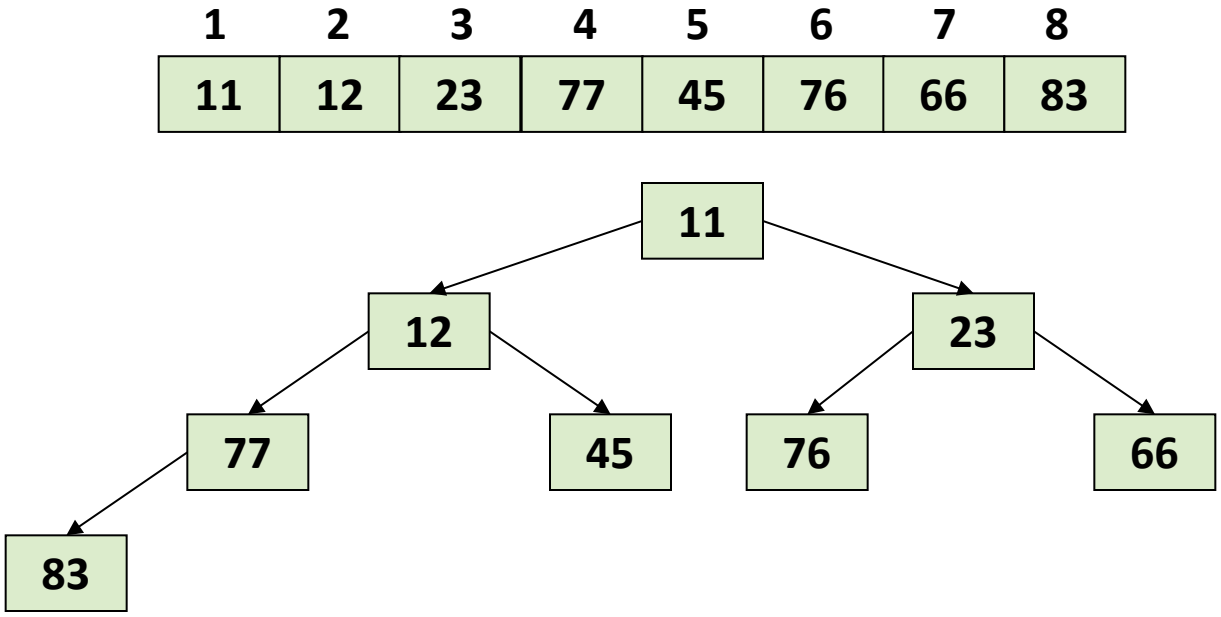
- **Selection Sort**: Sortierung durch fortgesetzte Minimumsauswahl (s.o.)
- N Minima sind auszuwählen, jede Auswahl kostet im Mittel $\mathcal{O}(N/2) = \mathcal{O}(N)$ Schritte, daher **Gesamtlaufzeit $\mathcal{O}(N^2)$**
- Idee bei Heapsort: **Beschleunigung** der Minimumsauswahl
- Betrachte folgende Anordnung der Array-Elemente (Array-Indizes **laufen hier von 1 … N** statt $0 \ldots N-1$):

1	2	3	4	5	6	7	8
11	12	23	77	45	76	66	83

- „Unsortiert“, aber folgende **Eigenschaft** für jedes `A[i]`:
  - `A[i]` ≤ `A[2i]`, falls `2i` ≤ `N`
  - `A[i]` ≤ `A[2i+1]`, falls `2i+1` ≤ `N`

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Veranschaulichung als Baumstruktur



- Jedes Baumelement `A[i]` besitzt 0, 1, oder 2 „**Kinder**“
- Kinder besitzen Arrayindex **`2i` bzw. `2i+1`**
- 12, 23 sind Kinder von 11; 77, 45 sind Kinder von 12 usw.

Def.: Gelten im Array `A[1...N]` für jedes `A[i]` die Eigenschaften `A[i]` ≤ `A[2i]` (falls `2i` ≤ `N`) und `A[i]` ≤ `A[2i+1]` (falls `2i+1` ≤ `N`), so ist `A` ein **(Minimums-)Heap**.

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

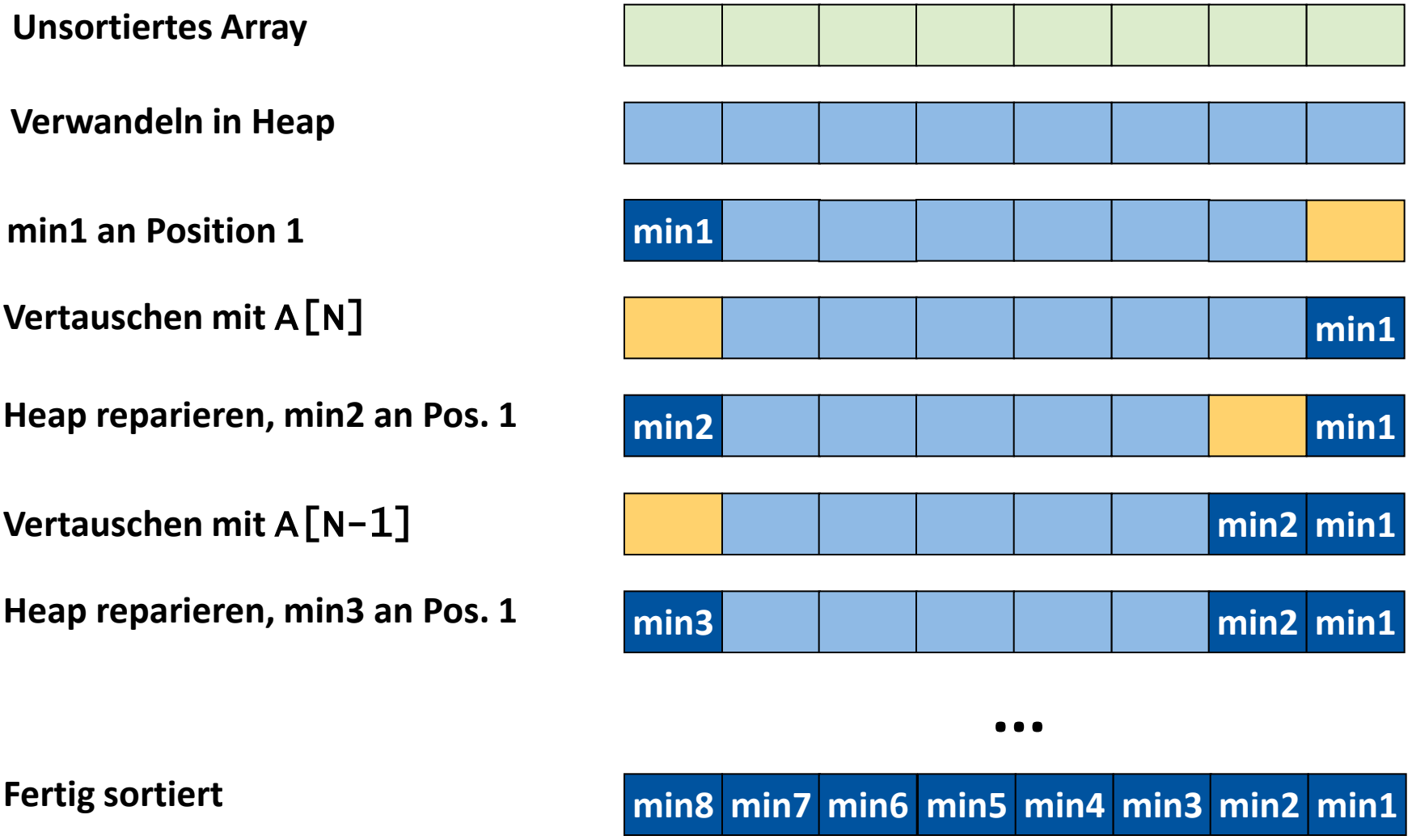
Heapsort-Prinzip

- **Heap-Eigenschaft**: Das (globale) **Array-Minimum** befindet sich stets ganz oben im Heap (der sog. **Wurzel**)
- **Folgerung**: Entnahme des Minimums aus einem Heap ist trivial (immer in `A[1]`, d.h. Laufzeit $\mathcal{O}(1)$)
- **Ablauf von Heapsort**:
  1. Verwandeln des unsortierten Arrays `A[1...N]` in einen Heap, danach ist Wurzel `A[1]` das Minimum
  2. Vertausche Wurzel und `A[N]`
  3. Min. befindet sich danach an der richtigen Stelle `N` (hier: **absteigende** Sortierreihenfolge!)
  4. `A[2...N-1]` ist noch korrekter Heap, jedoch neues Element `A[1]` evtl. falsch
  5. Wiederherstellen der Heap-Eigenschaft von `A[1...N-1]`
  6. Fortsetzen ab Schritt 2 mit nächstem Element `N-1` und neuer Wurzel

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Heapsort-Prinzip

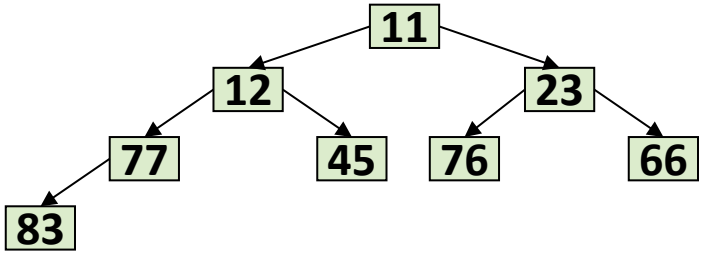
min i := i-tes kleinstes Element im Array

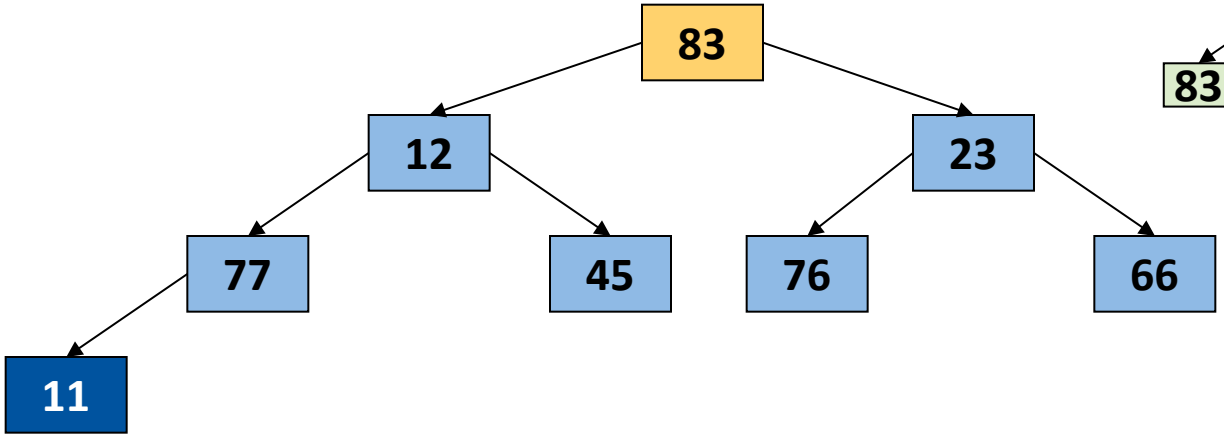


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

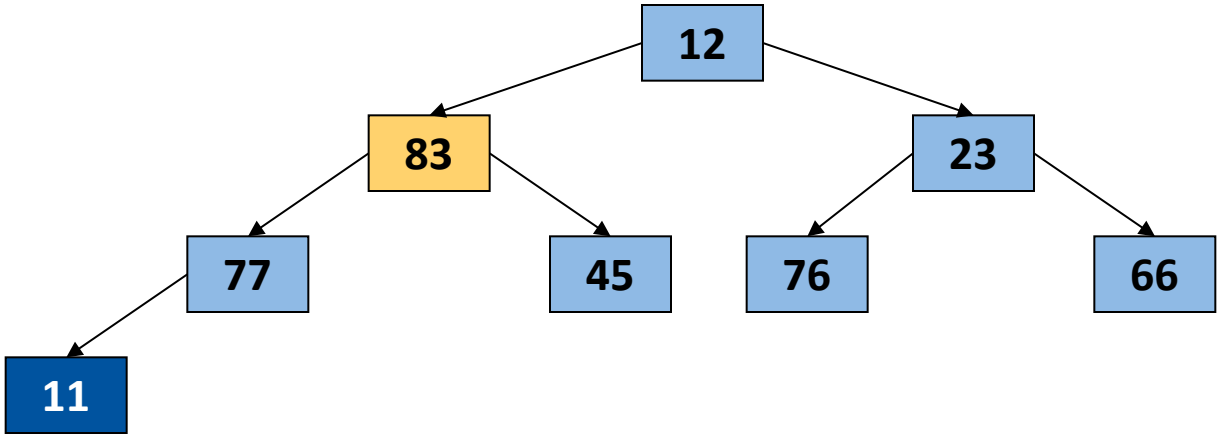
Reparatur des Heaps

- Situation nach erster Vertauschung





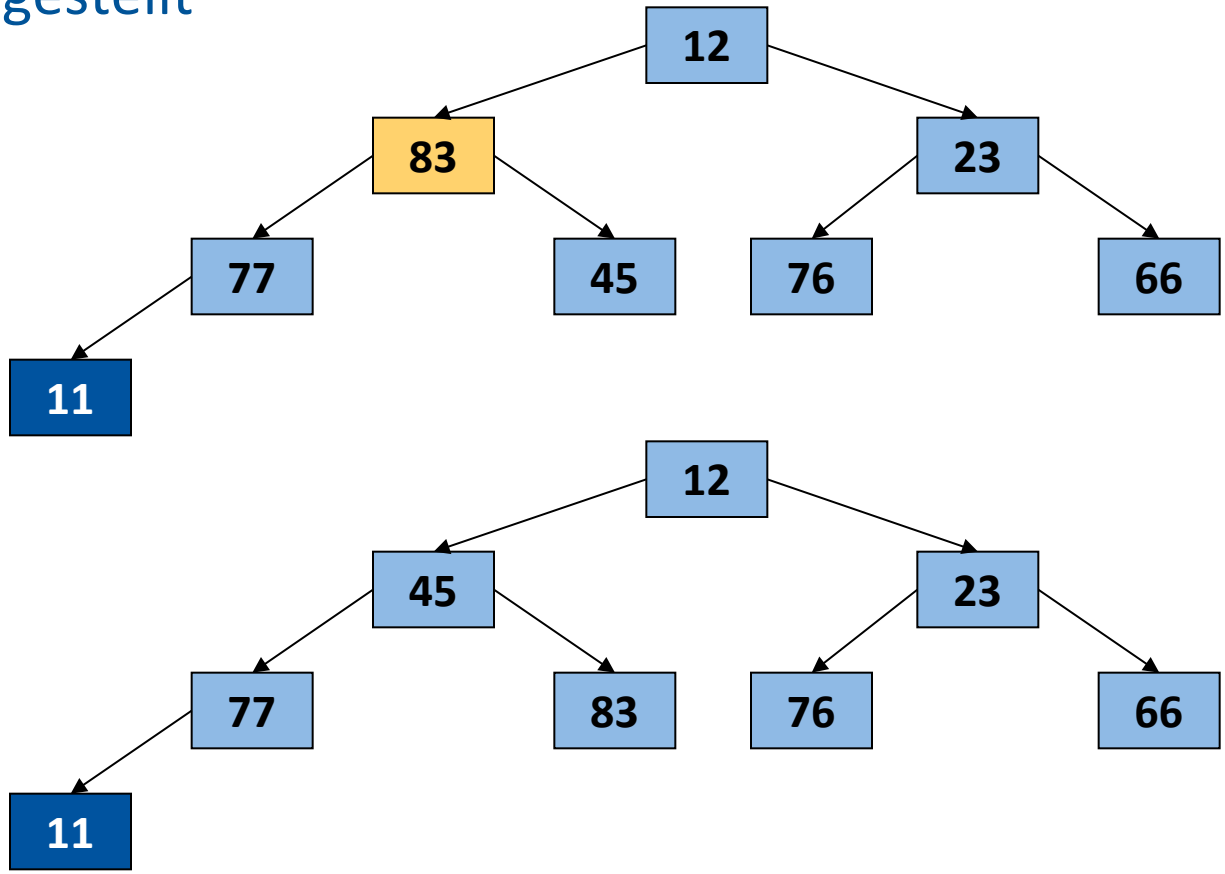
- „**Einsinken lassen**“ der Wurzel: Vertausche Wurzel mit kleinerem der beiden Kinder



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Reparatur des Heaps

- **Fortsetzen abwärts im Baum**, bis keine kleineren Kinder mehr vorhanden
- Vernachlässige **bereits sortierten Bereich** am Ende von A
- **Heap-Eigenschaft** des unsortierten Bereichs von A wieder hergestellt



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Heap-Reparatur (Einsinken lassen der Wurzel)

```
void sink(int A[], int k, int N) // A[0] nicht benutzt
{
  int child; // speichert kleinstes Kind falls vorhanden
  while (1) { // durchlaufen bis Abbruch
    if (2*k > N) break;  // Knoten k hat kein Kind
    if (2*k+1 <= N) {    // Knoten k hat 2 Kinder
      if (A[2*k] < A[2*k+1]) child = 2*k;
      else child = 2*k+1;
    } else child = 2*k;    // 2k <= N, Knoten k hat 1 Kind
    if (A[k] > A[child]) { // Vertauschen notwendig?
      exchange(A[k], A[child]);
      k = child;    // Knoten k evtl. weiter sinken lassen
    } else break; // sonst: richtige Pos. für k gefunden
  }
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Heapsort-Algorithmus

- Verwendung der Funktion **sink** auch zum einmaligen Heap-Aufbau zu Beginn
- Elemente mit Index > N/2 haben **keine Kinder**
- Dann **Vertauschen** und **Einsinken lassen** wie oben gezeigt

```
void heap_sort(int A[], int N) // A[0] nicht benutzt
{
  int i;
  for (i = N/2; i > 0; i--) { // Heap aufbauen
    sink(A,i,N);
  }
  for (i = N; i > 1; i--) {
    exchange(A[1], A[i]); // Min. ans akt. Heap-Ende setzen
    sink(A,1,i-1);        // Heap reparieren
  }
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Laufzeitanalyse

```
void heap_sort(int A[], int N)
{
  int i;
  for (i = N/2; i > 0; i--) {
    sink(A,i,N); // Heap aufbauen
  }
  for (i = N; i > 1; i--) {
    exchange(A[1],A[i]);
    sink(A,1,i-1);
  }
}
```

$\mathcal{O}(N)$ Iterationen

$\mathcal{O}(N)$ Iterationen

$\mathcal{O}(1)$

- Betrachtung der **worst case** Laufzeit
- 1. Schleife: $\mathcal{O}(N)$ Aufrufe von `sink`
- 2. Schleife: $\mathcal{O}(N)$ Aufrufe von `sink` + $\mathcal{O}(N)$
- Gesamt: $\mathcal{O}(N) \cdot T_{\text{sink}} + \mathcal{O}(N)$

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Laufzeitanalyse für T_{sink}

```
void sink(int A[], int k, int N)
{
  int child;
  while (1) {
    if (2*k > N) break;
    if (2*k+1 <= N) {
      if (A[2*k] < A[2*k+1])
        child = 2*k;
      else child = 2*k+1;
    } else child = 2*k;
    if (A[k] > A[child]) {
      exchange(A[k],A[child]);
      k = child;
    } else break;
  }
}
```

$\mathcal{O}(???)$ **Iterationen**

Alle Operationen jeweils $\mathcal{O}(1)$, also auch insgesamt $\mathcal{O}(1)$

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Laufzeitanalyse

- In jedem Durchlauf der `while`-Schleife wird `k` mindestens **verdoppelt** (`child = 2k bzw. 2k+1; k = child;`)
- Z.B. `k = 1, 2, 4, 8, 16, 32, ...`
- 32 nach 5 (= log 32) Schritten erreicht
- **Allgemein**: k kann höchstens **$(\log N)$-mal verdoppelt** werden, bis Arraygrenze `N` erreicht wurde und `sink` terminiert
- **Gesamtlaufzeit**:

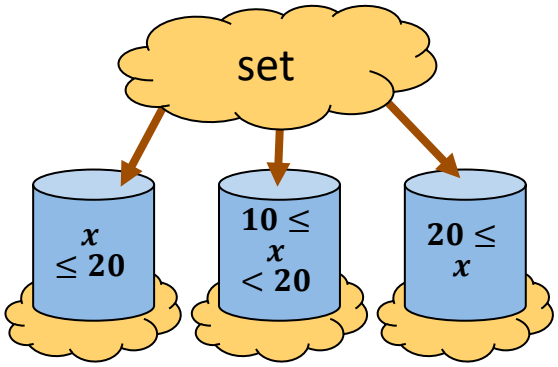
$$
\begin{aligned}
& \mathcal{O}(N) \cdot T_{\text{sink}} + \mathcal{O}(N) \\
= \; & \mathcal{O}(N) \cdot \big(\log N \cdot \mathcal{O}(1)\big) + \mathcal{O}(N) \\
= \; & \mathcal{O}(N) \cdot \mathcal{O}(\log N) + \mathcal{O}(N) \\
= \; & \mathcal{O}(N \log N) + \mathcal{O}(N) \\
= \; & \mathcal{O}(N \log N)
\end{aligned}
$$

- Heapsort braucht auch im **worst case** nur $\mathcal{O}(N \log N)$ Schritte

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Fazit zu Sortierverfahren

- **Man kann zeigen**: Jeder allgemeine Sortieralgorithmus benötigt im worst case **mindestens Laufzeit $\mathcal{O}(N \log N)$**
- D.h. Heapsort ist der „**bestmögliche**“ Sortieralgorithmus bis auf nichtsignifikante Terme + Konstanten
- In der **Praxis** können jedoch andere Verfahren (z.B. Quicksort und sogar Bubblesort für kleine N) durchaus **schneller** sein
- In **Spezialfällen** kann man noch schneller sortieren
- Sortierverfahren mit Verwendung zusätzlicher Zwischenspeicher
  - Bucket Sort, Radix Sort, Counting Sort (nächste Folie)



m-Buckets → $\mathcal{O}(m)$

Stabile Sortierung je Radix (Buchstabe, Ziffer, …)
Key → [K][e][y]

$m \times$ Sortieren von Radix r → $\mathcal{O}(m) \cdot \mathcal{O}(f(r))$

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Schlüsselindiziertes Zählen

- Lineare Komplexität in der Laufzeit
  - Idee: „Schlüsselindiziertes Zählen“
  - $m \coloneqq$ Anzahl möglicher Schlüssel
  - Beispiele
    - PLZ: $m_{\text{PLZ}} < 10^5$
    - Name (20 Buchstaben): $m_{\text{name}} = 26^{20} < 2^{5^{20}} = 2^{100} \approx 10^{30}$

Beispiel: `A[] := {'a','b','a','c','a','e','f','a'}`

```
C[m]              C[0] := 0
'a': C[1] =       C[1] += C[0] =
'b': C[2] =       C[2] += C[1] =
'c': C[3] =       C[3] += C[2] =
'd': C[4] =       C[4] += C[3] =
'e': C[5] =       C[5] += C[4] =
'f': C[6] =
```

```
for( all letters in A )
   C[A[i]-'a'+1]++;
for( all C \ C[0] )
   C[i] += C[i-1];
for( all letters in A )
   B[C[A[i]-'a']++] = A[i];
```

→ `B[] : {'a',‘a','a',‘a',‘b‘,‘c',‘e',‘f‘}`

© T. Gemmeke, IDS, RWTH

8. Lineare Datentypen

8.1 Listen

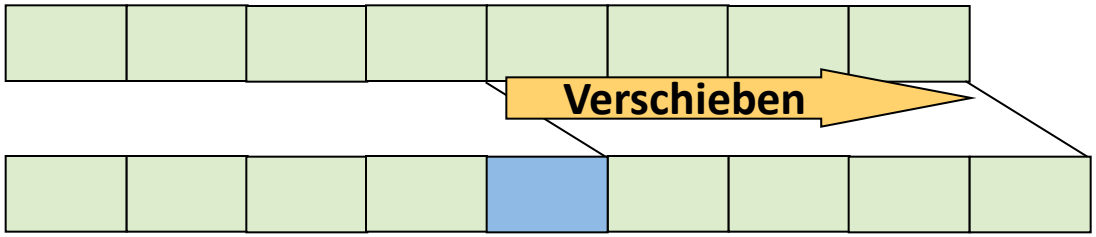
8.2 Stacks

8.3 Queues

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Mengendarstellung

- **Problemstellung**: Verwaltung einer Menge von Objekten $A = \{a_1, \ldots, a_n\}$ vom Typ `T`
- Gewünschte **Operationen**:
  - Suchen, Einfügen und Entfernen von Elementen
  - Vereinigung und Durchschnitt zweier Mengen A und B
- Prinzipiell wäre ein **Array** als Datenstruktur geeignet
- Jedoch zwei Probleme:
  - **Einfügen** eines neuen Elementes an einer bestimmten Position erfordert aufwendige **Umordnung**
  - **Größe** der Menge muss **statisch** feststehen (Bspw. Deklaration: `T A[100];` erlaubt max. 100 Elemente)



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

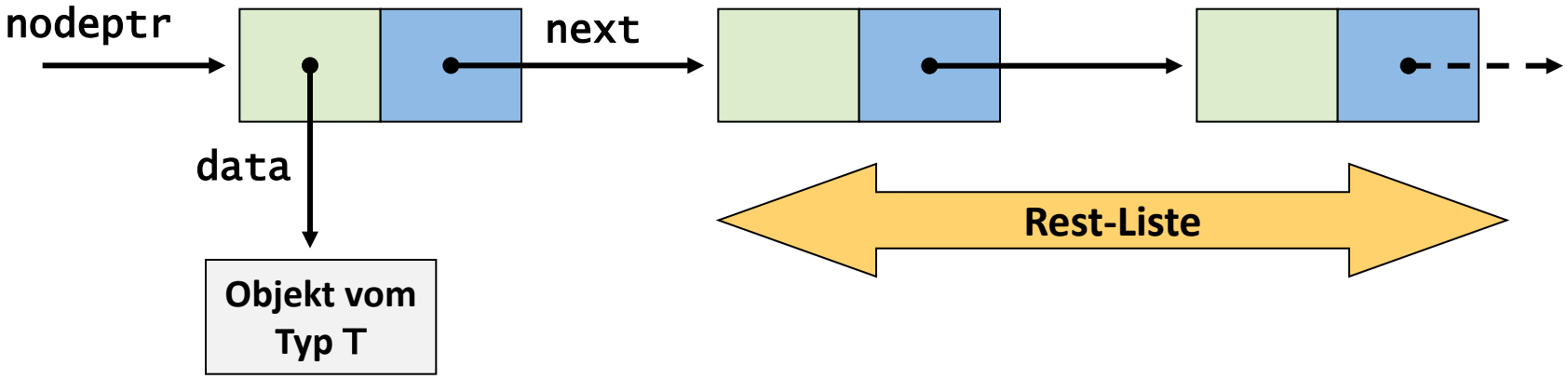
Verkettete Listen

Def.: Eine **(verkettete) Liste** ist entweder leer oder besteht aus einer Referenz auf einen **Knoten**, der ein Element und eine Referenz auf eine verkettete Liste enthält.

- Verwendung von **Zeigern** (siehe Kap. 3)
- Modellierung eines **Knotens** für ein Element vom Typ `T`:

```
typedef struct node {
  T* data; struct node* next; } *nodeptr;
```

- **`data`** ist Zeiger auf Listenelement vom Typ `T`
- **`next`** ist Zeiger auf Nachfolgerknoten



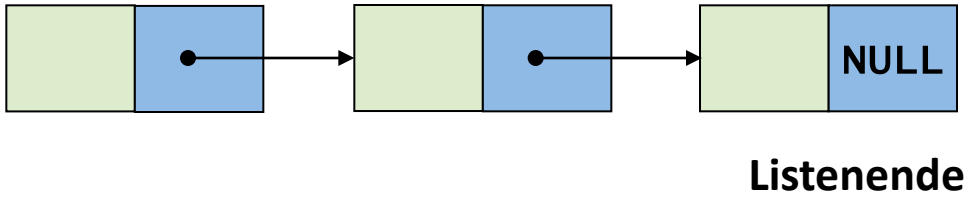
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

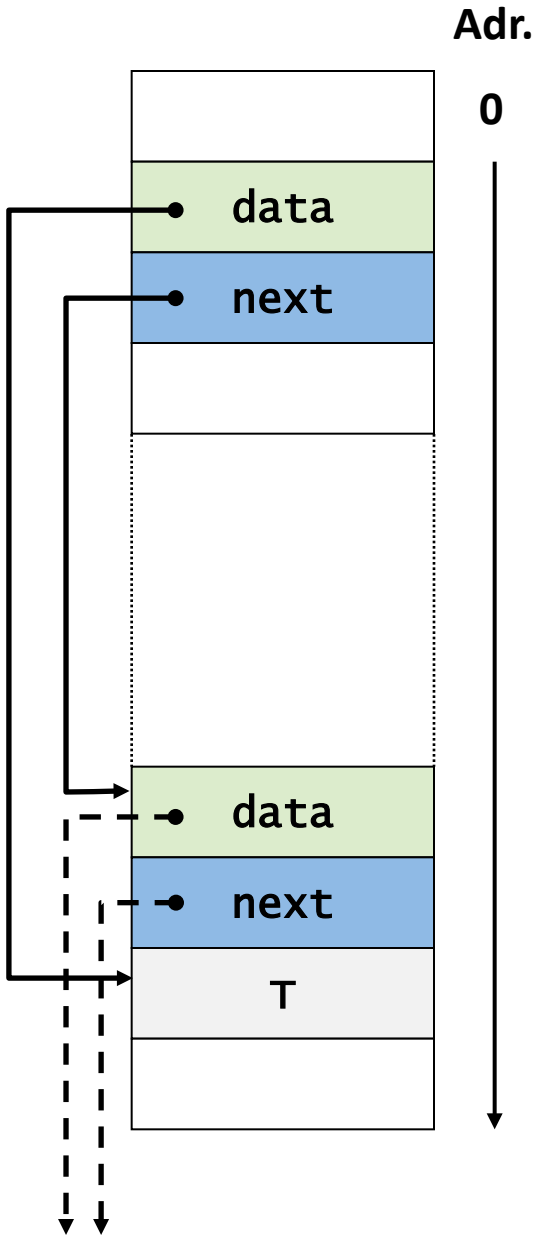
Verkettete Listen

- Liste im **Speicher** (vgl. Kap. 3)
- Liste = 1. Element + Restliste
- Darstellung einer leeren Liste:

```
#define NULL 0
```

- Zeigerwert **NULL** bedeutet, dass der Zeiger „nirgendwohin“ zeigt
- Speicherzugriff per NULL-Zeiger ist **unzulässig**, führt i.A. zum Programmabsturz





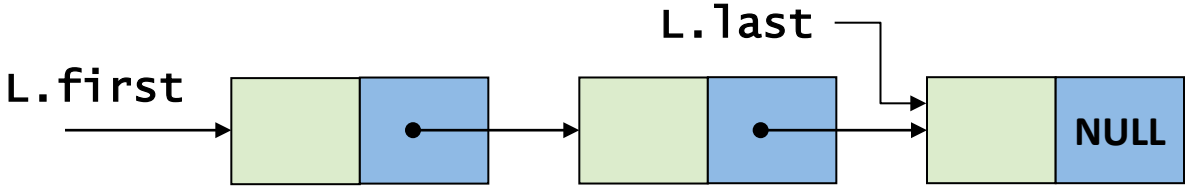
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Listenoperationen

- Definition des Datentyps **Liste**, Elemente vom Typ T

```
typedef struct node {
  T* data;
  struct node* next; } Node, *nodeptr;

typedef struct list {
  nodeptr first, last; } List, *listptr;
```



- Erzeugung einer **leeren Liste** und **Leerheitstest**

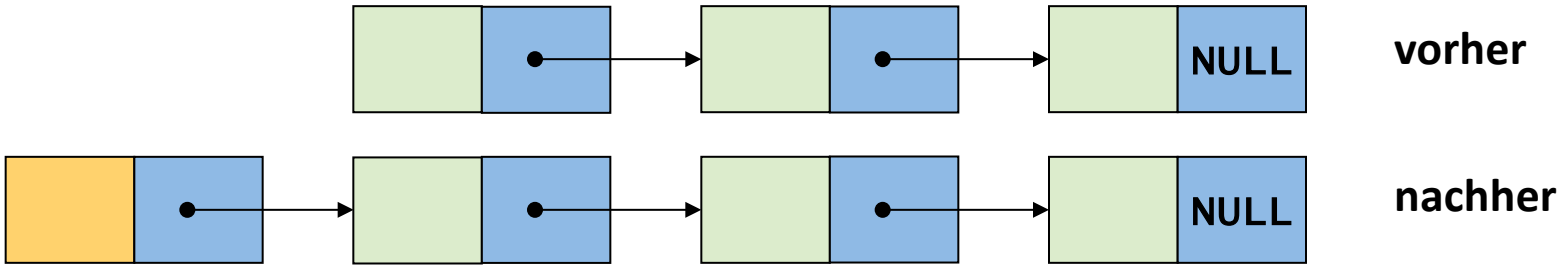
```
List L;
void Init(listptr L)
{
  L->first = NULL; L->last = NULL;
}

int IsEmpty(List L)
{
  return (L.first == NULL && L.last == NULL);
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Listenoperationen

- **Einfügen** eines neuen Listenelements vom Typ T am Listenanfang



- Funktion „`malloc`“ (siehe Kap. 3) stellt dynamisch Speicherplatz auf dem Heap bereit
- Funktion „`sizeof`“ berechnet die erforderliche Anzahl von Bytes (hier z.B. insges. 8 Bytes bei 32-Bit-Zeigern)

```
nodeptr newnode(T* item)
{
  nodeptr np;

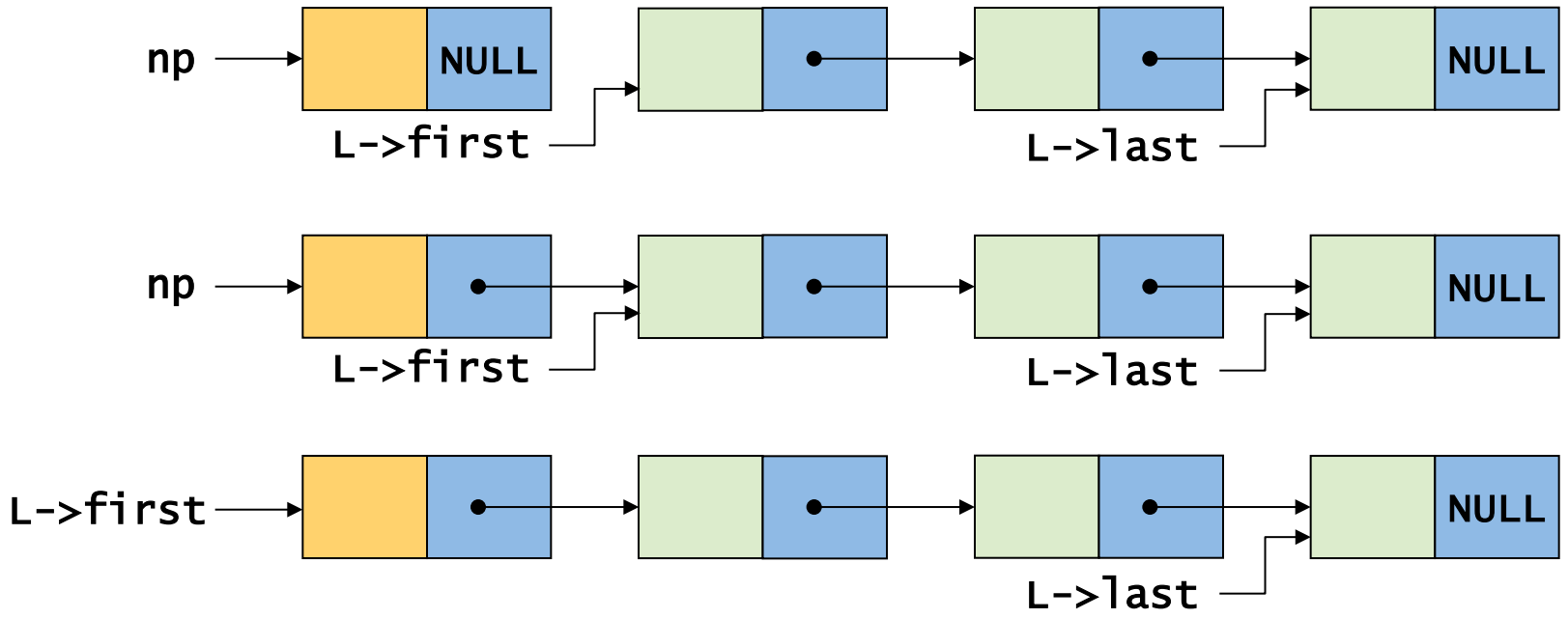
  np = (nodeptr)malloc(sizeof(Node));
  np->data = item;
  np->next = NULL;

  return np;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Einfügen am Listenanfang

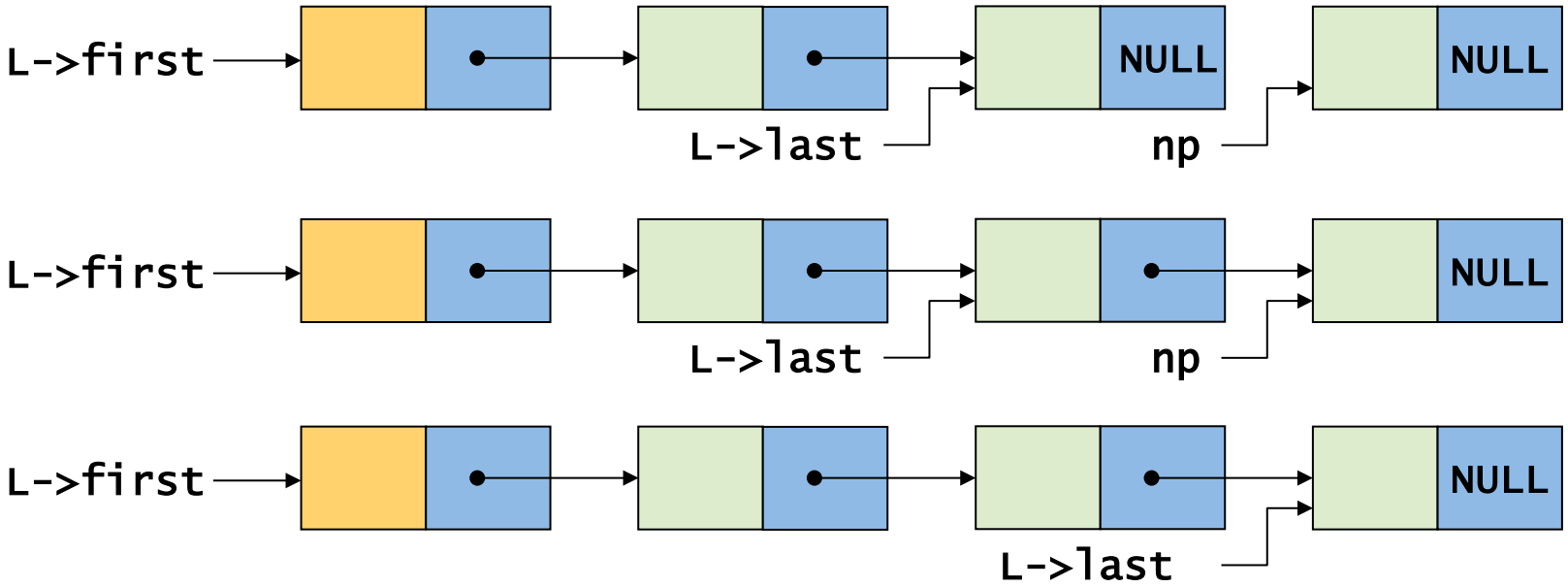
```
void AppendFirst(T* item, listptr L)
{
  nodeptr np = newnode(item);
  if (IsEmpty(*L)) {
    L->first = np; L->last = np;
  } else {
    np->next = L->first;
    L->first = np;
  }
}
```



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Einfügen am Listenende

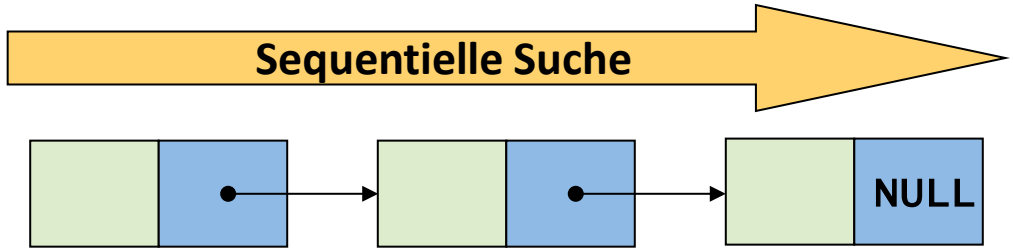
```
void AppendLast(T* item, listptr L)
{
  nodeptr np = newnode(item);
  if (IsEmpty(*L)) {
    L->first = np; L->last = np;
  } else {
    L->last->next = np;
    L->last = np;
  }
}
```



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Prüfen ob Element in Liste enthalten

```
int IsIn(T* item, List L)
{
  nodeptr np;
  if (IsEmpty(L)) return 0; // false
  np = L.first;
  while (np != NULL) {
    if (Equal(np->data, item)) return 1; // true
    np = np->next;
  }
  return 0; // false
}
```



- **Laufzeit** bei N-elementiger Liste:
  - Best case: O(1)
  - Worst case: O(N)
  - Average case: O(N/2) = O(N)

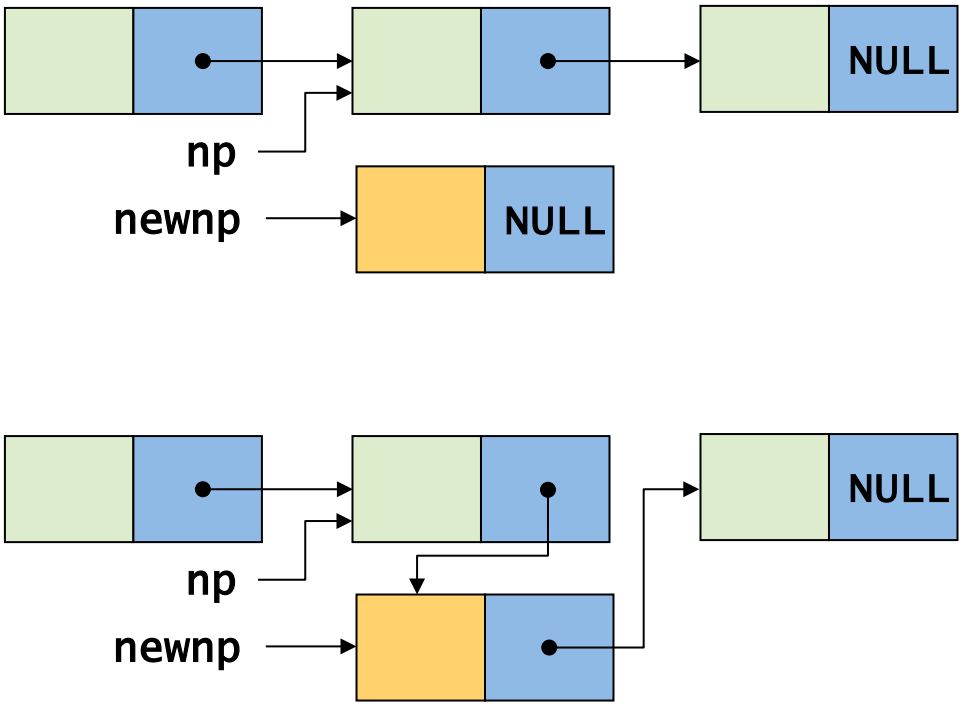
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Einfügen hinter ein bestimmtes Listenelement

```
// item2 hinter item1 einfügen
void InsertBehind(T* item1,
  T* item2, listptr L)
{
  nodeptr np, newnp;

  if (!IsIn(item1,*L)) {
    printf(„Fehler!“); return;
  }

  np = L->first;
  while (np != NULL) {
    if (Equal(np->data,item1)) {
      newnp = newnode(item2);
      newnp->next = np->next;
      np->next = newnp;
      if (np == L->last) {
        L->last = newnp;
      }
      break;
    }
    np = np->next;
  }
}
```

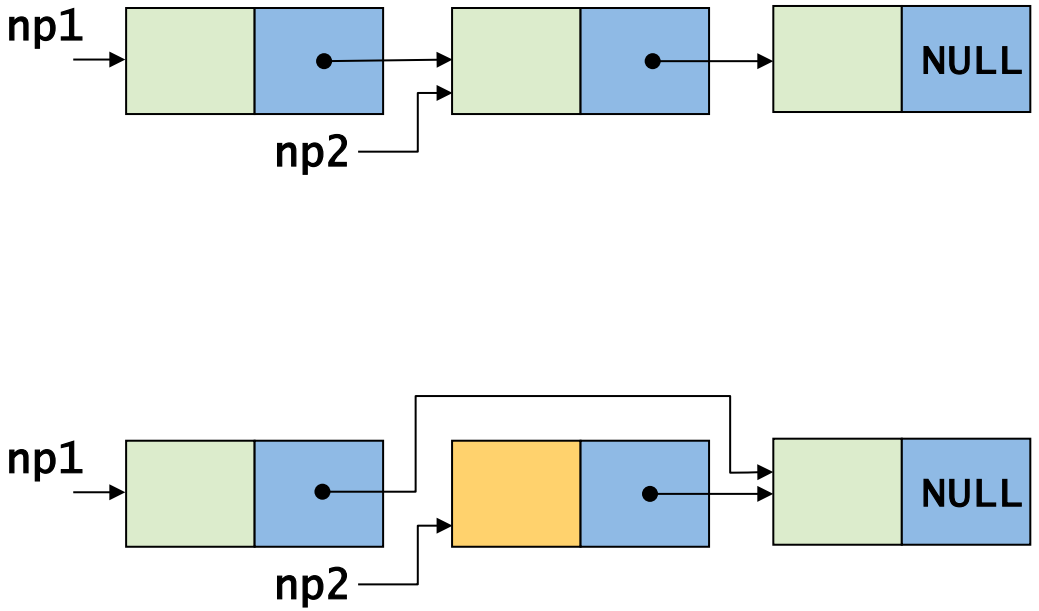


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Löschen eines Listenelementes

```
void Delete(T* item, listptr L)
{
  nodeptr np1, np2;

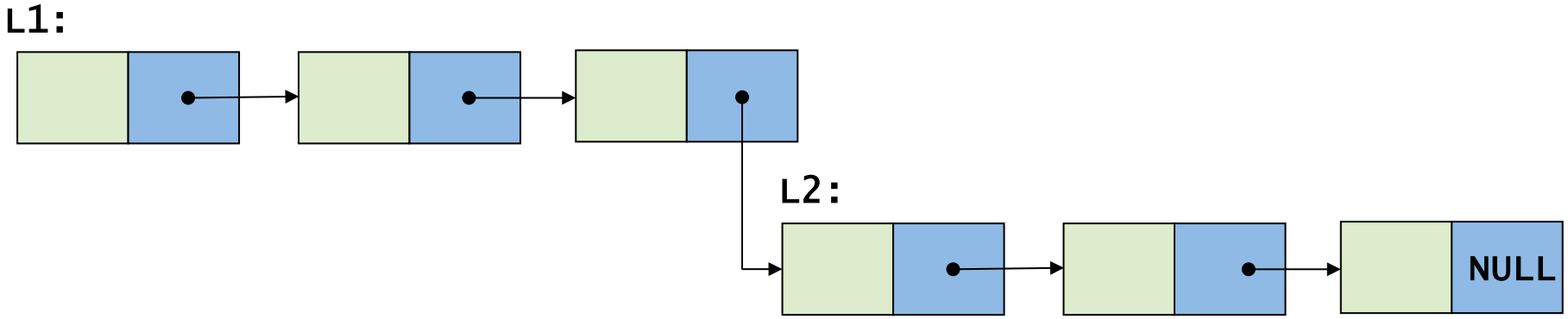
  if (IsEmpty(*L)) return;
  np1 = L->first;
  if (Equal(np1->data,item)) {
    L->first = np1->next;
    if (L->first == NULL)
      L->last = NULL;
    free(np1);
    return;
  }
  np2 = np1->next;
  while (np2 != NULL) {
    if (Equal(np2->data,item)) {
      np1->next = np2->next;
      if (np2 == L->last)
        L->last = np1;
      free(np2);
      break;
    }
    np1 = np2;
    np2 = np2->next;
  }
}
```



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Zusammenfügen zweier Listen

- Falls Liste als Mengendarstellung: entspricht **Vereinigung**
- Evtl. **Mehrfachvorkommen** von Elementen ausschließen




```
listptr Union(listptr L1, listptr L2)
{
  if (IsEmpty(*L2)) return L1;
  if (IsEmpty(*L1)) return L2;

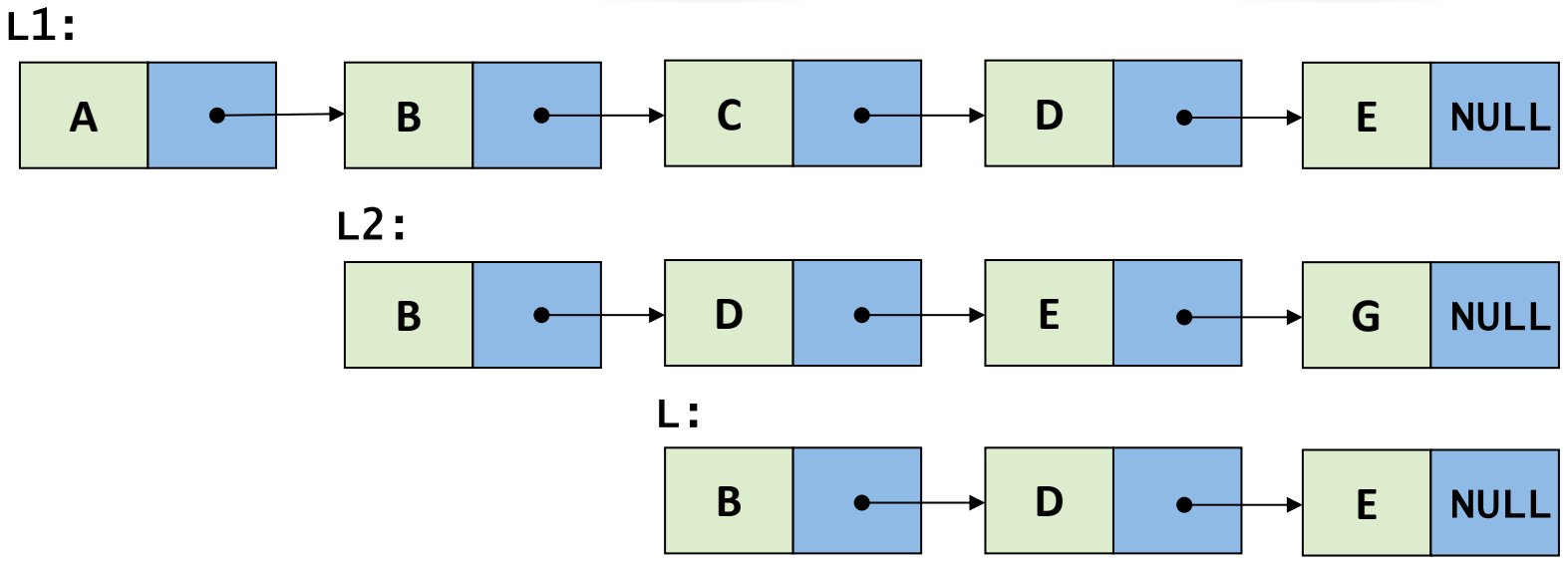
  L1->last->next = L2->first;
  L1->last = L2->last;

  return L1;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Schnittmengenberechnung mittels Listen

```
listptr Intersect(listptr L1, listptr L2)
{
  nodeptr np;
  listptr L = (listptr)malloc(sizeof(List));
  Init(L);
  np = L1->first;
  while (np != NULL) {
    if (IsIn(np->data, *L2)) AppendLast(np->data, L);
    np = np->next;
  }
  return L;
}
```



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

8. Lineare Datentypen

8.1 Listen

8.2 Stacks

8.3 Queues

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Stacks

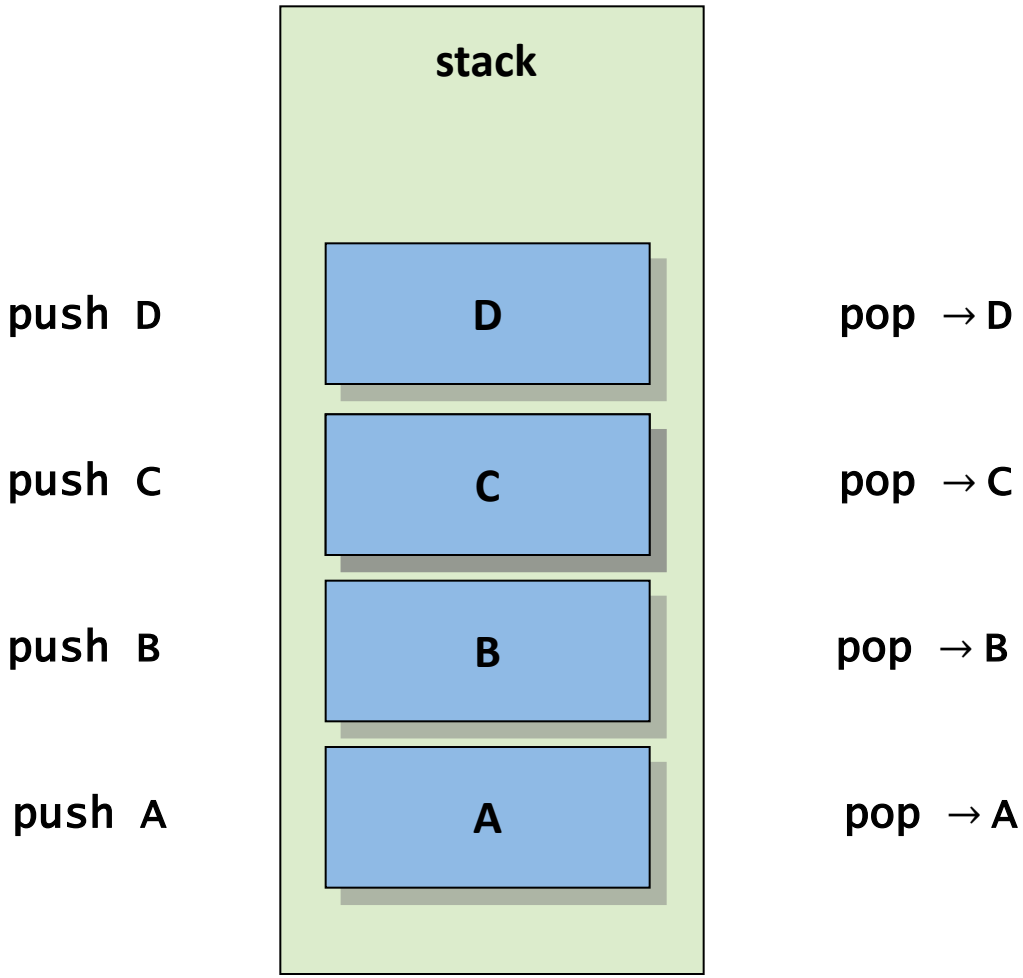
Def.: Ein **Stack** (Stapel) ist ein ADT, der folgende Grundoperationen unterstützt:

1. Einfügen eines neuen Elementes („Push“)
2. Entfernen des zuletzt eingefügten Elementes („Pop“)

- Stacks sind eine **grundlegende Datenstruktur** in vielen Anwendungen
  - Ablagesystem auf Professorenschreibtisch
  - Compiler-Frontend (Analyse des Quellprogramms)
  - Abarbeitung von C-Funktionsaufrufen
- Aufgrund der Pop-Operation auch als **LIFO-Datenstruktur** bezeichnet (*last in, first out*)
- Stack als ADT, d.h. **verschiedene Implementierungen** möglich (z.B. Listen und Arrays)

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

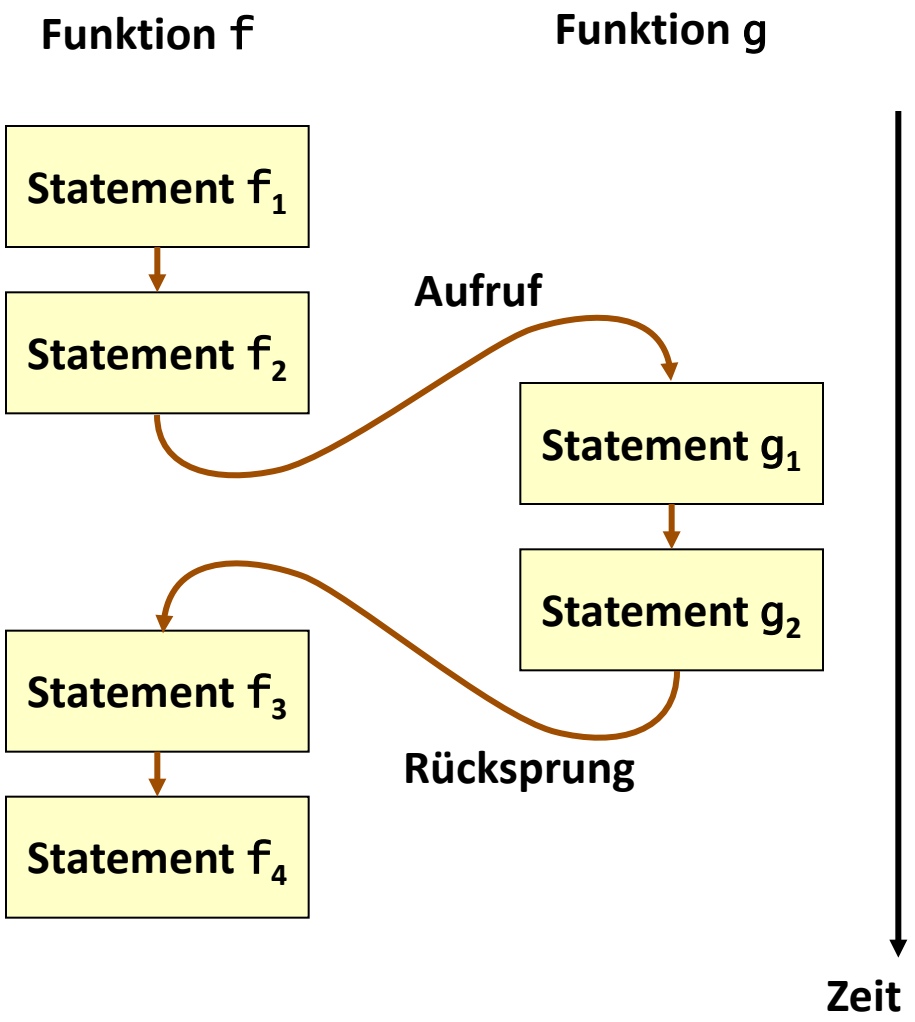
Push und Pop



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel: Laufzeit-Stack für C-Funktionsaufrufe

- Siehe auch Kap. 5
- Funktion f wird durch Aufruf von g **unterbrochen** und später **fortgesetzt**
- g kann ihrerseits **weitere Funktionen** aufrufen
- Umgebung zur Programmausführung muss einen **Stack noch aktiver Funktionen** verwalten
- **Funktionsaufruf** = Push
- **Funktionsende** = Pop



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel: Laufzeit-Stack für C-Funktionsaufrufe

```
void f(void)
{
  g();
}

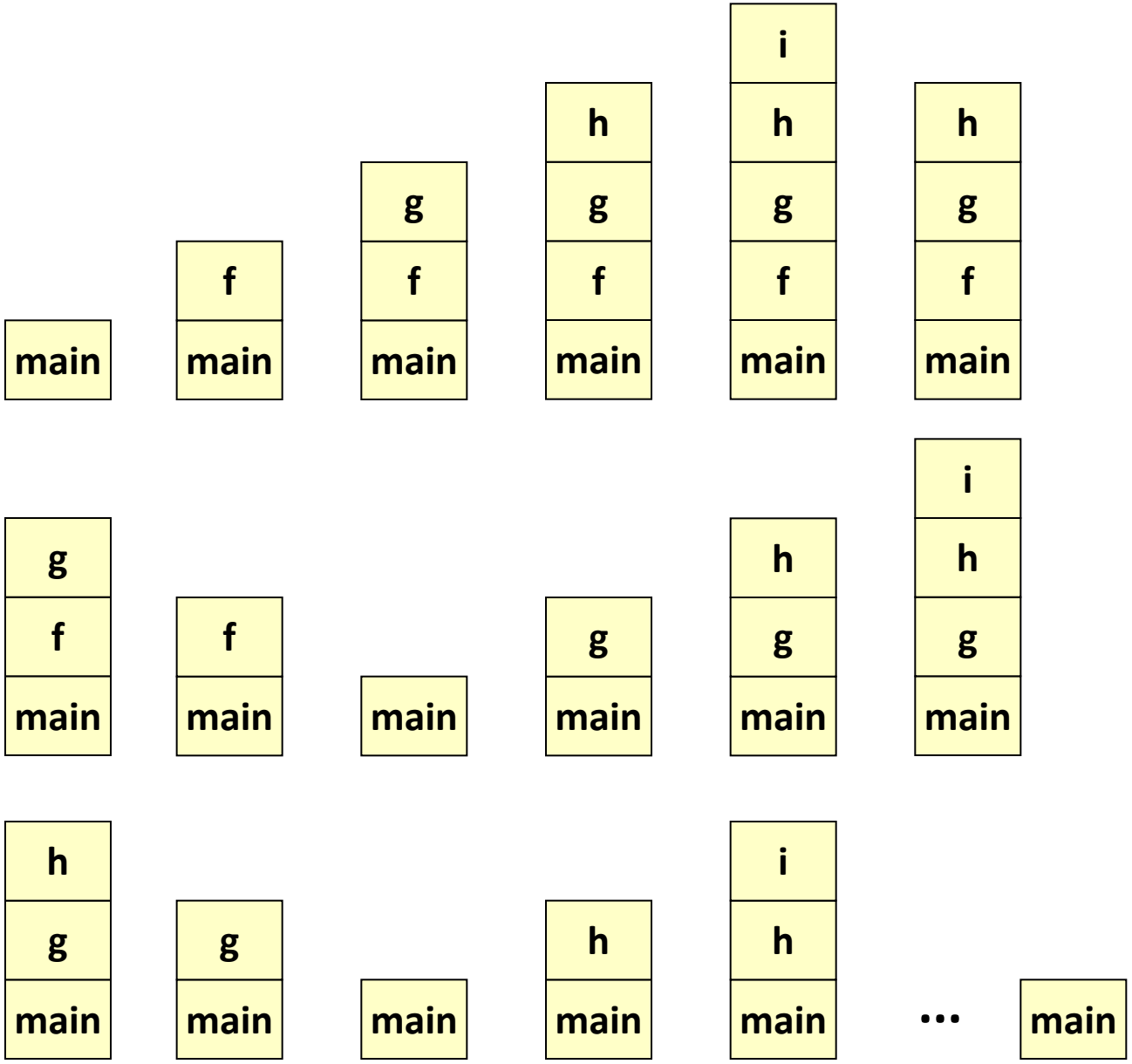
void g(void)
{
  h();
}

void h(void)
{
  i();
}

void i(void)
{
}

int main(void)
{
  f();
  g();
  h();

  return 0;
}
```



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Stack-Implementierung durch Listen

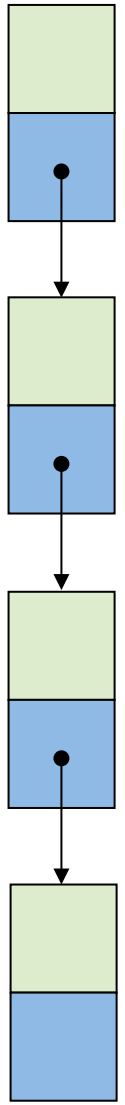
- Elemente auf Stack = Listenelemente
- **Push** = Einfügen am Listenanfang
- **Pop** = Entfernen des ersten Elementes
- Stack für beliebigen Elementtyp `T`:

```
List L;

void Push(T* item,listptr L)
{
  AppendFirst(item, L);
}

T* Pop(listptr L)
{
  T* item;
  if (IsEmpty(*L)) return NULL;

  item = L->first->data;
  Delete(item, L);
  return item;
}
```



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Stack-Implementierung durch Arrays

- Vorgabe der **max. Stack-Größe** notwendig
- Elemente auf Stack = Arrayelemente
- „**top**“ verweist stets auf das nächste freie Element (top = „**Stack-Pointer**“)
- **Push** = Einfügen und Erhöhen von top um 1
- **Pop** = Entfernen und Erniedrigen von top um 1
- Prüfen auf **Überlauf** (im Gegensatz zu Listen!)

```
#define MAX 100
typedef struct stk
{ T* elems[MAX]; int top; } stack;

void Init(stack* s)
{ s->top = 0; }

int IsEmpty(stack s)
{ return (s.top == 0); }

void Push(T* item,stack* s)
{
  if (s->top == MAX)
    printf(„Stack voll!“);
  s->elems[s->top] = item;
  s->top++;
}

T* Pop(stack* s)
{
  if (IsEmpty(*s)) return NULL;
  s->top--;
  return s->elems[s->top];
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel-Anwendung

- Auswertung geklammerter **arithmetischer Ausdrücke**
  - z.B. ((6 * (4 * 28)) + (9 - ((12 / 4) * 2)))
- **Innere Ausdrücke** vor äußeren auszuwerten
- Speicherung von Zwischenergebnissen notwendig
- **Zwei Stacks**: Operanden- und Operatorstack
- Abarbeitung eines Ausdrucks **von links nach rechts**:
  - „(“: keine Aktion
  - Operand: Push auf Operandenstack
  - Operator: Push auf Operatorstack
  - „)“: Pop des obersten Operators, Pop der obersten beiden Operanden, Ausdruck auswerten, Push Ergebnis auf Operandenstack

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel

((6 * (4 * 28)) + (9 - ((12 / 4) * 2)))

Operandenstack	Operatorstack
6	
6 4	*
6 4 28	* *
6 112	*
672	(leer)
672 9	+
672 9 12	+ -
672 9 12 4	+ - /
672 9 3	+ -
672 9 3 2	+ - *
672 9 6	+ -
672 3	+
675	(leer)

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

8. Lineare Datentypen

8.1 Listen

8.2 Stacks

8.3 Queues

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Queues

Def.: Eine **Queue** (Warteschlange) ist ein ADT, der folgende Grundoperationen unterstützt:

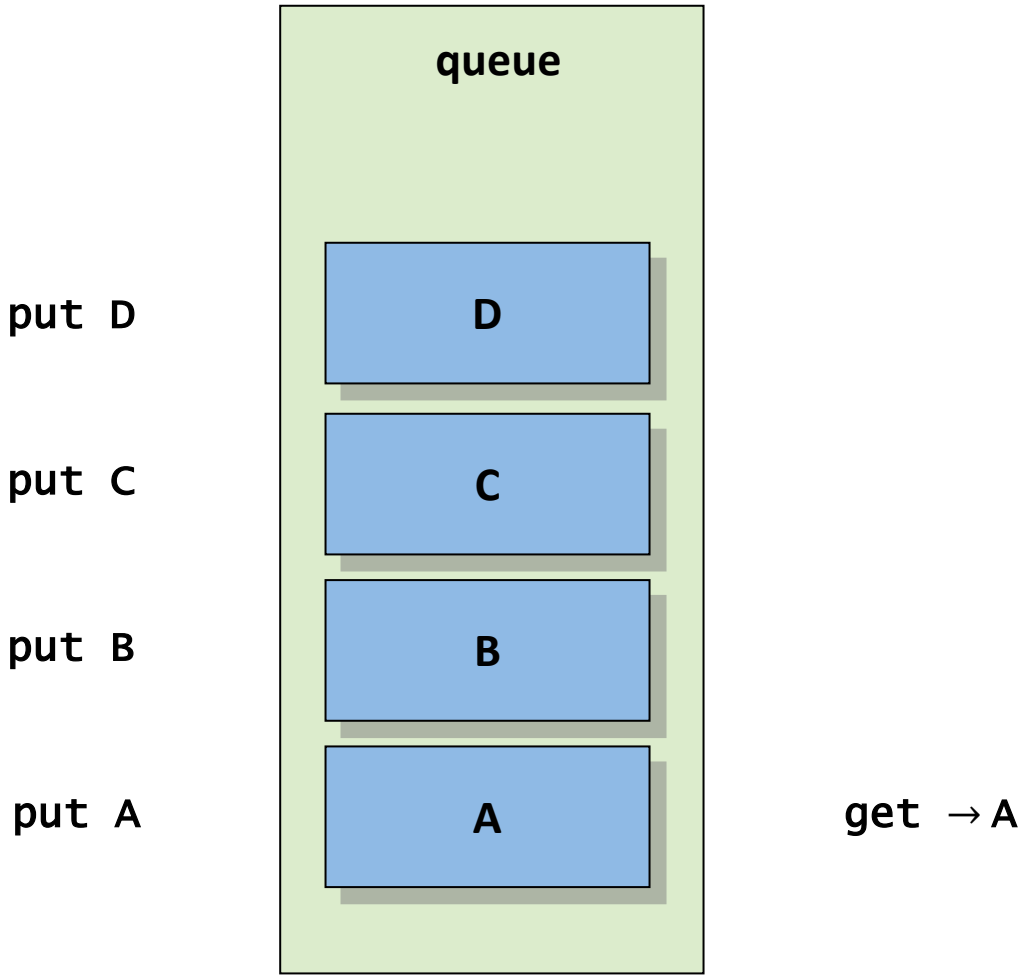
1. Einfügen eines neuen Elementes („Put“)
2. Entfernen des zuerst eingefügten Elementes („Get“)

- Verbreitete Datenstruktur, z.B.
  - Warteschlange von auszuführenden Prozessen
  - Kommunikationspuffer z.B. für Druckaufträge
- **FIFO-Prinzip**: *first in, first out* („zuerst gekommen, zuerst bedient“)



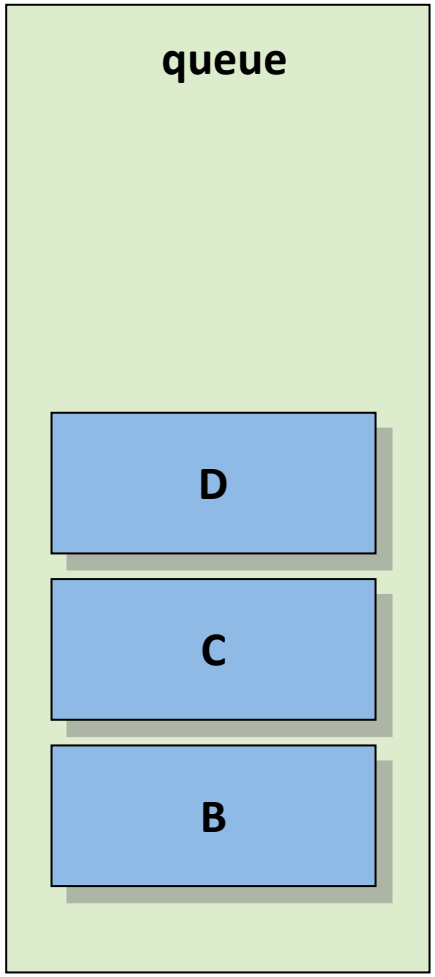
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Put und Get



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

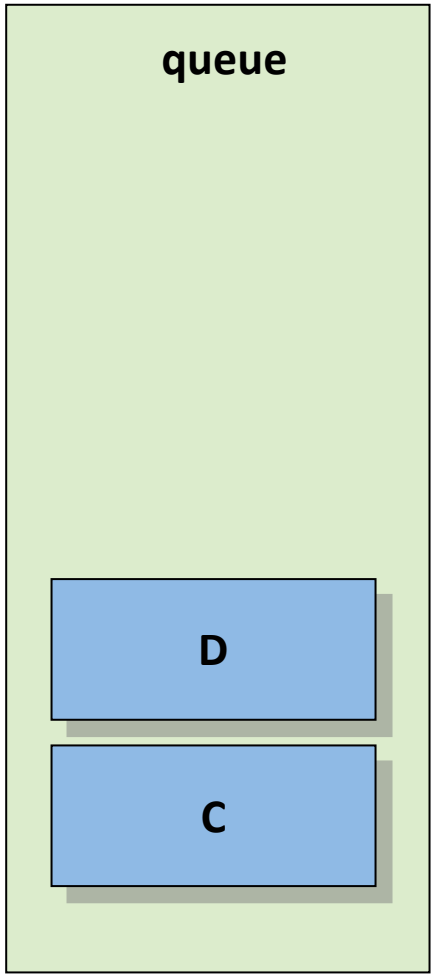
Put und Get



`get` → `B`

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Put und Get



`get` → `C`

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Queue-Implementierung durch Listen

- Queue-Elemente = Listenelemente
- **`Put`** = Einfügen am Listenende
- **`Get`** = Entfernen des ersten Elementes
- Queue für beliebigen Elementtyp `T`:

```
List L;

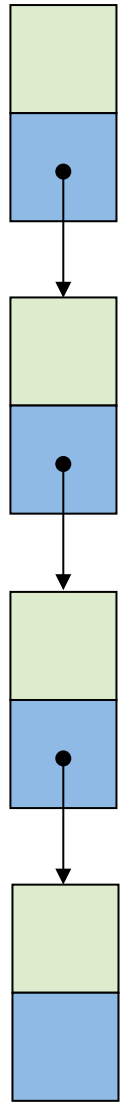
void Put(T* item,listptr L)
{
  AppendLast(item, L);
}

T* Get(listptr L)
{
  T* item;

  if (IsEmpty(*L)) return NULL;

  item = L->first->data;
  Delete(item, L);

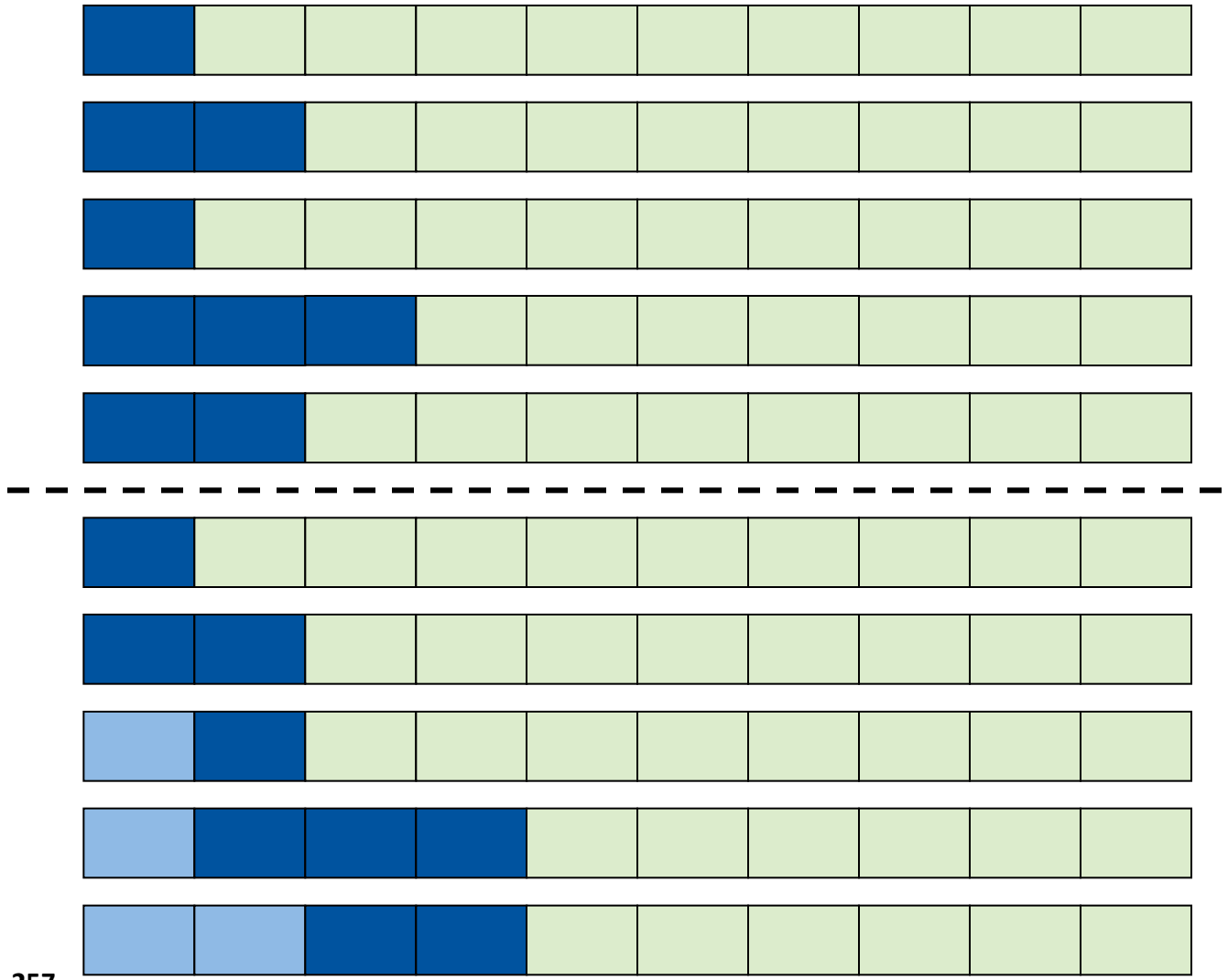
  return item;
}
```



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Queue-Implementierung durch Arrays

- Betrachte **Dynamik** von Queues im Vergleich zu Stacks in Array-Implementierung:



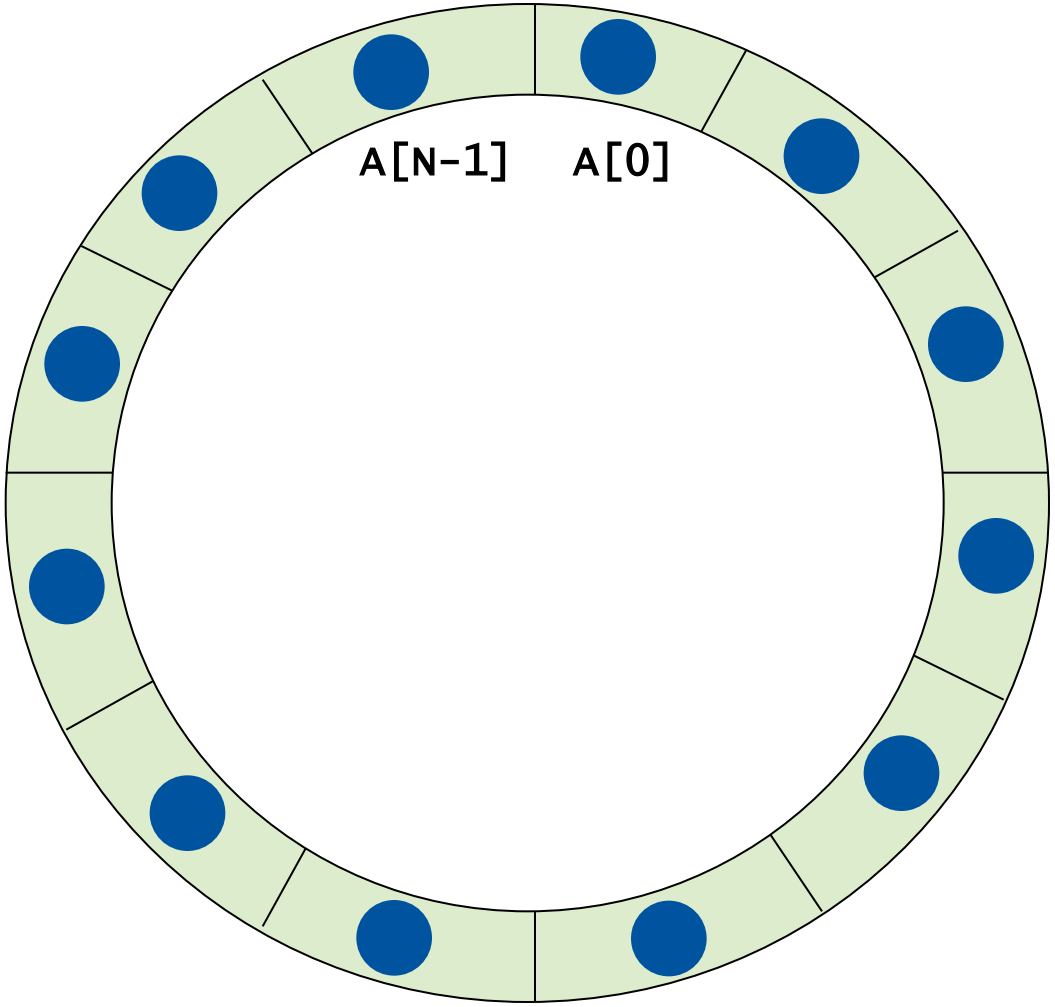
Stack: verwendet tendenziell immer den gleichen Speicherbereich

Queue: „wandert“ tendenziell immer weiter nach rechts, daher schlechte Speicherausnutzung

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Queue-Implementierung durch Arrays

- Lösung: **ringförmiges** Array bzw. **zyklische** Adressierung



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Queue-Implementierung durch Arrays

- Zwei **Zeiger** (Array-Indizes) `first` und `last` verweisen auf aktuellen Anfang (-1) und Ende (+1) der Queue
- Verwendung des Modulo-Operators (%) zur **zyklischen Adressierung**
- Prüfung auf **Über- und Unterlauf** (*overflow*, *underflow*) erforderlich

```
#define MAX 100

typedef struct q
{ T* elems[MAX];
  int first, last; } queue;

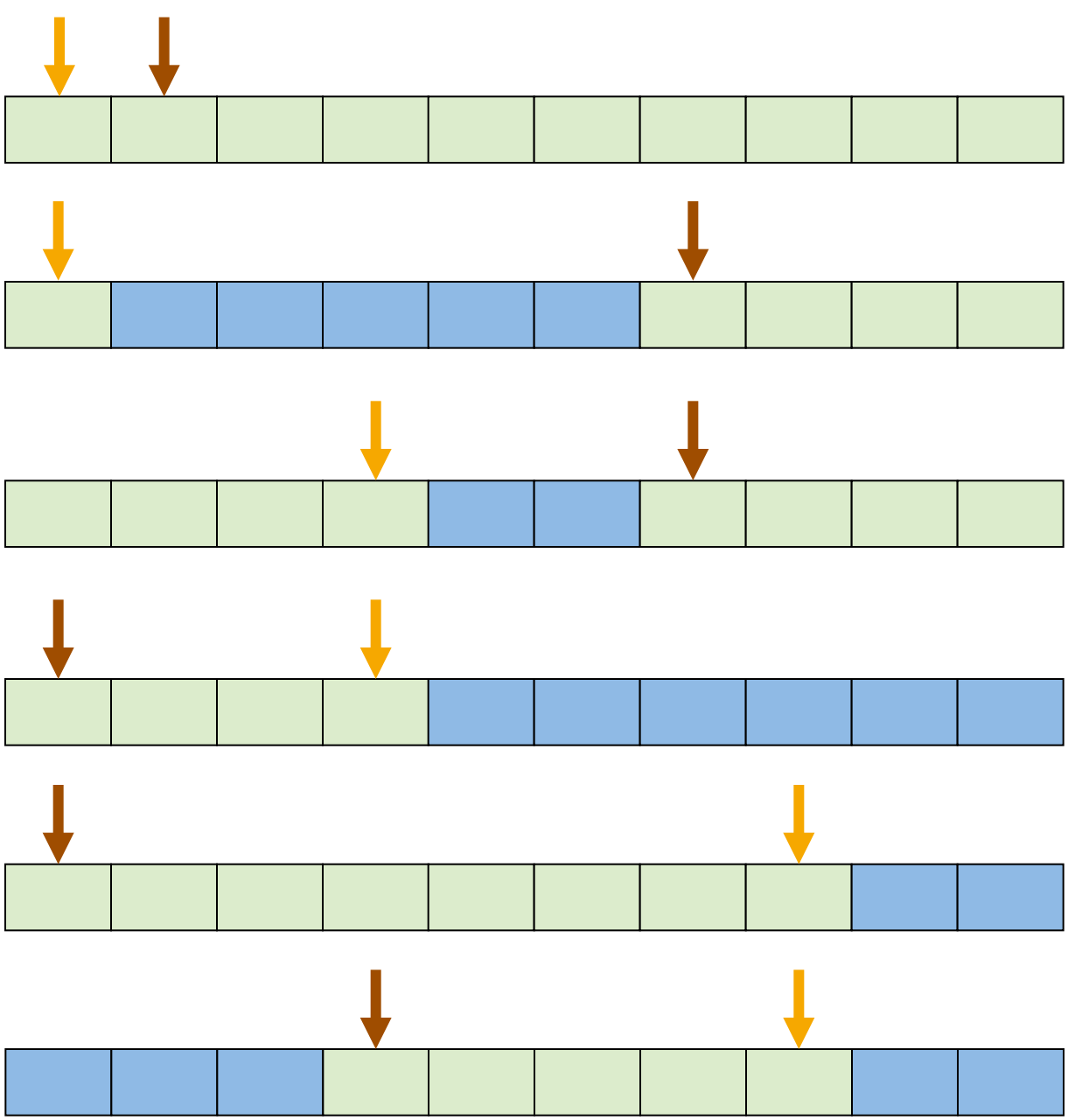
void Init(queue* q)
{
  q->first = 0;
  q->last = 1;
}

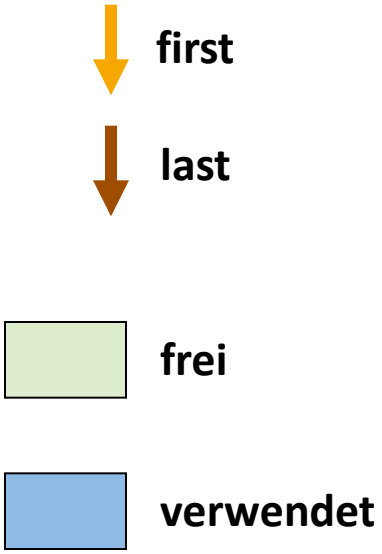
void Put(T* item,queue* q)
{
  if (q->last == q->first) overflow();
  else {
    q->elems[q->last] = item;
    q->last = (q->last+1) % MAX;
  }
}

T* Get(queue* q)
{
  q->first = (q->first+1) % MAX;
  if (q->first == q->last) underflow();
  else return q->elems[q->first];
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel-Ablauf





© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

9. Bäume

9.1 Definition und Darstellung

9.2 Baumdurchläufe

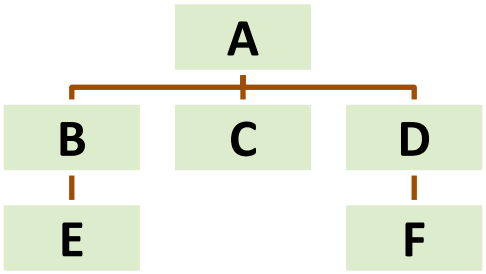
9.3 Suchbäume

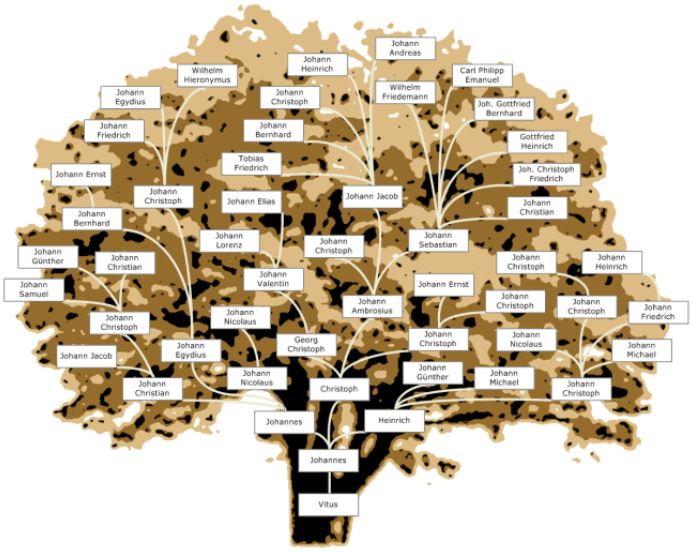
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

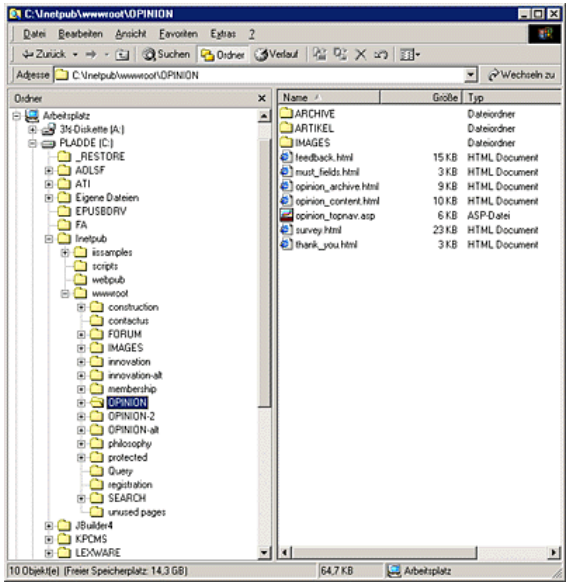
Bäume

Def.: Ein **Baum** besteht aus einem ausgezeichneten Knoten r (**Wurzel**, *root*) und $k \geq 0$ disjunkten Bäumen $T_1, \ldots, T_k$

- Bäume sind eine äußerst verbreitete Datenstruktur mit einer **Vielzahl von Anwendungen**
- Z.B. Darstellung von **Hierarchien** (Firmen-Organigramm), Eltern-Kind-Beziehung (Stammbaum), Dateisysteme u.v.m.
- Siehe auch Baumdarstellung von Arrays in **Heapsort**





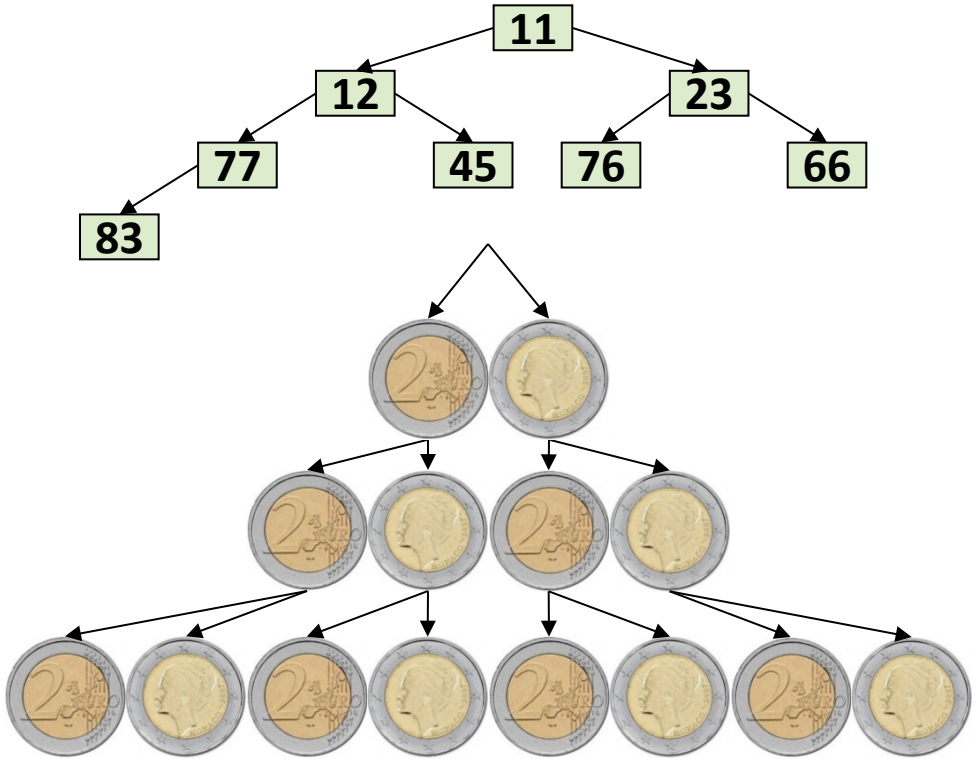


Binärbäume

- Spezialfall: Bäume mit max. **zwei Teilbäumen**

Def.: Ein **Binärbaum** $B = (r, B_L, B_R)$ ist entweder leer oder besteht aus einer Wurzel r sowie einem linken und rechten Teilbaum B_L bzw. B_R

- Z.B. **Heap** im Heapsort-Algorithmus: jeder Knoten besitzt max. zwei „Kinder“
- Z.B. geschachteltes `if-then-else`-Statement
- Z.B. graphische Darstellung eines **Münzwurf-Prozesses**

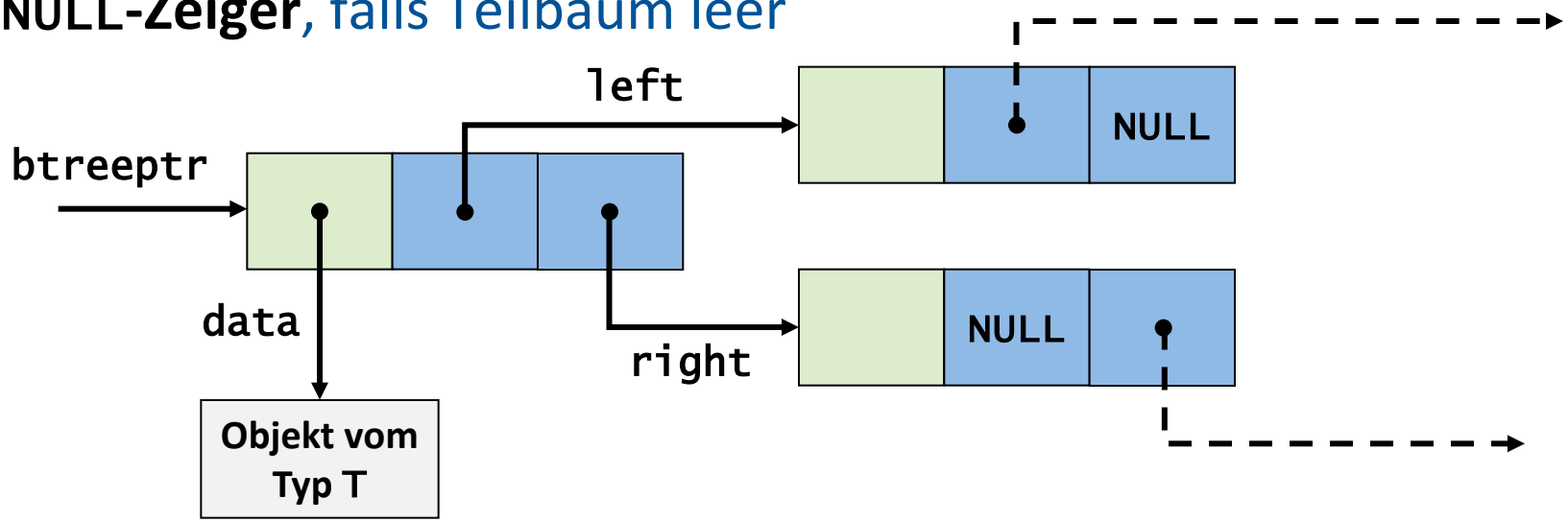


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Darstellung als C-Datentyp

```
typedef struct tr {
  T* data;
  struct tr *left,*right;
} btree, *btreeptr;
```

- Bäume als dynamische Datenstruktur, ähnlich zu Listen: Zeiger `data` auf **Nutzdaten** eines beliebigen Typs T
- Zeiger `left` und `right` verweisen auf **linken** und **rechten** Teilbaum
- **NULL-Zeiger**, falls Teilbaum leer



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

9. Bäume

9.1 Definition und Darstellung

9.2 Baumdurchläufe

9.3 Suchbäume

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Baumdurchläufe

- Häufige Anwendung: **Durchlaufen** eines Baumes, d.h. Besuchen/Verarbeiten aller Knoten in bestimmter Reihenfolge
- Z.B. beim Kopieren von Dateisystemen
- Sei $B = (r, B_L, B_R)$
- L bezeichnet Durchlauf des **linken** Teilbaums B_L
- R bezeichnet Durchlauf des **rechten** Teilbaums B_R
- W bezeichnet Bearbeitung der **Wurzel** r
- Drei prinzipielle Durchlaufschemata:
  - **Inorder**: L W R
  - **Preorder**: W L R
  - **Postorder**: L R W
- In/Pre/Post bzgl. des Zeitpunkts der Wurzel-Bearbeitung

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Rekursive Implementierung der Durchläufe

```
void visit(T* item)
{ // Verarbeitung von item
}

void preorder(btreeptr b)
{
  if (b == NULL) return;

  visit(b->data);
  preorder(b->left);
  preorder(b->right);
}

void inorder(btreeptr b)
{
  if (b == NULL) return;

  inorder(b->left);
  visit(b->data);
  inorder(b->right);
}
```

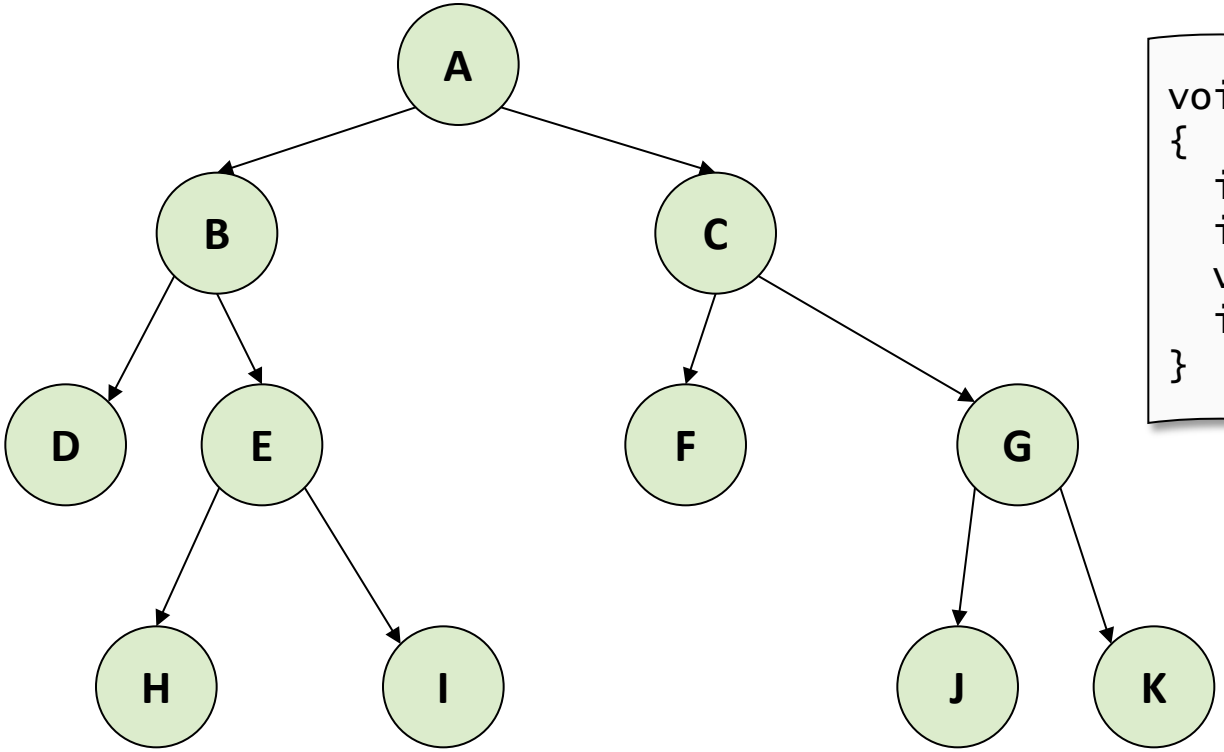
```
void postorder(btreeptr b)
{
  if (b == NULL) return;

  postorder(b->left);
  postorder(b->right);
  visit(b->data);
}
```

- Laufzeit: $\mathcal{O}(\#\,\text{Knoten})$
- Interner Ablauf der Rekursion: **Laufzeit-Stack** für Funktionsaufrufe (vgl. Kap. 8)
- **Übung**: nicht-rekursive Implementierung mittels expliziter Stack-Datenstruktur

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel: Inorder-Durchlauf (L W R)




```
void inorder(btreeptr b)
{
  if (b == NULL) return;
  inorder(b->left);
  visit(b->data);
  inorder(b->right);
}
```

- Inorder-Reihenfolge: D B H E I A F C J G K
- Preorder-Reihenfolge: A B D E H I C F G J K
- Postorder-Reihenfolge: D H I E B F J K G C A

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

9. Bäume

9.1 Definition und Darstellung

9.2 Baumdurchläufe

9.3 Suchbäume

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Mengendarstellung durch Binärbäume

- **Mengenoperationen** (vgl. Listen, Kap. 8): Einfügen, Löschen, Suchen (gemäß **Suchschlüssel** oder key), Schnitt, Vereinigung
- In vielen Anwendungen insbes. schnelle Implementierung von **Einfügen, Löschen, Suchen** erforderlich
- Beispiel: **Symboltabelle** im C-Compiler
  - Symboltabelle verwaltet **Informationen zu den Identifiern** (Variablen, Funktionen, etc.) eines Programms, z.B. Datentyp und Speicherbedarf
  - **Variablendeklaration** für ID bewirkt **neuen Eintrag** in Symboltabelle mit Suchschlüssel ID
  - **Variablenverwendung** bewirkt **Suchen** nach ID
  - Variable **nicht mehr benötigt**: ID aus Symboltabelle **löschen**

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Suchbäume

- **Komplexität** der Mengenoperationen bei Listenimplementierung typischerweise $\mathcal{O}(N)$ bei N Elementen
- Zu **aufwendig** für häufigen Gebrauch
- Sortiertes Array: Suchen in Zeit $\mathcal{O}(\log N)$ per **binärer Suche**, jedoch Einfügen und Löschen aufwendig
- Binäre Suche: Berechnung eines **Entscheidungsbaums**, d.h. in jedem Schritt Verzweigung in linkes oder rechtes Teilarray
- **Idee bei binären Suchbäumen**: Entscheidungsbaum als explizite Datenstruktur

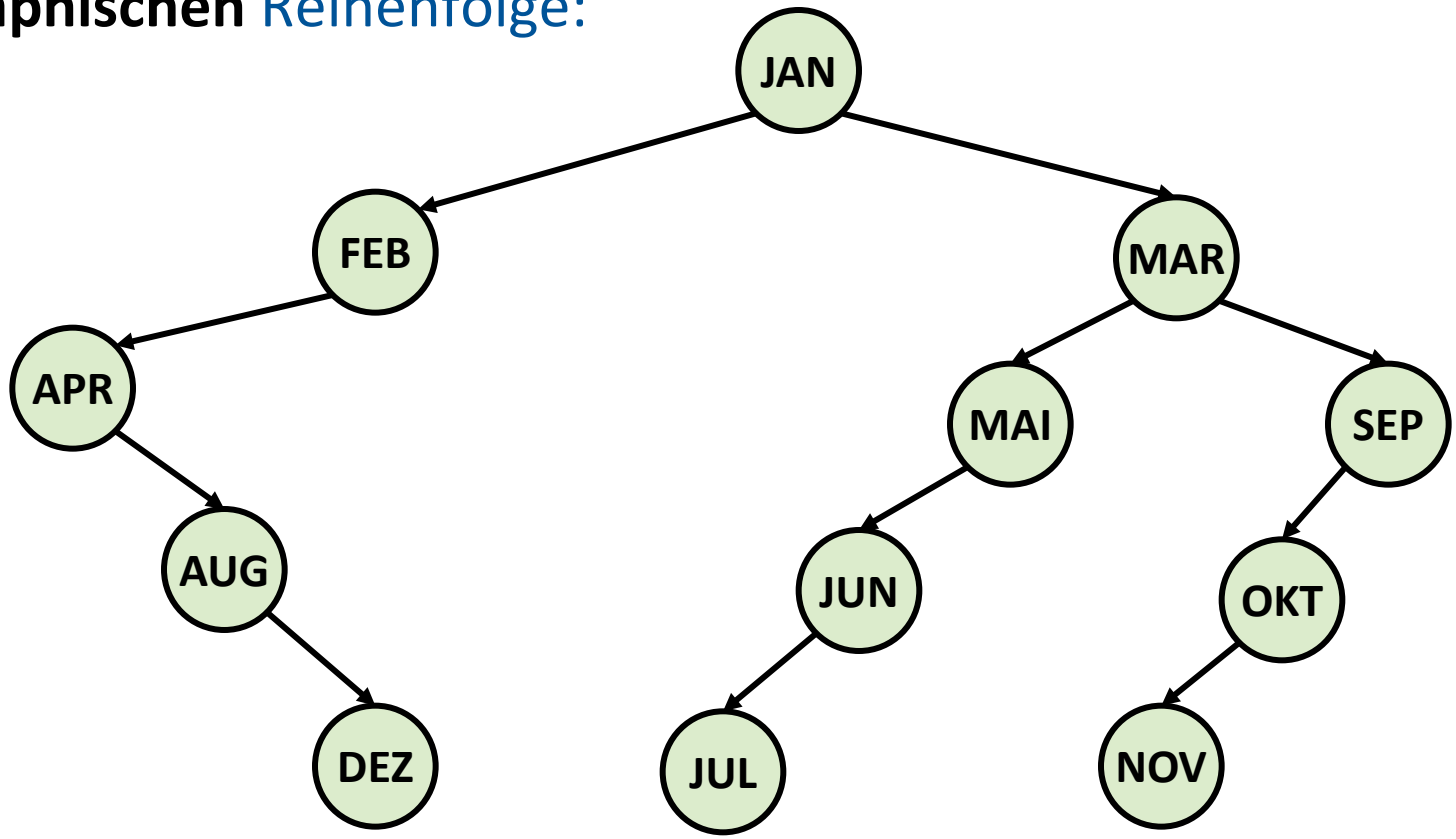
Def.: Sei $B = (r, B_L, B_R)$ ein Binärbaum und bezeichne $\text{key}(n)$ den Suchschlüssel für jeden Knoten n.
B heißt **Suchbaum**, falls gilt:

1. Für alle $n \in B_L$: $\text{key}(n) < \text{key}(r)$
2. Für alle $n \in B_R$: $\text{key}(n) > \text{key}(r)$
3. B_L und B_R sind Suchbäume

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel

- Suchbaum für Monatsnamen, geordnet entsprechend der **lexikographischen** Reihenfolge:



- **Übung**:
  - Welcher Baum ergibt sich bei umgekehrter Reihenfolge?
  - Wie kann man mittels Suchbäumen sortieren?

Einfügen eines Elementes

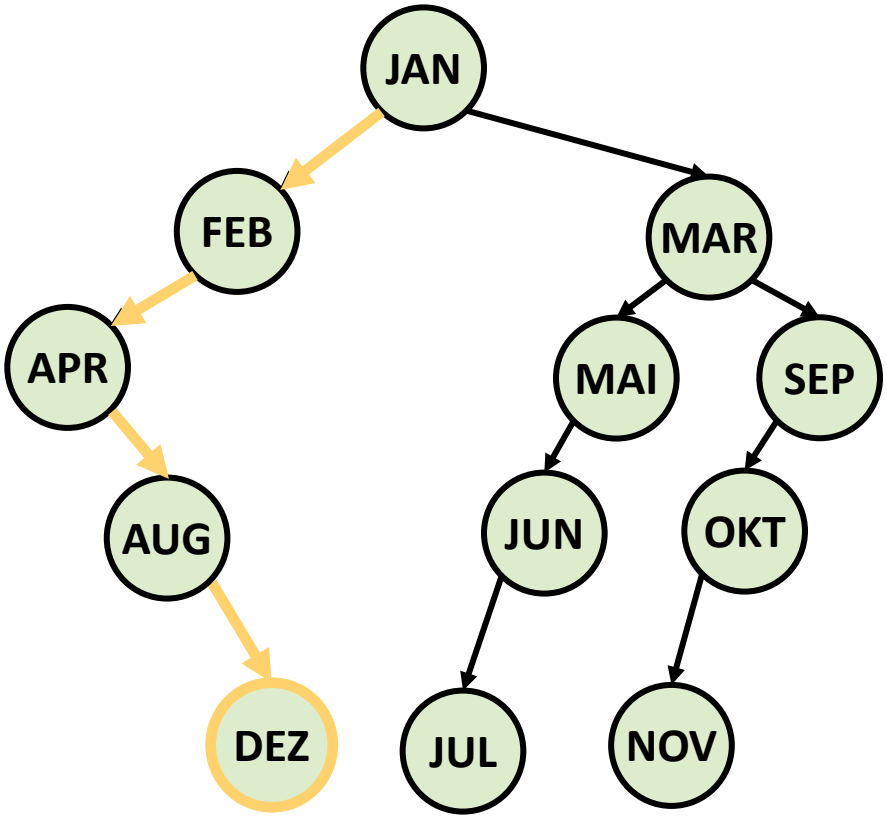
```
btreeptr insert(T* item, btreeptr b)
{
  btreeptr n;

  if (b == NULL) {
    n = (btreeptr)malloc(sizeof(btree));
    n->data = item;
    n->left = NULL; n->right = NULL;
    return n;
  }

  if (key(item) < key(b->data))
   b->left = insert(item,b->left);

  if (key(item) > key(b->data))
   b->right = insert(item,b->right);

  return b;
}
```



- Beispiel: Einfügen von „DEZ“

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

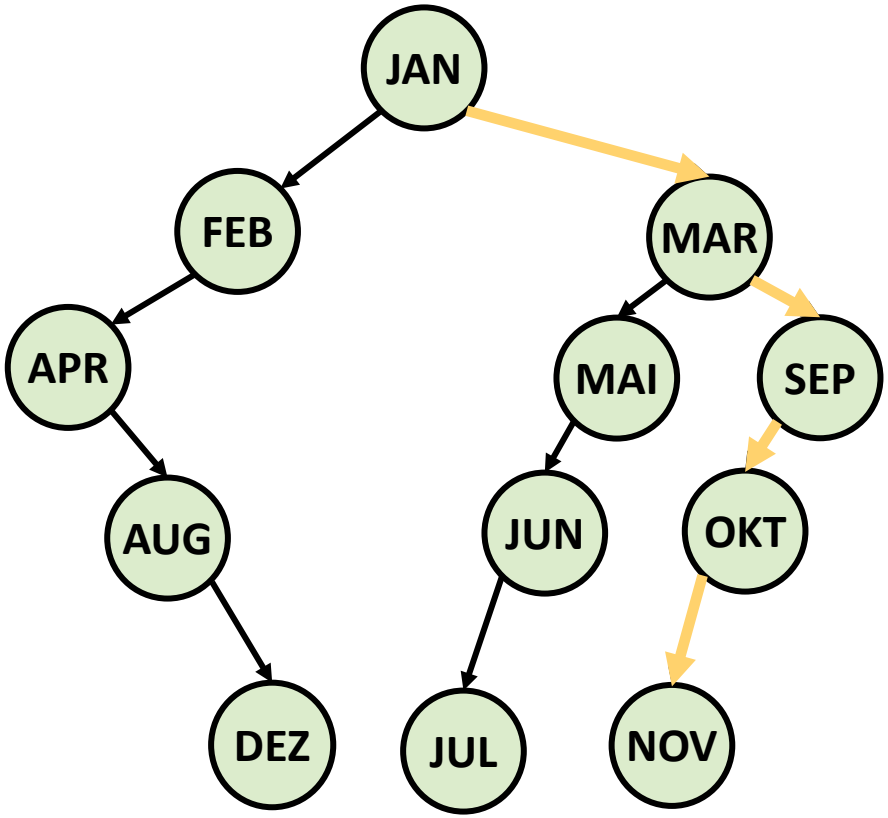
Suchen eines Elementes

```
T* search(T* item, btreeptr b)
{
  if (b == NULL) return NULL;

  if (key(item) == key(b->data))
    return b->data;

  if (key(item) < key(b->data))
    return search(item,b->left);

  if (key(item) > key(b->data))
    return search(item,b->right);
}
```

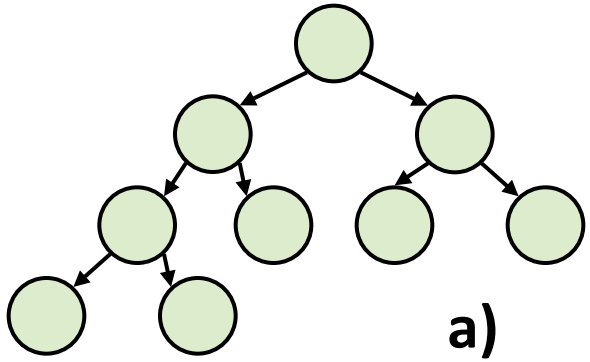


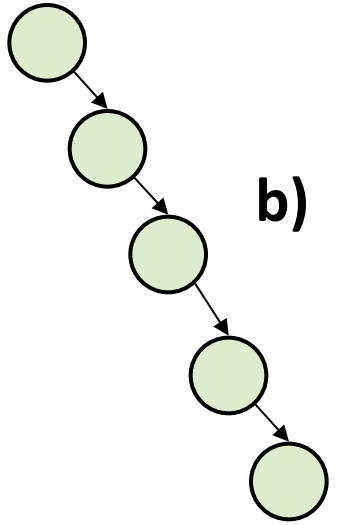
- **Beispiel**: Suchen von „NOV“
- **Aufwand**: Länge des Pfades von der Wurzel zum gesuchten Element (bzw. zu der Stelle, an der sich das Element befinden müsste)

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Laufzeitanalyse für binäre Suchbäume

- **Löschen eines Elementes**: gleiches Prinzip, jedoch etwas aufwendiger beim Löschen von inneren Baumknoten (Wiederherstellung der Baumstruktur notwendig)
- Aufwand bestimmt durch **max. Pfadlänge** im Baum
- Diese ist wiederum bestimmt durch die **Reihenfolge** der Einfügeoperationen (siehe Bsp. Monatsnamen)
- Extremfälle:
  a) **ausgeglichener Baum**: Höhe der Teilbäume unterscheidet sich max. um 1, auf jeder Ebene verdoppelt sich die Anzahl der Knoten, daher Suchzeit $\mathcal{O}(\log n)$ bei n Knoten
  b) **entarteter Baum**: wie Liste, d.h. $\mathcal{O}(n)$





© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Laufzeitanalyse für binäre Suchbäume

- Man kann zeigen: Bei einem aus **zufälligen Schlüsseln** aufgebauten binären Suchbaum liegt die Suchzeit im *average case* bei $\mathcal{O}(\log n)$
- **Jedoch**: nicht garantiert, *worst case* ist stets $\mathcal{O}(n)$
- Ausgeglichene binäre Suchbäume (**AVL-Bäume**): $\mathcal{O}(\log n)$ *worst case* für Suchen, Einfügen, Löschen garantiert
- Weitere Suchbaumstrukturen für **spezielle Anwendungen**
- Z.B. Verwaltung großer Mengen in **externen Dateien** (Files)
  - **Zugriffszeit** = Zeit für Positionierung im File + Zeit für Übertragung der Daten in den Hauptspeicher
  - Daher Übertragung von großen **zusammenhängenden Datenblöcken** statt einzelner Knoten sinnvoll
  - Datenstruktur: **B-Bäume**

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

10. Graphen

10.1 Definition und Grapheigenschaften

10.2 Berechnung kürzester Pfade

10.3 Datenstrukturen zur Graphrepräsentation

10.4 Graphdurchläufe

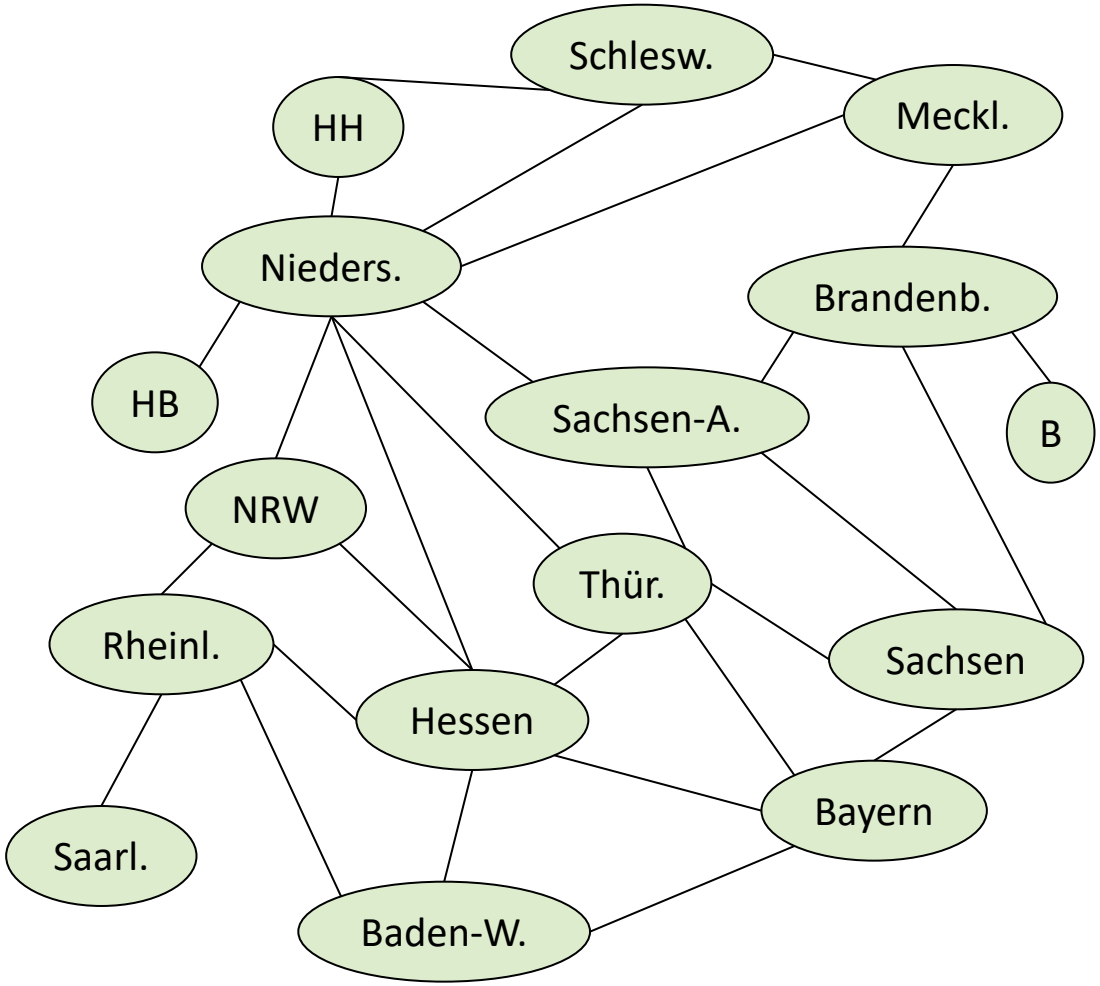
10.5 Spannbäume

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Graphen

- Graph: graphische Darstellung einer **Menge von Objekten und deren Beziehungen** (d.h. Relation)
- Bsp.: **topologische Information** einer Landkarte





© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Graphen

Def.: Eine (binäre) **Relation** R ist eine Teilmenge des kartesischen Produktes zweier Mengen: $R \subseteq A \times B$

- Bei Graphen meist $R \subseteq A \times A$ für gegebene Menge A ("**Knoten**")
- "aRb" als Schreibweise für $(a, b) \in R$, "**Kante**" zwischen a und b im Graphen
- Spezialfall symmetrische Relation: **aRb** $\Rightarrow$ **bRa**

Def.: Ein **(gerichteter) Graph** $G = (V, E)$ besteht aus
einer Menge V von **Knoten** (*vertices*) und
einer Menge E von **Kanten** (*edges*) mit $E \subseteq V \times V$.

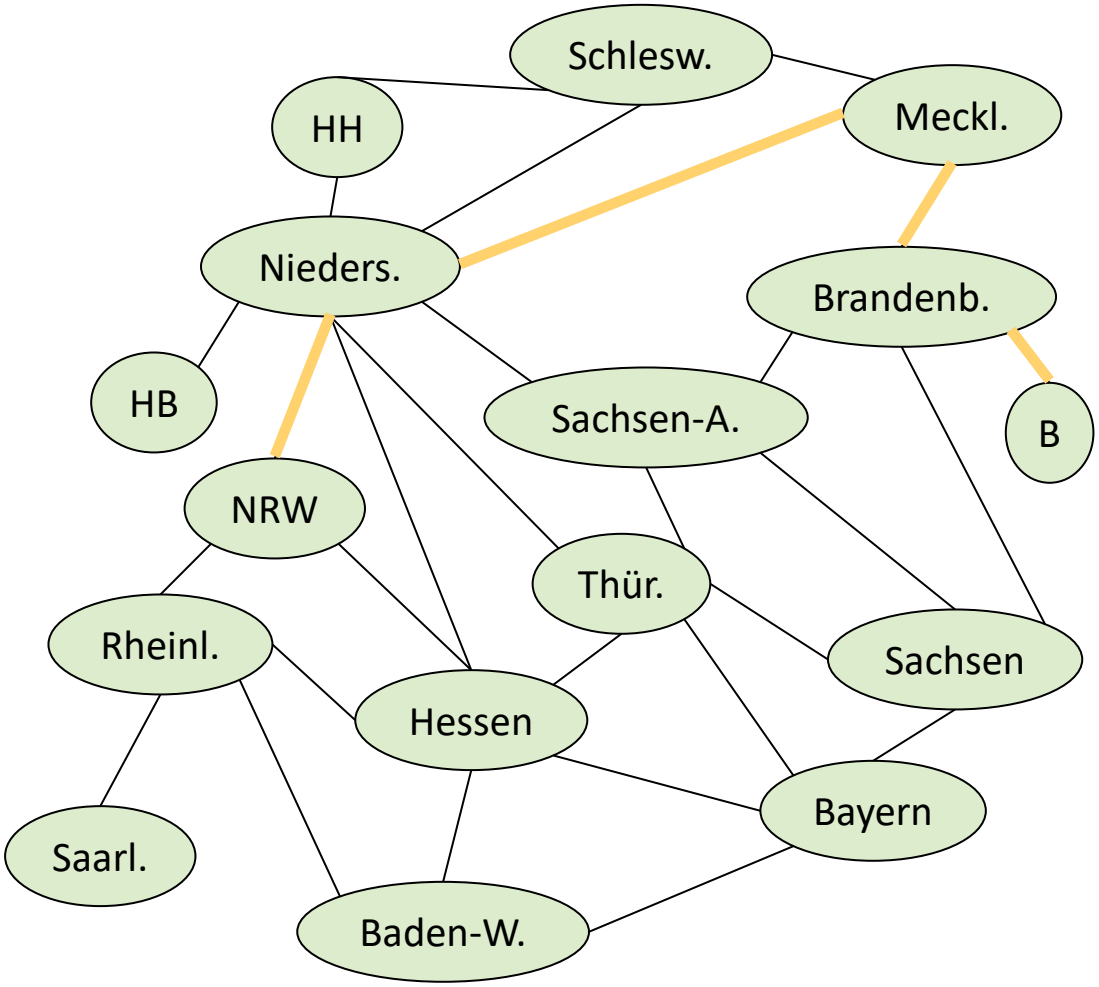
Def.: Gilt für alle Kanten (u, v) eines Graphen $G = (V, E)$
$(u, v) \in E \Rightarrow (v, u) \in E$
so heißt G **ungerichtet**. **Schreibweise**: $\{u, v\} \in E$.

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Pfade in Graphen

Def.: Eine Folge $(v_1, \ldots, v_n)$ von Knoten eines Graphen $G = (V, E)$ heißt **Pfad**, falls für alle $i = 1 \ldots n-1$: $\{v_i, v_{i+1}\} \in E$.

- Bsp. Landkarte: Knoten = Bundesländer
- **Ungerichteter Graph**, da Nachbarschafts-Beziehung symmetrisch
- **Pfad** = „Reiseweg“ von einem Land in ein anderes
- Im folgenden: Betrachtung **ungerichteter** Graphen

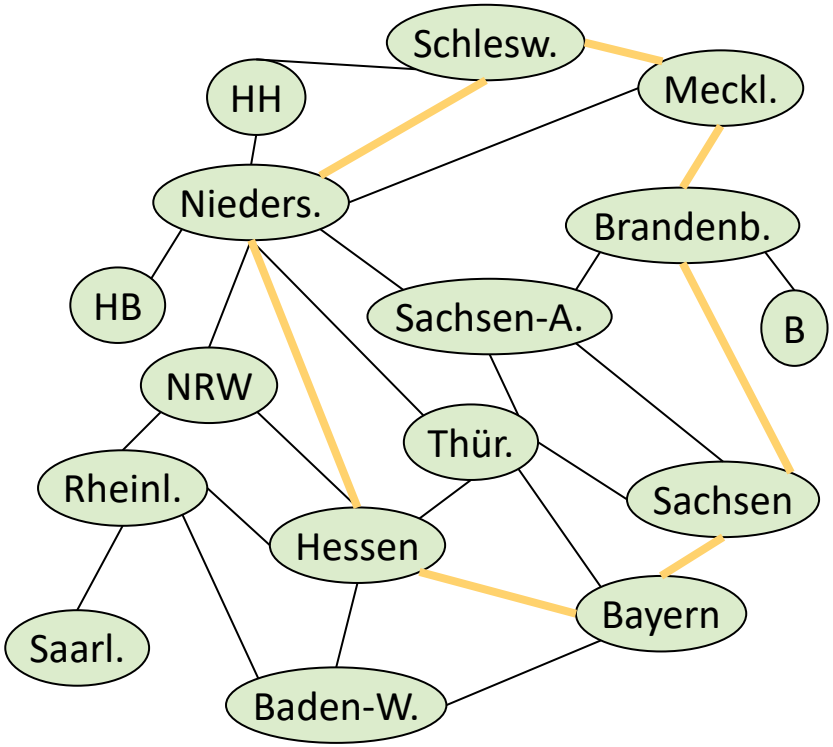


Zusammenhang und Zyklen

Def.: Ein Graph $G = (V, E)$ heißt **zusammenhängend**, falls zwischen zwei beliebigen Knoten $u, v \in V$ mindestens ein Pfad existiert.

Def.: Ein Graph $G = (V, E)$ heißt **zyklenfrei**, falls zwischen zwei beliebigen Knoten $u, v \in V$ höchstens ein Pfad existiert.

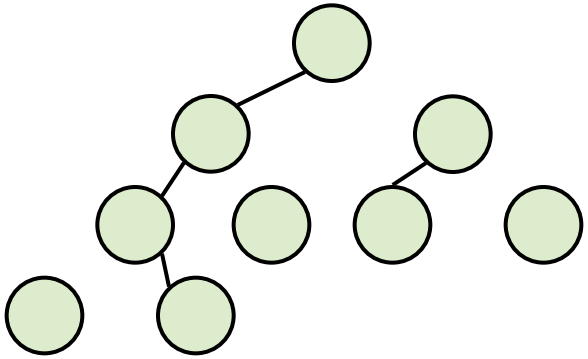
- Bsp. Landkarte:
  - zusammenhängend
  - nicht zyklenfrei



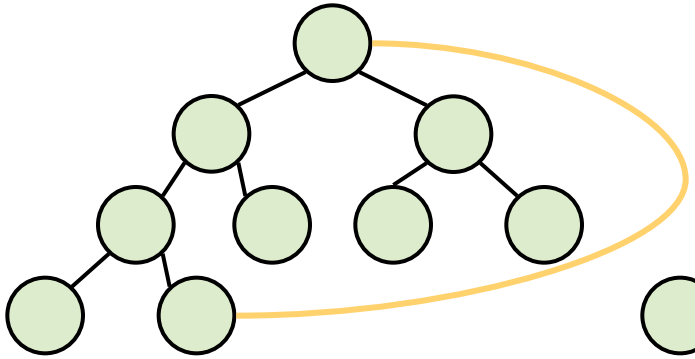
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Graphen und Bäume, Kantengewichte

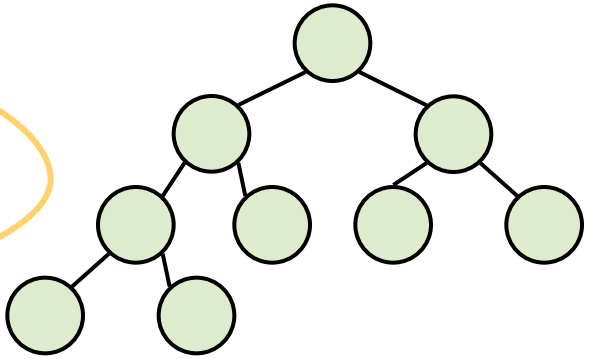
Def.: Ein zusammenhängender, zyklenfreier Graph $G = (V, E)$ heißt **Baum**.



Nicht zusammenhängend

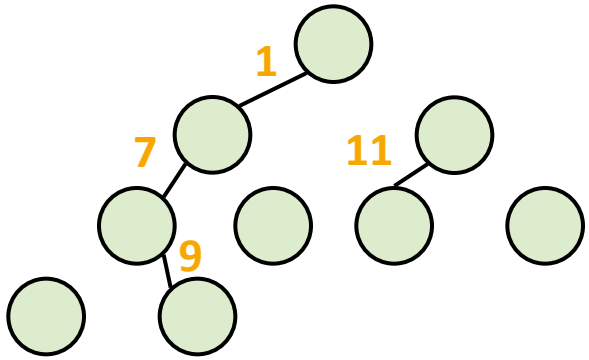


Nicht zyklenfrei



Baum

Def.: Ein Graph $G = (V, E, w)$ mit einer Abbildung $w\colon E \to \mathbb{R}$ heißt **(kanten)gewichteter Graph**.



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

10. Graphen

10.1 Definition und Grapheigenschaften

10.2 Berechnung kürzester Pfade

10.3 Datenstrukturen zur Graphrepräsentation

10.4 Graphdurchläufe

10.5 Spannbäume

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel-Anwendung: Navigationssystem





https://ndla.no/

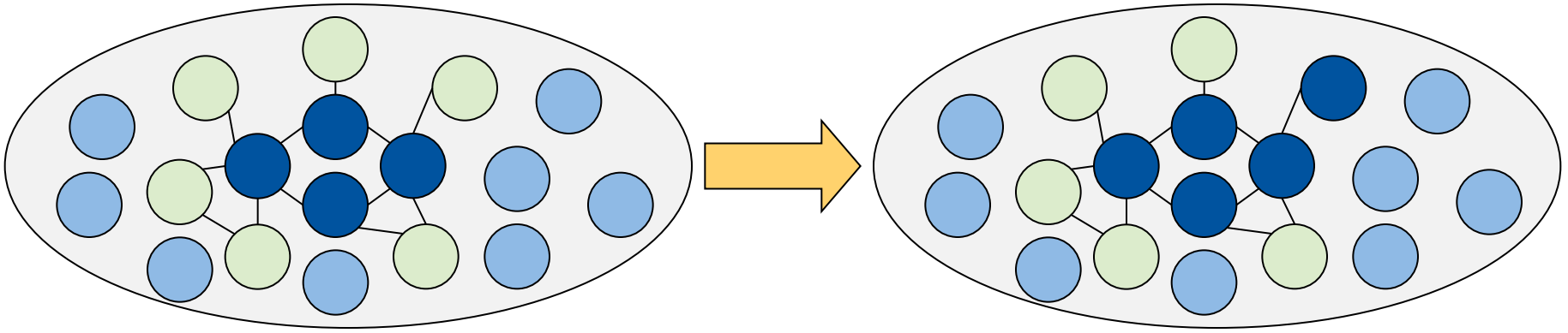
- **Knoten** = Geographische Orte
- **Kanten** = Straßenverbindungen
- **Kantengewichte** = Länge bzw. Fahrtdauer
- **Gesucht**:

  kürzester Pfad von A nach B gemäß Summe der Kantengewichte

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Bestimmung kürzester Pfade

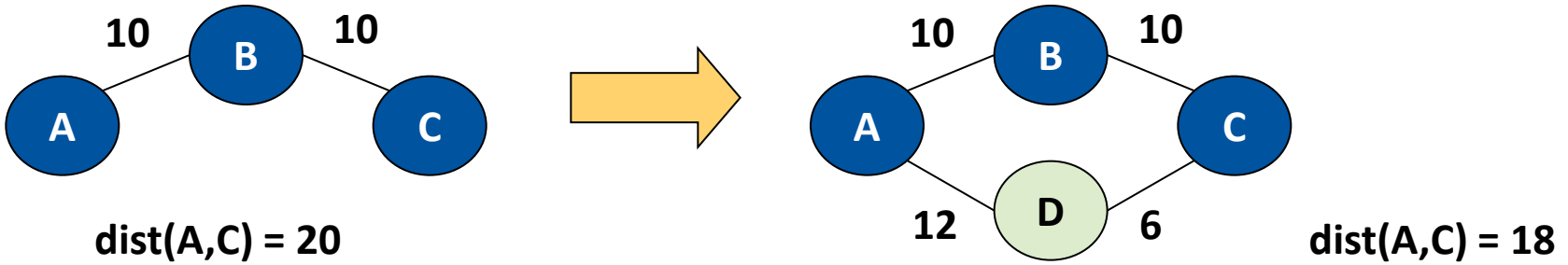
- **Dijkstra-Algorithmus** (*single source shortest path*)
- Bestimmung der kürzesten Pfade von einem **Startknoten v_0** aus zu allen anderen Graphknoten
- Idee:
  - In jedem Schritt Betrachtung einer „**Nachbarschaft**“ N von v_0, innerhalb derer die kürzesten Pfade zu v_0 bereits bekannt sind
  - Schrittweise **Erweiterung von N** um den jeweils am nächsten an N liegenden Knoten



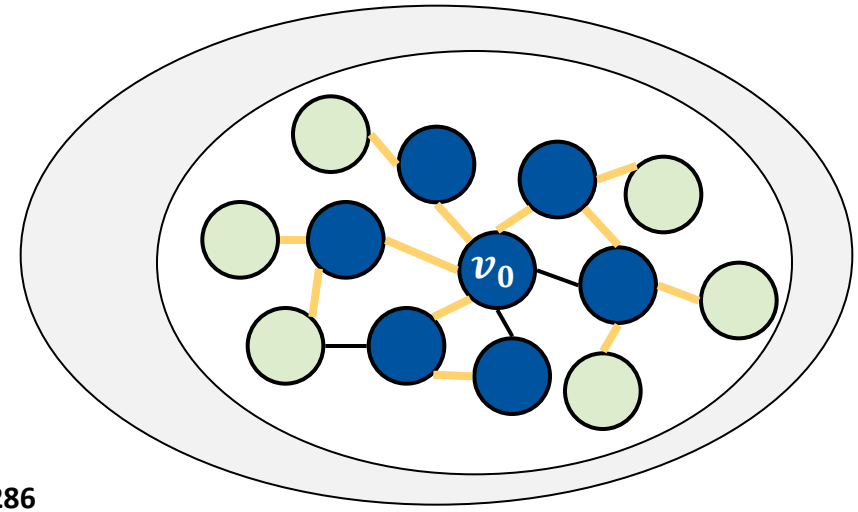
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

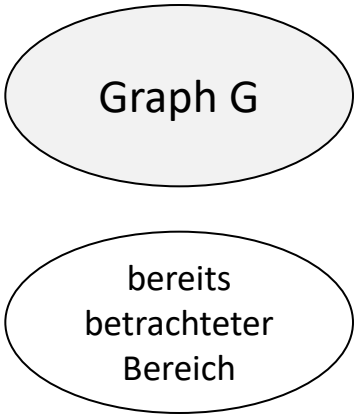
Ablauf des Dijkstra-Algorithmus

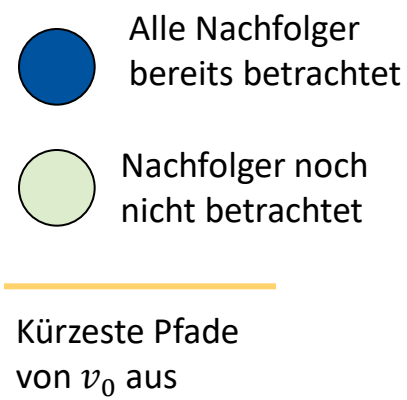
- **Bisher bekannter** min. Abstand *dist(u,v)* zwischen zwei Knoten kann durch neue Zwischenknoten **verkürzt** werden



Def.: Ist $G = (V, E)$ ein Graph und $\{u, v\} \in E$, so heißt v **Nachfolger** von u, und u heißt **Vorgänger** von v.







© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Dijkstra-Algorithmus

- **Pseudo-Code**: abstrakter Code für Algorithmus, nicht direkt compilier- und ausführbar!
- Vorauss.: Datenstruktur für **Graphen** (*graph*) und **Mengen** (*set*) verfügbar
- *graph* unterstützt Zugriff auf Knoten und Kanten (s. später)
- *set* unterstützt übliche Mengenoperationen
- Variablen:
  - `dist[v]` bezeichnet den aktuell min. Abstand von Knoten v zum Startknoten
  - `cost(u,v)` bezeichnet Gewichtung der Kante $\{u, v\}$
  - Mengen **BLUE, GREEN und ORANGE** bezeichnen blaue und grüne Knoten sowie orangene Kanten
- Orangene Kanten bilden stets einen **Baum von aktuell kürzesten Pfaden** vom Startknoten aus

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Dijkstra-Algorithmus

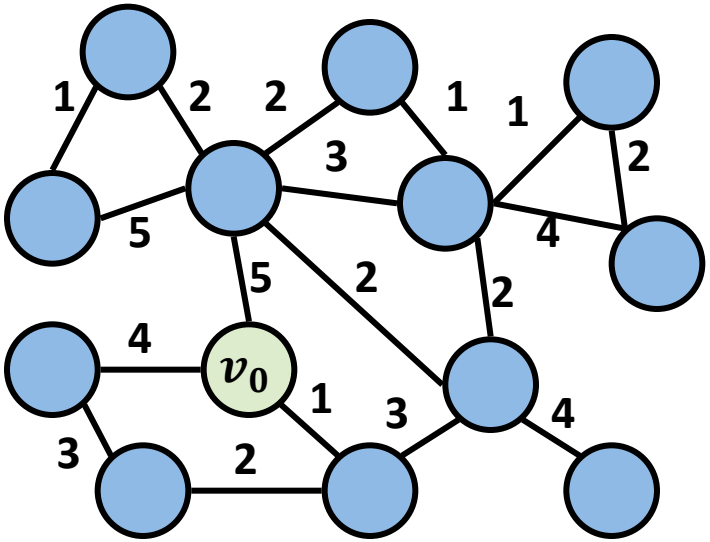
```
void shortest_path(graph G = (V = {v0,...,vn},E))
{
  set BLUE = {}, GREEN = {v0}, ORANGE = {}; dist[v0] = 0;

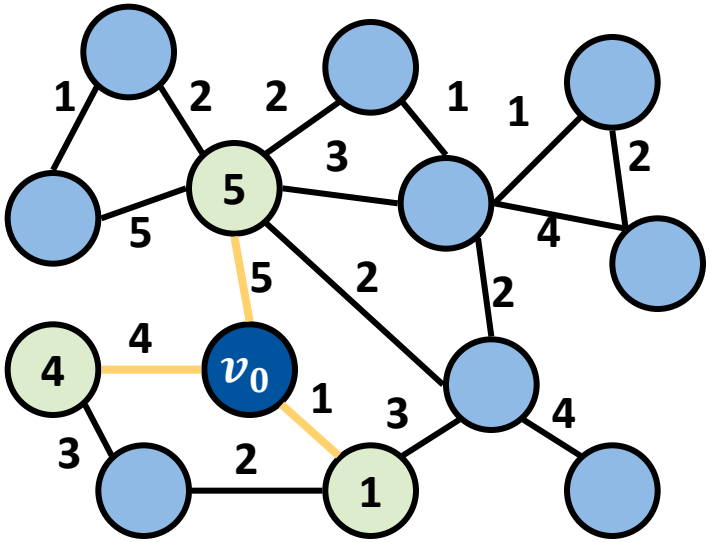
  while (GREEN != {}) { // solange noch unbearbeitete Knoten
    v = MinDist(GREEN); // v ∈ GREEN mit min. dist-Wert
    Insert(v, BLUE);   // BLUE = BLUE ∪ {v}
    Delete(v, GREEN);   // GREEN = GREEN \ {v}

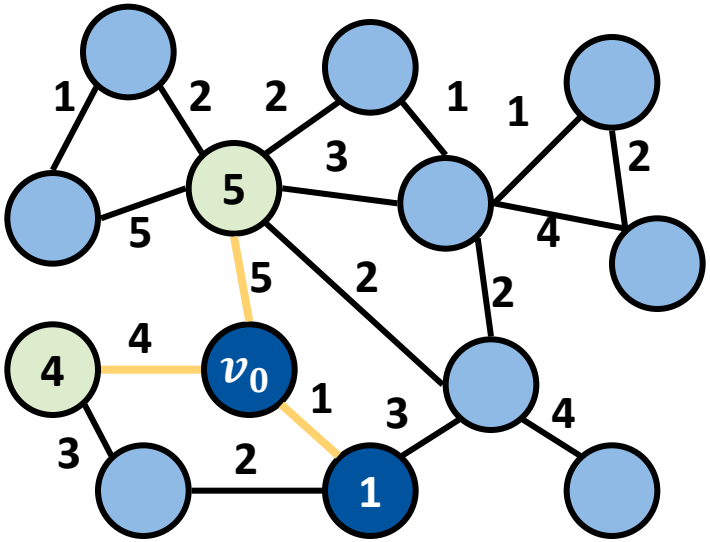
    for (u ∈ Succ(v)) { // für alle Nachfolger u von v
      if (!(u ∈ GREEN ∪ BLUE)) { // neuer Knoten erreicht
        Insert({u,v}, ORANGE);
        Insert(u, GREEN);
        dist[u] = dist[v] + cost(u,v);
      } else if (u ∈ GREEN) {    // u erneut erreicht
        if (dist[v] + cost(u,v) < dist[u]) {
          Insert({u,v}, ORANGE);
          e = PreviousEdge(u);   // vorher orangene Kante zu u
          Delete(e, ORANGE);     // ORANGE = ORANGE \ {e}
          dist[u] = dist[v] + cost(u,v);
        }
      }
    }
  }
}
```

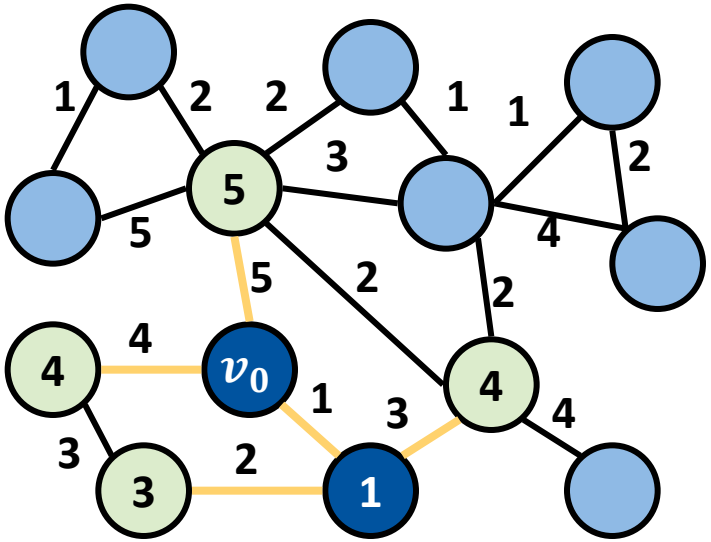

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel



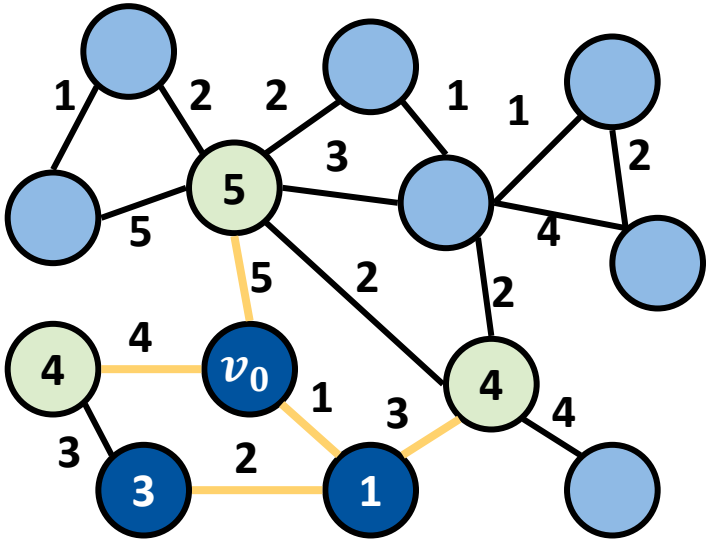


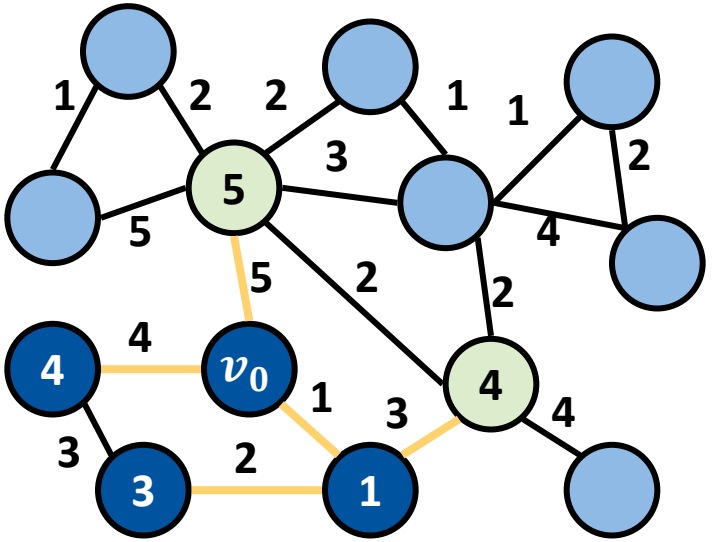


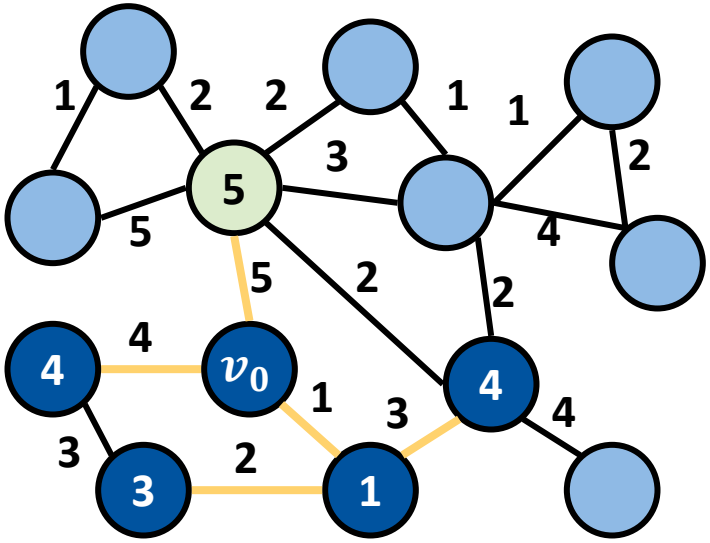


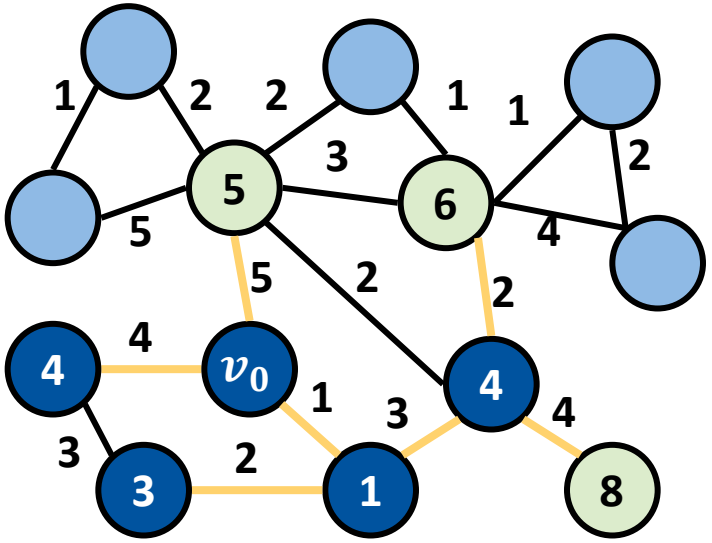
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel (Fortsetzung)



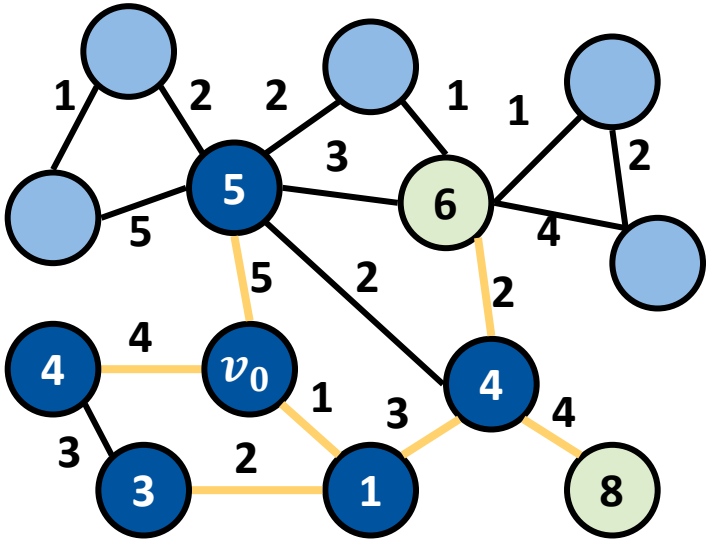


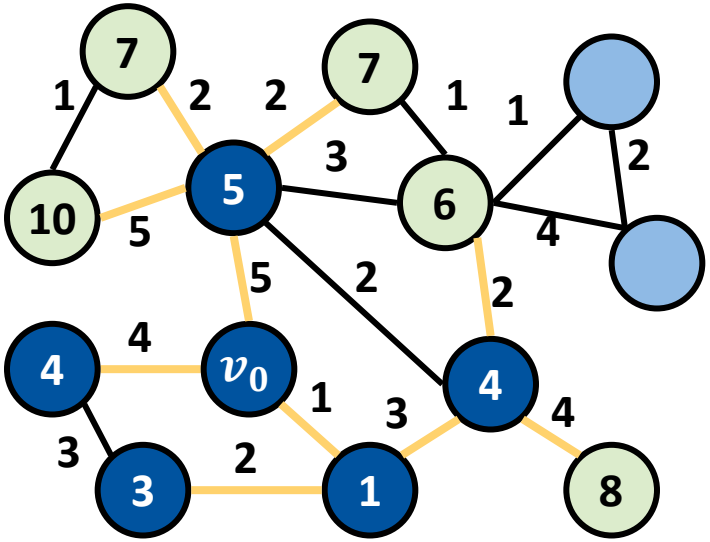


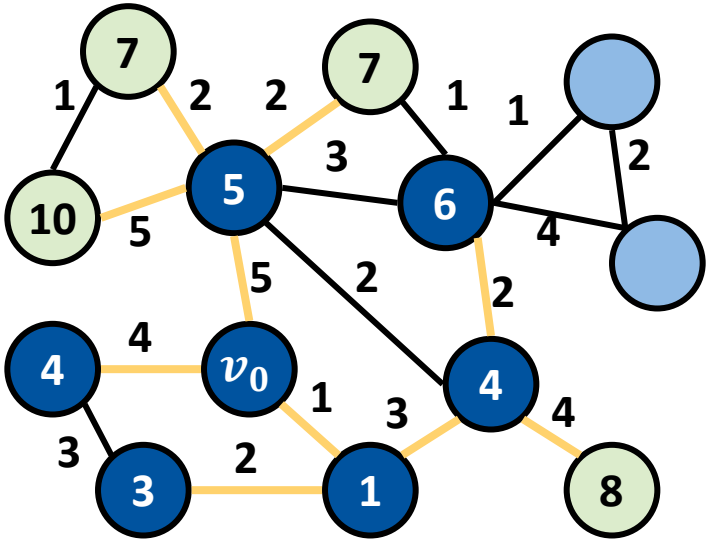


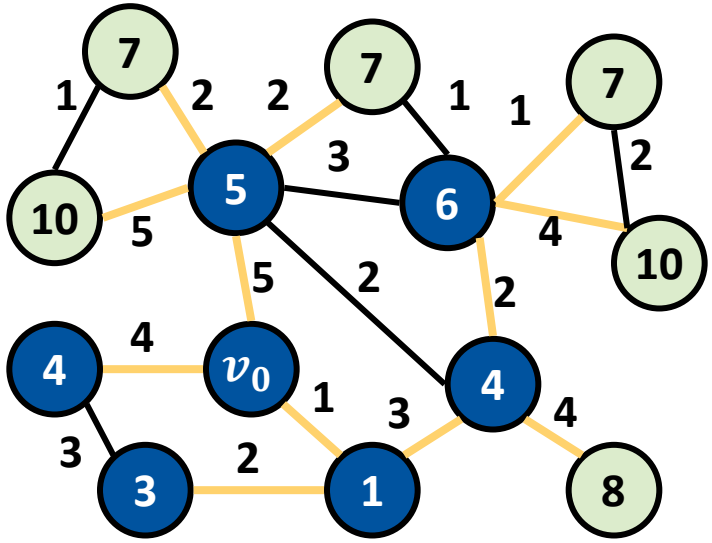
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel (Fortsetzung)



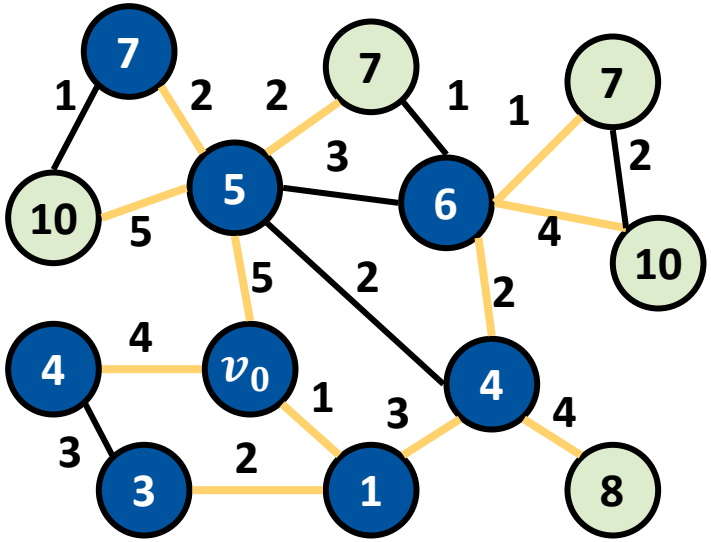


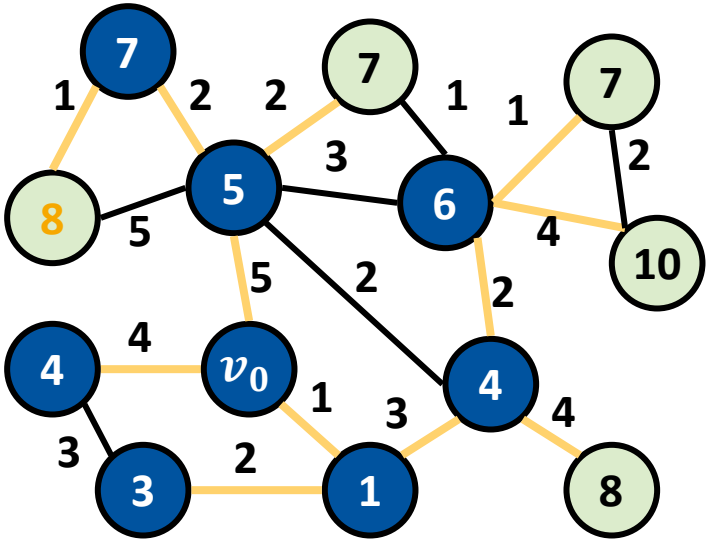


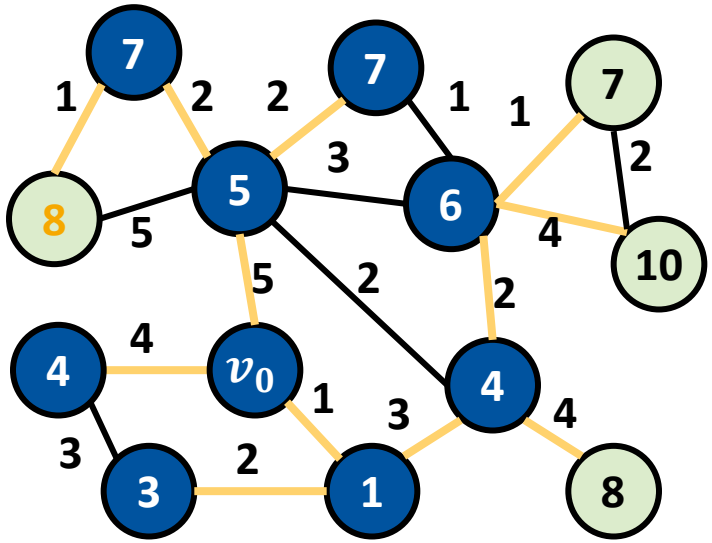


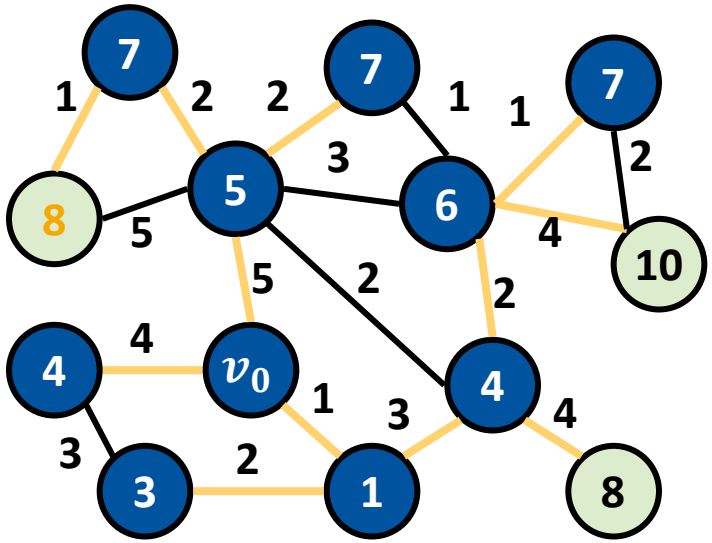
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel (Fortsetzung)



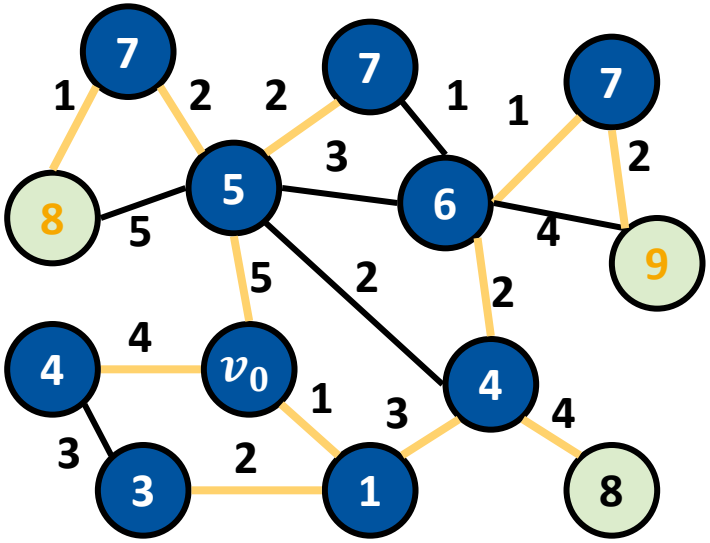




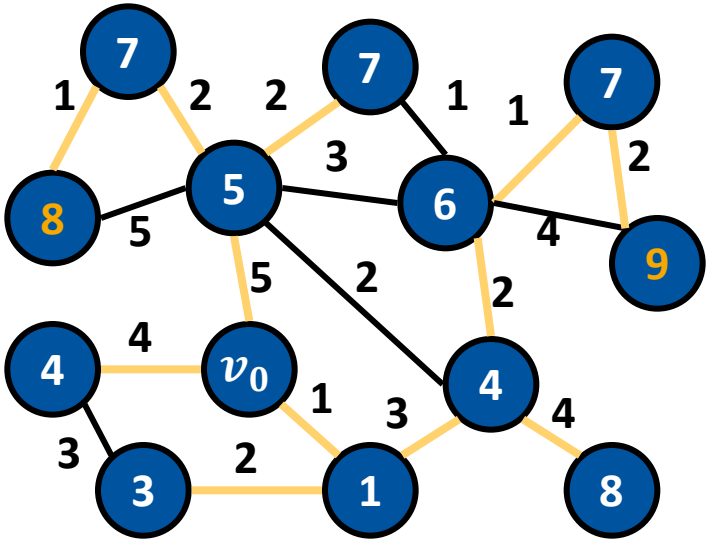


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel (Fortsetzung), Laufzeit



...



- In jedem Durchlauf der **while-Schleife** wird ein Knoten „blau“
- Aufwand **für jeden Knoten**:
  - Bestimmung des bzgl. *dist* min. „grünen“ Knotens: $\mathcal{O}(|V|)$
  - Behandlung aller $\mathcal{O}(|V|)$ Nachfolger: jeweils $\mathcal{O}(1)$
- Gesamt: $\boldsymbol{\mathcal{O}(|V|^2)}$
- **Geschicktere** Implementierung möglich: $\mathcal{O}(|E| \log|V|)$, d.h. besser im Fall $|E| \ll |V|^2$ ($|E|$ stets beschränkt durch $|V|^2$)

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

10. Graphen

10.1 Definition und Grapheigenschaften
10.2 Berechnung kürzester Pfade
10.3 Datenstrukturen zur Graphrepräsentation
10.4 Graphdurchläufe
10.5 Spannbäume

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Graphrepräsentation

- Datenstrukturen zur **internen Speicherung** von Graphen
- Gebräuchlich vor allem: **Adjazenzmatrix** und **Adjazenzlisten**
- Existiert zwischen zwei Graphknoten u und v eine Kante $e = \{u, v\}$, so heißen u und v **adjazent**

Def.: Ist $G = (V, E)$ ein Graph mit Knotenmenge $V = \{v_1, \ldots, v_n\}$, so heißt die $(n \times n)$-Matrix $A = (a_{ij})$ mit

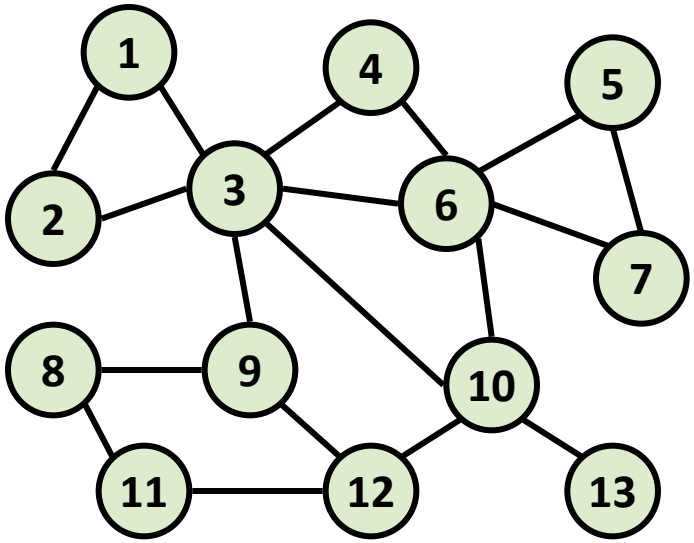
$$a_{ij} = 1, \quad \text{falls } \{v_i, v_j\} \in E$$
$$a_{ij} = 0, \quad \text{sonst}$$

Adjazenzmatrix zu G.

- $a_{ij} = 1$ zeigt an, dass zwischen zwei Knoten v_i und v_j **eine Kante** existiert
- $a_{ij} = 0$ zeigt an, dass **keine Kante** vorhanden ist
- Knoten werden **implizit** gespeichert

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel für Adjazenzmatrix



	1	2	3	4	5	6	7	8	9	10	11	12	13
1	0	1	1	0	0	0	0	0	0	0	0	0	0
2	1	0	1	0	0	0	0	0	0	0	0	0	0
3	1	1	0	1	0	1	0	0	1	1	0	0	0
4	0	0	1	0	0	1	0	0	0	0	0	0	0
5	0	0	0	0	0	1	1	0	0	0	0	0	0
6	0	0	1	1	1	0	1	0	0	1	0	0	0
7	0	0	0	0	1	1	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	1	0	1	0	0
9	0	0	1	0	0	0	0	1	0	0	0	1	0
10	0	0	1	0	0	1	0	0	0	0	0	1	1
11	0	0	0	0	0	0	0	1	0	0	0	1	0
12	0	0	0	0	0	0	0	0	1	1	1	0	0
13	0	0	0	0	0	0	0	0	0	1	0	0	0

- Bei **kantengewichteten** Graphen:
  - Kantengewicht statt 0/1 in Matrix eintragen
  - spezieller Wert (z.B. ∞), falls keine Kante vorhanden

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Eigenschaften und Implementierung

- **Vorteil**: Adjazenz zweier Knoten kann in Zeit $\mathcal{O}(1)$ geprüft werden
- **Nachteil**: Hoher Platzbedarf ($\mathcal{O}(|V|^2)$), unabhängig von $|E|$
- C-Implementierung:

```
#define N 100

int adj_matrix[N][N];

int adjacent(int i, int j)
{
  return adj_matrix[i][j] == 1 ? 1 : 0;
}
```

- **Bedingter Ausdruck** in C:

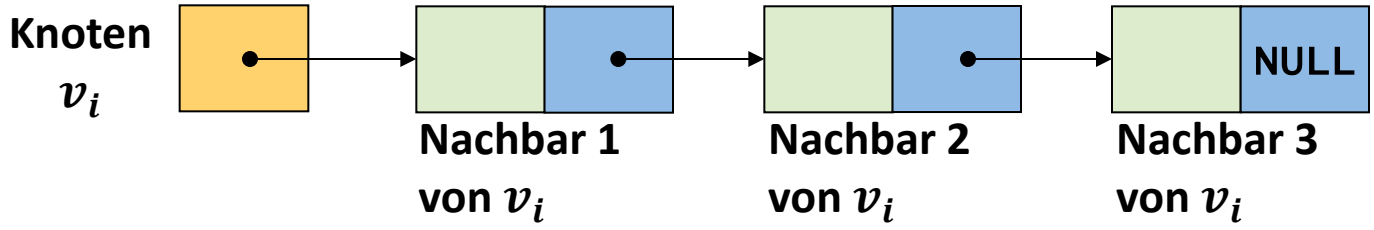
  Bedingung „?“ `then`-*Ausdruck* „`:`“ `else`-*Ausdruck*

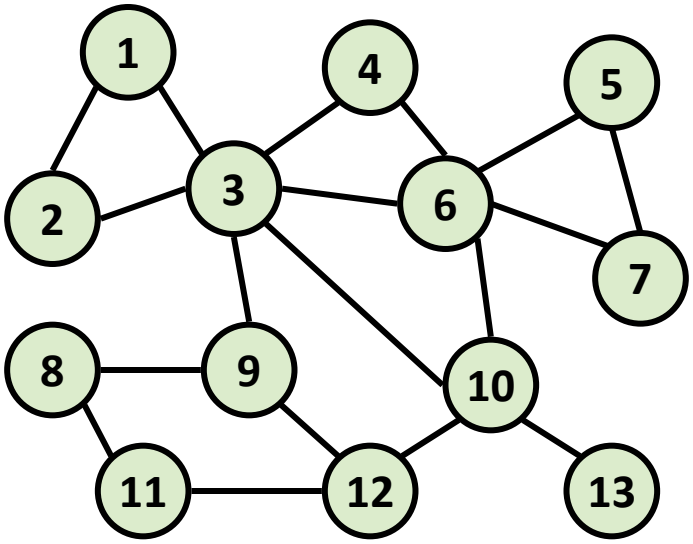
- Kürzere Schreibweise als `if-then-else`

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Adjazenzlisten

- Für jeden Knoten **Liste seiner Nachbarn** speichern
- **Array** von verketteten **Listen**





1: 2, 3
2: 1, 3
3: 1, 2, 4, 6, 9, 10
4: 3, 6
5: 6, 7
6: 3, 4, 5, 7, 10
7: 5, 6
8: 9, 11
9: 3, 8, 12
10: 3, 6, 12, 13
11: 8, 12
12: 9, 10, 11
13: 10

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Eigenschaften und Implementierung

- **Vorteil**: Speicherplatzbedarf nur $\mathcal{O}(|V| + |E|)$
- **Nachteil**: Test auf Adjazenz erfordert im *worst case* $\mathcal{O}(|V|)$

```
#define N 100

List adj_list[N];

int adjacent(int i, int j)
{
  List L;

  L = adj_list[i];
  return IsIn(j, L);
}
```

- Auch **Mischformen** und **alternative Graphrepräsentationen** möglich
- Auswahl je nach **Häufigkeit** der gewünschten Graphoperationen

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

10. Graphen

10.1 Definition und Grapheigenschaften
10.2 Berechnung kürzester Pfade
10.3 Datenstrukturen zur Graphrepräsentation
10.4 Graphdurchläufe
10.5 Spannbäume

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Graphdurchläufe

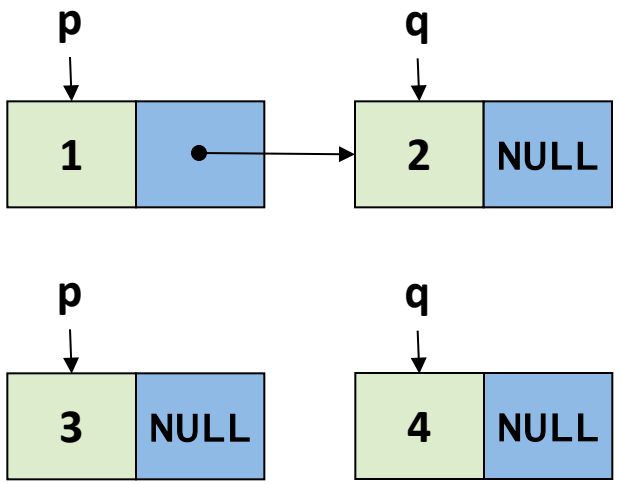
- **Gesucht**: Algorithmus, der alle Knoten eines Graphen der Reihe nach „besucht“
- Anwendungen u.a.:
  - Systematisches **Bearbeiten** aller Graphknoten
  - Prüfen ob Graph **zusammenhängend** ist
- Beispiel: **garbage collection**
- Aufsammeln und Freigabe von nicht mehr benötigtem dynamisch angeforderten Heap-Speicher (*garbage*)
- Als **Graph-Modell**:
  - Speicherblöcke = Knoten
  - Verweis (Zeiger) auf Speicherblock = Kante
- **Idee**: Block von Programmvariablen aus nicht mehr erreichbar
  → Block nicht mehr benötigt, daher Freigabe

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Entstehung von garbage

```
void f(T* item)
{
  nodeptr p,q;

  p = newnode(item);
  q = newnode(item);
  p->next = q;
  p = newnode(item);
  q = newnode(item);
}
```



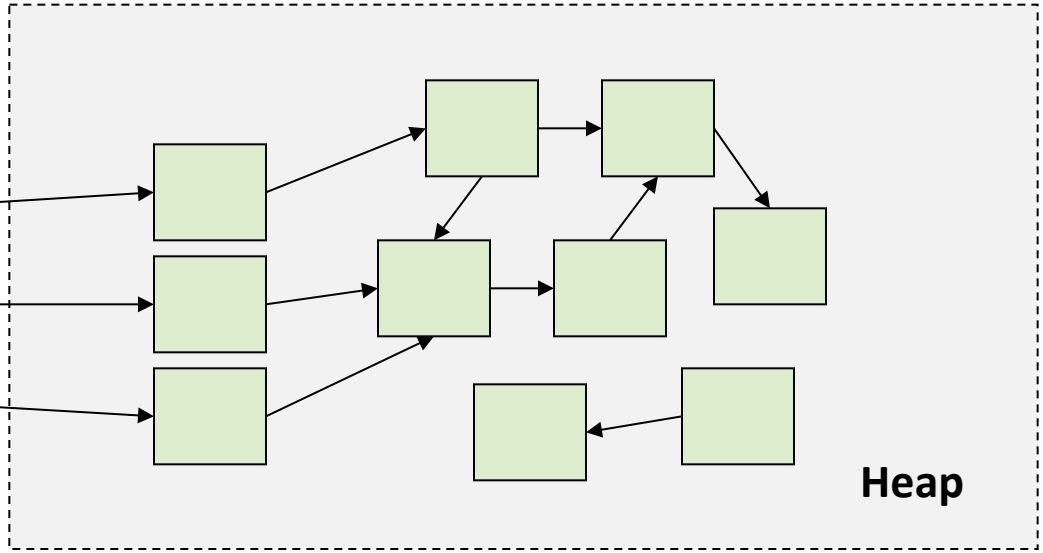
- Blöcke 1 und 2 anschließend **nicht mehr erreichbar**
- Jedoch nicht als „frei“ markiert, da nicht mittels `free` explizit **freigegeben**
- Falls `malloc` irgendwann im Programmverlauf **keinen freien Speicherplatz** mehr liefert, so muss eine *garbage collection* durchgeführt werden
- **Besserer Programmierstil**: immer Verwendung von `free`

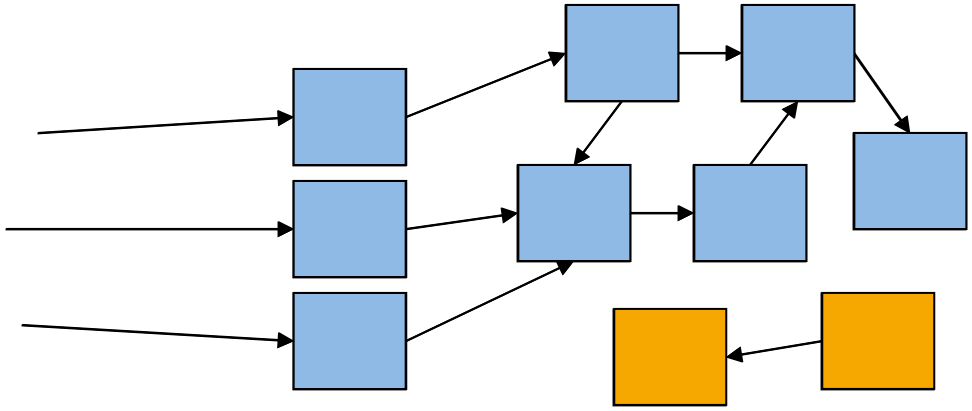
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Garbage collection

Im Programm deklarierte Zeigervariablen

```
p = ...
q = ...
r = ...
```



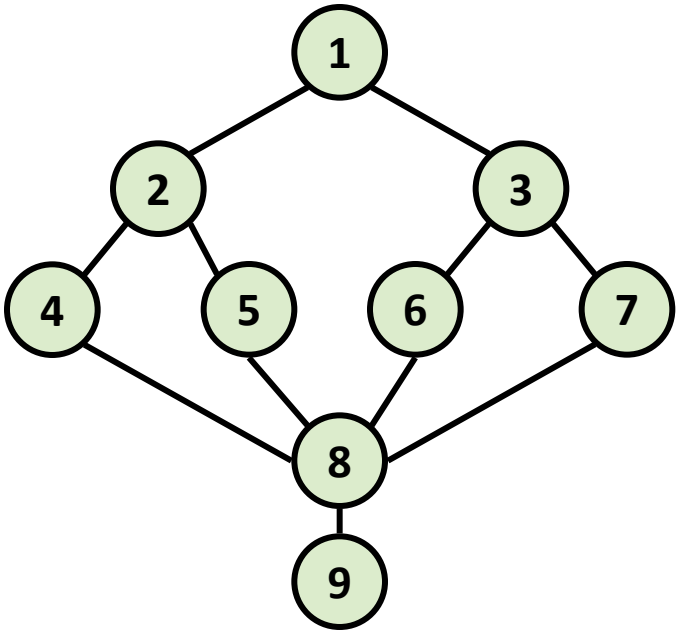


Nicht erreichte Knoten freigeben

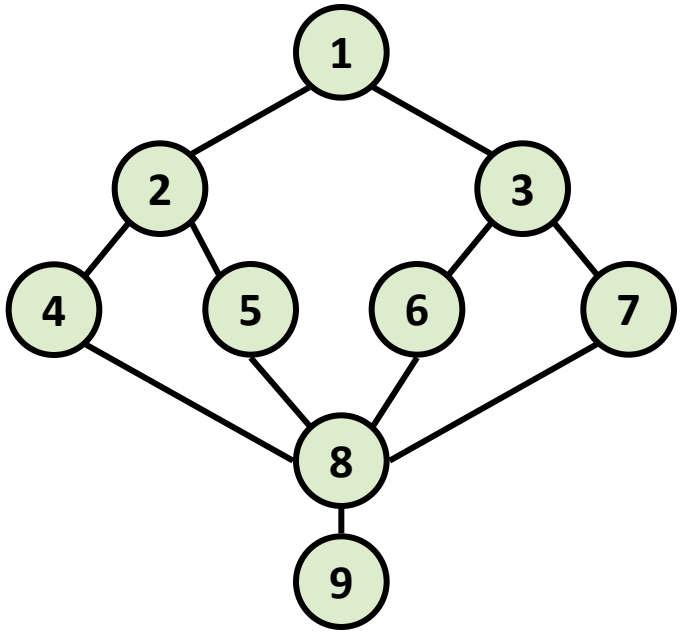
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Graphdurchläufe

- Prinzipiell zwei Möglichkeiten für Graphdurchlauf:
  - **Depth First Search** (DFS, **Tiefensuche**): ausgehend vom Startknoten möglichst lange Pfade verfolgen, oder
  - **Breadth First Search** (BFS, **Breitensuche**): ausgehend vom Startknoten weitere Knoten in der Reihenfolge wachsenden Abstands vom Startpunkt besuchen



DFS
1
2
4
8
5
6
3
7
9



BFS
1
2
3
4
5
6
7
8
9

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

DFS - Depth first search

- Algorithmus als **Pseudo-Code**
- Vorauss.:
  - Graphimplementierung verfügbar
  - Knoten durch integer-Zahlen identifiziert

```
int visited[N]; // implizit mit 0 initialisiert

void dfs(int v) // DFS beginnend bei v
{ int w;
  visited[v] = 1; // markiere v als besucht
  process(v);     // Verarbeitung von v
  for ({v,w} ∈ E) // für alle Nachbarn w
  { // rekursiver Aufruf von dfs:
    if (!visited[w]) dfs(w);
  }
}

int main(void)
{
  int v;
  for (v = 0; v < N; v++)
    if (!visited[v]) dfs(v);

  return 0;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

BFS – Breadth First Search

- In BFS werden die Knoten „ebenenweise“ besucht
- Verwendung einer **Queue** (Kap. 8)
- Queue speichert **noch zu bearbeitende Knoten auf nächster Ebene**, während „Geschwisterknoten“ auf selber Ebene gerade bearbeitet werden

```
int visited[N];

void bfs(int v) // BFS ab Knoten v
{
  int w; Queue Q;
  visited[v] = 1;
  process(v);
  Put(v,Q);
  while (!IsEmpty(Q)) {
    v = Get(Q);
    for ({v,w} ∈ E) {
      if (!visited[w]) {
        visited[w] = 1;
        process(w);
        Put(w,Q);
      }
    }
  }
}

void main()
{
  int v;
  for (v = 0; v < N; v++)
    if (!visited[v]) bfs(v);
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Laufzeitanalyse

- Annahme: Graphdarstellung als **Adjazenzlisten**
- **DFS**:
  - Aufruf für alle Knoten aus `main`: $\mathcal{O}(|V|)$
  - Einmalige Bearbeitung aller Kanten in `for`: $\mathcal{O}(|E|)$
  - **Gesamt**: $\mathcal{O}(|V| + |E|)$
- **BFS**
  - Aufruf für alle Knoten aus `main`, jeder Knoten einmal in der Queue: $\mathcal{O}(|V|)$
  - Einmalige Bearbeitung aller Kanten in `for`: $\mathcal{O}(|E|)$
  - **Gesamt**: $\mathcal{O}(|V| + |E|)$

```
void dfs(int v)
{
  int w;
  visited[v] = 1;
  process(v);
  for ({v,w} ∈ E) {
    if (!visited[w]) dfs(w);
  }
}
```

```
void bfs(int v)
{
  int w; Queue Q;
  visited[v] = 1;
  process(v);
  Put(v,Q);
  while (!IsEmpty(Q)) {
    v = Get(Q);
    for ({v,w} ∈ E) {
      if (!visited[w]) {
        visited[w] = 1;
        process(w);
        Put(w,Q);
      }
    }
  }
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

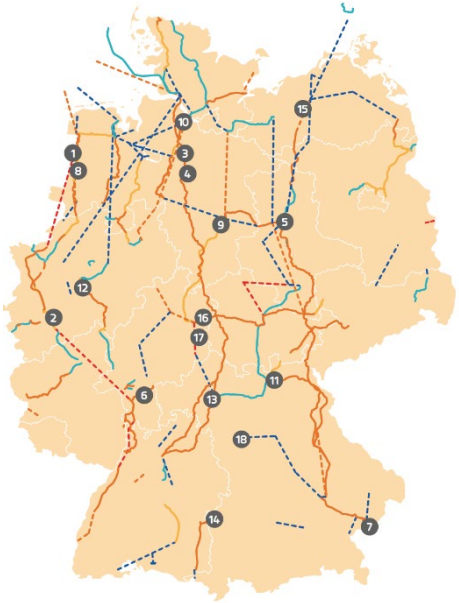
10. Graphen

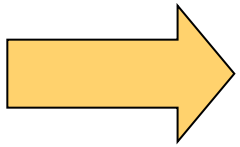
10.1 Definition und Grapheigenschaften
10.2 Berechnung kürzester Pfade
10.3 Datenstrukturen zur Graphrepräsentation
10.4 Graphdurchläufe
10.5 Spannbäume

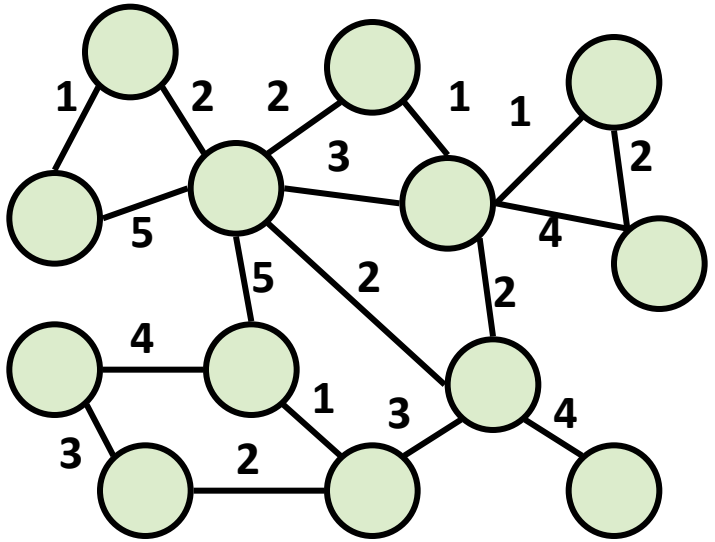
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Spannbäume

- Anwendung: z.B. **Leitungsplanung** zur Elektrifizierung entlegener Ortschaften
- **Graphmodellierung**:
  - Ortschaften = Knoten
  - Entfernungsmaß = Kantengewichte
- Zur **Vernetzung** müssen nicht alle möglichen Leitungen realisiert werden, es müssen nur alle Knoten erreicht werden
- Mit **möglichst geringen** Gesamt-Leitungskosten





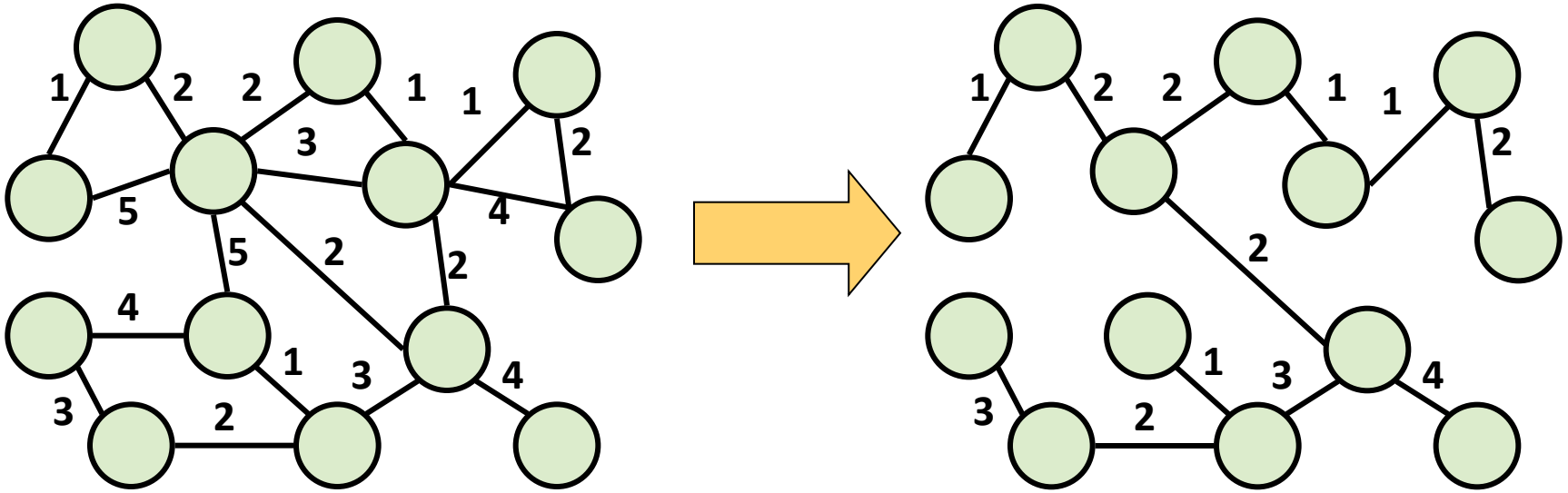


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Minimale Spannbäume

Def.: Ein **Spannbaum** in einem ungerichteten, zusammenhängenden und kantengewichteten Graphen $G = (V, E, w)$ ist ein zyklenfreier, zusammenhängender Teilgraph $G' = (V', E', w)$ mit $V' = V$ und $E' \subseteq E$.

Def.: Ein **minimaler Spannbaum** in einem ungerichteten, zusammenhängenden und kantengewichteten Graphen $G = (V, E, w)$ ist ein Spannbaum mit minimaler Kantengewichtssumme unter allen Spannbäumen zu G.



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Kruskal-Algorithmus

- **Idee**:
  - fortgesetzte Auswahl von Kanten mit geringem Gewicht
  - dabei Zyklen vermeiden

1. Beginne mit **leerem** Spannbaum T
2. Bilde Liste L der Graphkanten von G, **sortiert nach aufsteigendem Gewicht** (s. Sortieralgorithmen)
3. Betrachte jeweils **nächstgrößere** Kante e in L
4. Falls Einfügen von e in T einen **Zyklus** verursachen würde: e als ungültig betrachten und verwerfen
5. Sonst: e in T **einfügen**
6. Fortsetzen ab 3, bis T insgesamt $\boldsymbol{n-1}$ **Kanten** enthält

T bildet dann einen **minimalen Spannbaum** zu G

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Kruskal-Algorithmus

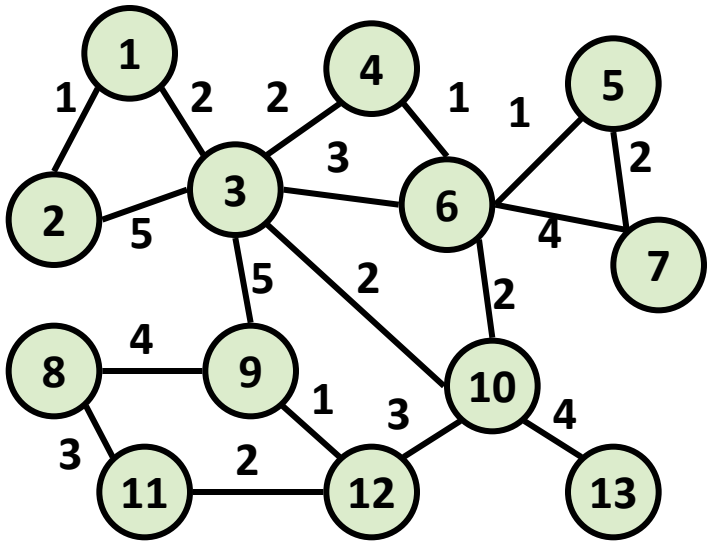
```
Graph MinSpanningTree(Graph G = (V,E))
{
  Graph T; List L; Edge e;
  T = {};
  L = Sort(E); // sortierte Kantenliste

  while (#Kanten in T < #Knoten in G – 1 && !IsEmpty(L)) {
    e = FirstElem(L); // nächst-günstigste Kante
    Delete(e,L);
    if (!Cycle(e,T)) { // kein Zyklus in T ∪ {e} ?
      T = T ∪ {e};
    }
  }

  if (#Kanten in T < #Knoten in G – 1) {
    /* keine verfügbaren Kanten mehr, d.h.
       G ist nicht zusammenhängend */
    printf("Fehler – kein Spannbaum möglich!");
  }
  return T;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

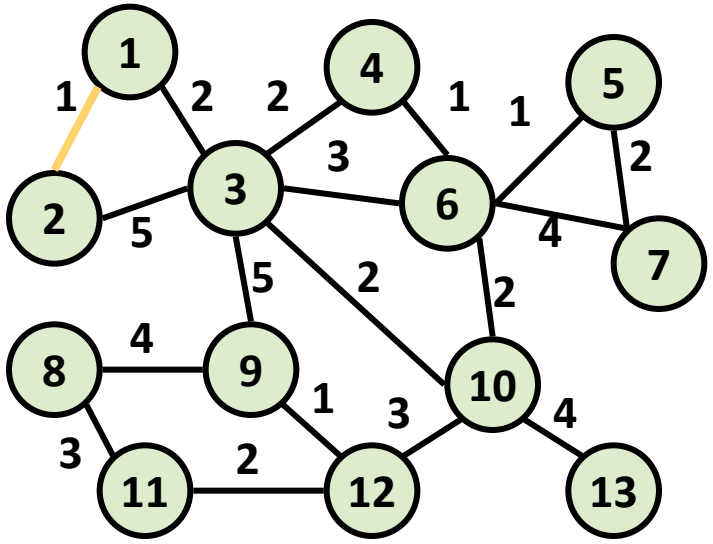
Beispiel



$T = \{\}$

Sortierte Kantenliste:

$L = \{\{1,2\},\{4,6\},\{5,6\},\{9,12\},\{1,3\},\{3,4\},$
$\{5,7\},\{3,10\},\{6,10\},\{11,12\},\{3,6\},$
$\{8,11\},\{10,12\},\{6,7\},\{8,9\},\{10,13\},$
$\{2,3\},\{3,9\}\}$

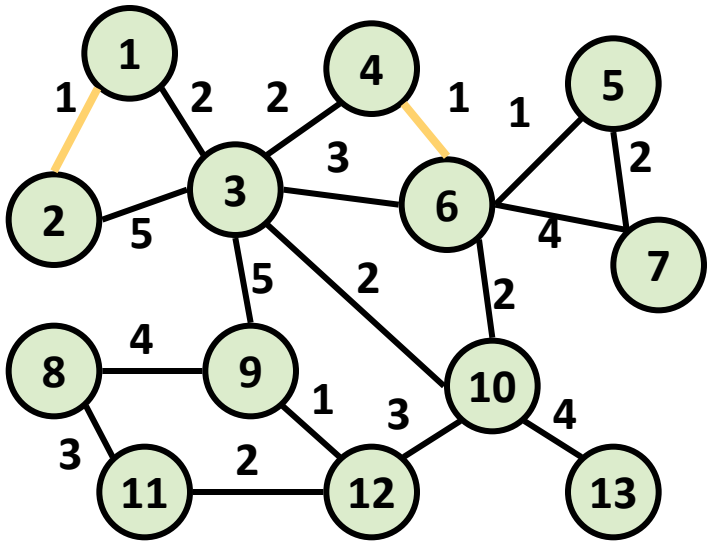


$T = \{\{1,2\}\}$

$L = \{\{4,6\},\{5,6\},\{9,12\},\{1,3\},\{3,4\},$
$\{5,7\},\{3,10\},\{6,10\},\{11,12\},\{3,6\},$
$\{8,11\},\{10,12\},\{6,7\},\{8,9\},\{10,13\},$
$\{2,3\},\{3,9\}\}$

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

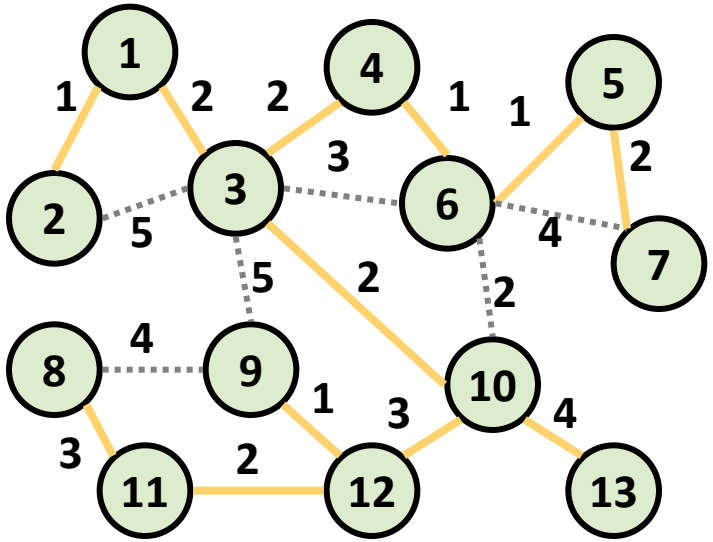
Beispiel



$T = \{\{1,2\},\{4,6\}\}$

$L = \{\{5,6\},\{9,12\},\{1,3\},\{3,4\},$
$\{5,7\},\{3,10\},\{6,10\},\{11,12\},\{3,6\},$
$\{8,11\},\{10,12\},\{6,7\},\{8,9\},\{10,13\},$
$\{2,3\},\{3,9\}\}$

...



$\{6,10\},\{3,6\},\{6,7\},\{8,9\}$ bilden Zyklen !

$T = \{\{1,2\},\{4,6\},\{5,6\},\{9,12\},\{1,3\},\{3,4\},$
$\{5,7\},\{3,10\},\{11,12\},\{8,11\},\{10,12\},$
$\{10,13\}\}$

$L = \{\{2,3\},\{3,9\}\}$

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Laufzeitanalyse

- **Sortieren**: $\mathcal{O}(|E| \log |E|)$
- **`while`-Schleife**:
  - max. $|E|$ Durchläufe
  - Entnahme und Löschen von e: $\mathcal{O}(1)$
  - Test auf Zyklus: bei Verwendung spezieller Datenstruktur („**union/find**“) praktisch in $\mathcal{O}(1)$
- **Gesamtlaufzeit**:
  $\mathcal{O}(|E| \log |E|)$

```
Graph MinSpanningTree(Graph G = (V,E))
{
  Graph T; List L; Edge e;

  T = {};
  L = Sort(E);

  while (#Kanten in T < #Knoten in G – 1 &&
         !IsEmpty(L)) {
    e = FirstElem(L);
    Delete(e, L);
    if (!Cycle(e, T)) { T = T ∪ {e}; }
  }

  if (#Kanten in T < #Knoten in G – 1) {
    printf("Fehler – kein Spannbaum möglich!");
  }

  return T;
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

11. Techniken zum Algorithmenentwurf

11.1 Rekursion und Iteration
11.2 Backtracking
11.3 Kombinatorische Optimierung
11.4 Schwierige Probleme und Heuristiken
11.5 Dynamische Programmierung
11.6 Branch and Bound

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Rekursion und Iteration

- **Rekursion**: lat. *recurrere* (zurücklaufen)
  - Definition einer Funktion `f` mittels `f` selbst
  - Im Programm: Berechnung von `f` durch erneuten Aufruf von `f` (mit anderen Parametern)
  - Häufig elegante und kompakte Funktionsdefinition
  - Nachteile: Höherer Speicherbedarf durch Laufzeit-Stack sowie evtl. Gefahr von Endlosschleifen
- **Iteration**: lat. *iterare* (wiederholen)
  - Berechnung einer Funktion `f` durch einzelne Schritte auf bestimmter Datenstruktur
  - Typischerweise als Schleife (z.B. `for`) realisiert
  - Vor- und Nachteile komplementär zur Rekursion

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel: Fakultätsfunktion

- Siehe auch Kap. 3
- $\mathbf{fact}(n) = 1 \times 2 \times \ldots \times n$
- Laufzeit für beide Algorithmen $\mathcal{O}(n)$
- Rekursive Variante ist **kompakter**
- Wird mittels **Laufzeit-Stack** abgearbeitet (s. Kap. 8)
- D.h. **Zusatzaufwand** für Stack-Bereitstellung und „Aufräumen“
- Daneben: Compiler können Schleifen oft gut **optimieren**
- Iterative Variante daher praktisch meist **schneller**

Iterativ

```
int factorial(int n)
{
  int i, fact;

  fact = 1;
  for (i = 2; i <= n; i++) {
    fact *= i;
  }

  return fact;
}
```

Rekursiv

```
int factorial(int n)
{
  if (n <= 1)
    return 1;
  else return
    n * factorial(n-1);
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel: Fibonacci-Zahlen

- $\mathbf{fib}(n) = \begin{cases} n, & \text{für } n \leq 1 \\ \text{fib}(n-1) + \text{fib}(n-2), & \text{für } n > 1 \end{cases}$

  (d.h. $\text{fib}(0) = 0, \quad \text{fib}(1) = 1$)

- **Fibonacci-Folge**:

  $(0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \ldots)$

- Viele Anwendungen in der **Mathematik**
- Auch in der **Natur** zu beobachten
- Siehe http://www.youtube.com/watch?v=iPKUe-69PdA

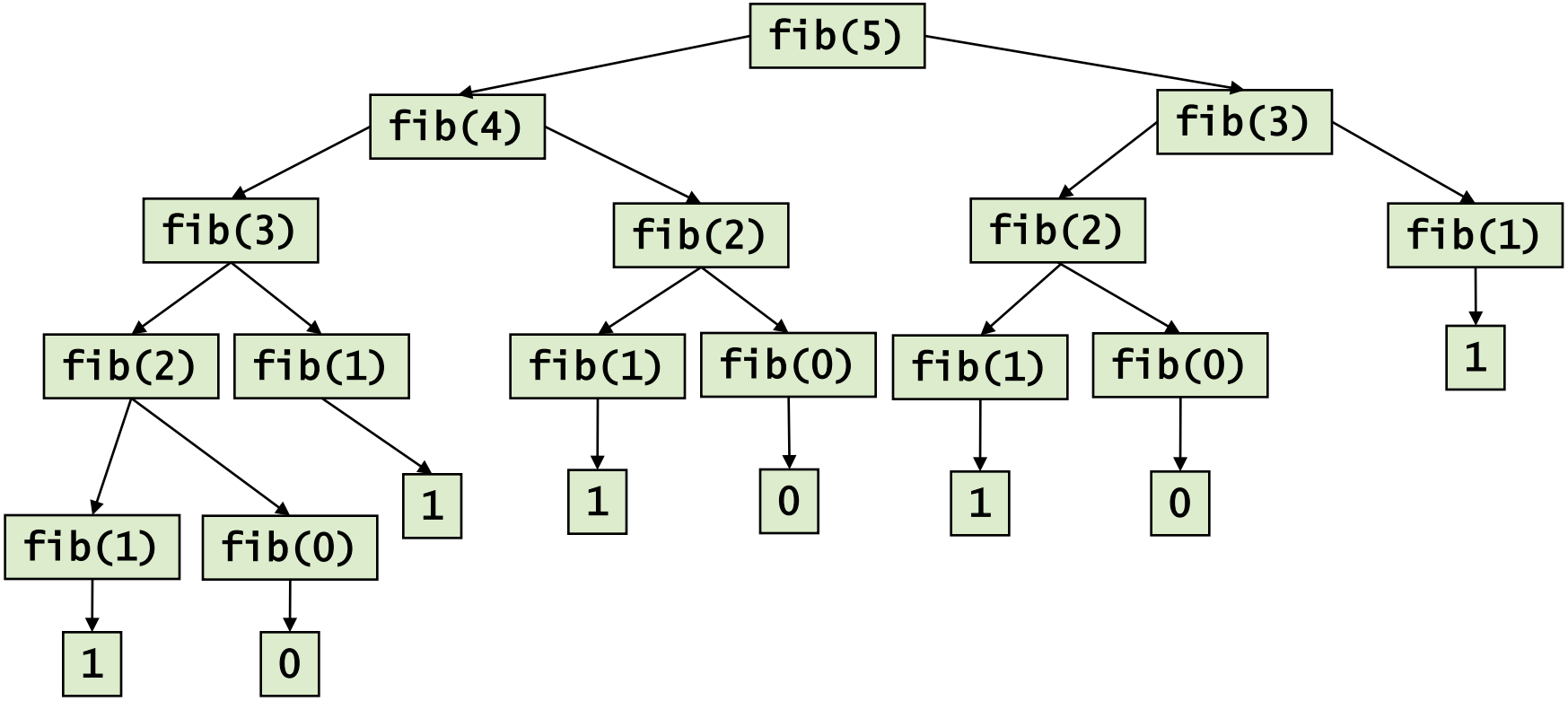


„Naive“ Rekursion:

```
int fib(int n)
{
  if (n <= 1)
    return 1;
  else
    return fib(n-1) + fib(n-2);
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Laufzeitanalyse



- Aufruf von `fib` erfordert i.a. **zwei rekursive Aufrufe** von `fib`
- Rekursionsbaum wächst **exponentiell**!
- Zeitbedarf für Aufruf **`fib(n)`**:
  - $\mathcal{O}(1)$ für $n \leq 1$
  - Zeit für `fib(n-1)` + Zeit für `fib(n-2)` + 1 für Addition

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Laufzeitanalyse

- $\boldsymbol{T(n) = T(n-1) + T(n-2) + 1}$
- Grobe **Abschätzung**:

$$
\begin{aligned}
T(n) &\geq \text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2) \\
&\geq 2 \times \text{fib}(n-2) \\
&= 2 \times \big(\text{fib}(n-3) + \text{fib}(n-4)\big) \\
&= 2 \times \text{fib}(n-3) + 2 \times \text{fib}(n-4) \\
&\geq 4 \times \text{fib}(n-4) \\
&\ldots \\
&\geq 2^{k-1} \times \text{fib}\big(n-2(k-1)\big) \quad [\text{ setze } n = 2k \text{ }] \\
&= 2^{k-1} \times \text{fib}\big(n-2(\tfrac{1}{2}n-1)\big) = 2^{k-1} \times \text{fib}(2) \\
&= 2^{k-1} \\
&= 2^{n/2-1} = 1/2 \times 2^{n/2}
\end{aligned}
$$

- Also $T(n) = \boldsymbol{\mathcal{O}(2^n)}$ (mindestens)
- Zusätzlicher Overhead für **Stackverwaltung** (konst. Faktor)

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Iterative Berechnung der Fibonacci-Folge

- **Problem** der rekursiven Version: Bei Berechnung von $\mathrm{fib}(n)$ wird Teilfolge $\mathrm{fib}(n-2)$ stets unnötig **doppelt** berechnet
- **Iterative** Version:

```
int fib(int n)
{
  int f1, f2, f;
  if (n <= 1) return 1;

  f1 = 1;
  f2 = 0;
  for (i=2; i<=n; i++) {
    f = f1 + f2;
    f2 = f1;
    f1 = f;
  }

  return f;
}
```

- Laufzeit $\boldsymbol{\mathcal{O}(1)}$ für $n \leq 1$
- Laufzeit $\boldsymbol{\mathcal{O}(n)}$ sonst

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

11. Techniken zum Algorithmenentwurf

11.1 Rekursion und Iteration

11.2 Backtracking

11.3 Kombinatorische Optimierung

11.4 Schwierige Probleme und Heuristiken

11.5 Dynamische Programmierung

11.6 Branch and Bound

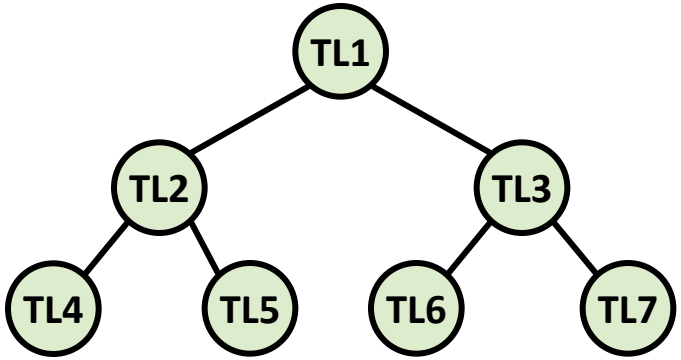
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Backtracking

- „Rücksetzverfahren“
- **Grundprinzip**:
  - Ausbau von Teillösung zu Gesamtlösung
  - „Raten“, falls Weg zur Gesamtlösung nicht feststeht
  - Rücknahme früherer Entscheidungen, falls keine Gesamtlösung erreichbar
- **Eigenschaften**:
  - Existierende Lösung wird garantiert gefunden, indem notfalls alle Möglichkeiten ausprobiert werden
  - Dieser Fall führt jedoch zu schlechtem Laufzeitverhalten

Analogie:

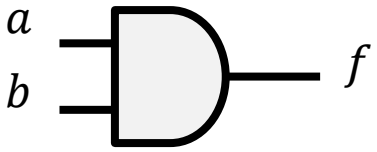
- DFS in Graphen
- TL = Teillösung



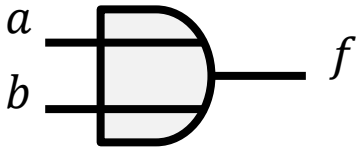
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel aus der technischen Informatik

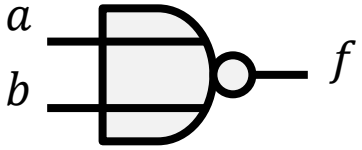
- Schaltkreise für Boolesche Funktionen
- **Boolesche Funktion** ist eine Abbildung $f : \{0,1\}^2 \rightarrow \{0,1\}$
- **Schaltkreis** = Hardware-Implementierung einer Booleschen Funktion
- Einfache Funktionen ($f : \{0,1\} \times \{0,1\} \rightarrow \{0,1\}$) werden durch **Logikgatter** realisiert, z.B. AND, OR, NOR
- Komplexe Funktionen durch **Zusammensetzen** von Gattern



AND: $f = 1 \Leftrightarrow a = 1 \text{ UND } b = 1$



OR: $f = 1 \Leftrightarrow a = 1 \text{ ODER } b = 1$

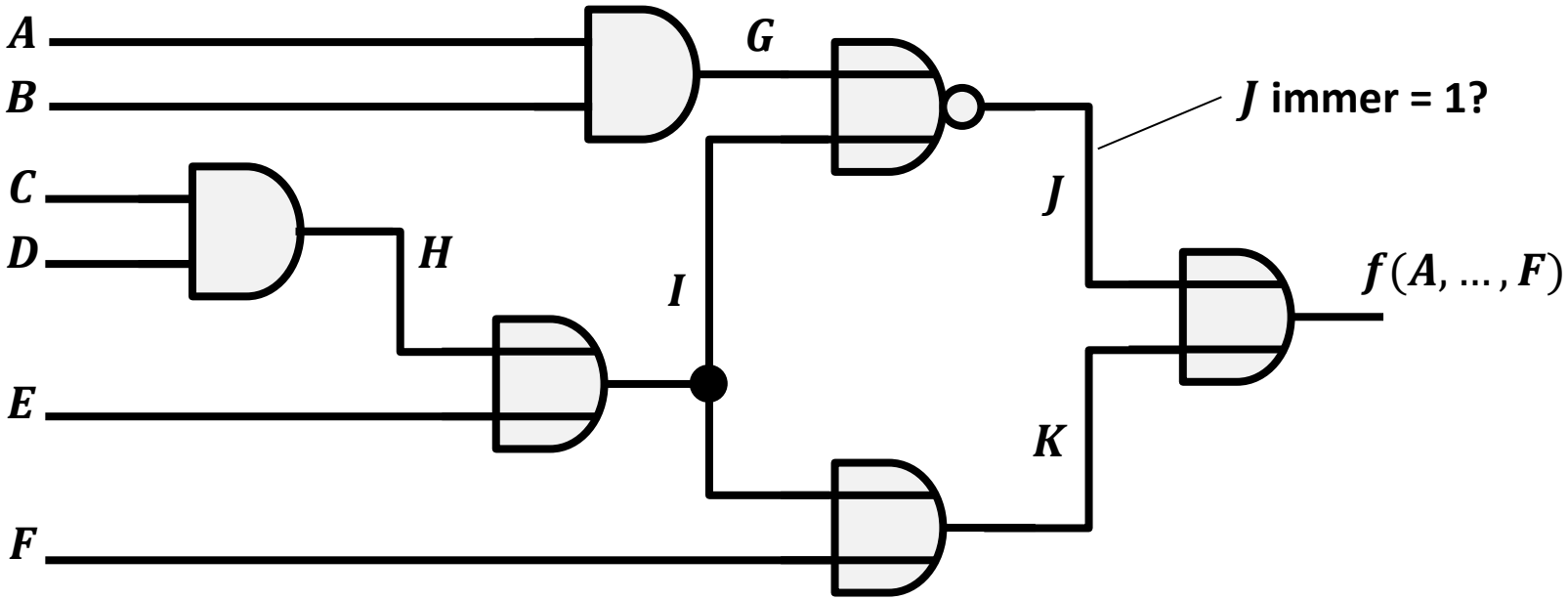


NOR: $f = 1 \Leftrightarrow \text{NICHT}(a = 1 \text{ ODER } b = 1)$

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Testen von Schaltkreisen

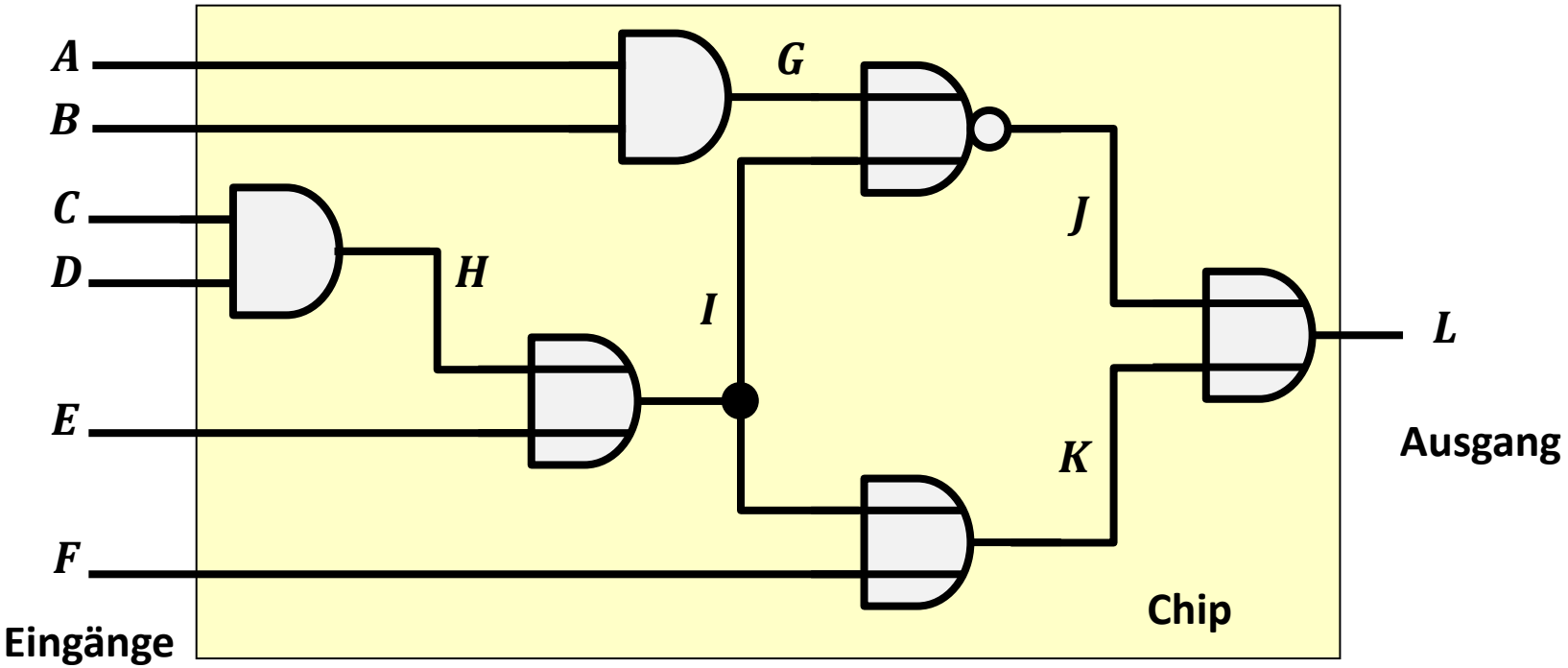
- Bei der Herstellung von **integrierten Schaltkreisen** (*integrated circuit*, IC) als Silizium-Chips können Fehler auftreten
- **Fehlermodell**: eine beliebige Leitung im Schaltkreis ist (z.B. durch Kurzschluss) immer konstant 0 oder 1
- Beispiel:



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Testen von Schaltkreisen

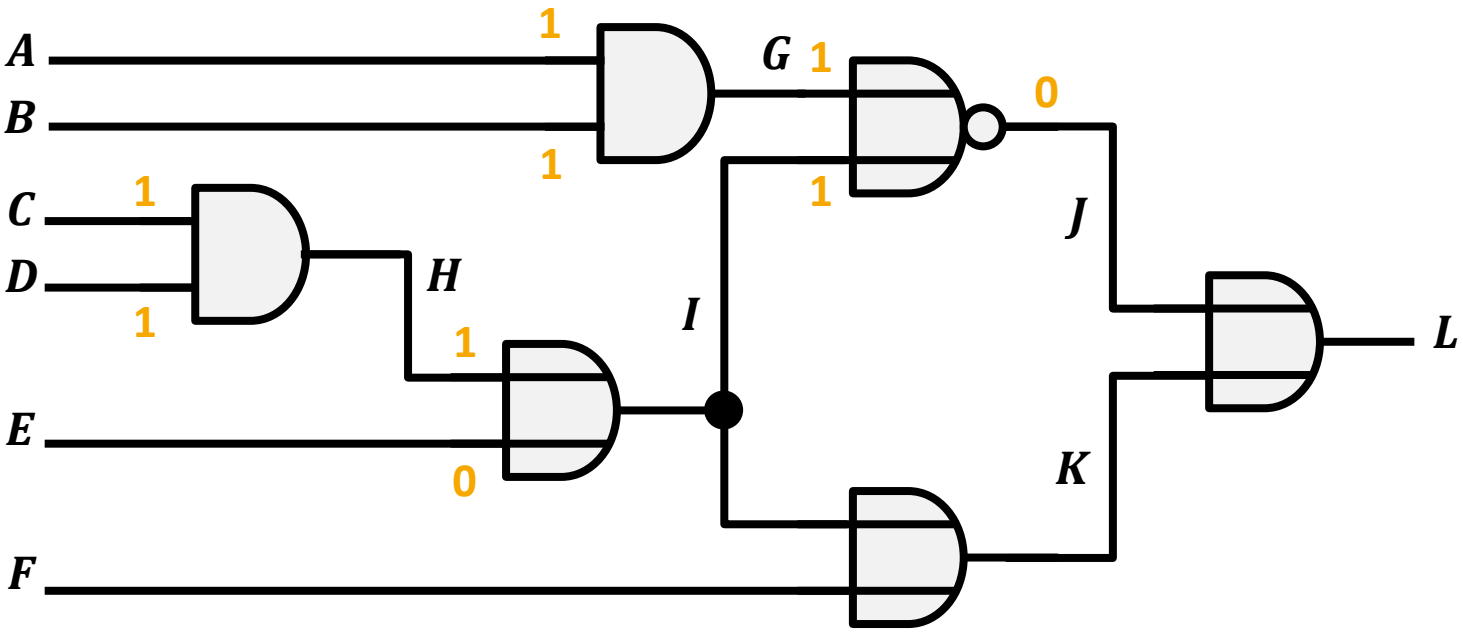
- Nur **Ein- und Ausgänge** der Schaltung können gesetzt bzw. gemessen werden, jedoch keine internen Leitungen
- Wie kann $J = 1$ getestet werden?
- Geeigneten binären **Eingabevektor** $(A - F)$ wählen, so dass im korrekten Fall $J = 0$
- Beobachten der **Ausgabe** L



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Bestimmung des Eingabevektors

- Setze $\boldsymbol{J = 0}$, sonst Fehler nicht messbar
- $J = G$ NOR I, also z.B. $G = 1, I = 1$
- $G = A$ AND B, also $A = 1, B = 1$
- $I = H$ OR E, also z.B. $H = 1, E = 0$
- $H = C$ AND D, also $C = 1, D = 1$
- Dieser Vektor $A - E$ erzwingt $J = 0$, **falls Schaltkreis korrekt**

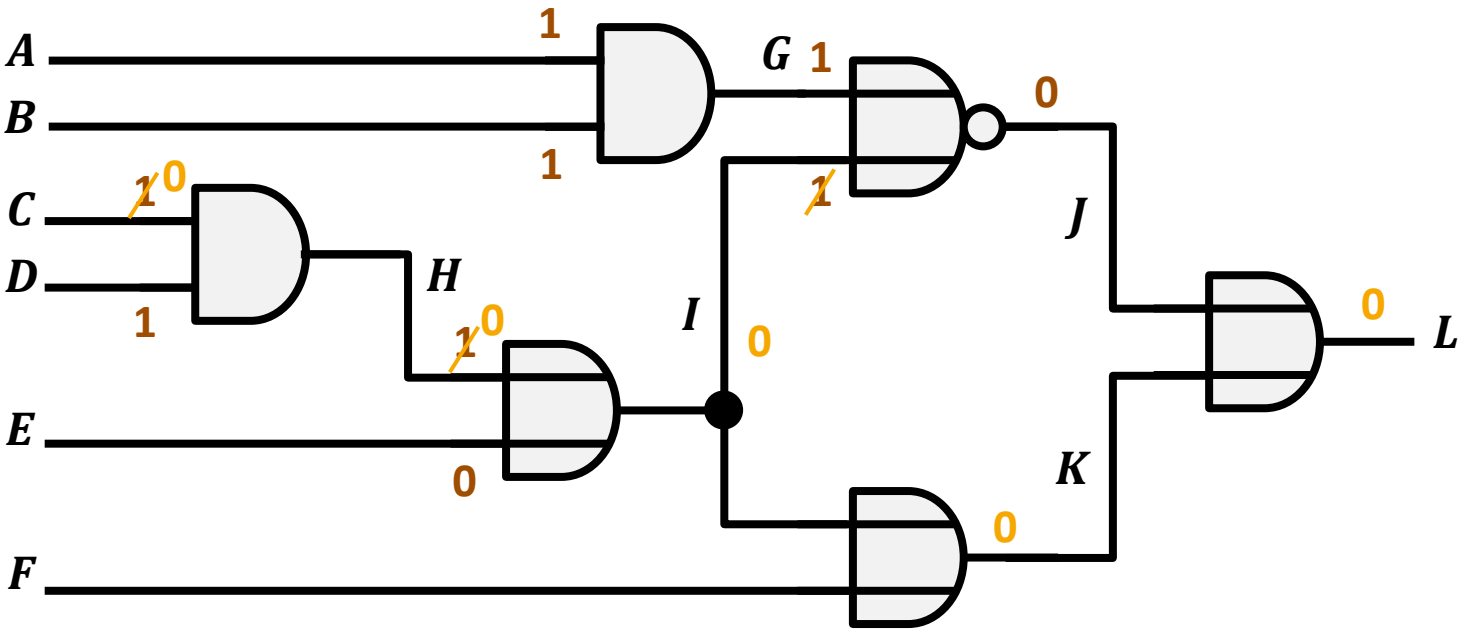


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beobachtung der Ausgabe

- $K = 1$ impliziert $L = 1$, unabhängig von J
- Also $K = 0$ zu setzen
- $K = I \text{ OR } F$, also $F = 0,\ I = 0$ (Widerspruch zu $I = 1$)
- Backtracking:

  $I = 0,\ H = 0,\ C = 0$
- Dann ist $\boldsymbol{L = 0}$ $\Leftrightarrow$ **Fehler $J = 1$ liegt nicht vor**



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

11. Techniken zum Algorithmenentwurf

11.1 Rekursion und Iteration

11.2 Backtracking

11.3 Kombinatorische Optimierung

11.4 Schwierige Probleme und Heuristiken

11.5 Dynamische Programmierung

11.6 Branch and Bound

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Kombinatorische Optimierung

- Konstruktion einer **Lösung eines Problems** durch Auswahl einer **Teilmenge** einer **Menge von diskreten Elementen**
- Jede gültige Lösung muss einer Menge von **Randbedingungen** genügen
- **Gesucht**: Lösung, die bzgl. einer **Zielfunktion** optimal ist (d.h. minimal oder maximal, je nach Problemstellung)
- **Beispiel**: minimaler Spannbaum in einem Graphen
  - Menge der diskreten Elemente:
    - alle Graphkanten
  - Gültige Lösung:
    - Alle Knoten werden genau einmal erfasst
    - Zusammenhängend und keine Zyklen
  - Zielfunktion:
    - Summe der Kantengewichte → min

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Optimierung durch Greedy-Algorithmen

- *Greedy* = gierig
- Ausgehend von Teillösung wähle Fortsetzung, welche zum **aktuellen Zeitpunkt den größten Gewinn verspricht**
- Eigenschaften:
  - Meist **geringe Laufzeit**
  - Optimale Lösung evtl. **nicht gefunden**
- Beispiel: **Kruskal-Algorithmus**
  - Greedy: wähle stets nächst-billigste Kante
  - Laufzeit $\mathcal{O}(|E|\log|E|)$
  - Optimum garantiert gefunden

```
Graph MinSpanningTree(Graph G = (V,E))
{
  Graph T; List L; Edge e;

  T = {};
  L = Sort(E);

  while (#Kanten in T < #Knoten in G-1
      && !IsEmpty(L)) {
    e = FirstElem(L);
    Delete(e,L);
    if (!Cycle(e,T)) {
      T = T ∪ {e};
    }
  }
}
```


© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel: Münzwechselproblem

- Landeswährung enthält Münzen zu $\boldsymbol{n_1, \ldots, n_k}$ **Einheiten**
- $n_1 > n_2 \ \ldots \ > n_k$
- Stets $\boldsymbol{n_k = 1}$, sonst nicht alle Beträge wechselbar
- Z.B. **Euro/Cent**:
  $n_1 = 200, \ \ n_2 = 100, \ \ n_3 = 50,$
  $n_4 = 20, \ \ n_5 = 10, \ \ n_6 = 5,$
  $n_7 = 2, \ \ n_8 = 1$
- **Problem**: Auszahlung von Betrag N mit verfügbaren Münzen
- **Diskrete Elemente**: Münzen
- **Gültige Lösung**:
  Menge von Münzen mit Gesamtwert N
- **Zielfunktion**:
  möglichst wenige Münzen verwenden



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Greedy-Algorithmus

- **Strategie**: immer möglichst viele „große“ Münzen wählen, bevor man zu „kleineren“ greift
- „Untere Gaußklammerfunktion“: $\mathbf{floor}(x)$ = größte ganze Zahl, die nicht größer ist als x
- Führt bei „guten“ Stückelungen immer zum **Optimum**
- **Schlechte Stückelung**: z.B. $n_1 = 2n_2 + 1, \;\; n_2 > 1, \;\; n_3 = 1$

Greedy-Münzwechsel

```
void change(int N)
{
  int W, i;

  W = N;
  for (i=1; i<=k; i++) {
    f = floor(W/n_i);
    wähle f Münzen
     der Einheit n_i;
    W -= n_i * f;
  }
}
```

Schlechter Fall, z.B. $N = 3n_2$

z.B. $n_2 = 10, \;\; N = 30, \; n_1 = 21$

1 Münze zu Wert 21

dann $W = 30 - 21 = 9$

9 Münzen zu Wert 1

Zusammen 10 Münzen

Ausreichend wären 3 Münzen Wert 10

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

11. Techniken zum Algorithmenentwurf

11.1 Rekursion und Iteration

11.2 Backtracking

11.3 Kombinatorische Optimierung

11.4 Schwierige Probleme und Heuristiken

11.5 Dynamische Programmierung

11.6 Branch and Bound

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Komplexität von Optimierungsproblemen

- Für viele wichtige kombinatorische Optimierungsprobleme ist bekannt, dass sie **nicht in akzeptabler Zeit optimal lösbar** sind, da ihre Laufzeit (höchstwahrscheinlich) mindestens in $\mathcal{O}(2^n)$ liegt
- Sog. **NP-vollständige** Probleme
- **Rechenbeispiel**: Problemgröße $n = 64$, schneller Rechner führt 2^{32} Operationen/sec aus
- **Rechenzeit**: 2^{32} sec ≈ 136 Jahre
- **Prinzipielle Möglichkeiten**:
  1. Optimale Lösung (*brute force*) für kleine Werte von n
  2. Optimale Lösung, und dafür sorgen, dass *worst case* möglichst selten eintritt
  3. Gute, aber nicht notwendig optimale Lösung berechnen

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiele NP-vollständiger Probleme

- **Traveling salesman problem**:
  - Modell: Graph mit Kantengewichten
  - Gesucht: Kürzester Rundweg (Zyklus), der alle Knoten genau einmal enthält
- **Graph-Färbung**:
  - Modell: ungerichteter Graph
  - Gesucht: Färbung der Knoten mit min. Anzahl Farben, so dass benachbarte Knoten unterschiedliche Farben haben
- **Erfüllbarkeit Boolescher Funktionen**:
  - Gibt es für Funktion f eine Eingabe x mit $f(x) = 1$?



Kürzester Rundweg über die 15 größten Städte Deutschlands (von > 43 * 10^9 möglichen) [Wikipedia]

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

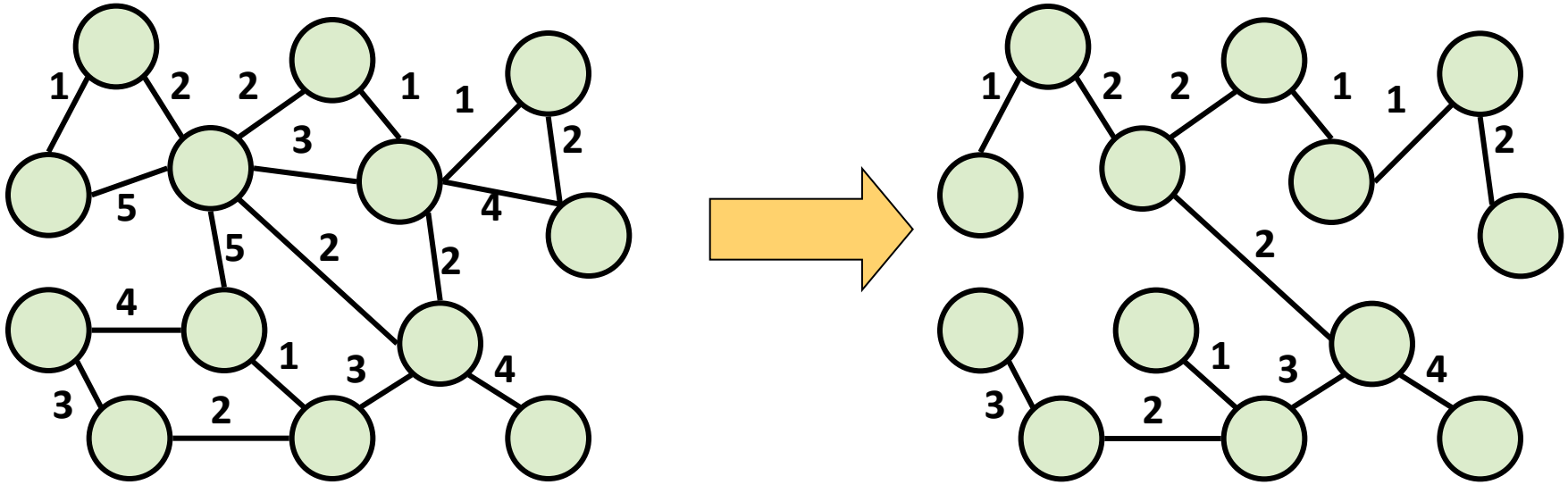
Komplexität von Optimierungsproblemen

- Zur Kategorie 3 gehören **heuristische Algorithmen**
- **Heuristik**:
  - Kompromiss zwischen Rechenaufwand und Lösungsqualität
  - Basierend auf **problemspezifischem Wissen** und „Daumenregeln“
  - **Greedy-Algorithmen** sind oft Heuristiken
  - Gewünscht: möglichst oft sehr gute Lösung
  - **Nachteil**: Lösung kann auch beliebig schlecht sein
- Außerdem möglich: **Approximationsalgorithmen**
  - Garantie, dass Lösungsqualität nicht um mehr als bestimmten Faktor vom Optimum abweicht
  - Faktor = „**Güte**“ des Approximationsalgorithmus

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel für Heuristik

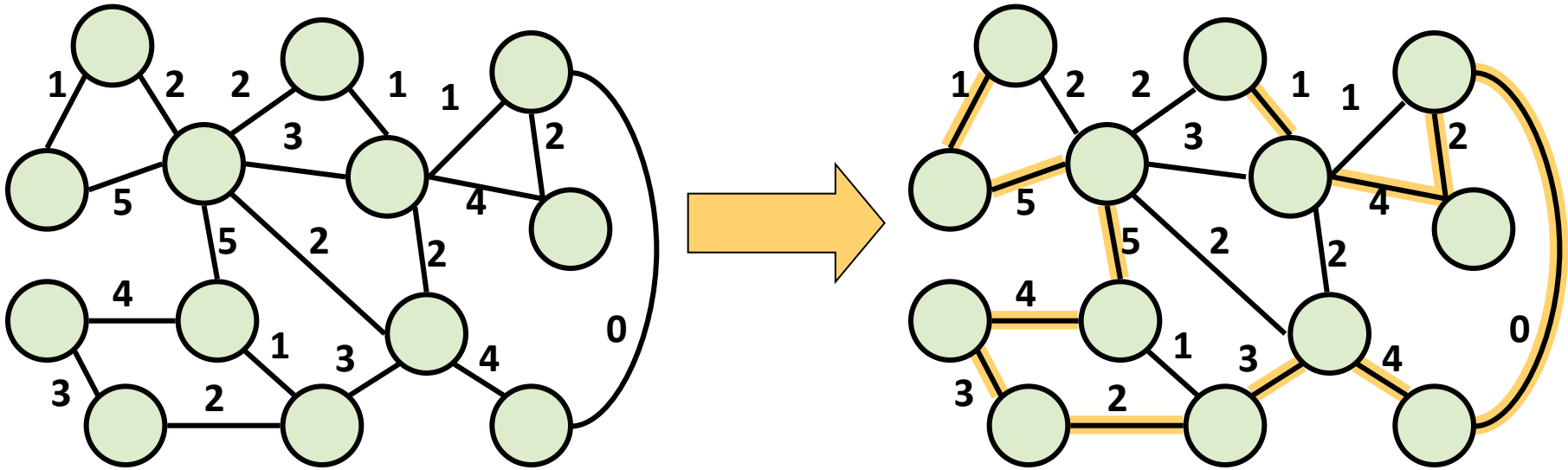
- **Gegeben**: kantengewichteter **vollständiger** Graph $G = (V, E, w)$, d.h. für alle Knoten u, v gilt $\{u, v\} \in E$
- **Gesucht**: Pfad $P = (v_1, \ldots, v_n)$ in G, der
  - jeden Knoten in V genau einmal enthält und
  - eine maximale Kantengewichtssumme besitzt
- Vergleiche minimaler (bzw. maximaler) Spannbaum:



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Maximal gewichteter Pfad

- Man kann zeigen: Problem ist **NP-vollständig**
- Ausnutzung der **Ähnlichkeit** zum Spannbaumproblem
  - Pfad = „entarteter“ (d.h. linearer) Spannbaum
  - Modifikation des **Kruskal-Algorithmus**: nächste Kante e nur gültig, falls
    - Kein Zyklus entsteht
    - **Keine Verzweigungen** entstehen (d.h. kein „echter“ Baum)
  - Erzeugt heuristisch **gute Lösungen** in kurzer Zeit



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

11. Techniken zum Algorithmenentwurf

11.1 Rekursion und Iteration

11.2 Backtracking

11.3 Kombinatorische Optimierung

11.4 Schwierige Probleme und Heuristiken

11.5 Dynamische Programmierung

11.6 Branch and Bound

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

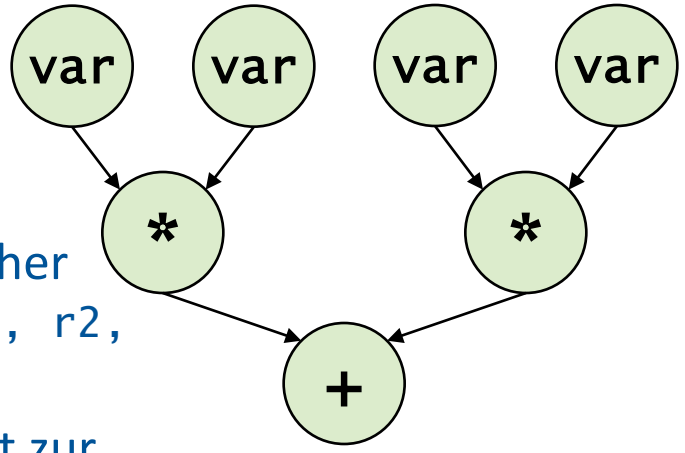
Dynamische Programmierung

- **Effizientes exaktes Lösen** von Optimierungsproblemen mit folgender Eigenschaft:
  - Die optimale Lösung des Gesamtproblems kann effizient aus **optimalen Lösungen von Teilproblemen** berechnet werden
- Beispiel aus dem **Compilerbau**:
  - Rechner besitzt elementare **Maschinenbefehle** (Assemblerbefehle, s. Kap. 1), z.B. Addition oder Multiplikation zweier Zahlen
  - **Optimale Befehlsauswahl**: Wie kann ein komplexer arithmetischer Ausdruck in der Programmiersprache C am schnellsten mit den verfügbaren Assemblerbefehlen berechnet werden?

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel: Befehlsauswahl

- Ausdruck: `(a * b) + (c * d)`
- Befehle:
  - Modell:
    - Rechner lädt Variablen (`var`) aus dem Speicher (`load`) und speichert Werte in Registern (`r1, r2, r3`)
    - Jeder Befehl besitzt ein Kostenmaß (z.B. Zeit zur Ausführung)
  - **`load`**: `r1 = var,` Kosten 1
  - **`add`**: `r1 = +(r1,r2),` Kosten 2
  - **`mul`**: `r1 = *(r1,r2),` Kosten 2
  - **`copy`**: `ri = rj,` Kosten 1
  - **`muladd`**: `r1 = +(*(r1,r2),r3),` Kosten 3

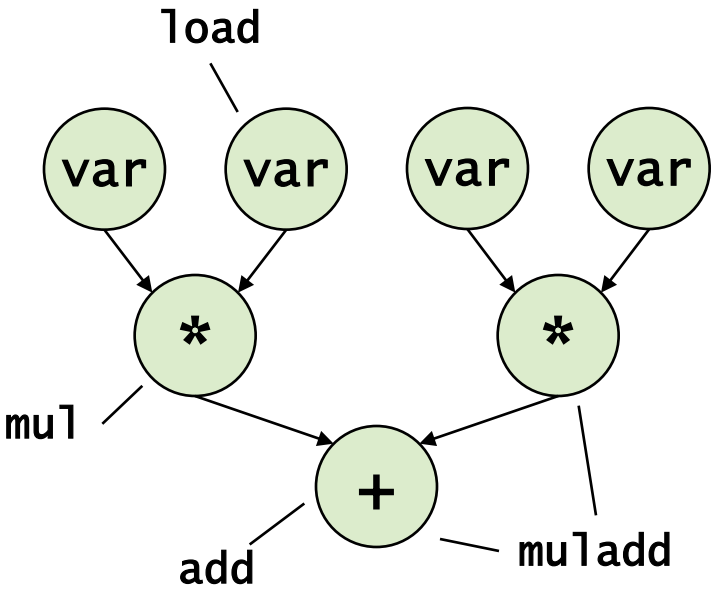


Abstrakte Darstellung des Ausdrucks als Baum

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel: Befehlsauswahl

- **Ansatz**:
  - Ausdrucksbaum mit möglichst „billiger“ Menge von Befehlen überdecken
  - Lösung für den gesamten Ausdrucksbaum aus optimalen Lösungen für seine Teilbäume bestimmen
- **Für jeden Teilbaum merken**:
  - Welcher Befehl an dessen Wurzel passt (z.B. „+“ mittels „add“ zu überdecken, „*“ mittels „`mul`“)
  - In welchem Register das Ergebnis des Teilbaums gespeichert wird
  - Welche Gesamtkosten hierbei entstehen



`var`-Knoten:

Überdeckung mit „`load`“

Ergebnis dann in Register `r1`

Kosten: 1

Überdeckung mit „`copy r1, r2`“

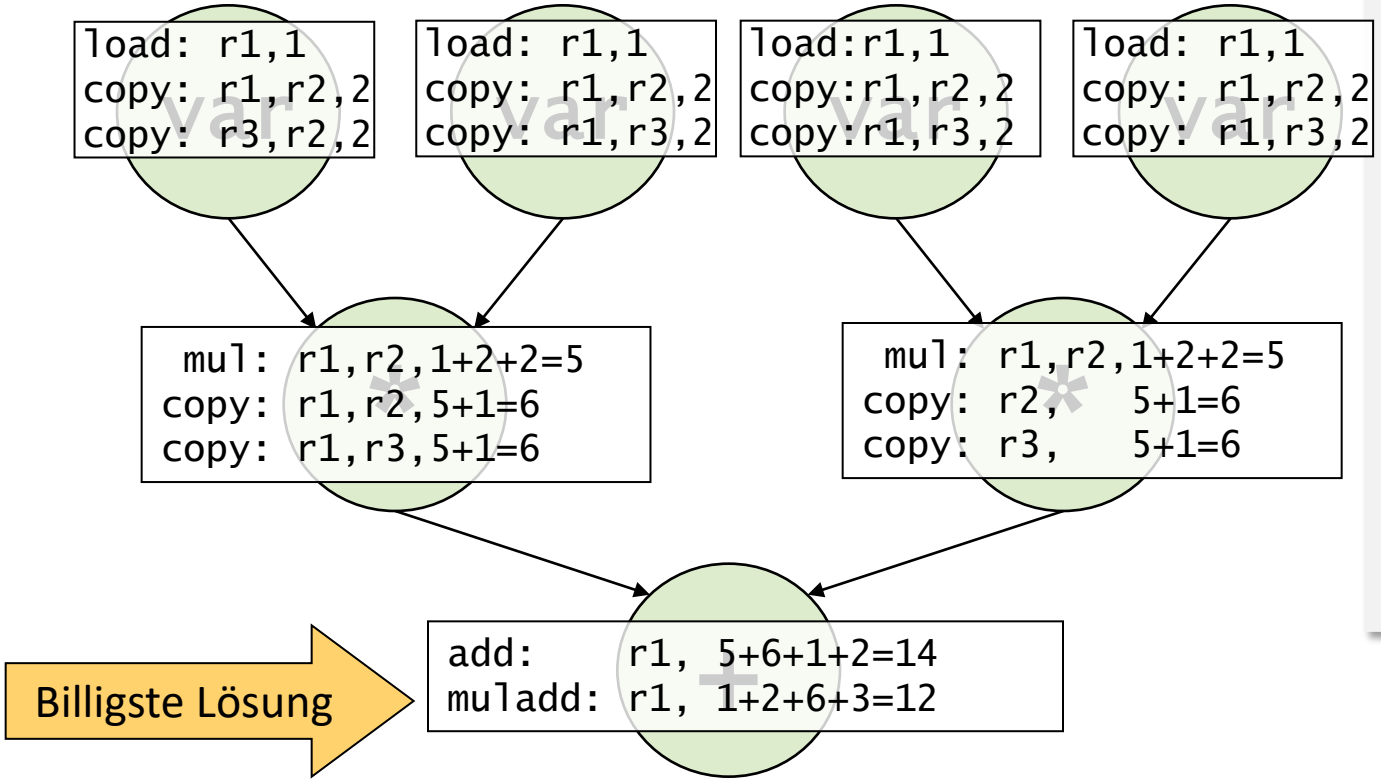
Ergebnis dann in Register `r2`

Kosten: 2

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel: Befehlsauswahl

Überdeckung des Ausdrucksbaums



Befehle

```
load:
r1  var
  → Kosten 1
add:
r1  +(r1,r2)
  → Kosten 2
mul:
r1  *(r1,r2)
  → Kosten 2
copy:
ri rj
  Kosten 1
muladd:
r1  +(*(r1,r2), r3)
  → Kosten 3
```

- Aus dieser Lösung kann die **optimale Assembler-Befehlsfolge** unmittelbar konstruiert werden
- **Laufzeit**: $\mathcal{O}(n)$ bei n Baumknoten

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

11. Techniken zum Algorithmenentwurf

11.1 Rekursion und Iteration

11.2 Backtracking

11.3 Kombinatorische Optimierung

11.4 Schwierige Probleme und Heuristiken

11.5 Dynamische Programmierung

11.6 Branch and Bound

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

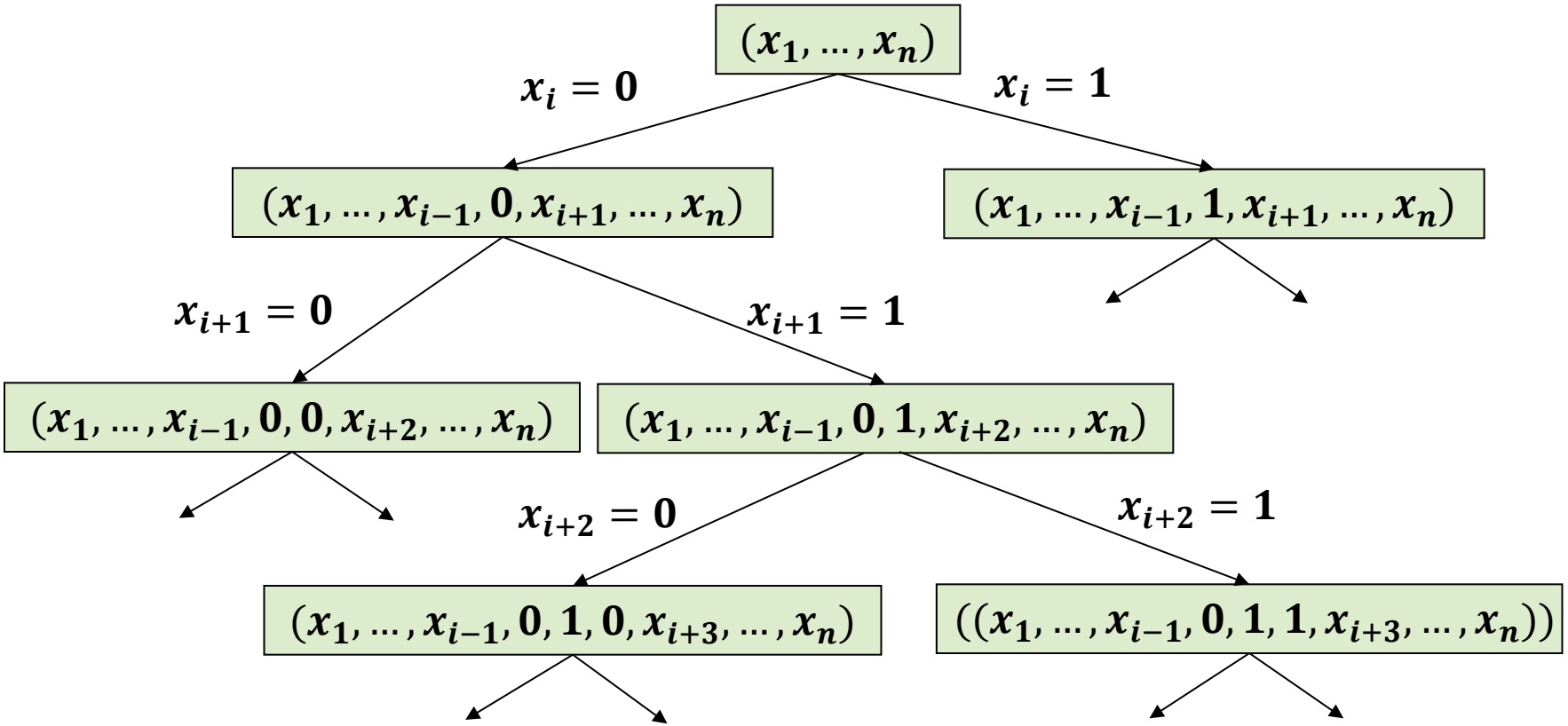
Branch and Bound

- „Verzweigung und Schranke“
- **Exaktes Lösen** schwieriger (NP-vollständiger) Optimierungsprobleme
- **Idee**:
  - Lösen eines „schwierigen“ kombinatorischen Optimierungsproblems entspricht Durchlauf eines **Entscheidungsbaumes**
  - Entscheidungsbaum typischerweise **exponentiell groß**
  - „**Beschneidung**“ des Baumes verringert Laufzeit ohne Verlust an Optimalität
- Ähnliches Konzept auch bei **Spielbäumen** (TicTacToe, Dame, Schach, …)
- **Charakteristik**:
  - Im *worst case* exponentielle Laufzeit
  - Im *average case* wesentlich schneller

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Entscheidungsbaum

- **Annahme**: Vektor $V = (x_1, \ldots, x_n)$ von 0/1-Variablen so zu wählen, dass eine Kostenfunktion $f(V)$ minimiert wird
- **Entscheidungsbaum**: Festlegen jeweils einer Variable



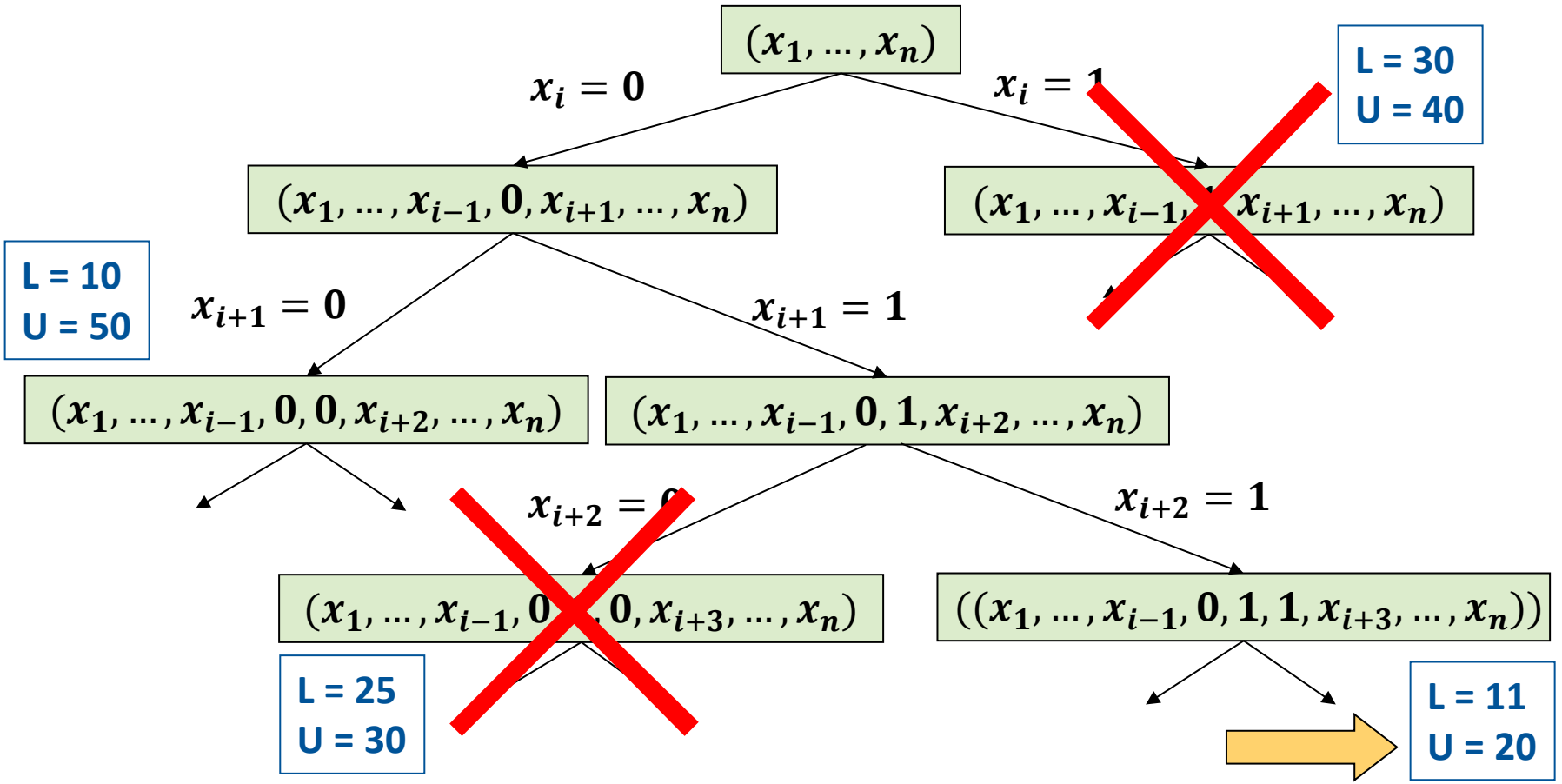
© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Schranken

- Während der Verzweigung: Berechnung von **oberen** (*upper bound*, U) und **unteren Schranken** (*lower bound*, L) für die Kostenfunktion f an jedem Knoten K, so dass im Teilbaum unterhalb von K
  - in jedem Fall **mindestens Kosten L** entstehen
  - garantiert eine Lösung mit **Kosten höchstens U** existiert
- **Bound**: Besitzt Knoten K eine **untere Schranke L, welche größer ist als das kleinste bisher bekannte U**, dann kann der Teilbaum unterhalb von K von der Suche **ausgeschlossen** werden, ohne dass eine optimale Lösung verloren geht
- **Wichtig**:
  - Schranken sind schnell zu berechnen (sonst zu aufwendig)
  - Schranken sind „scharf“ (sonst kein effektives Bound)

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

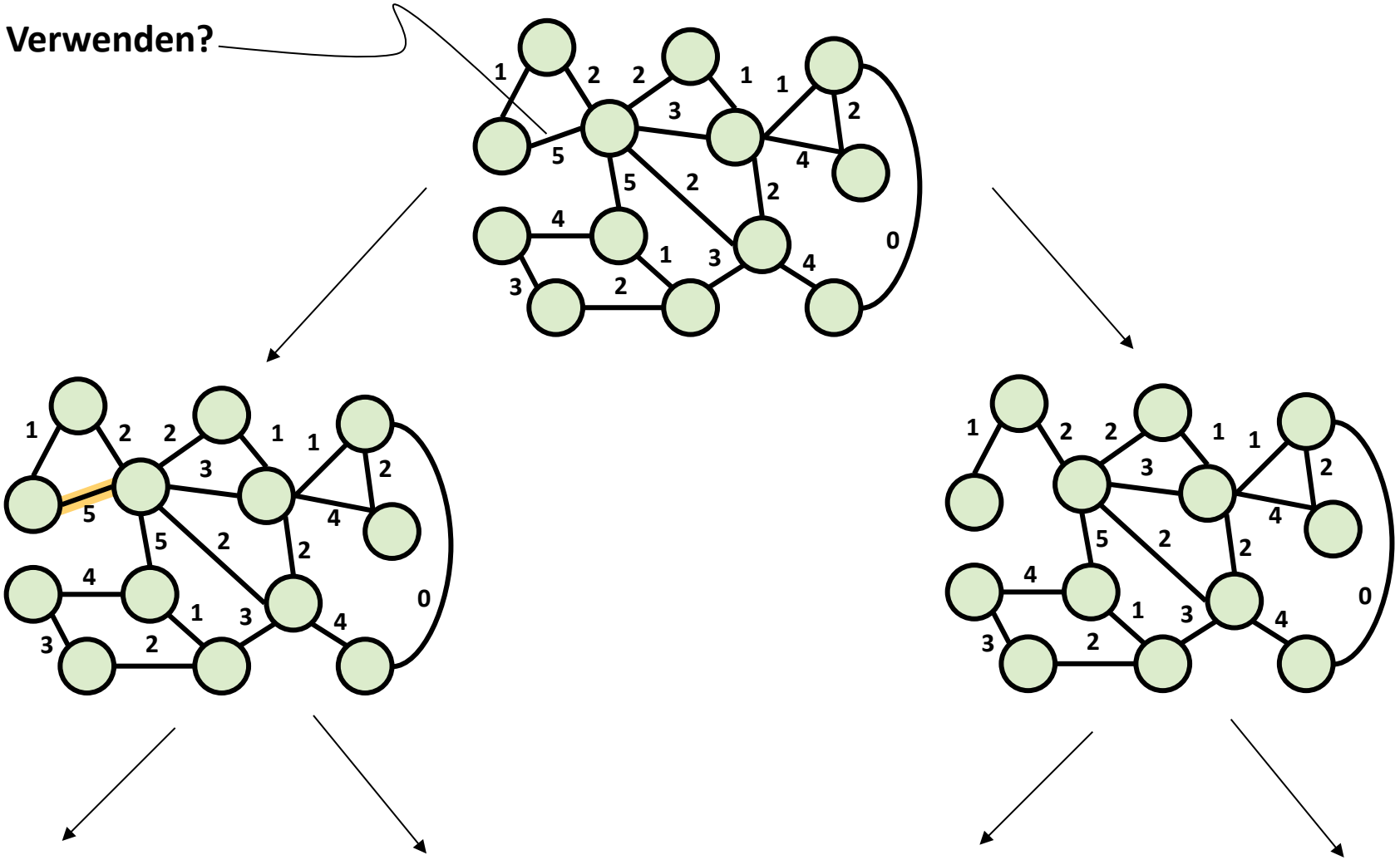
Beispiel für Bound



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel: max. gewichteter Pfad in Graphen

- **Branch**: bestimmte Kante wird in den Pfad aufgenommen (sofern gültig) oder aber explizit ausgeschlossen



© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Beispiel: max. gewichteter Pfad in Graphen

- Effiziente Berechnung der Schranken?
- **Gegeben**: Graph mit n Knoten und Teillösung mit k bereits fest gewählten Kanten
- **Obere Schranke U**:
  - Im besten Falle könnten noch die $n - k - 1$ „schwersten“ unter den noch nicht betrachteten Kanten hinzugefügt werden
  - Optimale Lösung kann höchstens schlechter sein
- **Untere Schranke L**:
  - Setze Teillösung mittels Heuristik durch Hinzunahme von $n - k - 1$ Kanten zu Gesamtlösung fort
  - Optimale Lösung kann höchstens besser werden
- **Maximierungsproblem**: Falls für einen Knoten der Wert von U kleiner ist als bisher bestes L, so kann der Teilbaum entfallen

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Bildnachweise

#20 Petr Kratochvil , „frau-sitzt-mit-laptop “, publicdomainpictures.net

„PC-CPU-Feld-Vektor-Bild“, publicdomainvectors.org

#23 Ldorfman - According to a work of Johanna Pung made for Wikimedia Deutschland – Own work, CC BY-SA 3.0,

commons.wikimedia.org/w/index.php?curid=21837408

#32 Max Pixel, “Medical Cold Fever Thermometer Temperature Digital“, maxpixel.net

#143 Tim Reckmann, „Kreditkarte“, CC-BY 2.0, ccnull.de

#163 MartinStr, „blasen-wasser-sprudelnd-lufblasen-230014“, pixabay.com

#213 ACBahn, „Warteschlange Eisdiele“, commons.wikimedia.org, CC GNU 1.2

#245 „gps-navigation-garmin-gerät-304842“, pixabay.com

openstreetmap.de, Open Database Licence (ODbL) 1.0

#270 „Grundsätzliche Informationen zum Netzausbau”, www.buergerdialog-stromnetz.de

#280 wirestock, „closeup-bee-sunflower-field-sunligh”, freepik.com

#294 „money_coins_loose_change_euro_cent_specie_metal_money_finance”, pxhere.com

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Literaturnachweise

- B.W. Kernighan, D.M. Ritchie: Programmieren in C, 2. Auflage, Hanser, 1990, ISBN 3-446-15497-3
- R. Sedgewick: Algorithmen in C++, Teil 1-4, Addison-Wesley, 2002, ISBN 3-8273-7026-4
- K. Loudon: Mastering Algorithms with C, 1. Auflage, O'Reilly, 1999, ISBN 978-1-56592-453-6

© T. Gemmeke, IDS, RWTH; R. Leupers, SSS

Stichwortregister zu C++ Sprachelementen

- Abstrakte Datentypen 138
- Ausgabe 39
- Funktionen
  - Lambda 119
  - Overloading 153
- Klassen 140
  - (De)Constructor 142
  - Public vs. Private 144
  - Inheritance 147
- Libraries 34
  - Using & namespace 40
- Loops
  - Iterators 129
- Polymorphie
  - Statisch 113
  - Dynamisch 149
- Smart Pointer 70
  - Unique 71
  - Shared 72
  - Weak 73
- Streams 39, 155
- Template 117

© T. Gemmeke, IDS, RWTH